



> **cours**

8 • Console AWS

9 • Services AWS essentiels

10 • Bonnes pratiques

11 • Points d'attention

Ressources

Quiz du chapitre

Vocabulaire du chapitre

Terme	Définition
Cloud Computing	Modèle de fourniture de ressources informatiques accessibles par le réseau, disponibles à la demande et mesurées selon l'utilisation.
Scalabilité	Capacité d'un système à augmenter ou réduire sa capacité pour absorber une variation durable de charge.
Élasticité	Ajustement rapide, souvent automatique, des ressources à la charge observée.

Région AWS	Zone géographique indépendante dans laquelle AWS exploite plusieurs zones de disponibilité.
Zone de disponibilité (AZ)	Ensemble d'un ou plusieurs centres de données isolés des autres AZ d'une même région et reliés par un réseau à faible latence.
Service managé	Service dont AWS exploite une partie des composants techniques, par exemple le système hôte ou le moteur de base de données.
API	Interface de programmation qui permet à un logiciel de demander une opération à un service.
Haute disponibilité	Conception visant à maintenir un service accessible malgré la défaillance d'un composant prévu par l'architecture.

:::info Cette formation débute par les fondations théoriques indispensables avant de manipuler la console AWS : sans comprendre ce qu'est réellement le Cloud Computing, ses modèles économiques et ses responsabilités, il est impossible de faire des choix d'architecture pertinents par la suite.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Expliquer** ce qu'est AWS, son historique et son positionnement sur le marché du Cloud
- **Identifier** les cinq caractéristiques fondamentales du Cloud Computing selon le NIST
- **Distinguer** le modèle économique CAPEX du modèle OPEX et en expliquer l'impact sur les entreprises
- **Situer** les responsabilités respectives d'AWS et du client dans le modèle de responsabilité partagée
- **Différencier** les modèles de service IaaS, PaaS et SaaS
- **Comparer** les modèles de déploiement Cloud public, privé et hybride
- **Expliquer** les principes de la virtualisation, de la conteneurisation et des microservices
- **Décrire** l'infrastructure mondiale AWS (régions, zones de disponibilité, edge locations)
- **Appliquer** les six piliers du AWS Well-Architected Framework à un cas simple
- **Naviguer** dans l'AWS Management Console et identifier ses principales fonctionnalités
- **Utiliser** les outils de suivi budgétaire AWS (Billing Dashboard, Budgets, Cost Explorer) pour maîtriser les coûts :::

1. Introduction

1.1 Qu'est-ce qu'AWS et pourquoi le découvrir ?

Amazon Web Services (AWS) est une plateforme de cloud computing dont les premiers services largement disponibles, S3 et EC2, ont été lancés en 2006. Son catalogue couvre notamment le calcul, le stockage, le réseau, les bases de données, la sécurité, l'analyse de données et l'intelligence artificielle.

Les organisations utilisent ces services pour construire, héberger ou exploiter des systèmes informatiques sans posséder toute l'infrastructure physique sous-jacente. Le choix d'AWS ne dispense pas d'architecture, de sécurité ni de gouvernance : il déplace une partie des responsabilités vers le fournisseur.

Pourquoi apprendre AWS ?

- AWS fait partie des principaux fournisseurs mondiaux de cloud public.
- AWS fait partie des plateformes fréquemment rencontrées dans les environnements cloud.
- Ses services permettent d'étudier des concepts transférables : automatisation, élasticité, responsabilité partagée et conception résiliente.
- Son catalogue évolue régulièrement ; la documentation officielle reste la référence pour les fonctions disponibles.

Vous trouverez AWS dans **quasiment tous les secteurs** : startups qui testent une idée en quelques jours, grandes banques qui hébergent leurs systèmes critiques, hôpitaux qui gèrent les données médicales, gouvernements, universités, agences spatiales...

1.2 Quelques dates clés

Année	Événement	Explication pédagogique
2006	Lancement d'AWS avec S3 et EC2	AWS débute avec deux services fondamentaux : S3 pour le stockage objet (sauvegardes, fichiers, images) et EC2 pour le calcul (machines virtuelles). Ces deux services incarnent les piliers du cloud : stockage et puissance de calcul à la demande.
2009	Introduction de VPC et RDS	AWS ajoute VPC (Virtual Private Cloud) pour créer des réseaux privés isolés, et RDS (Relational Database Service)

		pour gérer des bases de données relationnelles sans administrer les serveurs. Cela marque l'arrivée des services réseau et des bases gérées.
2012	Lancement de DynamoDB	AWS introduit DynamoDB , une base NoSQL scalable et sans schéma, adaptée aux applications modernes (web, mobile, IoT). C'est le tournant vers les architectures serverless et les microservices.
2014	Lancement de AWS Lambda	Lambda exécute du code à la demande sans exposer au client l'administration des serveurs sous-jacents. Le terme <i>serverless</i> décrit cette abstraction, pas l'absence de serveurs.
Aujourd'hui	Plateforme de services étendue	AWS propose des services couvrant notamment le calcul, le stockage, le réseau, les données, la sécurité, l'automatisation et l'intelligence artificielle. Le catalogue évolue régulièrement.

Ces dates montrent comment AWS a évolué d'un simple fournisseur de serveurs et de stockage vers une **plateforme cloud complète**, capable de répondre à tous les besoins informatiques : hébergement, sécurité, automatisation, intelligence artificielle, etc.

1.3 Certifications AWS

AWS classe ses certifications en quatre catégories : **Foundational**, **Associate**, **Professional** et **Specialty**. Ces catégories indiquent la profondeur et le domaine évalués ; AWS n'impose ni formation préalable ni ordre obligatoire entre les examens.

Le niveau **Foundational** comprend notamment Cloud Practitioner et AI Practitioner. Il valide une compréhension générale du cloud ou de l'intelligence artificielle, sans cibler un rôle technique unique.

Le niveau **Associate** cible les professionnels qui déploient et opèrent concrètement des infrastructures AWS au quotidien. Il se décline selon le métier visé : **Solutions Architect – Associate** pour la conception d'architectures, **CloudOps Engineer – Associate (SOA-C03)** pour l'exploitation et la supervision — ce nom remplace SysOps Administrator depuis 2025 — et **Developer – Associate** pour l'intégration d'applications avec les API et services AWS. Le catalogue comprend également des certifications Associate orientées données et machine learning.

Le niveau **Professional** évalue des tâches techniques complexes. Le catalogue officiel comprend Solutions Architect, DevOps Engineer et Generative AI Developer au niveau Professional.

Le niveau **Specialty** valide une expertise approfondie dans un domaine. Le catalogue officiel actuel comporte Advanced Networking et Security. Comme les codes et examens évoluent, la page officielle doit être consultée avant toute inscription.

Les notions du programme recoupent une partie du périmètre de **Solutions Architect – Associate (SAA-C03)**, notamment IAM, EC2, S3, VPC, RDS et les principes Well-Architected. La formation ne se substitue toutefois ni au guide d'examen courant ni à l'expérience pratique recommandée.

 [Certifications AWS](#)

2. Fondamentaux du Cloud Computing

2.1 Qu'est-ce que le Cloud Computing ?

Le **Cloud Computing** est une évolution majeure dans la manière dont les systèmes d'information sont conçus, déployés et exploités.

Pendant des décennies, les entreprises ont investi massivement dans leurs propres infrastructures : datacenters internes, salles machines, serveurs physiques, systèmes de climatisation, sécurité, licences logicielles, équipes d'exploitation dédiées.

Ce modèle repose sur des investissements lourds (**CAPEX**) et des cycles de décision longs.

Le Cloud vient bouleverser cette logique en offrant un modèle **flexible, scalable** et **orienté services** : les ressources sont **louées à la demande** et **payées à l'usage**.

 [Documentation officielle AWS — What is Cloud Computing](#)

2.2 Les cinq caractéristiques fondamentales du Cloud (NIST)

Le **NIST (National Institute of Standards and Technology)** a défini cinq caractéristiques fondamentales du Cloud Computing. Ces critères permettent de distinguer une véritable solution Cloud d'une simple externalisation :

1. Libre-service à la demande (On-demand self-service)

Un utilisateur peut provisionner des ressources informatiques (par exemple, une machine virtuelle) sans intervention humaine du fournisseur.

En pratique avec AWS : Vous cliquez sur « Lancer une instance EC2 », configurez les paramètres, et en quelques secondes, votre serveur est actif et prêt à l'emploi.

2. Mise en commun des ressources (Resource pooling)

Les ressources physiques (serveurs, stockage, réseau) sont partagées entre plusieurs clients mais restent isolées logiquement.

En pratique avec AWS : Vos instances EC2 exécutent sur du matériel physique partagé avec d'autres clients, mais AWS garantit une isolation complète — vos données restent vos données.

3. Élasticité et scalabilité (Rapid elasticity)

Les ressources peuvent être augmentées ou réduites dynamiquement en fonction de la demande.

En pratique avec AWS : Votre application reçoit soudain 10 fois plus de visiteurs. AWS augmente automatiquement la capacité de calcul, puis la réduit quand le trafic baisse.

4. Mesurabilité et facturation à l'usage (Measured service)

Chaque ressource est mesurée et facturée selon la consommation réelle : CPU, mémoire, bande passante, stockage, etc.

En pratique avec AWS : Vous payez par heure pour une instance EC2, par Go stocké sur S3, par requête sur Lambda. Pas de frais fixes, pas de ressources inutilisées payées « pour rien ».

5. Accès étendu au réseau (Broad network access)

Les utilisateurs gèrent leurs ressources via une console Web, des API ou des outils en ligne de commande (CLI).

En pratique avec AWS : Vous ne contactez pas AWS pour dire « besoin d'un serveur ». Vous vous connectez à la console, créez ce que vous voulez, et c'est instantané.

 **NIST — The NIST Definition of Cloud Computing** [↗](#)

2.3 CAPEX vs OPEX — Comprendre le changement de modèle économique

Historiquement, les entreprises ont fonctionné sur un modèle **CAPEX** (*Capital Expenditures*) : elles achetaient leurs équipements, les installaient dans leurs propres locaux et les exploitaient pendant

plusieurs années.

Avec le Cloud, elles passent à un modèle **OPEX** (*Operational Expenditures*) : elles louent des ressources à la demande et paient uniquement pour leur consommation réelle.

Modèle	Définition	Exemple typique	Gestion
CAPEX	Investissement initial important amorti sur plusieurs années	Achat de serveurs physiques	Budget fixe
OPEX	Dépenses variables basées sur l'usage	Paiement horaire d'une instance EC2	Budget flexible

Exemple concret :

CAPEX (Avant le Cloud) : Une entreprise achète 10 serveurs physiques pour héberger une application interne. Coût initial : 50 000 €. Ils fonctionnent en permanence, même en période creuse, avec des coûts d'entretien et de maintenance élevés (climatisation, électricité, renouvellement matériel tous les 5-7 ans).

OPEX (Avec AWS) : La même entreprise utilise **Amazon EC2 (Elastic Compute Cloud)**, un service AWS permettant de lancer et gérer des **machines virtuelles**. Elle ne paie que les heures réellement consommées (ex. 100 € par mois en moyenne) et peut arrêter les instances lorsqu'elles ne sont pas nécessaires.

La différence ? Flexibilité, prévisibilité des coûts, et liberté d'adapter l'infrastructure en temps réel.

Attention aux coûts AWS — le modèle OPEX peut surprendre : Une instance EC2 laissée tournante 24h/7j sans utilisation reste facturée. Contrairement à un investissement CAPEX amorti sur plusieurs années, les coûts OPEX s'accumulent en temps réel. Activez toujours des **alertes de budget (AWS Budgets)** dès le premier jour. :::

 [AWS Economics Center](#)

2.4 La rupture culturelle du libre-service

Dans les environnements informatiques traditionnels, **les équipes métiers et techniques** devaient **passer par des processus centralisés** pour obtenir des ressources :

- Demande de serveurs à l'équipe IT
- Validation budgétaire et technique

- Commande physique, installation, configuration... 👉 Résultat : **des délais de plusieurs semaines** avant de pouvoir simplement démarrer un projet.

Avec **le Cloud AWS**, ce modèle est **complètement bouleversé**.

1. Libre-service immédiat

- Les équipes peuvent **provisionner elles-mêmes** des ressources (VM, bases de données, stockage...) **en quelques clics ou via des API**.
- Plus besoin d'attendre une autre équipe pour « débloquer » les moyens techniques.
- Cela change profondément la manière de travailler : les développeurs deviennent **autonomes**.

2. Responsabilisation et gouvernance

- Le libre-service ne signifie pas « anarchie » : cela s'accompagne de **règles, quotas, politiques IAM, budgets**, etc.
- Les organisations doivent mettre en place une **gouvernance cloud** qui encadre cette autonomie.

3. Accélération de l'innovation

- Cette autonomie permet de tester une idée en quelques heures plutôt qu'en plusieurs semaines.
- Les équipes peuvent **échouer vite et pas cher**, puis pivoter ou étendre si le projet fonctionne.

4. Une vraie rupture culturelle

- Le passage au cloud AWS **n'est pas qu'une question de technologie**.
- Il faut **changer les mentalités** :
 - Déléguer le pouvoir de créer des ressources.
 - Accepter la rapidité et parfois le désordre initial.
 - Former et responsabiliser les équipes pour qu'elles deviennent **consommatrices actives** du cloud.

« Avant, il fallait une demande, une validation, une livraison. Aujourd'hui, on clique — ou on lance une commande API — et on a une infrastructure prête en quelques secondes. Cette autonomie est la clé du Cloud, mais elle exige une nouvelle culture : celle de la confiance, de la responsabilisation et de la gouvernance bien pensée. »

2.5 Les limites et contraintes du Cloud

Aucun modèle n'est parfait. Le Cloud introduit aussi plusieurs défis importants :

- **Dépendance au fournisseur (vendor lock-in)** : migrer entre AWS, Azure ou GCP peut nécessiter une réarchitecture importante.

- **Connexion Internet critique** : sans réseau, pas d'accès aux ressources hébergées dans le cloud.
- **Sécurité partagée** : certaines responsabilités ne sont **pas déléguées** au fournisseur.
- **Maîtrise des coûts** : une mauvaise configuration peut générer des dépenses importantes et rapides.

Nous allons détailler **la contrainte liée à la sécurité partagée**, car elle est souvent **mal comprise** par les entreprises débutant dans le cloud.

2.6 Le modèle de responsabilité partagée AWS

Le tableau compare EC2, RDS et Lambda. Plus le service est **managé** — c'est-à-dire exploité techniquement par AWS — plus AWS prend en charge de couches techniques. Le client reste toutefois responsable de la classification de ses données et des autorisations qu'il accorde.

Le modèle de responsabilité partagée AWS répartit la sécurité entre deux parties, et confondre les deux moitiés est l'une des erreurs les plus fréquentes chez les entreprises qui découvrent le cloud.

AWS porte la responsabilité de la **sécurité du cloud**, c'est-à-dire de tout ce qui constitue l'infrastructure sous-jacente sur laquelle les clients construisent leurs services. Cela couvre la sécurité physique des data centers — accès contrôlé, vidéosurveillance, alimentation redondante — que le client n'a jamais à gérer lui-même. Cela couvre aussi le matériel et la couche de virtualisation qui isole chaque client des autres sur un même serveur physique, ainsi que la disponibilité globale de la plateforme, avec ses multiples régions et zones de disponibilité conçues pour survivre à la panne d'un data center entier.

Le **client**, de son côté, porte la responsabilité de la **sécurité dans le cloud** : tout ce qu'il configure et déploie au-dessus de cette infrastructure reste de son ressort. Il doit gérer les accès et les identités via IAM — qui a le droit de faire quoi sur son compte — et sécuriser ses données par le chiffrement et la rotation régulière des clés. Il doit configurer correctement son réseau (VPC, Security Groups, NACL) pour éviter d'exposer par erreur des ressources sensibles sur Internet. Et lorsqu'il utilise un service de type IaaS comme EC2, où AWS fournit la machine virtuelle mais pas ce qui tourne dessus, il reste responsable des sauvegardes, des correctifs de sécurité et des mises à jour du système d'exploitation.

Cela signifie que même si AWS est hautement sécurisé, **une mauvaise configuration côté client peut compromettre la sécurité** (par ex. un bucket S3 public par erreur).

⚠️ **Responsabilité client — erreurs fréquentes en production :**

- Un bucket S3 configuré en **accès public par erreur** expose toutes vos données sur Internet.
- L'utilisation du **compte root** pour les opérations quotidiennes est une faille de sécurité majeure.
- Un **Security Group ouvert sur 0.0.0.0/0 port 22** expose vos instances SSH au monde entier.
- L'absence de **MFA** sur le compte root est la première cause de compromission de compte AWS.

AWS ne peut pas vous protéger de vos propres erreurs de configuration — c'est votre responsabilité. ⚠️

2.7 Sécurité de l'information dans le Cloud

La **sécurité de l'information** repose sur trois piliers fondamentaux (CID) :

Pilier	Définition	Exemple AWS
Confidentialité	Seules les personnes autorisées doivent accéder aux données.	IAM, KMS, Bucket Policies
Intégrité	Les données ne doivent pas être altérées ou corrompues.	Checksums, versioning S3
Disponibilité	Les systèmes doivent être accessibles à tout moment.	Multi-AZ, Load Balancer, snapshots

Ce tableau cite trois exemples AWS dont vous ne verrez le détail que plus tard dans la formation — c'est normal à ce stade : **IAM** (gestion des identités et des accès) est le sujet entier du Chapitre 2, **KMS** (Key Management Service, le service de gestion des clés de chiffrement) et les **Bucket Policies** (règles d'accès attachées à un bucket S3) seront présentés respectivement en section 6 de ce chapitre et au Chapitre 3. Retenez pour l'instant seulement le principe : chaque pilier CID s'appuie sur des outils AWS concrets que vous manipulerez au fil de la formation.

AWS fournit des outils et des certifications (ISO 27001, SOC 2, PCI-DSS...), mais **l'entreprise reste responsable de son usage**.

Exemples concrets de responsabilités du client :

- Un bucket S3 mal configuré (public par erreur) = **fuite de données** → responsabilité **du client**.
- Un mot de passe faible ou MFA non activée = **intrusion possible** → responsabilité **du client**.
- Absence de sauvegarde / réplication = **perte de disponibilité** → responsabilité **du client**.
- Firewall mal ouvert (0.0.0.0/0 sur SSH) = exposition à Internet → responsabilité **du client**.

C'est pourquoi **les bonnes pratiques de configuration** sont **aussi importantes que les services AWS eux-mêmes**.

En résumé :

Sécurité du cloud (AWS)	Sécurité dans le cloud (Client)
--------------------------------	--

Data centers, matériel

Gestion des accès et des rôles (IAM)

Virtualisation et
réseau global

Sécurisation des données et des flux

Résilience physique

Mise à jour, chiffrement, surveillance

Conformité et certifications

Respect des politiques internes et
réglementations sectorielles

2.8 Externalisation du Système d'Information (SI)

Pourquoi externaliser ?

Externaliser le SI vers le Cloud permet de :

- **Réduire les coûts** d'infrastructure et de maintenance.
- **Accéder à une expertise** et à des technologies de pointe.
- **Améliorer la disponibilité et la continuité d'activité.**

Mais cela implique aussi une **réévaluation des risques** :

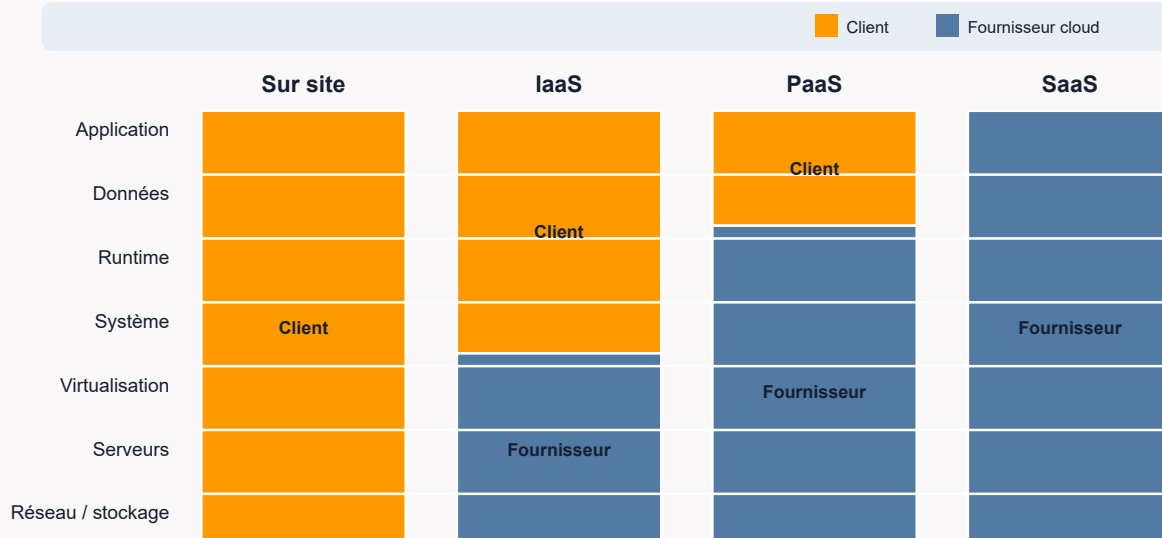
- Dépendance contractuelle envers le fournisseur.
- Nécessité de revoir les politiques de sauvegarde et d'accès.
- Conformité légale (RGPD, hébergement des données, localisation géographique).

Conseil : Avant toute migration, réalisez une **analyse d'impact** (juridique, technique et organisationnelle). AWS met à disposition un outil appelé **AWS Migration Readiness Assessment (MRA)**.

 [AWS Migration Hub](#) 

3. Modèles de services : IaaS, PaaS et SaaS

Qui gère chaque couche ?



Lecture du schéma. De gauche à droite, le fournisseur prend en charge davantage de couches. Le client conserve néanmoins la responsabilité de ses données, de ses identités, de ses configurations et de la manière dont il utilise le service. Le schéma décrit une répartition générale : le contrat précis dépend du service choisi.

Les modèles de services sont essentiels pour comprendre **comment les entreprises consomment le Cloud**. Ils structurent les **couches techniques** (infrastructure, plateforme, application) et déterminent qui — du client ou du fournisseur — est responsable de chaque couche.

AWS Cloud Service Models [↗](#)

Ces modèles sont une clé de lecture indispensable pour architecturer correctement une solution Cloud.

3.1 Infrastructure as a Service (IaaS)

Infrastructure as a Service (IaaS) désigne un modèle dans lequel le fournisseur met à disposition des ressources informatiques de base : **puissance de calcul, stockage, réseau et sécurité**. Le client gère et configure ces ressources selon ses besoins.

Caractéristiques clés :

- Accès à des **machines virtuelles** configurables.
- Gestion des disques, du réseau et de la sécurité.
- Contrôle total sur l'OS et les applications.
- Évolutivité à la demande.

Exemples AWS :

- **Amazon EC2 (Elastic Compute Cloud)** : service qui permet de créer et gérer des machines virtuelles dans le Cloud.
- **Amazon EBS (Elastic Block Store)** : stockage en mode bloc attaché aux instances EC2.
- **Amazon VPC (Virtual Private Cloud)** : réseau virtuel isolé et configurable.
- **Elastic Load Balancing (ELB)** : répartition automatique du trafic entre plusieurs ressources.

Analogie : L'IaaS est comparable à la location d'un terrain pour construire une maison — le client choisit tout, mais doit aussi tout gérer.

3.2 Platform as a Service (PaaS)

Platform as a Service (PaaS) décharge le client de la gestion de l'infrastructure et du système d'exploitation. Le fournisseur gère les couches basses, le client se concentre sur le développement et la mise en production.

Caractéristiques clés :

- Pas de gestion des serveurs ni de patching système.
- Environnements prêts à l'emploi pour le déploiement applicatif.
- Scalabilité intégrée.

Exemples AWS :

- **AWS Elastic Beanstalk** : service de déploiement et d'orchestration d'applications Web.
- **AWS Lambda** : service serverless qui exécute du code sans gérer de serveur.
- **Amazon RDS (Relational Database Service)** : base de données relationnelle gérée (patches, sauvegardes, monitoring automatisés).

 [Documentation AWS Elastic Beanstalk](#)

Le PaaS permet de **réduire la charge opérationnelle** et d'accélérer les cycles de développement.

3.3 Software as a Service (SaaS)

Software as a Service (SaaS) est un modèle dans lequel le fournisseur gère toute la pile technique — infrastructure, plateforme et application — et livre un service clé en main.

Caractéristiques clés :

- Aucune gestion technique côté client.
- Accès via un navigateur ou une API.
- Maintenance et sécurité assurées par le fournisseur.

Exemples AWS :

- **AWS WorkMail** : service de messagerie hébergée.
- **Salesforce** : CRM en ligne.
- **Microsoft 365, Google Workspace** : solutions collaboratives.

Software as a Service sur AWS [↗](#)

Retenons les cas d'usage typiques du SaaS — messagerie, CRM, outils de collaboration, ERP.

3.4 Comparatif synthétique des modèles

Élément	IaaS	PaaS	SaaS
Flexibilité	Très élevée	Moyenne	Faible
Temps de déploiement	Plus long	Rapide	Instantané
Cas d'usage typique	Migration, environnements complexes	Déploiements rapides, automatisation	Solutions métiers clés en main

4. Modèles de déploiement : Public, Privé et Hybride

Les **modèles de déploiement** définissent *où* sont hébergées les ressources Cloud et *qui* les gère. Ce choix a un impact direct sur la **gouvernance**, la **sécurité**, la **performance** et le **coût global** d'une solution Cloud.

Il existe trois grands modèles de déploiement reconnus par le NIST et adoptés par la majorité des entreprises : **Cloud Public**, **Cloud Privé** et **Cloud Hybride**.

Le modèle de déploiement choisi dépend souvent d'un compromis entre agilité, sécurité, conformité et maîtrise de l'environnement technique.

4.1 Le Cloud Public

Le **Cloud Public** est un modèle dans lequel les ressources sont hébergées dans les **datacenters du fournisseur Cloud** et partagées entre plusieurs clients. Les infrastructures sont **mutualisées**, mais chaque client bénéficie d'un environnement isolé logiquement.

Caractéristiques :

- Hébergement sur l'infrastructure du fournisseur.
- Mutualisation des ressources physiques.
- Facturation à l'usage.
- Évolutivité quasi illimitée.
- Accès global via Internet sécurisé.

Avantages :

- Pas de gestion matérielle.
- Déploiement rapide.
- Coût réduit grâce à la mutualisation.
- Accès à un large éventail de services.

Inconvénients :

- Moins de contrôle sur l'infrastructure physique.
- Dépendance au fournisseur.
- Besoins accrus en gouvernance et sécurité.

Exemple AWS :

AWS Public Cloud repose sur l'infrastructure mondiale d'**Amazon Web Services (AWS)**, composée de **régions** (Regions), **zones de disponibilité** (Availability Zones, AZ) et **points de présence** (Edge Locations).

- Une *région* est une zone géographique (par exemple *eu-west-3* pour Paris).
- Une *zone de disponibilité* (AZ) est un datacenter isolé dans cette région, assurant redondance et tolérance aux pannes.
- Les *points de présence* servent principalement aux services de diffusion de contenu comme Amazon CloudFront.

Services AWS typiques dans le Cloud public :

- **Amazon EC2** : instances de calcul à la demande
- **Amazon S3** : stockage objet scalable
- **AWS Lambda** : exécution de code sans serveur
- **Amazon RDS** : bases de données relationnelles gérées

- **Amazon CloudFront** : CDN mondial pour accélérer les contenus

 **AWS Global Infrastructure** 

4.2 Le Cloud Privé

Le **Cloud Privé** est un modèle dans lequel l'entreprise déploie ses propres ressources Cloud sur une **infrastructure dédiée**, souvent hébergée dans ses propres locaux ou dans un datacenter tiers, mais **non mutualisée** avec d'autres clients.

Caractéristiques :

- Infrastructure dédiée à une seule organisation.
- Contrôle complet sur l'environnement.
- Peut utiliser des technologies de virtualisation similaires à celles du Cloud public.
- Peut être automatisé et orchestré comme un Cloud public.

Avantages :

- Meilleure maîtrise de la sécurité et de la conformité.
- Contrôle total sur les configurations.
- Plus de personnalisation possible.

Inconvénients :

- Coût d'investissement plus élevé (proche d'un modèle CAPEX).
- Moins de flexibilité et de scalabilité.
- Nécessite des équipes internes pour la gestion.

Pourquoi le Cloud Privé existe-t-il encore ?

Avec la transformation digitale — certaines entreprises conservent des charges sensibles en interne pour des raisons réglementaires.

Exemple : le RGPD (Règlement Général sur la Protection des Données)

Le **RGPD** impose aux entreprises européennes une **maîtrise stricte** de la localisation, de la protection et de la gouvernance des données personnelles. De ce fait, certaines entreprises — notamment dans les secteurs **banque**, **santé** ou **secteur public** — choisissent de **conserver certaines données ou systèmes sensibles en interne** ou sur des infrastructures "souveraines" pour :

- Garantir que les données ne quittent pas l'Espace Économique Européen,
- Mieux contrôler les accès et traitements,

- Répondre aux obligations de **traçabilité** et de **confidentialité**.

Exemples concrets :

- **Système bancaire français** hébergeant les données de ses clients sur une infrastructure locale ou un cloud “de confiance” pour respecter :
 - L'article 44 du RGPD (transferts hors UE),
 - Les obligations de minimisation et de limitation de traitement (articles 5 et 6),
 - Les recommandations de la CNIL sur l'hébergement de données sensibles.
- **Code de la santé publique** → données de santé hébergées uniquement chez un **Hébergeur de Données de Santé (HDS)** agréé en France. Les hôpitaux et laboratoires doivent conserver certaines données sur site ou dans un cloud certifié HDS.
- **Secteur de la Défense** → obligations de **souveraineté et de sécurité** imposées par l'ANSSI (ex. SecNumCloud), qui limitent l'externalisation à des fournisseurs labellisés.

En résumé :

Législation / Réglementation	Secteur impacté	Conséquence sur l'hébergement
RGPD	Tous secteurs manipulant des données personnelles	Contrôle de localisation, transferts restreints
HDS	Santé	Hébergement uniquement chez des prestataires certifiés
ANSSI / SecNumCloud	Secteurs sensibles / OIV	Cloud souverain ou on-prem obligatoire pour certaines charges

Ce type de réglementation **explique pourquoi certaines entreprises gardent encore des charges sensibles “on-premise”** ou sur des **infrastructures dédiées** plutôt que dans le cloud public pur.

⚠ **RGPD et localisation des données AWS** : Par défaut, AWS peut stocker vos données dans n'importe quelle zone de disponibilité de la région choisie. Pour les données personnelles de citoyens européens, vous devez impérativement choisir une **région européenne** (ex. `eu-west-3` Paris, `eu-central-1` Francfort) et vérifier que les services utilisés ne transfèrent pas les données hors UE sans votre accord explicite. ⚠

AWS s'est adapté :

- Un groupe bancaire peut déployer un **Cloud privé** sur sa propre infrastructure virtualisée avec VMware ou OpenStack, tout en automatisant les déploiements comme dans AWS.

 [VMware Cloud Foundation](#)  [OpenStack](#)

Services AWS typiques pour le Cloud privé :

- **AWS Outposts** : infrastructure AWS déployée dans le datacenter du client
- **VMware Cloud on AWS** : environnement VMware géré dans AWS
- **Amazon VPC** : réseau virtuel isolé
- **AWS Direct Connect** : liaison réseau privée entre le client et AWS

4.3 Le Cloud Hybride

Le **Cloud Hybride** combine le **Cloud Privé** et le **Cloud Public**, en permettant aux applications et données de circuler entre les deux environnements. C'est aujourd'hui le modèle **le plus répandu** dans les grandes organisations.

Caractéristiques :

- Combinaison de ressources internes et publiques.
- Interopérabilité entre les environnements.
- Optimisation des coûts et de la sécurité.
- Support de scénarios de migration progressive.

Avantages :

- Flexibilité maximale.
- Répartition intelligente des charges de travail.
- Sécurité renforcée pour les données sensibles.
- Possibilité d'exploiter le meilleur des deux mondes.

Inconvénients :

- Gestion plus complexe.
- Nécessite des compétences solides en architecture et interconnexion.
- Risque de dépendances multiples.

Exemple AWS :

AWS Direct Connect est un service AWS qui permet de **créer une liaison réseau privée dédiée** entre l'infrastructure interne d'une entreprise et AWS. Cela permet de combiner la sécurité d'un Cloud privé avec la puissance d'un Cloud public.

Services AWS typiques pour le Cloud hybride :

- AWS Storage Gateway : intégration du stockage local avec S3
- AWS Transit Gateway : interconnexion de VPC et réseaux sur site
- AWS Systems Manager : gestion centralisée des ressources hybrides
- AWS Backup : sauvegarde unifiée pour ressources cloud et sur site

 [AWS Direct Connect](#)

Le Cloud hybride est souvent une **étape transitoire** vers une adoption plus large du Cloud public.

4.4 Comparatif synthétique

Élément	Public Cloud	Private Cloud	Hybrid Cloud
Propriété de l'infrastructure	Fournisseur (ex. AWS)	Entreprise	Mixte
Isolation	Mutualisée, logique	Dédiée, physique	Mixte
Coûts	OPEX, à la demande	CAPEX, investissement initial	Mixte
Sécurité	Standardisée, contrôlée par fournisseur	Complète, contrôlée par l'entreprise	Partagée
Scalabilité	Très élevée	Limitée	Élevée
Gouvernance	Fournisseur	Entreprise	Mixte

4.5 Recommandations par contexte

Modèle	Recommandé pour...	Explication simplifiée
--------	--------------------	------------------------

Cloud public	Startups, PME, environnements Dev/Test, charges variables	AWS gère toute l'infrastructure. Vous utilisez les services à la demande, sans rien installer ni maintenir. Idéal pour démarrer vite, tester, ou adapter les ressources selon le trafic.
Cloud privé	Organisations réglementées, données sensibles, applications critiques	L'environnement est dédié à une seule entreprise. Plus de contrôle, plus de sécurité, mais aussi plus de configuration. Utilisé quand les données ne doivent pas sortir d'un périmètre défini (ex : santé, finance).
Cloud hybride	Entreprises en transition, contraintes réglementaires, intégration avec systèmes existants	Combine les deux : une partie des ressources reste sur site ou en cloud privé, l'autre est dans le cloud public. Permet de migrer progressivement, tout en respectant les contraintes métier ou techniques.

AWS permet aux entreprises de choisir le **modèle le plus adapté à leur contexte** :

- Le **cloud public** est parfait pour démarrer rapidement, tester des idées, ou gérer des pics de trafic.
- Le **cloud privé** offre un **contrôle maximal** pour les secteurs sensibles ou réglementés.
- Le **cloud hybride** est une **solution intermédiaire** qui permet de **connecter l'ancien et le nouveau**, en douceur.

Cette flexibilité est l'un des grands atouts d'AWS : vous pouvez commencer petit, évoluer progressivement, et adapter votre architecture à vos besoins métier, techniques et réglementaires.

5. Fondamentaux techniques : Virtualisation et Conteneurs

Derrière le Cloud se cachent des **technologies fondamentales** qui le rendent possible. Deux piliers en particulier ont permis l'essor massif des infrastructures à la demande :

- La **virtualisation**, qui permet de découper une machine physique en plusieurs machines virtuelles indépendantes.

- La **conteneurisation**, qui permet d'isoler et d'exécuter des applications de manière légère et flexible.

Comprendre ces concepts est essentiel pour appréhender le fonctionnement d'**Amazon Web Services (AWS)** et de nombreux autres fournisseurs Cloud.

 [Documentation Amazon EC2](#)  [Introduction to Virtualization — VMware](#)  [Docker Documentation](#) 

Insistons ici sur la **différence entre virtualisation et conteneurisation**, car cette distinction conditionne la compréhension des services comme Amazon EC2, ECS ou EKS.

5.1 Virtualisation — Principe et rôle dans le Cloud

Définition

La **virtualisation** est une technologie qui permet d'exécuter plusieurs **machines virtuelles (VM)** sur un seul serveur physique. Chaque VM fonctionne comme une machine indépendante avec son propre système d'exploitation et ses ressources allouées (CPU, mémoire, stockage, réseau).

La virtualisation est gérée par une couche logicielle appelée **hyperviseur**.

Types d'hyperviseurs

- **Type 1 (bare-metal)** : installé directement sur le matériel physique. Exemples : VMware ESXi, Microsoft Hyper-V, KVM.
- **Type 2 (hosted)** : installé au-dessus d'un système d'exploitation. Exemples : VirtualBox, VMware Workstation.

Avantages de la virtualisation :

- Meilleure utilisation des ressources matérielles.
- Isolation entre les environnements.
- Déploiement rapide.
- Maintenance simplifiée.

Virtualisation et AWS

Sur AWS, la virtualisation est au cœur de nombreux services, en particulier :

- **Amazon EC2 (Elastic Compute Cloud)** : service permettant de créer et gérer des machines virtuelles dans le Cloud AWS.

- **Amazon EBS (Elastic Block Store)** : service de stockage en mode bloc utilisé par les instances EC2.
- **Amazon VPC (Virtual Private Cloud)** : service qui fournit un réseau virtuel isolé dans AWS pour héberger les ressources.

5.2 Paravirtualisation et Émulation

Deux modes techniques sont utilisés dans la virtualisation Cloud :

- **Émulation** : le matériel est simulé intégralement par logiciel. Plus flexible mais moins performant.
- **Paravirtualisation** : le système invité est conscient qu'il est virtualisé et interagit directement avec l'hyperviseur, ce qui offre de meilleures performances.

AWS utilise principalement des technologies de **paravirtualisation** et de **virtualisation matérielle assistée** pour offrir des performances quasi natives.

 **AWS Nitro System** — La plateforme AWS Nitro est une combinaison de matériel dédié et de logiciels légers qui améliore la sécurité et les performances de la virtualisation EC2.

5.3 Conteneurisation — Une approche plus légère

Définition

La **conteneurisation** est une méthode d'isolation applicative qui permet d'exécuter plusieurs applications sur le même système d'exploitation, dans des environnements isolés appelés **conteneurs**.

Contrairement à une machine virtuelle, un conteneur ne contient pas son propre OS complet — il partage le noyau de l'hôte, ce qui le rend **beaucoup plus léger**.

Avantages des conteneurs :

- Démarrage quasi instantané.
- Consommation réduite de ressources.
- Facilité de portabilité et de déploiement.
- Standardisation des environnements.

Outils et services associés

- **Docker** : technologie de conteneurisation la plus répandue, permettant de créer, packager et exécuter des conteneurs.

- **Amazon ECS (Elastic Container Service)** : service AWS permettant de déployer et gérer des conteneurs Docker à grande échelle.
- **Amazon EKS (Elastic Kubernetes Service)** : service AWS qui fournit une plateforme Kubernetes managée pour orchestrer des conteneurs.

 [Amazon ECS Documentation](#)  [Amazon EKS Documentation](#)

Il est important de comprendre la différence clé — VM = OS complet, Conteneur = application isolée. C'est une **rupture technologique** qui a permis l'essor de l'architecture microservices.

5.3 Microservices — Architecture distribuée et scalable

Les **microservices** sont une approche architecturale qui consiste à **découper une application monolithique en petits services indépendants**, chacun responsable d'une fonction métier spécifique.

Exemple : Une application de e-commerce ne sera plus UNE grosse application, mais plusieurs petits services :

- Service d'authentification
- Service de gestion de panier
- Service de paiement
- Service de recommandations
- Service de livraison

Avantages des microservices

Le monolithe (une application unique de 500k lignes, une seule technologie, un déploiement lent et une scalabilité limitée) s'oppose aux microservices (Auth, Cart, Payment, Reco découpés en services indépendants), qui autorisent des technologies différentes par service, un déploiement rapide et une scalabilité flexible par équipe.

- **Agilité** : chaque équipe développe son service indépendamment.
- **Scalabilité** : seul le service de paiement reçoit beaucoup de trafic ? Scalez juste ce service.
- **Résilience** : si le service de recommandations est down, le reste de l'app fonctionne.
- **Technologie** : chaque service peut utiliser Node.js, Python, Java... selon le besoin.

Microservices et conteneurs

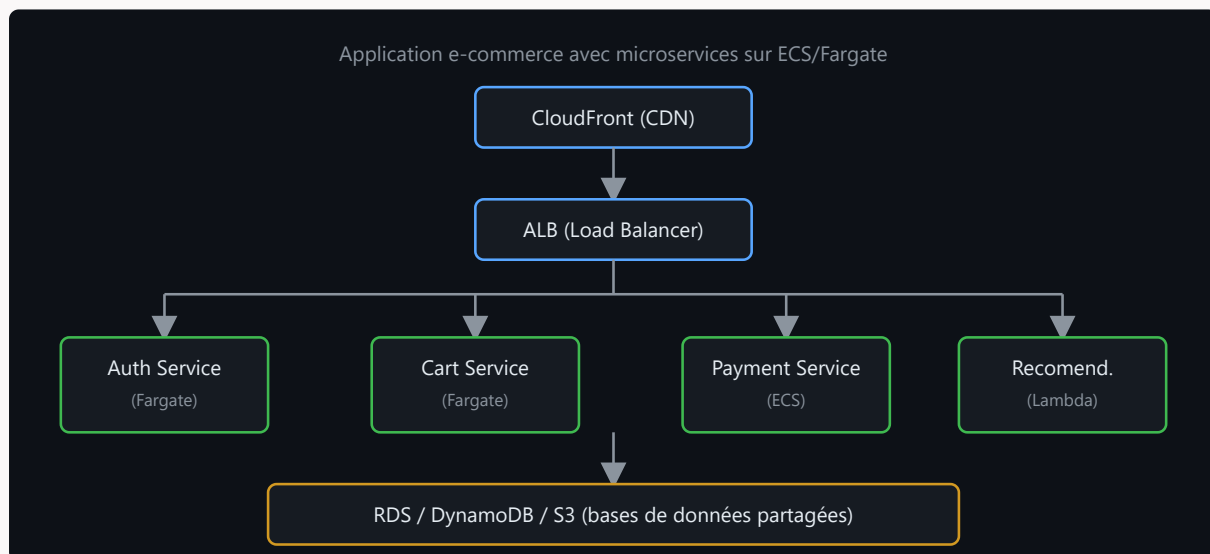
Les microservices **sans conteneurs** seraient très compliqués à gérer. C'est pourquoi **Docker + Kubernetes** (ou **AWS ECS**) sont devenus essentiels.

Chaque microservice est empaqueté dans son propre **conteneur Docker**, puis orchestré par un outil qui gère les déploiements, la scalabilité et la résilience.

Services AWS pour microservices :

- **Amazon ECS (Elastic Container Service)** : orchestration native AWS, plus simple que Kubernetes.
- **Amazon EKS (Elastic Kubernetes Service)** : Kubernetes managé, si vous avez une flotte massive.
- **AWS Fargate** : exécution serverless de conteneurs (sans gérer les instances EC2).
- **AWS Lambda** : exécution de fonctions sans serveur (cas de microservices très légers).

Un exemple en image



Lecture du schéma. Les requêtes entrent par CloudFront, puis l'Application Load Balancer les répartit vers plusieurs microservices exécutés séparément. Chaque service accède uniquement au stockage adapté à son besoin. Les flèches représentent les flux réseau ; elles ne signifient pas que tous les services partagent automatiquement les mêmes droits IAM.

 [Microservices Documentation AWS](#)

5.4 Virtualisation vs conteneurisation — tableau comparatif

Élément	Virtualisation (VM)	Conteneurisation
Système	OS complet par machine virtuelle	Noyau partagé, application isolée
Démarrage	Lent (minutes)	Rapide (secondes)
Consommation de ressources	Élevée	Faible
Portabilité	Moins flexible	Très flexible

Cas d'usage typiques

Migration d'applications legacy,
serveurs complets

Microservices, CI/CD,
déploiements rapides

Ce tableau permet de bien fixer les différences entre VM et conteneurs.

6. Présentation d'AWS et de son écosystème

6.1 Vue d'ensemble

Amazon Web Services (AWS) est l'un des principaux fournisseurs mondiaux de cloud public. Lancé en 2006, AWS est une filiale d'Amazon qui propose une **plateforme Cloud complète** couvrant :

- L'infrastructure (serveurs, stockage, réseau),
- Les plateformes de développement et déploiement applicatif,
- Les services managés et serverless,
- L'intelligence artificielle, la data et l'IoT.

AWS est présent dans la quasi-totalité des secteurs d'activité : industrie, banque, administration publique, éducation, santé, etc.

 [Site officiel AWS](#)  [AWS Overview Documentation](#) 

6.2 Les applications dans les offres Cloud

AWS n'est pas seulement une plateforme d'infrastructure : c'est aussi un **écosystème d'applications et de solutions métiers**.

Quelques exemples typiques :

- **Hébergement web et API** : via Amazon EC2, Elastic Beanstalk ou Lightsail.
- **Stockage et sauvegarde** : avec S3, EBS et Glacier.
- **Analyse de données** : grâce à Redshift et Athena.
- **Sécurité et conformité** : via IAM, CloudTrail et Security Hub.

Ces briques sont interconnectées. Une **application complète AWS** s'appuie généralement sur plusieurs de ces services, orchestrés ensemble pour offrir disponibilité, sécurité et performance.

6.3 Les principaux acteurs du marché du Cloud

Le Cloud Computing est un marché concurrentiel dominé par **trois géants**, avec quelques acteurs spécialisés ou souverains :

Panorama comparatif simplifié

Avant de décortiquer AWS, il est utile de comprendre son positionnement face à ses concurrents principaux.

Fournisseur	Origine	Points forts	Contexte d'utilisation	Position sur le marché
AWS	Amazon (2006)	Catalogue étendu, écosystème, documentation et communauté	Organisations de toutes tailles	Fournisseur de cloud public généraliste
Microsoft Azure	Microsoft (2010)	Intégration avec l'écosystème Microsoft et services hybrides	Organisations utilisant déjà les technologies Microsoft	Fournisseur de cloud public généraliste
Google Cloud Platform (GCP)	Google (2008)	Services de données, d'analyse et d'intelligence artificielle	Applications, données et apprentissage automatique	Fournisseur de cloud public généraliste
OpenStack (Open Source)	Communauté	Infrastructure privée, totale maîtrise, coût réduit	Cloud privé, institutions académiques, hébergeurs	Marché de niche
OVHcloud (Européen)	OVH (France)	Souveraineté données EU, prix compétitifs	PME EU, données sensibles, RGPD stricte	Marché de niche EU

IBM Cloud, Oracle Cloud	IBM / Oracle	Spécialisés (IA, bases données massives)	Grandes entreprises, solutions legacy	Marché de niche
----------------------------	--------------	--	---------------------------------------	-----------------

Comparatif détaillé : AWS vs Azure vs GCP

Aspect	AWS	Azure	GCP
Étendue du catalogue	Calcul, réseau, données, sécurité, IA et exploitation	Calcul, réseau, données, sécurité, IA et exploitation	Calcul, réseau, données, sécurité, IA et exploitation
Services de calcul	EC2, Lambda, Fargate	VMs, App Services, AKS	Compute Engine, Cloud Functions
Bases de données	RDS, DynamoDB, Aurora	SQL Database, Cosmos DB	Cloud SQL, Firestore
Données et IA	SageMaker, Glue	Azure ML, Synapse	BigQuery, Vertex AI 🌟
Implantation	Infrastructure mondiale organisée en régions et zones	Infrastructure mondiale organisée en régions et zones	Infrastructure mondiale organisée en régions et zones
Matériel propriétaire	AWS Nitro	Optimisé	Processeurs TPU (IA)
Force communautaire	Très forte	Forte (corporations)	Croissante (startups)
Apprentissage	Dépend des services et du rôle étudiés	Facilités possibles pour les équipes Microsoft	Facilités possibles pour les équipes data et Google Cloud
Certifications	Cloud Practitioner, CloudOps Engineer, Solutions Architect	AZ-900, AZ-104	Cloud Digital Leader, Associate Cloud Engineer, Professional

Ce que signifient les lignes moins évidentes de ce tableau :

- **Matériel propriétaire** : chaque fournisseur a développé du matériel sur mesure pour ses data centers plutôt que d'utiliser des composants standards du marché. **AWS Nitro** (déjà présenté en section 5.2 de ce chapitre) est un système matériel et logiciel conçu par AWS qui décharge la virtualisation, le réseau et la sécurité du CPU principal du serveur physique — cela signifie que quasiment 100 % de la puissance de la machine physique est disponible pour vos instances EC2, au lieu qu'une partie soit consommée par l'hyperviseur lui-même. Chez GCP, les **TPU (Tensor Processing Units)** sont des puces conçues spécifiquement pour accélérer les calculs de machine learning (entraînement et inférence de modèles d'IA) — un TPU va beaucoup plus vite qu'un CPU classique sur ce type de calcul précis, mais ne sert à rien pour un usage généraliste.
- **Force communautaire** : désigne le volume et l'activité de la communauté d'utilisateurs autour du fournisseur — tutoriels, forums (Stack Overflow, Reddit), projets open source, meetups. Plus cette communauté est active, plus il est facile de trouver de l'aide en cas de blocage. AWS bénéficie ici de son avance historique (premier arrivé en 2006) : davantage de contenus accumulés sur près de 20 ans.
- **Apprentissage** : la courbe d'apprentissage dépend du rôle, des services étudiés et des compétences existantes. Une équipe déjà familière de l'écosystème Microsoft, Google ou AWS peut retrouver plus rapidement certains concepts, sans que cela rende un fournisseur objectivement plus simple dans tous les contextes.
- **Certifications** : chaque fournisseur a son propre parcours de certification, avec des noms et des codes différents mais un principe similaire (un premier niveau généraliste, puis des spécialisations). Côté Azure, **AZ-900** est la certification fondamentale (équivalent du AWS Cloud Practitioner) et **AZ-104** la certification Administrator Associate (gestion quotidienne des ressources Azure, équivalent du AWS SysOps). Les certifications AWS sont détaillées à la section 1.3 de ce chapitre.

Pourquoi les organisations comparent-elles plusieurs fournisseurs ?

La comparaison doit partir des contraintes techniques et organisationnelles :

1. services disponibles dans les régions nécessaires ;
2. compétences déjà présentes dans l'organisation ;
3. intégration avec l'identité, le réseau et les outils existants ;
4. conformité, support, réversibilité et structure de coûts ;
5. dépendance acceptable envers des services propriétaires.

Les parts de marché varient selon la source, la période et le périmètre mesuré. Elles ne constituent pas, à elles seules, un critère d'architecture.

Solutions open source

Enfin, une mention importante : **OpenStack**.

OpenStack est une plateforme **open source complète** qui permet à toute organisation de **construire son propre cloud privé**, sans dépendre d'un fournisseur unique. Elle est utilisée par :

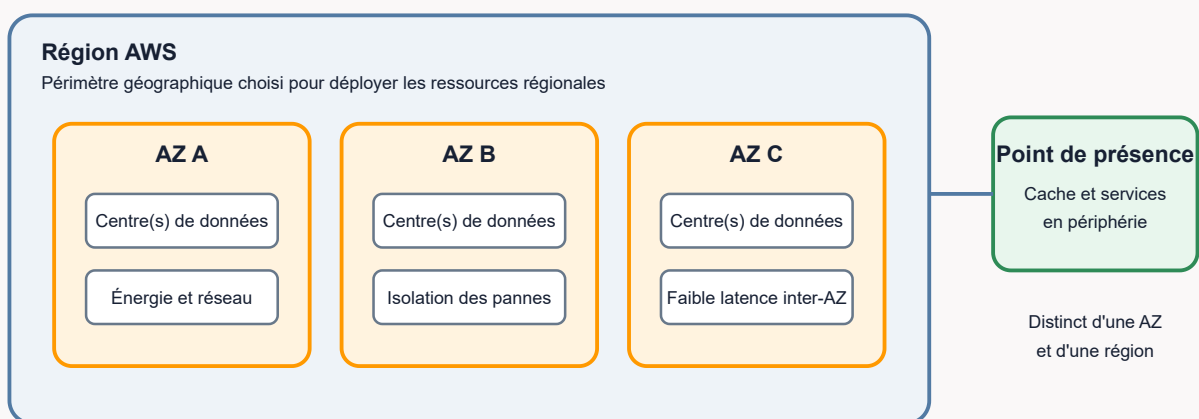
- Universités et instituts de recherche
- Gouvernements (souveraineté)
- Hébergeurs et fournisseurs de services cloud alternatifs
- Organisations ayant besoin de total contrôle

OpenStack ne joue pas sur le même terrain qu'AWS/Azure/GCP (qui sont des services managés) — c'est une brique technologique que vous installez et maintenez vous-même.

 **Gartner Magic Quadrant 2024 – Cloud Infrastructure and Platform Services** [↗](#)

6.4 L'infrastructure mondiale d'AWS

Régions, zones de disponibilité et points de présence



Lecture du schéma. Une région est le périmètre géographique sélectionné pour de nombreux services. Elle contient plusieurs zones de disponibilité isolées les unes des autres. Un point de présence sert notamment à rapprocher certains services des utilisateurs ; ce n'est ni une région ni une zone dans laquelle on déploie arbitrairement les mêmes ressources.

Le schéma suivant part d'une requête utilisateur et montre pourquoi une région ne suffit pas, à elle seule, à garantir la disponibilité. Le trafic doit être distribué vers plusieurs zones indépendantes et les ressources zonales doivent réellement être dupliquées.

Les trois AZ appartiennent à la même région et communiquent par le réseau régional AWS, mais constituent des domaines de défaillance distincts. Déployer une instance dans `eu-west-3a` et une base uniquement dans cette même AZ reste une architecture mono-AZ, même si la région en propose trois.

L'un des piliers majeurs d'AWS est son **infrastructure mondiale** extrêmement étendue et redondée. Elle est structurée en trois grandes composantes :

1. Régions AWS (Regions)

- Une *région* est une zone géographique physique (exemple : `eu-west-3` pour Paris).
- Chaque région contient plusieurs datacenters appelés Zones de disponibilité (AZ).
- Les services AWS sont déployés par région et choisis par l'utilisateur.

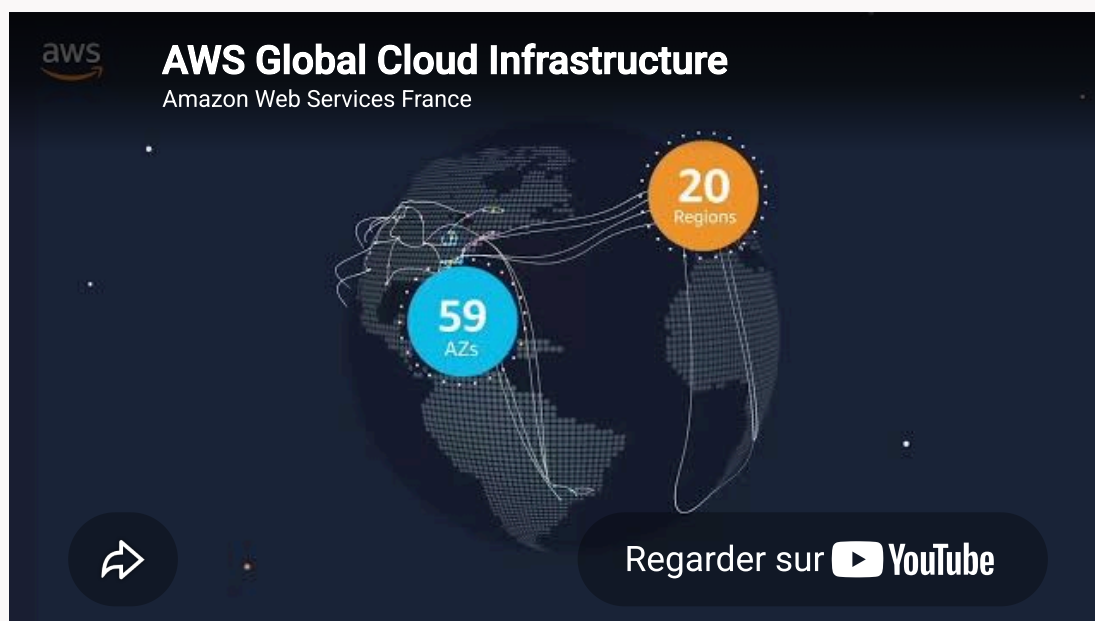
2. Zones de disponibilité (Availability Zones - AZ)

- Une AZ correspond à un ou plusieurs datacenters indépendants reliés entre eux par des liens à très faible latence.
- Les AZ sont conçues pour **garantir la haute disponibilité et la tolérance aux pannes**.

3. Points de présence (Edge Locations)

- Répartis dans le monde entier, ils permettent d'accélérer la diffusion de contenu et d'optimiser les performances réseau via des services comme **Amazon CloudFront** (réseau de diffusion de contenu — CDN).

Vidéo explicative : infrastructure AWS



Retenons que le concept de *région* et de *localisation géographique des données* est souvent déterminant pour des raisons légales et de performance.

6.5 Principes de tarification AWS

L'un des aspects stratégiques de l'adoption du Cloud est la **tarification à l'usage**. AWS applique un modèle de facturation souple et transparent basé sur la consommation réelle.

Principes clés :

- **Pay-as-you-go** : paiement en fonction de l'utilisation réelle des ressources.
- **Sans engagement initial** : pas de coût d'entrée, contrairement au modèle CAPEX.
- **Économies d'échelle** : les tarifs peuvent diminuer avec l'usage.
- **Facturation par service** : chaque service (EC2, S3, RDS...) est facturé séparément selon ses métriques propres.

Selon le service et l'engagement pris, plusieurs modes de tarification existent — à la demande, instances réservées, Savings Plans, Spot Instances. Le Chapitre 3 détaille chacun de ces modes avec des cas métier chiffrés, au moment où vous configurerez vos premières instances EC2.

 [AWS Pricing Overview](#)  [AWS Pricing Calculator](#) 

7. AWS Well-Architected Framework

Le **Well-Architected Framework** est une méthodologie officielle AWS qui permet de **concevoir des architectures Cloud robustes**, sécurisées, performantes, optimisées en coûts et **durables**.

C'est une **philosophie de design** qui s'applique à chaque projet AWS, du plus petit au plus grand.

Ce framework repose sur **six piliers**, souvent représentés par le sigle **SOPREC** :

- 🏠 **Security** → Sécurité, IAM, chiffrement
- ⚙️ **Operational Excellence** → Monitoring, automatisation, processus
- 💪 **Reliability** → Résilience, haute disponibilité
- ⚡ **Performance** → Efficacité des ressources
- 💰 **Cost Optimization** → Optimisation des dépenses
- 🌱 **Sustainability** → Impact environnemental, efficacité

👉 **Vérification AWS CLI — liste des piliers WAF disponibles pour une review :**

```
1 aws wellarchitected list-lenses --lens-type AWS_OFFICIAL
```

```
1 {
2     "LensSummaries": [
3         { "LensAlias": "wellarchitected", "LensName": "AWS Well-Architected Fra
```



```
4         "LensVersion": "2023-04-10", "Description": "Six pillars review" },
5     { "LensAlias": "serverless", "LensName": "Serverless Lens", "LensVersio
6     { "LensAlias": "saas", "LensName": "SaaS Lens", "LensVersion": "1.0" }
7 ]
8 }
```

Le Framework Well-Architected est disponible depuis la console et via API. Vous pouvez lancer une revue avec **AWS Well-Architected Tool** : les questions sont organisées selon les six piliers. Ce service ne doit pas être confondu avec **AWS WAF**, le pare-feu applicatif web. :::

1. Security (Sécurité)

Mettre en œuvre des mesures de protection fortes à tous les niveaux — **IAM, chiffrement, sécurité réseau, MFA, audit**.

En pratique :

- Créer des rôles IAM spécifiques plutôt que d'utiliser le compte root.
- Chiffrer les données au repos et en transit.
- Utiliser des security groups et NACL pour isoler les ressources.
- Activer CloudTrail pour auditer les accès.

2. Operational Excellence (Excellence opérationnelle)

Surveiller et améliorer continuellement les opérations — **mise en place de processus, d'automatisation et de monitoring**.

En pratique :

- Utiliser CloudWatch pour surveiller les ressources.
- Automatiser les déploiements avec CloudFormation ou Terraform.
- Documenter les procédures d'incident.
- Faire des reviews régulières de l'architecture.

3. Reliability (Fiabilité / Résilience)

Assurer la **résilience** et la **tolérance aux pannes** — **redondance, sauvegarde, failover automatique**.

En pratique :

- Déployer les applications dans plusieurs **Availability Zones** pour la haute disponibilité.
- Mettre en place des sauvegardes automatiques (EBS snapshots, RDS backups).
- Utiliser des auto-scaling groups pour ajuster le nombre d'instances en cas de charge.
- Tester régulièrement les scénarios de récupération d'urgence (RTO/RPO).

4. Performance Efficiency (Efficacité des performances)

Utiliser les ressources de manière **efficace et évolutive** — **choix des bonnes instances, optimisation des requêtes.**

En pratique :

- Choisir le bon type d'instance EC2 selon le workload (t3 pour web, m5 pour applications, c5 pour calcul).
- Utiliser des caches (ElastiCache, CloudFront) pour réduire la latence.
- Optimiser les requêtes de bases de données.
- Utiliser AWS Compute Optimizer pour recommander les bonnes tailles.

5. Cost Optimization (Optimisation des coûts)

Éviter les dépenses inutiles et **ajuster la capacité à la demande** — **monitoring des coûts, réservation d'instances, suppression des ressources non utilisées.**

En pratique :

- Utiliser des **Reserved Instances** ou **Savings Plans** pour réduire les coûts.
- Mettre en place des **budgets d'alerte** dans AWS Billing.
- Arrêter les ressources non utilisées (instances EC2, RDS développement).
- Analyser les logs de coûts avec Cost Explorer ou AWS Cost Anomaly Detection.

6. Sustainability (Durabilité) — Le pilier le plus récent ✨

Minimiser l'**impact environnemental** de votre infrastructure Cloud — **efficacité énergétique, utilisation optimale des ressources, choix des régions.**

En pratique :

- Utiliser les régions AWS qui utilisent des énergies renouvelables (ex : `eu-west-1` en Irlande, alimentée à >80% par des renouvelables).
- Éteindre les ressources inutilisées plutôt que de les laisser tourner.
- Utiliser le **Compute Optimizer** pour ajuster les tailles d'instances et réduire la consommation.
- Préférer les architectures **serverless** (Lambda, Fargate) qui consomment moins que les EC2 permanentes.
- Monitorer l'empreinte carbone de vos déploiements via le **AWS Customer Carbon Footprint Tool**.

C'est un enjeu de plus en plus important pour les entreprises souhaitant atteindre leurs **objectifs de développement durable (ODD)** et de **net-zéro** (carbon-neutral).

Appliquer le Framework au quotidien

Lors de **chaque conception d'architecture AWS**, on doit se poser ces questions :

-  **Sécurité** : Les données sont-elles protégées ? IAM bien configuré ?
-  **Opérations** : Comment monitorer et alerter ? Comment déployer sans risque ?
-  **Résilience** : Que se passe-t-il si un service tombe ? Avons-nous des sauvegardes ?
-  **Performance** : Les ressources sont-elles bien dimensionnées ? Y a-t-il des goulots ?
-  **Coûts** : Payons-nous ce que nous utilisons réellement ? Pouvons-nous optimiser ?
-  **Durabilité** : Quel est notre impact environnemental ? Pouvons-nous le réduire ?

Cette approche sera **reprise au fil des jours**, notamment lors des ateliers EC2, S3, RDS et VPC.

8. AWS Management Console

La **console AWS** est une interface Web centralisée qui permet de gérer tous les services AWS depuis un seul endroit.

8.1 Principales fonctionnalités

La console web regroupe en un seul endroit tout ce dont un administrateur a besoin pour piloter son compte AWS au quotidien, et c'est justement ce qui la rend accessible aux débutants malgré la richesse du catalogue de services.

Elle donne d'abord accès à l'ensemble des services AWS disponibles dans la région sélectionnée : EC2, S3, RDS, Lambda et les centaines d'autres services se retrouvent derrière une barre de recherche unique, sans avoir à mémoriser des noms de commandes ou des endpoints d'API. Elle permet ensuite de visualiser en temps réel l'état des ressources déjà créées — combien d'instances EC2 tournent, quels buckets S3 existent, comment est configuré le VPC — ce qui en fait le point d'entrée naturel pour diagnostiquer un problème avant de creuser en ligne de commande. Elle sert aussi de porte d'entrée vers **AWS IAM (Identity and Access Management)**, où se gèrent les comptes, les utilisateurs et leurs permissions : c'est depuis la console que l'on crée les premiers utilisateurs IAM et qu'on leur attribue des droits, avant même de savoir écrire une policy JSON. Enfin, elle donne un accès direct aux métriques d'usage et à la facturation, ce qui permet de surveiller les coûts sans quitter l'interface — un réflexe indispensable pour éviter les mauvaises surprises en fin de mois.

8.2 Structure de la console

Zone	Rôle
Barre de recherche	Trouver rapidement un service (ex : EC2, S3, IAM)
Panneau de navigation	Accéder aux sections : Services, Ressources, Facturation
Région	Indique la localisation actuelle (ex : Paris eu-west-3)
Compte utilisateur	Informations, factures, connexions IAM
Notifications	Alertes et mises à jour
Support	Documentation, centre d'aide, tickets AWS

 [AWS Console](#)  [Guide de l'interface AWS Management Console](#)

:::success **Vérification AWS CLI — lister les régions disponibles :**

```
1 aws ec2 describe-regions --output table
```

```

1  -----
2  |                      DescribeRegions                      |
3  +-----+-----+
4  |  RegionName      |  Endpoint                      |
5  +-----+-----+
6  |  eu-west-3       |  ec2.eu-west-3.amazonaws.com  | ← Paris
7  |  eu-west-1       |  ec2.eu-west-1.amazonaws.com  | ← Irlande
8  |  eu-central-1    |  ec2.eu-central-1.amazonaws.com| ← Francfort
9  |  us-east-1       |  ec2.us-east-1.amazonaws.com  | ← Virginie du Nord
10 |  ap-southeast-1   |  ec2.ap-southeast-1.amazonaws.com| ← Singapour
11 +-----+-----+
```

La région **eu-west-3** (Paris) est votre région de travail par défaut pour cette formation. Vérifiez toujours que vous êtes bien dans la bonne région avant de créer une ressource — une instance EC2 créée dans **us-east-1** par erreur sera difficile à retrouver. :::

Rappelons que la console est un moyen parmi d'autres d'interagir avec AWS — il existe également des API, des SDK et la **CLI AWS** pour les automatisations.

8.3 Bonnes pratiques de base

- Toujours vérifier la **région active** avant de créer des ressources.
- Fermer les ressources non utilisées pour éviter des coûts inutiles.
- Utiliser **IAM** pour créer des comptes utilisateurs distincts plutôt que d'utiliser le compte root — rappel : voir l'encart sur le compte root en section 2.6 de ce chapitre.
- Activer la **MFA (Multi-Factor Authentication)** pour sécuriser les accès.
- Surveiller les coûts dans **Billing & Cost Management**.

 [Best Practices AWS IAM](#)

8.4 Gestion du budget et des coûts AWS

L'un des **risques majeurs** du cloud est la **perte de contrôle des coûts**. Un bucket S3 public par erreur, une instance EC2 oubliée, un transfert de données non optimisé — et la facture peut exploser en quelques jours.

AWS fournit des outils puissants pour **monitorer, budgéter et optimiser** les coûts.

Principes clés de la tarification AWS

1. **Pay-as-you-go** : payer ce qu'on utilise réellement.
2. **Granularité** : chaque service a ses propres métriques — EC2 facturé par heure (ou seconde), S3 par Go stocké + requêtes, RDS par heure + stockage + transfert.
3. **Réductions possibles** : Reserved Instances (engagement 1 ou 3 ans), Savings Plans (engagement flexible), Spot Instances (capacité excédentaire à -70%).
4. **AWS Free Tier** : les nouveaux clients peuvent recevoir 100 USD de crédits à l'inscription et jusqu'à 100 USD supplémentaires via des activités. Le plan gratuit s'arrête après six mois ou lorsque les crédits sont épuisés. Dans un plan gratuit, AWS n'inscrit pas de frais sur la facture avant passage au plan payant ; dans un plan payant, l'utilisation non couverte ou supérieure au solde de crédits est facturée au tarif normal.

👉 **Exemple de commande AWS CLI pour vérifier votre consommation du Free Tier :**

```
1 aws ce get-cost-and-usage \  
2   --time-period Start=2024-01-01,End=2024-01-31 \  
3   --granularity MONTHLY \  
4   --metrics BlendedCost \  
5   --group-by Type=DIMENSION,Key=SERVICE
```

```
1 {
```

```

2      "ResultsByTime": [{
3          "TimePeriod": { "Start": "2024-01-01", "End": "2024-01-31" },
4          "Groups": [
5              { "Keys": ["Amazon EC2"], "Metrics": { "BlendedCost": { "Amount": "
6              { "Keys": ["Amazon S3"], "Metrics": { "BlendedCost": { "Amount": "0
7              { "Keys": ["Amazon RDS"], "Metrics": { "BlendedCost": { "Amount": "
8          ]
9      }]
10 }

```

Dans cet exemple fictif, les crédits promotionnels absorbent encore la consommation EC2 et RDS. S3 affiche un coût car l'usage concerné n'est plus entièrement couvert. Le résultat réel dépend de la date de création du compte, du plan choisi, du solde de crédits, de la région et des services utilisés : il faut toujours vérifier la page **Free Tier** et la facturation du compte actif. :::

Outils de monitoring des coûts

AWS Billing Dashboard

Accessible depuis la console AWS, le **Billing Dashboard** affiche :

- Facture du mois en cours
- Prévisions de coûts (estimé) jusqu'à fin du mois
- Services coûtant le plus

C'est un premier coup d'œil rapide, mais peu détaillé.

AWS Cost Explorer

Cost Explorer permet une **analyse fine des dépenses** :

- Filtrer par service (EC2, S3, RDS...)
- Filtrer par période (jour, mois, année)
- Filtrer par région
- Voir l'historique et les tendances

Cas d'usage : « Pourquoi ma facture a-t-elle explosé en janvier ? Quel service en est responsable ? »

AWS Budgets

AWS Budgets permet de **définir des seuils d'alerte** :

- Budget mensuel maximum (ex : 500 €/mois)

- Alertes si dépassement imminent
- Alertes si usage atypique détecté

Cas d'usage : Recevoir une notification avant d'exploser le budget plutôt qu'une mauvaise surprise à la fin du mois.

AWS Cost Anomaly Detection

Utilise l'**apprentissage automatique** pour détecter les **anomalies de dépenses** :

- Si vous dépensez soudain 10x plus que normal dans une région, alerter.
- Si un service coûte étrangement cher, signaler.

C'est très utile pour détecter des ressources oubliées ou des fuites de données.

AWS Pricing Calculator

AWS Pricing Calculator permet d'**estimer les coûts** avant de déployer :

- Configurez votre architecture (1 instance EC2 t3.micro, 100 Go S3...)
- Voir le coût mensuel/annuel
- Comparer différentes configurations

C'est idéal pour les **POC (Proof of Concept)** et les **RFQ** (appels d'offre).

Bonnes pratiques pour maîtriser les coûts

Pratique	Description	Impact
Utiliser des Reserved Instances	Engagement 1 ou 3 ans pour réduction tarifaire (~30-60%)	Très utile pour charges stables 24/7
Utiliser Spot Instances	Capacité excédentaire à -70% (mais peut être interrompue)	Idéal pour tâches batch, tests
Éteindre les ressources non utilisées	Arrêter EC2 dev/test en fin de journée, supprimer RDS inutilisés	Économies rapides et simples
Auto-scaling	Adapter le nombre d'instances au trafic	Évite surprovisionnement

Optimiser le stockage	S3 Standard pour accès fréquent, S3 Glacier pour archive	Différences tarifaires importantes
Utiliser des réductions volantes	Savings Plans, commitments, consolidated billing	~30-40% de réduction possible
Monitorer régulièrement	Cost Explorer, Budgets, anomaly detection	Détection précoce des dérives

Exemple concret : budget mal maîtrisé

Imaginons une startup qui lance une application web sur AWS.

Scénario optimiste :

- Petite instance EC2 dont la consommation est couverte par le solde de crédits du compte de démonstration
- 50 Go S3 (~1 € / mois)
- **Coût mensuel : ~1 €**

Scénario problématique (sans monitoring) :

- Instance EC2 mal fermée le week-end : +50 €
- Bucket S3 avec version history excessive : +100 €
- Transfert de données cross-région oublié : +200 €
- **Coût mensuel : ~350 € !**

Avec les outils AWS Budgets :

- Budget défini : 50 €/mois
- Alerte à 40 € : Action rapide
- Découverte de l'instance oubliée → suppression
- Découverte des versions excessives → nettoyage
- **Coût final : 5 €/mois**

👉 **Résultat attendu — alerte AWS Budgets reçue par email :**

```

1 De : no-reply@notifications.aws
2 Objet : [AWS Budgets] Alerte – Mon Budget Mensuel (80% atteint)
3
4 Bonjour,
```



```
5
6 Votre budget "Mon Budget Mensuel" a atteint 80% de votre seuil d'alerte.
7
8 Budget : 50,00 USD
9 Dépensé : 40,23 USD
10 Prévu ce mois : 52,31 USD
11
12 Service le plus coûteux : Amazon EC2 (28,50 USD)
13 Recommandation : vérifiez les instances EC2 actives dans toutes les régions.
14
15 → Accéder au Cost Explorer : https://console.aws.amazon.com/cost-management/
```

Grâce à cette alerte précoce, vous pouvez arrêter l'instance oubliée avant que la facture n'explose. Sans ce budget configuré, vous n'auriez découvert le problème qu'à la réception de la facture mensuelle. :::

C'est pourquoi **mettre en place un monitoring budgétaire dès le départ est essentiel.**

 [AWS Billing and Cost Management](#)  [AWS Cost Explorer Guide](#)  [AWS Cost Optimization Best Practices](#)

9. Les services AWS les plus utilisés

AWS propose un catalogue étendu et évolutif couvrant le calcul, le stockage, le réseau, les bases de données, la sécurité et de nombreux services applicatifs. Il est plus utile de comprendre les familles et les critères de choix que de mémoriser un nombre de services rapidement périmé. Sa philosophie est simple : offrir à chaque entreprise, quelle que soit sa taille, **la même puissance technologique** qu'Amazon utilise pour ses propres opérations mondiales.

Domaine	Service	Explication	Cas d'usage	Analogie
Calcul	EC2	Machines virtuelles configurables (CPU, RAM, OS)	Hébergement web, backend, batch	Louer un serveur dans le cloud
	Lambda	Exécution de code sans	Microservices, automatisation,	Interrupteur intelligent :

		serveur, déclenchée par événements	API	déclenche une action sans infrastructure
Stockage	S3	Stockage d'objets scalable et durable	Sauvegardes, fichiers, hébergement statique	Entrepôt numérique : chaque fichier est une boîte
	EBS	Stockage bloc attaché à EC2, persistant	Bases de données, stockage système	Disque dur virtuel
	EFS	Système de fichiers partagé pour EC2	Dossiers partagés, applications multi- instances	Dossier réseau partagé
Bases de données	RDS	Base relationnelle gérée (MySQL, PostgreSQL...)	ERP, CRM, applications transactionnelles	Serveur SQL géré par AWS
	DynamoDB	Base NoSQL scalable sans schéma	IoT, mobile, jeux, logs	Carnet de notes sans structure fixe
Réseau	VPC	Réseau privé isolé avec sous- réseaux et sécurité	Isolation d'environnements, segmentation	Immeuble privé avec étages et portiers
	CloudFront	CDN mondial pour accélérer les contenus	Sites web, vidéos, fichiers statiques	Réseau de relais pour livrer plus vite
Sécurité	IAM	Gestion des identités et des accès	Contrôle des permissions, MFA, rôles	Badge d'accès pour chaque utilisateur

	KMS	Gestion des clés de chiffrement	Sécurité des données, RGPD, PCI-DSS	Coffre-fort numérique pour les clés
DevOps	CloudFormation	Déploiement d'infrastructure via des fichiers YAML/JSON	Automatisation, standardisation	Plan d'architecte pour construire automatiquement
	CodePipeline	Orchestration CI/CD pour le déploiement	Intégration continue, tests, livraison	Chaîne de montage automatisée
IA/ML	SageMaker	Plateforme de machine learning gérée	Modèles prédictifs, classification, NLP	Laboratoire de data science automatisé
	Rekognition	Analyse d'images et vidéos par IA	Détection faciale, modération, OCR	Caméra intelligente qui comprend les images
Analyse	Athena	Requêtes SQL sur des fichiers S3	Exploration de données, logs, BI	Loupe SQL sur vos fichiers
	Redshift	Entrepôt de données pour BI et reporting	Tableaux de bord, KPIs, analytique	Base de données XXL pour l'analyse

AWS Products [↗](#)

Nous n'entrons pas ici dans les détails techniques de chaque service — cela sera traité dans les jours suivants (IAM, S3, EC2, RDS, VPC...).

9.1 Automatisation et orchestration : Chef, Puppet et OpsWorks

Au-delà des services de base, AWS propose des outils pour **automatiser le déploiement et la gestion de l'infrastructure**. Ces outils sont essentiels dans une approche **DevOps** et **Infrastructure as Code (IaC)**.

AWS OpsWorks — Orchestration managée par AWS

AWS OpsWorks était une famille de services AWS destinée au déploiement et à la gestion automatisés d'applications et d'infrastructures.

Il supporte deux moteurs d'automation populaires :

- **Chef** (propriétaire de OpsWorks Chef)
- **Puppet** (partenariat AWS)

⚠ **Toute la famille OpsWorks est désormais en fin de vie et désactivée.** OpsWorks for Chef Automate et OpsWorks for Puppet Enterprise ont été arrêtés le 5 mai 2024 ; OpsWorks Stacks l'a été le 26 mai 2024. Cette section sert uniquement à reconnaître une infrastructure historique et à comprendre les principes de Chef et Puppet. Pour une nouvelle architecture AWS, privilégiez notamment AWS Systems Manager, les images automatisées, les services de conteneurs ou une solution de gestion de configuration encore maintenue. ⚠

Cas d'usage typiques :

- Provisionner automatiquement des serveurs EC2 avec une stack applicative complète.
- Gérer les configurations à grande échelle sans intervention manuelle.
- Assurer la conformité des serveurs (tous les serveurs web ont la même configuration).
- Automatiser les déploiements continus (CI/CD).

Chef — Automation as Code

Chef est un outil d'**automatisation de configuration** très populaire. Il fonctionne sur un modèle de **recettes (recipes)** écrites en Ruby qui décrivent **l'état souhaité** d'une machine.

```
1 # Exemple simplifié d'une recipe Chef
2 package 'apache2' do
3   action :install
4 end
5
6 service 'apache2' do
7   action [:enable, :start]
8 end
9
10 template '/var/www/html/index.html' do
```

```
11 source 'index.html.erb'
12 end
```

:::success **Résultat attendu — exécution de `chef-client` :**

```
1 [2024-01-15T09:12:34+00:00] INFO: Starting Chef Infra Client Run
2 [2024-01-15T09:12:35+00:00] INFO: Installing package apache2
3 [2024-01-15T09:12:42+00:00] INFO: package[apache2] installed version 2.4.57
4 [2024-01-15T09:12:43+00:00] INFO: service[apache2] enabled and started
5 [2024-01-15T09:12:43+00:00] INFO: template[/var/www/html/index.html] created fi
6 [2024-01-15T09:12:43+00:00] INFO: Chef Infra Client Run complete in 9.123 secon
```

Apache est installé, démarré et la page d'accueil est en place. Si vous relancez `chef-client`, rien ne change — c'est l'idempotence en action. :::

Intérêt : Au lieu de cliquer dans une UI ou d'écrire des scripts bash, vous décrivez le **résultat attendu** (Apache installé, service actif, fichiers à jour). Chef **idempotent** — si vous exécutez la recipe 10 fois, le résultat sera toujours le même.

 [Chef Documentation](#) 

Puppet — Configuration Management

Puppet est un outil comparable à Chef, mais avec une approche **déclarative** différente. Au lieu de recettes, Puppet utilise un **langage déclaratif** qui décrit l'état souhaité en termes de ressources. Voici le même objectif (Apache installé et actif) exprimé en syntaxe Puppet :

```
1 # Exemple simplifié de Puppet
2 package { 'apache2':
3   ensure => present,
4 }
5
6 service { 'apache2':
7   ensure => running,
8   enable => true,
9 }
```

:::success **Résultat attendu — exécution de `puppet agent --test` :**

```
1 Info: Caching catalog for node01.example.com
2 Info: Applying configuration version '1705312800'
3 Notice: /Stage[main]/Main/Package[apache2]/ensure: created
```

```
4 Notice: /Stage[main]/Main/Service[apache2]/ensure: ensure changed 'stopped' to
5 Notice: Applied catalog in 8.54 seconds
```

Puppet a appliqué l'état souhaité : `apache2` installé et le service actif. Tout est tracé dans les logs Puppet Master. :::

Intérêt : Comme Chef, il permet d'automatiser des déploiements complexes à grande échelle.

 [Puppet Documentation](#)

Ansible — Automatisation agentless et modules AWS

Ansible est aujourd'hui l'un des outils d'automatisation les plus utilisés dans les environnements Cloud, notamment en **complément de Terraform** pour la gestion de configuration.

Contrairement à Chef ou Puppet, Ansible est **agentless** : il n'installe rien sur les machines cibles — il se connecte via SSH (Linux) ou WinRM (Windows) et exécute les tâches définies dans des **playbooks** YAML. Voici un playbook typique pour configurer un serveur Apache sur une instance EC2 :

```
1  # Exemple de playbook Ansible – installation Apache sur EC2
2  ---
3  - name: Configurer un serveur web Apache sur EC2
4    hosts: ec2_instances
5    become: yes
6
7    tasks:
8      - name: Installer Apache
9        apt:
10          name: apache2
11          state: present
12
13      - name: Démarrer et activer le service
14        service:
15          name: apache2
16          state: started
17          enabled: yes
18
19      - name: Copier la page d'accueil
20        copy:
21          src: index.html
22          dest: /var/www/html/index.html
```

:::success **Résultat attendu** — `ansible-playbook site.yml -i inventory.ini` :

```
1  PLAY [Configurer un serveur web Apache sur EC2] *****
2
3  TASK [Gathering Facts] *****
4  ok: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]
5
6  TASK [Installer Apache] *****
7  changed: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]
8
9  TASK [Démarrer et activer le service] *****
10 changed: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]
11
12 TASK [Copier la page d'accueil] *****
13 changed: [ec2-18-234-56-78.eu-west-3.compute.amazonaws.com]
14
15 PLAY RECAP *****
16 ec2-18-234-56-78.eu-west-3.compute.amazonaws.com : ok=4  changed=3  unreachable
```

Apache est installé et opérationnel sur l'instance EC2. La ligne `changed=3` confirme que les trois tâches ont apporté des modifications. Si vous relancez le playbook, vous verrez `changed=0` — c'est l'idempotence Ansible. :::

Ansible propose également une **collection dédiée AWS** (`amazon.aws`) permettant de piloter directement les ressources AWS depuis un playbook :

```
1  # Exemple : créer une instance EC2 avec le module Ansible AWS
2  - name: Lancer une instance EC2
3    amazon.aws.ec2_instance:
4      name: "mon-serveur-web"
5      instance_type: t3.micro
6      image_id: ami-0c55b159cbfafa1f0
7      region: eu-west-3
8      key_name: ma-cle-ssh
9      security_groups:
10        - sg-xxxxxxxxx
11      tags:
12        Env: production
13        Owner: formation
```

:::success **Résultat attendu** — `ansible-playbook create-ec2.yml` :

```

1 TASK [Lancer une instance EC2] *****
2 changed: [localhost]
3
4 PLAY RECAP *****
5 localhost : ok=1  changed=1  unreachable=0  failed=0
6
7 Instance créée : i-0a1b2c3d4e5f67890
8 IP publique    : 15.236.142.87
9 Région         : eu-west-3
10 Statut        : running

```

L'instance EC2 `mon-serveur-web` est lancée en `eu-west-3`. Elle est tagguée `Env: production` et visible dans la console AWS sous EC2 > Instances. Vous pouvez vous y connecter via SSH : `ssh -i ma-cle-ssh.pem ubuntu@15.236.142.87` .:::

Intérêt dans un contexte AWS :

- Complémentaire à **Terraform** : Terraform crée l'infrastructure (VPC, EC2, RDS...), Ansible configure les instances après leur lancement.
- Pas d'agent à installer sur les instances EC2 — idéal pour les environnements éphémères.
- Intégration native avec les services AWS via la collection `amazon.aws`.
- Utilisé dans les pipelines CI/CD avec CodePipeline ou Jenkins.

 [Ansible Documentation officielle](#)  [Collection Ansible AWS \(amazon.aws\)](#)

CloudFormation — Infrastructure as Code AWS-native

Enfin, il existe **AWS CloudFormation**, l'outil **native AWS** pour déployer l'infrastructure via du code YAML/JSON. Ce template déclare deux ressources : une instance EC2 et un bucket S3. AWS les crée dans le bon ordre, sans commande manuelle :

```

1 # Exemple CloudFormation : créer une instance EC2 et un bucket S3
2 AWSTemplateFormatVersion: '2010-09-09'
3 Resources:
4   MyInstance:
5     Type: AWS::EC2::Instance
6     Properties:
7       ImageId: ami-0c55b159cbfafa1f0
8       InstanceType: t2.micro
9
10  MyBucket:
11    Type: AWS::S3::Bucket

```



```
12 Properties:
13 BucketName: my-app-bucket
```

:::success **Résultat attendu — après `aws cloudformation deploy --template-file stack.yaml --stack-name ma-stack` :**

```
1 Waiting for changeset to be created...
2 Waiting for stack create/update to complete...
3
4 Successfully created/updated stack - ma-stack
5
6 Stack ID    : arn:aws:cloudformation:eu-west-3:123456789012:stack/ma-stack/abc12
7 Status     : CREATE_COMPLETE
8 Resources  :
9   - MyInstance → i-0abc123def456789 (AWS::EC2::Instance) CREATE_COMPLETE
10  - MyBucket   → my-app-bucket       (AWS::S3::Bucket)   CREATE_COMPLETE
```

Les deux ressources ont été créées par CloudFormation dans le bon ordre. En cas de suppression, `aws cloudformation delete-stack --stack-name ma-stack` supprimera l’EC2 et le bucket ensemble — ce qui garantit qu’il ne reste pas de ressources orphelines. :::

Intérêt : CloudFormation est intégré nativement à AWS. Vous versionnez votre infrastructure comme du code et pouvez la recréer en quelques clics.

Comparaison simplifiée

Outil	Modèle	Langage	Cas d’usage	Intégration AWS
Chef	Impératif (recipes)	Ruby	Configurations complexes ou parc historique	Solution Chef maintenue hors OpsWorks
Puppet	Déclaratif	Puppet DSL	Gestion de flotte à grande échelle	Solution Puppet maintenue hors OpsWorks

Ansible	Déclaratif (agentless)	YAML	Config management, post- provisioning	Via collection <code>amazon.aws</code>
CloudFormation	Déclaratif (IaC)	YAML/JSON	Déploiement d'infrastructure AWS	Native, recommandé
Terraform	Déclaratif (IaC)	HCL	Multi-cloud, plus flexible	Via AWS provider

Recommandation pratique :

- Pour **débuter avec AWS**, utiliser **CloudFormation** ou **Terraform**.
- Pour **configurer les instances après déploiement**, utiliser **Ansible** — agentless et très bien intégré à AWS.
- Pour **gérer des configurations** sur des centaines de serveurs, évaluer AWS Systems Manager ou une solution Chef/Puppet maintenue ; ne pas créer de dépendance à OpsWorks.
- Pour **Puppet**, privilégier dans les environnements d'entreprise très structurés.
- En pratique : **Terraform + Ansible** est le duo le plus courant aujourd'hui — Terraform provisionne, Ansible configure.

 [AWS CloudFormation Documentation](#)
 [Sortir d'AWS OpsWorks Stacks avant sa fin de vie](#)

 [Terraform AWS Provider](#)

10. Bonnes pratiques de démarrage

10.1 Repères pour la navigation dans la console AWS

Voici les éléments essentiels à repérer dans la console pendant la démonstration.

Éléments clés :

1. **Sélecteur de région** : situé en haut à droite, il permet de choisir la région AWS dans laquelle les ressources seront déployées.
2. **Barre de recherche des services** : permet d'accéder rapidement à n'importe quel service AWS (EC2, S3, VPC, IAM, RDS...).

3. **Tableau de bord (Dashboard)** : affiche les services récemment utilisés et les informations de facturation.
4. **Panneau IAM** : permet de gérer les utilisateurs, groupes, rôles et stratégies de sécurité.
5. **Billing & Cost Management** : donne accès au suivi de la consommation et à la facturation.

 [Guide AWS Management Console](#)  [AWS Billing Documentation](#)  [IAM Documentation](#)

Retenons que certaines ressources sont **spécifiques à une région** et que la facturation dépend de cette localisation.

⚠ Piège fréquent — mauvaise région active : Si vous créez des ressources dans une région autre que celle attendue (ex. `us-east-1` au lieu de `eu-west-3`), vous ne les verrez pas dans votre vue habituelle et continuerez à les payer. Vérifiez toujours le sélecteur de région en haut à droite de la console avant toute création de ressource.

11. Points importants et pièges fréquents

Piège	Réalité
« AWS, c'est juste des serveurs dans le cloud »	AWS propose un catalogue étendu couvrant notamment calcul, stockage, réseau, données, IA, sécurité et exploitation.
« Je suis en sécurité, AWS gère tout »	Non. AWS gère la sécurité <i>du</i> cloud, vous gérez la sécurité <i>dans</i> le cloud (IAM, chiffrement, configuration).
« Le Cloud public n'est pas conforme RGPD »	Faux. AWS est conforme RGPD. C'est votre usage qui doit être conforme — localisation des données, consentement, droit à l'oubli, etc.
« Les coûts AWS sont imprévisibles »	Avec une bonne gouvernance (budgets, alertes, AWS Cost Explorer), les coûts sont très maîtrisables.
« Un conteneur = une VM plus légère »	Non. Un conteneur ne contient pas d'OS complet — il partage le noyau de l'hôte. C'est une architecture radicalement différente.
« Je dois mettre toutes mes données en cloud »	Non. Certaines données peuvent rester on-premise pour des raisons légales, réglementaires ou métier. Le cloud hybride existe pour ça.

« IaaS, PaaS, SaaS — pareil pareil »

Non. Chaque modèle **décale les responsabilités**. En IaaS, vous gérez plus ; en SaaS, AWS gère presque tout.

« Je peux utiliser n'importe quelle région »

Non. Certaines régions n'ont pas tous les services, et les données sensibles doivent rester dans des zones spécifiques (ex. RGPD en UE).

Comprendre l'infrastructure ne suffit pas — encore faut-il la sécuriser. Le Chapitre 2 est entièrement consacré à la sécurité et à la gestion des identités dans AWS : comment **IAM** structure les droits et les accès, comment appliquer les bonnes pratiques de moindre privilège, et comment mettre en place une gouvernance solide.

Ressources

Documentation officielle AWS

- [AWS Documentation](#) 
- [AWS Well-Architected Framework](#) 
- [AWS Free Tier](#) 
- [Régions et zones de disponibilité AWS](#) 

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les questions et les propositions restent dans un ordre stable.

Le quiz interactif est disponible dans la version web du support.

Chapitre 2 — Sécurité des accès avec AWS IAM

Vocabulaire

1 · Identités et autorisations IAM

2 · MFA et politiques conditionnelles

3 · Fédération et SSO

4 · Identités applicatives avec Cognito

5 · Stratégie multi-comptes

6 · Traçabilité avec CloudTrail

7 · Points d'attention

8 · Choisir l'authentification

Ressources

Quiz du chapitre

Vocabulaire du chapitre

Terme	Définition
Identité	Entité connue d'un système d'authentification, par exemple une personne, une application ou un service.
Principal AWS	Identité qui signe et envoie une requête à AWS : utilisateur IAM, rôle, service ou session fédérée.
Authentification	Vérification de l'identité déclarée.
Autorisation	Décision qui permet ou refuse une action sur une ressource après authentification.
Politique IAM	Document JSON qui décrit des autorisations ou des refus au moyen d'actions, de ressources et de conditions.
Rôle IAM	Identité AWS sans identifiants permanents, assumée pour obtenir une session temporaire.
MFA	Authentification multifacteur : utilisation d'au moins deux catégories de preuves distinctes.
Fédération	Délégation de l'authentification à un fournisseur d'identité externe à AWS.
SSO	Authentification unique permettant d'accéder à plusieurs applications ou comptes après une seule connexion.

:::info Le chapitre précédent a posé les bases théoriques du Cloud Computing et présenté l'écosystème AWS ; ce chapitre entre dans le concret en abordant le premier pilier opérationnel de toute architecture AWS : la sécurité des accès.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Expliquer** le rôle stratégique d'IAM et ses concepts fondamentaux (users, groups, roles, policies)

- **Rédiger** une politique IAM au format JSON et comprendre la logique d'évaluation des permissions
- **Appliquer** le modèle de responsabilité partagée AWS au domaine de la sécurité
- **Sécuriser** un compte utilisateur avec le MFA et des politiques conditionnelles
- **Utiliser** AWS STS pour délivrer des identités temporaires (identity broker)
- **Mettre en œuvre** une fédération d'identité avec IAM Identity Center, SAML et AD FS
- **Distinguer** IAM et Amazon Cognito pour la gestion des identités applicatives
- **Concevoir** une stratégie multi-comptes avec AWS Organizations et des Service Control Policies (SCP)
- **Configurer** la traçabilité des activités avec AWS CloudTrail et la comparer à AWS Config
- **Créer** via la CLI des utilisateurs, groupes, rôles et politiques IAM, et activer le MFA :::

1. Introduction à IAM : Identity and Access Management

1.1 Définition et rôle stratégique

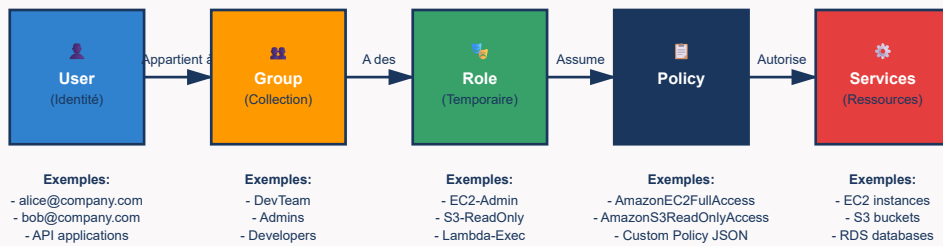
IAM (Identity and Access Management) est le service AWS qui permet de **gérer les identités, les permissions et les politiques d'accès** aux ressources AWS.

IAM est à la sécurité ce que la serrure est à une porte. Il constitue **le cœur de la gouvernance des accès** dans un environnement AWS.

- IAM contrôle **qui** peut faire **quoi** sur **quelle ressource**, et **dans quelles conditions**.
- Il permet de sécuriser les accès au niveau le plus granulaire possible.
- Il est un prérequis à toute architecture bien conçue sur AWS.

IAM fonctionne comme un contrôleur central placé entre deux mondes : d'un côté les **identités** (utilisateurs, groupes, rôles, services AWS), de l'autre les **ressources AWS** qu'elles cherchent à atteindre (S3, EC2, RDS, Lambda...). Chaque requête suit le même chemin : une identité émet un appel API, IAM évalue les **politiques JSON** qui lui sont associées, puis autorise (**Allow**) ou refuse (**Deny**) l'accès à la ressource visée.

Modèle de Contrôle d'Accès IAM — Chaîne de Responsabilité



Concept: Permission Boundary (Limite de Permissions)

Définition:

Une Permission Boundary est une limite maximale définissant le périmètre complet des permissions qu'un utilisateur/rôle peut obtenir. Elle agit comme un filtre : même si une policy accorde l'accès à 100 services, la boundary limite à 5 services.



💡 Conseil: Appliquer le Principe du Moindre Privilège (PoLP) — accorder juste ce qui est nécessaire, pas plus.

Lecture du schéma. Une identité authentifiée envoie une requête. IAM rassemble les politiques applicables, recherche d'abord un refus explicite, puis vérifie qu'une autorisation correspond à l'action et à la ressource. Sans autorisation applicable, la requête est refusée implicitement.

Ce schéma résume le principe : aucune ressource AWS n'est jamais contactée directement par une identité sans passer par cette évaluation IAM — c'est ce mécanisme, invisible mais systématique, qui rend possible le contrôle d'accès au niveau le plus granulaire.

🔗 [Documentation officielle AWS IAM](#)

1.2 Concepts fondamentaux d'IAM

Utilisateur IAM

Un **utilisateur IAM** représente une identité permanente dans AWS, associée à des identifiants (login/mot de passe ou clés d'accès). Il est utilisé pour représenter une personne ou une application qui interagit avec les services AWS.

C'est comme un badge nominatif d'employé : il donne un accès personnel et identifiable au système.

Groupe IAM

Un **groupe IAM** est une collection logique d'utilisateurs qui partagent les mêmes permissions. Il permet d'appliquer des politiques communes à plusieurs utilisateurs, comme les membres d'une équipe Dev, les administrateurs ou les lecteurs.

C'est comme un département dans une entreprise (ex. IT, Marketing, Finance) : tous les membres ont les mêmes règles d'accès.

Rôle IAM

Un **rôle IAM** est une identité temporaire que l'on peut assumer pour accéder à des ressources AWS. Il est utilisé par des services AWS (comme EC2 ou Lambda), ou pour permettre l'accès entre comptes ou via une fédération d'identités.

C'est comme un badge visiteur temporaire : il donne un accès limité dans le temps à certaines zones.

Politique IAM

Une **politique IAM** est un document JSON qui définit précisément les permissions accordées ou refusées. Elle permet de contrôler les actions autorisées sur les ressources AWS.

C'est comme un règlement intérieur : il définit ce qui est autorisé ou interdit pour chaque profil.

MFA (Multi-Factor Authentication)

Le **MFA** ajoute une couche de sécurité en exigeant un second facteur d'authentification, comme un code temporaire ou une clé physique, en plus du mot de passe.

C'est comme une double serrure : il faut une clé et un code pour ouvrir la porte.

STS (Security Token Service)

Le **STS** permet de générer des identifiants temporaires pour accéder à AWS, avec une durée limitée (15 minutes à 12 heures). Il est idéal pour déléguer des accès sans exposer de credentials permanents.

C'est comme un badge temporaire valable quelques heures : il expire automatiquement après usage.

Sans STS, chaque utilisateur devrait avoir des identifiants IAM permanents dans chaque compte AWS, avec des permissions fixes. Cela rendrait la gestion des accès lourde, risquée, et peu compatible avec les standards modernes d'authentification.

⚠️ danger Ne jamais utiliser le compte root pour les opérations courantes. Le compte root AWS dispose de tous les droits sans restriction — il n'est pas soumis aux politiques IAM. Toute

compromission du root expose l'intégralité du compte. Créez immédiatement un utilisateur IAM administrateur, activez MFA sur le root, puis verrouillez les credentials root. :::

1.3 Modèle de responsabilité partagée appliqué à la sécurité

Observons le modèle de responsabilité partagée appliqué à la sécurité.

Élément	Responsabilité AWS	Responsabilité Client
Infrastructure physique	✓	✗
Réseau global	✓	✗
Contrôle d'accès utilisateur	✗	✓
Stratégies IAM	✗	✓
Configuration du chiffrement	✗	✓
Gestion des identités externes	✗	✓

Ce tableau résume la philosophie d'AWS :

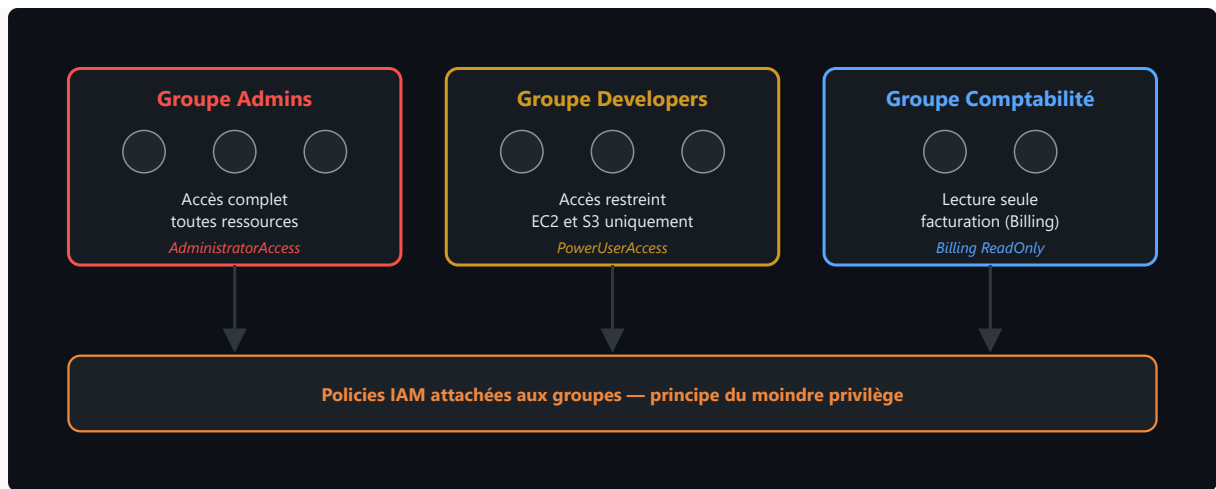
« AWS sécurise le cloud, vous sécurisez ce que vous y déployez ».

Il est important de rappeler qu'IAM ne se limite pas à la gestion d'utilisateurs. Il s'agit d'un **système d'autorisation distribué** qui s'applique aussi aux services AWS eux-mêmes, aux rôles inter-comptes et aux identités fédérées.

1.4 Exemple concret d'organisation des accès

Un administrateur met en place une **stratégie de sécurité structurée** :

- Groupe **Admins** → droits complets sur le compte AWS.
- Groupe **Developers** → accès restreint à **EC2** et **S3**.
- Groupe **Comptabilité** → lecture seule sur la facturation (Billing).



Lecture du schéma. Les utilisateurs sont rattachés à des groupes représentant une fonction. Les politiques attachées au groupe transmettent les mêmes autorisations à ses membres. Un groupe ne s’imbrique pas dans un autre groupe IAM et ne doit pas servir à représenter une charge de travail.

Chaque groupe reçoit une **policy JSON adaptée**, garantissant le **principe du moindre privilège** — le détail de la syntaxe JSON de ces politiques est développé juste après, en §1.5.

1.5 Structure d’une politique IAM — Exemple concret

Policy : Lecture seule sur S3

Cette politique illustre la syntaxe de base d’une policy IAM. Notez les quatre champs obligatoires : `Version`, `Statement`, `Effect`, `Action`, et `Resource`.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": [
7          "s3:GetObject",
8          "s3:ListBucket"
9        ],
10       "Resource": "*"
11     }
12   ]
13 }
```

Cette policy donne aux membres du groupe uniquement la possibilité de **lister et lire les objets S3**.

Policy : Accès restreint à un bucket spécifique

Pour restreindre l'accès à **un bucket précis** (et non à tout S3), on remplace `"Resource": "*" par l'ARN du bucket cible.`

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Action": [
7          "s3:GetObject",
8          "s3:ListBucket"
9        ],
10       "Resource": [
11         "arn:aws:s3:::mon-bucket-secret",
12         "arn:aws:s3:::mon-bucket-secret/*"
13       ]
14     }
15   ]
16 }
```

Cette policy autorise la lecture d'objets S3 dans le bucket `mon-bucket-secret`, **mais uniquement si l'utilisateur est connecté depuis une IP comprise dans `203.0.113.0/24`.**

1.6 IAM Policy Evaluation Logic

Le **IAM Policy Evaluation Logic** est le **mécanisme interne d'AWS** qui détermine **si une action est autorisée ou refusée** lorsqu'un utilisateur ou un rôle tente d'accéder à une ressource AWS (ex : S3, EC2, RDS...).

En d'autres termes : **c'est la logique de décision** qu'AWS applique pour savoir si une requête doit être acceptée ou rejetée.

Il est utilisé **à chaque fois qu'un utilisateur, un rôle ou un service** tente d'effectuer une action sur une ressource AWS.

Exemples :

- Un utilisateur veut lire un fichier dans S3 → AWS vérifie les politiques IAM
- Un rôle Lambda veut écrire dans DynamoDB → AWS vérifie les politiques IAM
- Un service EC2 veut accéder à un secret → AWS vérifie les politiques IAM

AWS suit **trois règles fondamentales**, dans cet ordre :

1. **Deny explicite** → priorité absolue

Si une politique dit **“Effect”: “Deny”**, l'accès est **refusé**, même si une autre politique dit “Allow”.

2. **Allow explicite** → si aucun Deny

Si une politique dit **“Effect”: “Allow”**, et qu'il n'y a pas de Deny, l'accès est **autorisé**.

3. **Pas de politique = refus implicite**

Si aucune politique ne couvre l'action demandée, l'accès est **refusé par défaut**.

 [Policy Evaluation Logic](#)

 [Policy Simulator](#)

⚠ **Principe du moindre privilège (Least Privilege)**. Ne jamais attribuer `"Action": "*" ou "Resource": "*" en production`. Accordez uniquement les permissions strictement nécessaires à la tâche. En cas de doute, commencez par un accès minimal et élargissez progressivement selon les besoins réels.

1.7 Services IAM complémentaires

Service	Rôle
AWS Organizations	Gestion centralisée de plusieurs comptes AWS (OU, SCP, facturation consolidée).
AWS STS (Security Token Service)	Génère des identifiants temporaires sécurisés.
Amazon Cognito	Gestion d'utilisateurs finaux (authentification applicative).
IAM Identity Center (ex-SSO)	Authentification unique (SSO) sur plusieurs comptes et applications.

 [AWS Organizations](#)

 [STS Documentation](#)

2. Sécuriser les accès IAM avec MFA et politiques conditionnelles

2.1 Pourquoi MFA et politiques conditionnelles ?

Dans tout système d'information, la sécurité ne repose pas seulement sur **qui est connecté**, mais aussi sur **comment** cette connexion est sécurisée.

Sur AWS, l'**authentification multifacteur (MFA)** est une mesure essentielle pour réduire les risques liés aux accès non autorisés — en particulier sur les comptes disposant de privilèges élevés (root, admin, DevOps...).

L'activation de MFA fait partie des **bonnes pratiques fondamentales** recommandées par AWS dès la création d'un compte.

Mais MFA n'est pas seulement un bouton à cocher : combinée aux **politiques conditionnelles IAM**, elle permet de mettre en place une **sécurité contextuelle et granulaire**.

2.2 Comprendre MFA

Définition

MFA (Multi-Factor Authentication) est un mécanisme de sécurité qui nécessite **au moins deux méthodes d'authentification indépendantes** :

1. **Quelque chose que vous savez** → mot de passe, clé d'accès.
2. **Quelque chose que vous possédez** → application MFA (ex : Authenticator) ou clé physique (YubiKey).
3. (Optionnel) **Quelque chose que vous êtes** → biométrie.

Dans AWS, la MFA s'applique :

- au compte **root**,
- aux **utilisateurs IAM**,
- aux **rôles** via AWS CLI ou API,
- aux **accès fédérés**.

Types de MFA supportés

Type	Description	Exemples
MFA virtuelle	Application mobile (TOTP)	Google Authenticator, Authy, Microsoft Authenticator
MFA matérielle	Clé physique dédiée	YubiKey, Gemalto
Passkey / FIDO2	Authentification sans mot de passe	Clé biométrique ou physique compatible FIDO

Gemalto est un fabricant (aujourd’hui filiale de Thales) de clés de sécurité physiques concurrentes de YubiKey — les deux fonctionnent selon le même principe : un petit boîtier USB ou NFC à brancher/approcher pour valider la connexion, sans code à recopier. **FIDO2** (Fast IDentity Online 2) est le standard ouvert sur lequel reposent ces clés biométriques ou physiques : il définit comment le navigateur, l’appareil et le service en ligne dialoguent pour vérifier votre identité sans jamais transmettre de mot de passe — c’est la même technologie qui permet de se connecter avec son empreinte digitale ou Face ID sur un site compatible.

2.3 Bonnes pratiques AWS sur MFA

- Toujours activer MFA sur le **compte root** (obligatoire en production).
- Exiger MFA pour tous les utilisateurs **ayant des privilèges élevés**.
- Automatiser la vérification de MFA via **AWS Config** ou **Security Hub**.
- Interdire les actions sensibles (ex : suppression d’instances, modification de politiques) sans MFA.

 [AWS Security Best Practices](#) 

⚠ danger MFA obligatoire sur le compte root et tous les comptes administrateurs. Une connexion sans MFA sur un compte à privilèges élevés est la principale cause de compromission de comptes AWS. AWS

Security Hub et IAM Access Analyzer peuvent détecter automatiquement les comptes sans MFA et générer des alertes. :::

2.4 Audit de MFA avec AWS Config

AWS Config permet de vérifier automatiquement que les utilisateurs IAM ont bien activé MFA.

Exemple de règle : `iam-user-mfa-enabled`

 [AWS Config Rules](#)

2.5 Politiques conditionnelles IAM

Une **policy conditionnelle** permet d'ajouter des **règles contextuelles** à une politique IAM classique.

Cela permet d'aller **au-delà des permissions statiques** en intégrant des critères comme :

- la présence ou non de MFA,
- l'adresse IP source,
- la région AWS,
- l'heure,
- le VPC Endpoint utilisé.

La clé `Condition` dans une policy JSON contrôle ces règles.

2.6 Exemples de politiques conditionnelles

Exiger MFA pour les actions sensibles

La clé `Condition` permet d'ajouter des critères supplémentaires à une policy. Voici comment bloquer **toutes les actions** si l'utilisateur n'a pas activé MFA pour sa session en cours :

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
```



```

6     "Action": "*",
7     "Resource": "*",
8     "Condition": {
9         "BoolIfExists": {
10            "aws:MultiFactorAuthPresent": "false"
11        }
12    }
13 }
14 ]
15 }

```

Cette policy **refuse toute action** si MFA n'est pas activée pour la session.

Elle peut être attachée à :

- un groupe d'administrateurs,
- un utilisateur,
- un rôle IAM.

Remarque : cette approche est **préférée** à un simple "Allow MFA" car elle couvre tous les cas par défaut.

Restreindre les accès à une IP spécifique

Pour n'autoriser les connexions que depuis un réseau d'entreprise (VPN ou bureau), on utilise la condition `aws:SourceIp`. Ici, `NotIpAddress` signifie "si l'IP n'est PAS dans cette plage, refuser" :

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": "*",
7        "Resource": "*",
8        "Condition": {
9          "NotIpAddress": {
10             "aws:SourceIp": "203.0.113.0/24"
11          }
12        }
13      }
14    ]

```

Cette policy **refuse toute action** si la connexion ne provient pas de l'adresse IP autorisée.

Combinée à MFA, elle renforce la **sécurité d'accès des administrateurs**.

Politique combinée IP + MFA

Les conditions se combinent avec un **ET logique** : toutes doivent être vraies pour que l'accès soit autorisé. C'est la politique la plus stricte — idéale pour les comptes administrateurs :

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": "*",
7        "Resource": "*",
8        "Condition": {
9          "BoolIfExists": {
10             "aws:MultiFactorAuthPresent": "false"
11           },
12          "NotIpAddress": {
13             "aws:SourceIp": "203.0.113.0/24"
14           }
15        }
16      }
17    ]
18  }
```

Ici, l'accès n'est autorisé que si :

- l'utilisateur se connecte **depuis l'IP autorisée** ET
- **MFA est activée**.

 [IAM Policy Elements - Condition](#)

2.7 Bonnes pratiques et points d'attention

- MFA ne remplace pas les bonnes politiques IAM, elle **les renforce**.
- Toujours combiner MFA avec une politique conditionnelle.
- Documenter les exceptions éventuelles (services automatisés, CI/CD).
- Tester systématiquement les politiques dans **Policy Simulator** avant déploiement.

2.8 STS (Security Token Service) — Identity Broker en détail

Qu'est-ce qu'une identité temporaire ?

Jusqu'à présent, nous avons parlé d'**identités permanentes** :

- Un utilisateur IAM a un **login/mot de passe** et des **clés d'accès** permanents.
- Un rôle IAM reçoit des permissions via une policy.

Mais dans une architecture moderne, il est souvent risqué de **distribuer des credentials permanents**. C'est où intervient **AWS STS (Security Token Service)**.

STS génère des identifiants temporaires (valides quelques minutes à quelques heures), qui expirent automatiquement sans intervention manuelle.

C'est comme un badge visiteur qui se détruit après 24 heures : il n'y a rien à récupérer après son expiration.

Comment fonctionne STS ?

Le processus est simple :

1. Un utilisateur ou service demande un token STS.
2. STS valide la demande (qui êtes-vous ? avez-vous le droit ?).
3. STS émet **un set de credentials temporaires** : AccessKeyId, SecretAccessKey, et SessionToken.
4. Ces credentials permettent d'accéder à AWS pendant leur durée de validité.
5. À l'expiration, les credentials deviennent inutilisables.

STS Identity Broker — Cas d'usage concret

Un **Identity Broker** est un système qui :

1. Authentifie un utilisateur via un système externe (AD, LDAP, SSO, application custom).
2. Contacte STS pour obtenir des credentials AWS temporaires.

3. Retourne ces credentials à l'utilisateur.

Cela permet aux utilisateurs d'accéder à AWS **sans avoir de compte IAM permanent**, et sans exposer d'accès clés stockés durablement.

Exemple : Intégration avec un système LDAP corporate

Une entreprise utilise **LDAP interne** pour gérer ses employés. Elle souhaite que ces employés accèdent à AWS **sans créer de comptes IAM**.

Approche sans STS (mauvaise) :

- Créer un compte IAM pour chaque employé.
- Distribuer des clés d'accès à chacun.
- Gérer les mots de passe dans deux systèmes (LDAP + IAM).
- Déploiement manuel, complexe, peu sécurisé.

Approche avec STS Identity Broker (correcte) :

1. L'employé se connecte à un **portail interne** avec ses identifiants LDAP.
2. Le portail authentifie l'utilisateur contre LDAP.
3. Le portail contacte STS via une API : « *Cet utilisateur veut un token AWS* ».
4. STS retourne des credentials temporaires.
5. Le portail affiche à l'employé un **lien direct vers la console AWS** avec ces credentials.
6. L'employé clique, se connecte à AWS, et accède aux ressources autorisées.
7. **Aucun compte IAM permanent n'a été créé.**

Flux technique de STS

```
1  Utilisateur (LDAP)
2      ↓
3      [Se connecte au portail interne]
4      ↓
5  Portail / Broker
6      ↓
7      [Appel API STS : sts:GetCallerIdentity ou sts:AssumeRole]
8      ↓
9  AWS STS
10     ↓
11     [Retourne : AccessKeyId, SecretAccessKey, SessionToken, Expiration]
12     ↓
```

```
13 Portail (reçoit les credentials)
14     ↓
15     [Génère un lien de connexion AWS Manager Console]
16     ↓
17 Utilisateur accède à AWS Console (valide 1 heure, puis expiration)
```

Types de requêtes STS courantes

Opération STS	Cas d'usage	Durée de validité
GetCallerIdentity	Vérifier qui vous êtes	N/A (détection)
AssumeRole	Un utilisateur IAM/fédéré assume un rôle	1 heure (configurable)
GetSessionToken	Obtenir des credentials temporaires pour l'utilisateur courant	1 heure à 36 heures
AssumeRoleWithSAML	Assumer un rôle via assertion SAML (fédération)	1 heure
AssumeRoleWithWebIdentity	Assumer un rôle via token JWT (OAuth/OIDC)	Configurable

Avantages de STS Identity Broker

- Zéro compte IAM permanent pour les utilisateurs fédérés.
- Credentials automatiquement révoquées à l'expiration.
- Audit centralisé via CloudTrail (qui a demandé un token, quand, d'où).
- Intégration facile avec les systèmes legacy (LDAP, SAP, Salesforce...).
- MFA possible au niveau du broker.

Exemple de code (Python) — Simple Identity Broker

```
1 import boto3
2 import sys
```

```

3
4 # Client STS
5 sts_client = boto3.client('sts')
6
7 # 1. Un utilisateur LDAP s'authentifie (simulé ici)
8 user = "alice"
9 role_arn = "arn:aws:iam::123456789012:role/FederatedUserRole"
10
11 try:
12     # 2. Appel STS pour obtenir credentials temporaires
13     response = sts_client.assume_role(
14         RoleArn=role_arn,
15         RoleSessionName=f"{user}-session",
16         DurationSeconds=3600 # 1 heure
17     )
18
19     # 3. Extraire les credentials
20     credentials = response['Credentials']
21
22     print(f"AccessKeyId: {credentials['AccessKeyId']}")
23     print("SecretAccessKey: <masquée volontairement>")
24     print(f"SessionToken: {credentials['SessionToken']}")
25     print(f"Expiration: {credentials['Expiration']}")
26
27 except Exception as e:
28     print(f"Erreur STS : {e}")

```

:::success **Résultat attendu — exécution du script Python STS :**

```

1 AccessKeyId: ASIA4EXAMPLESTEMP
2 SecretAccessKey: <masquée volontairement>
3 SessionToken: AQoDYXdzEJr//////////wEaoAK...HgC=
4 Expiration: 2024-01-15 11:40:00+00:00

```

Les credentials temporaires STS se distinguent par leur `AccessKeyId` commençant par **ASIA** (vs **AKIA** pour les clés permanentes). Ils expirent automatiquement à l'heure indiquée — aucune révocation manuelle nécessaire. :::

Ce code illustre comment un **broker** peut obtenir et distribuer des credentials temporaires.

3. Fédération d'identité et SSO avec IAM Identity Center

3.1 Introduction à la fédération d'identité

Lorsque les entreprises grandissent, elles disposent souvent **de systèmes d'authentification déjà en place** : annuaire Active Directory, LDAP, IdP externe comme Okta ou Azure AD...

Dans ce contexte, **créer des comptes IAM manuellement pour chaque utilisateur n'est ni scalable ni sécurisé**.

C'est là qu'intervient la **fédération d'identité**, un mécanisme permettant de déléguer l'authentification à un fournisseur d'identité existant.

Sur AWS, cette fédération est gérée via **IAM Identity Center** (anciennement AWS SSO) ou via des **rôles fédérés SAML**.

Grâce à cette approche :

- les utilisateurs conservent **leurs identifiants d'entreprise**,
- aucune **gestion manuelle des mots de passe** dans AWS,
- les administrateurs appliquent des **règles centralisées de sécurité**.

 [Documentation IAM Identity Center](#)

3.2 IAM Identity Center vs IAM classique — Clarification pour débutant

Beaucoup de stagiaires confondent ces deux concepts. Voici la différence **essentielle** :

Critère	IAM classique	IAM Identity Center
Pour qui ?	Comptes AWS individuels (admins, DevOps locaux)	Organisations / Entreprises avec plusieurs comptes AWS

Authentification	Login/mot de passe IAM natif	Fédération (AD, Azure AD, Okta...) ou gestion d'utilisateurs centralisée
Gestion centralisée	Non — chaque compte gère ses propres utilisateurs	Oui — un seul endroit pour gérer tous les accès
Multi-comptes	Difficile à gérer manuellement	Natif — accès simple entre comptes
MFA	À configurer par utilisateur	Automatisé, appliqué par l'IdP
Cas d'usage	Petit équipe, POC, environnement de test	Production, conformité, grande entreprise
Exemple	Un développeur solo crée un compte IAM pour lui	Une PME avec 5 comptes AWS (Prod, Dev, Finance, etc.)

En pratique :

- Petite équipe ? → IAM classique suffit.
- Entreprise avec AD ? → IAM Identity Center connecté à l'AD.
- Application web avec utilisateurs finaux ? → Cognito (section 5).

3.3 Définitions clés

Terme	Définition
Fédération d'identité	Mécanisme qui permet à des utilisateurs authentifiés par un fournisseur d'identité externe d'accéder à AWS sans compte IAM natif.
IdP (Identity Provider)	Service qui authentifie les utilisateurs (AD FS, Azure AD, Okta, PingIdentity...).
SP (Service Provider)	Service AWS qui consomme l'identité validée par l'IdP.
SAML (Security Assertion Markup Language)	Protocole standard pour la fédération d'identité entre systèmes d'entreprise et cloud.

OAuth / OIDC (OpenID Connect)

Protocole moderne de délégation d'accès, souvent utilisé pour les applications web et mobiles.

IAM Identity Center

Service AWS permettant de gérer l'accès fédéré et le Single Sign-On (SSO).

3.4 Architecture de la fédération d'identité avec AWS

1. L'utilisateur s'authentifie via l'IdP de l'entreprise (par exemple, Active Directory).
2. L'IdP émet une **assertion SAML** ou un **token OIDC**.
3. AWS IAM Identity Center valide cette assertion.
4. Un **rôle IAM fédéré** est attribué à l'utilisateur.
5. L'utilisateur accède à la console ou à l'API AWS selon ses permissions.

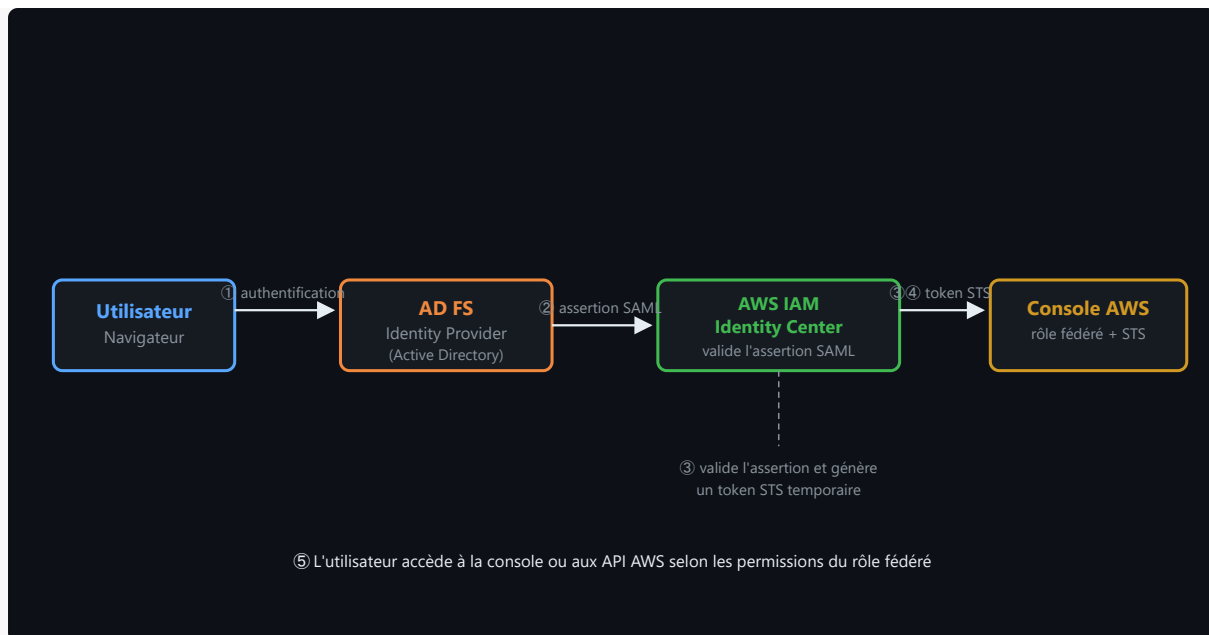
Le schéma détaillé de ce flux, avec l'implémentation concrète AD FS, est présenté juste après en §3.5.

3.5 Active Directory Federation Services (AD FS) et SAML — Implémentation pratique

AD FS est un service **Microsoft** qui joue le rôle d'**Identity Provider (IdP)** pour les entreprises utilisant **Windows Server Active Directory**.

Pour une entreprise ayant un **AD local** ou **Azure AD**, AD FS permet de créer un **pont de confiance** vers AWS via le protocole **SAML**.

Architecture générale : AD FS → AWS IAM



Lecture du schéma. L'utilisateur s'authentifie auprès d'AD FS, qui joue le rôle de fournisseur d'identité. AD FS émet une assertion SAML signée ; AWS la valide, associe l'utilisateur à un rôle autorisé et remet une session temporaire. Le mot de passe d'entreprise n'est pas transmis à AWS.

Étapes de mise en place (Vue d'ensemble pour débutant)

1. Configurer AD FS comme provider SAML

- En entreprise, l'administrateur AD crée une "application cloud" dans AD FS.
- Cette application est configurée pour accepter les demandes de connexion AWS.

2. Créer un fournisseur d'identité SAML dans AWS IAM

- Console AWS → IAM → Identity Providers → Create Provider.
- Type : SAML.
- Télécharger les **métadonnées SAML** d'AD FS (fichier XML).

3. Créer un rôle IAM fédéré

- Console AWS → IAM → Roles → Create Role.
- Type de confiance : **Federated Provider** (SAML).
- Sélectionner le provider créé à l'étape 2.
- Attacher des politiques (ex. : `AmazonS3ReadOnlyAccess` pour les développeurs).

4. Mapper les groupes AD aux rôles IAM

- Les groupes AD (ex. `AWS-Developers`, `AWS-Admins`) sont mappés à des rôles IAM.
- Quand un employé du groupe `AWS-Developers` se connecte, il reçoit automatiquement le rôle associé.

5. Distribuer le lien SSO aux utilisateurs

- Les utilisateurs reçoivent un **lien unique** (ex. : `https://monentreprise.com/adfs/ls/idpinitiatedsignon.aspx`).
- En cliquant, ils sont authentifiés contre l'AD et automatiquement connectés à AWS.

Exemple de Trust Policy SAML (pour IAM Role)

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Allow",
6        "Principal": {
7          "Federated": "arn:aws:iam::123456789012:saml-provider/ADFS"
8        },
9        "Action": "sts:AssumeRoleWithSAML",
10       "Condition": {
11         "StringEquals": {
12           "SAML:aud": "https://signin.aws.amazon.com/saml"
13         }
14       }
15     ]
16   }
17 }
```

Explication :

- **Principal** : indique qui peut assumer ce rôle (ici, le provider SAML **ADFS**).
- **Action** : l'action autorisée (**AssumeRoleWithSAML** pour les assertions SAML).
- **Condition** : vérification que la demande vient réellement d'AWS (protection).

Avantages de cette approche

- Les utilisateurs **n'ont aucun compte IAM** (zéro gestion manuelle).
- L'authentification reste **centralisée** en AD (changement de mot de passe une seule fois).
- **MFA en AD** s'applique automatiquement à AWS.
- Impossible de partager des credentials IAM (car ils n'existent pas).
- **Audit centralisé** : CloudTrail enregistre qui s'est connecté, quand, et depuis où.

Points de vigilance

- Les **métadonnées SAML** (certificats) ont une date d'expiration. Il faut les renouveler régulièrement.
- La **confiance de certificat** doit être validée côté AWS.
- Si AD FS tombe, les utilisateurs **ne peuvent plus accéder à AWS** (prévoir un backup ou un accès d'urgence).

⚠ **Single Point of Failure SAML**. Si l'IdP (AD FS, Azure AD, Okta) devient indisponible, tous les accès fédérés AWS sont coupés. Prévoyez toujours un compte IAM d'urgence ("break-glass account")

avec MFA, stocké en lieu sûr, pour récupérer l'accès en cas de panne de l'IdP. :::

 [Configuring AD FS as SAML Provider](#)

 [SAML Provider in IAM](#)

 [AssumeRoleWithSAML API](#)

3.6 Pourquoi utiliser la fédération d'identité ?

La fédération d'identité résout un problème très concret : dans une entreprise qui possède déjà un annuaire (Active Directory, LDAP, ou un fournisseur SSO cloud), créer un compte IAM distinct pour chaque employé signifie gérer deux systèmes d'identité en parallèle, avec le risque de désynchronisation que cela implique.

La gestion des identités et des mots de passe devient **centralisée** : c'est l'annuaire d'entreprise qui reste la source de vérité, et AWS ne fait que consommer les assertions d'authentification qu'il produit. Les politiques de sécurité déjà en place dans l'entreprise — complexité des mots de passe, durée de vie des sessions, restrictions d'accès — s'appliquent automatiquement à AWS sans duplication de configuration. Le nombre de comptes IAM permanents diminue fortement, puisque chaque connexion fédérée génère des identifiants temporaires plutôt qu'un compte durable : moins de comptes permanents, c'est mécaniquement moins de surface d'attaque en cas de fuite de identifiants. La traçabilité et la conformité s'en trouvent également améliorées, car chaque accès fédéré peut être rattaché à l'identité réelle de l'employé dans l'annuaire d'entreprise, plutôt qu'à un compte IAM générique partagé. Enfin, l'activation du MFA et des règles conditionnelles se fait souvent plus simplement au niveau de l'IdP (Identity Provider) qu'en la répétant pour chaque compte IAM individuel.

 [AWS Best Practices for SSO](#)

3.7 Exemple de scénario concret

Une entreprise dispose déjà d'un Active Directory local.

Elle souhaite que ses développeurs se connectent à la console AWS **avec leurs identifiants Windows**.

- Mise en place d'un **AD Connector** ou AWS Directory Service.
- Configuration d'AD FS comme IdP SAML.
- IAM Identity Center est configuré pour accepter les assertions SAML d'AD FS.
- Un rôle IAM fédéré "DeveloperRole" est mappé sur un groupe AD "AWS-Dev".

- Les développeurs accèdent à AWS en cliquant sur une URL SSO interne.

Résultat :

- Pas de création de comptes IAM individuels,
- Permissions gérées via AD,
- Accès tracé et sécurisé.

3.8 Intégration avec Azure AD et Okta

IAM Identity Center permet une intégration native avec :

- **Azure AD** via SAML ou SCIM
- **Okta** via SAML ou OIDC

Cela permet de synchroniser les groupes et utilisateurs automatiquement.

 [Azure AD Integration](#) 

 [Okta Integration](#) 

3.9 Points de vigilance et bonnes pratiques

- Préférer la fédération d'identité à la multiplication des comptes IAM.
- Activer MFA au niveau de l'IdP.
- Bien cartographier les groupes AD vers les rôles IAM.
- Tenir à jour les métadonnées SAML (certificats, endpoints).
- Surveiller les connexions via CloudTrail.

 [AWS IAM : comment gérer les permissions de ses équipes ?](#) 

4. Amazon Cognito : gestion d'identités applicatives

4.1 Différence : IAM vs Cognito

Souvent, les stagiaires confondent IAM et Cognito. C'est normal — ce sont tous les deux des services d'identité. Mais ils n'ont **pas le même public** :

- **IAM** = gestion des identités **administratives** (accès AWS pour l'équipe IT/DevOps).
- **Amazon Cognito** = gestion des identités **applicatives** (accès à une application web ou mobile pour les utilisateurs finaux).

Amazon Cognito est un service AWS qui permet de gérer l'authentification et l'autorisation des utilisateurs dans les applications web et mobiles. Il permet de créer des **pools d'utilisateurs** (User Pools) et de connecter des **fournisseurs d'identité externes** (Google, Facebook, SAML, etc.).

4.2 Cas d'usage typique

Une application web souhaite permettre à ses utilisateurs de se connecter avec leur compte Google ou via un login/mot de passe.

L'application utilise Cognito pour gérer les sessions, les jetons d'accès, et les droits d'accès aux ressources AWS.

4.3 Concepts clés

- **User Pool** : base d'utilisateurs gérée par Cognito (inscription, mot de passe, MFA, etc.).
- **Identity Pool** : permet d'obtenir des **credentials AWS temporaires** pour accéder à des services comme S3 ou DynamoDB.
- **Fédération d'identité** : possibilité de déléguer l'authentification à un fournisseur externe (SAML, OAuth2).

Cognito est souvent utilisé dans les architectures serverless ou mobiles. Il permet de sécuriser l'accès aux ressources AWS sans exposer de credentials statiques.

 [Amazon Cognito — Guide officiel](#)

5. Stratégie multi-comptes avec AWS Organizations

5.1 Problématique

Lorsqu'une entreprise évolue et déploie de plus en plus de workloads dans AWS, **gérer toutes les ressources dans un seul compte devient vite risqué et ingérable** :

- Risques de sécurité accrus (trop de permissions dans un même périmètre).
- Facturation difficile à segmenter.
- Difficile d'appliquer des politiques globales cohérentes.
- Problèmes de conformité et de cloisonnement des environnements.

5.2 Introduction à AWS Organizations

Pour répondre à ces enjeux, AWS propose **AWS Organizations**, un service qui permet de **centraliser la gouvernance de plusieurs comptes AWS** tout en conservant une séparation stricte des environnements.

La structure hiérarchique complète — Management Account, OU et SCP associées — est illustrée en §5.8 avec un exemple concret à 4 comptes.

 [Documentation officielle AWS Organizations](#)

5.3 Définition

AWS Organizations est un service qui permet de créer et de gérer plusieurs comptes AWS depuis une seule organisation centrale, d'appliquer des politiques globales et d'unifier la facturation.

Il facilite :

- la **structuration logique** des environnements (prod, dev, test...),
- l'application de **politiques de sécurité globales**,
- la **centralisation de la facturation**,
- la **gestion simplifiée des identités et des accès** à grande échelle.

5.4 Concepts clés d'AWS Organizations

Élément	Définition
Organisation	Ensemble de comptes AWS sous une gouvernance centralisée.
Management Account	Compte maître utilisé pour créer et gérer l'organisation.
Member Accounts	Comptes membres, rattachés et gérés via l'organisation.
OU (Organizational Unit)	Groupe logique de comptes AWS (par fonction ou par environnement).
SCP (Service Control Policy)	Politique globale qui définit les actions maximales autorisées dans les comptes membres.

 [AWS Organizations Concepts](#)

5.5 Exemple d'architecture multi-comptes

Une entreprise met en place une **stratégie à 4 comptes** :

- **Management** → Compte central de gouvernance et facturation.
- **Production** → Applications critiques.
- **Développement** → Tests et intégrations.
- **Sandbox** → Environnement libre pour expérimenter.

Ces comptes sont regroupés dans des OU distinctes et **sécurisés par des SCP** adaptées :

- SCP sur **Sandbox** → Interdire la suppression de budgets et d'alertes.
- SCP sur **Production** → Interdire toute modification réseau sans validation.
- SCP globale → Interdire l'utilisation de certaines régions AWS.



Lecture du schéma. Le compte de gestion pilote l'organisation. Les unités organisationnelles regroupent les comptes par environnement et reçoivent des SCP. Une SCP fixe la limite maximale des autorisations possibles ; elle n'accorde jamais à elle seule une permission IAM.

Ce schéma illustre la hiérarchie complète : le **Management Account** en racine centralise la gouvernance et la facturation consolidée de l'ensemble des comptes ; chaque OU porte sa propre SCP adaptée à son niveau de risque (Sandbox très restreinte sur la suppression de ressources de suivi budgétaire, Production verrouillée sur toute modification réseau) ; et une SCP globale attachée à la racine s'applique uniformément à tous les comptes, quelle que soit leur OU — c'est le niveau approprié pour une contrainte transverse comme l'interdiction de régions AWS non autorisées.

5.6 Service Control Policies (SCP) — Clarification

Les **SCP** sont des politiques qui **définissent la limite supérieure** des permissions dans un compte membre.

Elles **ne donnent pas de permissions** directement mais **restreignent** ce que les politiques IAM peuvent autoriser.

SCP vs IAM Policy — Différence essentielle

Beaucoup de stagiaires confondent **SCP** et **IAM Policy**. Voici la différence cruciale :

Aspect	IAM Policy	SCP
Fonction	DONNE les permissions	LIMITE les permissions (plafond)

Niveau	S'applique à un utilisateur, groupe ou rôle IAM	S'applique à tout un compte ou OU
Évaluation	"Puis-je faire cela ?"	"Le compte autorise-t-il cela ?"
Exemples	<code>Allow s3:GetObject</code> , <code>Deny ec2:RunInstances</code>	<code>Deny iam:CreateUser</code> , <code>Deny *:*</code> (sauf S3)
Cas bloqué	Un utilisateur sans policy = accès refusé	Un utilisateur avec Allow, mais SCP refuse = accès refusé

Analogie : Restaurant avec zones interdites

- IAM Policy = "Alice peut commander des plats du menu".
- SCP = "Personne dans le restaurant ne peut avoir d'alcool" (limite absolue).

Même si Alice a l'autorisation IAM, la SCP l'empêche de commander de l'alcool.

Ordre d'évaluation des permissions

```

1 1. SCP évalue : "Est-ce que le compte autorise cela ?"
2   - Si Deny → Accès refusé (STOP)
3   - Si Allow ou neutre → Continue
4
5 2. IAM Policy évalue : "Est-ce que l'utilisateur a la permission ?"
6   - Si Deny explicite → Accès refusé (STOP)
7   - Si Allow explicite → Accès autorisé
8   - Si rien → Accès refusé (par défaut)
9
10 Résultat final = SCP AND IAM Policy

```

Exemple concret :

- Un utilisateur a une IAM Policy : `Allow ec2:*` (tous les droits EC2).
- Mais l'OU a une SCP : `Deny ec2:RunInstances` (interdire lancer des instances).
- **Résultat** : L'utilisateur peut faire presque tout avec EC2 **sauf lancer des instances**.

5.7 Exemples SCP : Cas courants en entreprise

Exemple 1 : Interdire la création de ressources en dehors de eu-west-3

Contexte : L'entreprise doit rester conforme au RGPD (données en Europe).

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": "*",
7        "Resource": "*",
8        "Condition": {
9          "StringNotEquals": {
10             "aws:RequestedRegion": "eu-west-3"
11           }
12        }
13      }
14    ]
15  }
```

Effet : Les développeurs peuvent utiliser AWS, mais **toutes les ressources doivent être en région Paris** (eu-west-3). Les régions US/Asie sont bloquées.

Exemple 2 : Interdire la suppression de certaines ressources critiques

Contexte : Les stagiaires ne doivent pas pouvoir supprimer les RDS ou les VPC de production.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": [
7          "rds:DeleteDBInstance",
8          "rds:DeleteDBCluster",
9          "ec2:DeleteVpc",
10         "ec2:DeleteSubnet"
11        ],
12        "Resource": "*"
13      }
14    ]
```

```
15 }
```

Effet : Même un administrateur IAM du compte ne peut pas supprimer ces ressources (la SCP bloque).

Exemple 3 : Interdire les services coûteux (ex. Production only)

Contexte : Sur le compte Sandbox, on veut éviter les ressources chères (SageMaker, Redshift).

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Action": [
7          "sagemaker:*",
8          "redshift:*",
9          "elasticmapreduce:*",
10         "mediaconvert:*"
11       ],
12       "Resource": "*"
13     }
14   ]
15 }
```

Effet : Sur le compte Sandbox, ces services ne peuvent pas être utilisés. Idéal pour limiter les coûts d'expérimentation.

Exemple 4 : Interdire les actions sans MFA (Politique conditionnelle)

Contexte : Les administrateurs ne peuvent utiliser la console AWS que s'ils ont MFA activée.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "NotAction": [
7          "iam:CreateVirtualMFADevice",
8          "iam:EnableMFADevice",
9          "sts:GetSessionToken"
10       ],

```

```

11     "Resource": "*",
12     "Condition": {
13         "BoolIfExists": {
14             "aws:MultiFactorAuthPresent": "false"
15         }
16     }
17 }
18 ]
19 }

```

Effet : Tout est bloqué sauf création et activation du MFA, à moins que MFA ne soit déjà présente. Oblige à configurer MFA en premier.

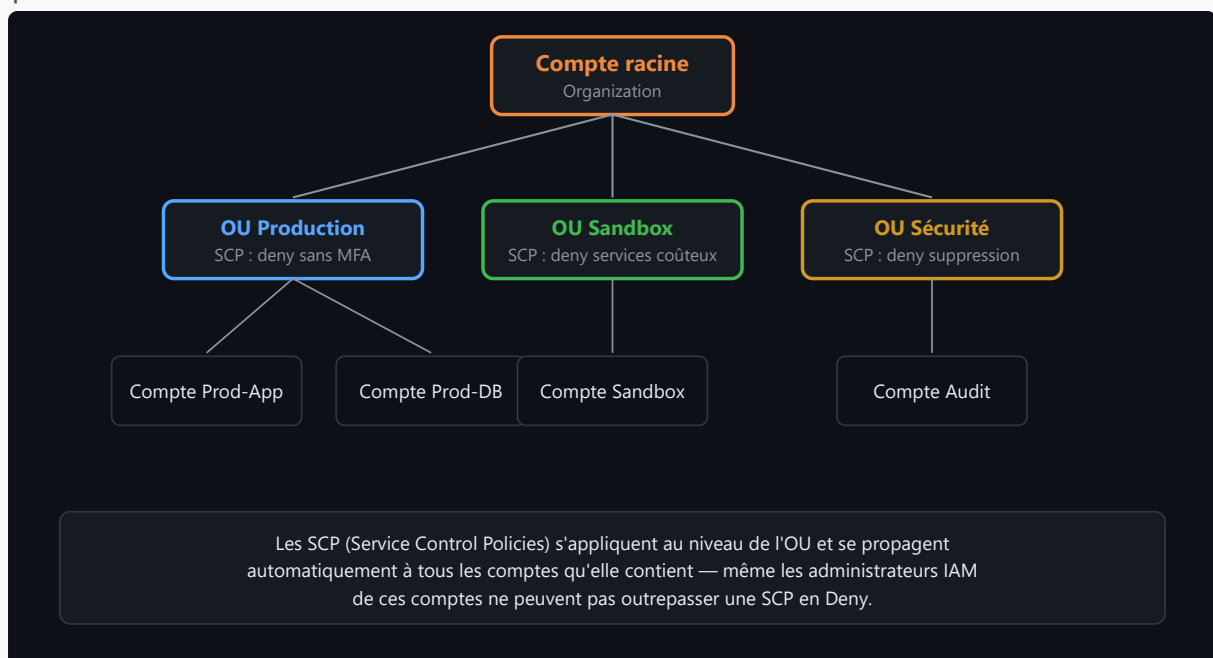
5.8 Où attacher les SCP ? — OU vs Comptes

Les SCP peuvent s'appliquer à différents niveaux :

1. **À une OU :** Affecte tous les comptes de l'OU et leurs enfants.
2. **À un compte individuel :** Affecte uniquement ce compte.
3. **À la racine :** Affecte l'organisation entière.

Bonne pratique : Préférer les OU plutôt que les comptes individuels, pour une gestion centralisée.

Exemple de structure :



Lecture du schéma. L'arbre représente l'héritage de la gouvernance : une règle placée sur la racine ou une unité organisationnelle s'applique aux comptes descendants. Les comptes restent toutefois des

frontières d'isolation distinctes avec leurs propres ressources et rôles.

5.9 Test et validation des SCP

IMPORTANT : Toujours tester les SCP dans un **compte non critique** avant de les appliquer globalement.

Utiliser le **IAM Policy Simulator** :

1. Console AWS → IAM → Policy Simulator.
2. Simuler une action (ex : `ec2:RunInstances`) avec un utilisateur.
3. Vérifier si elle est bloquée par SCP ou policy.

Cela évite les surprises en production.

 [Service Control Policies](#)

 [SCP Examples](#)

5.10 Avantages d'une stratégie multi-comptes

- Séparation claire des environnements (Prod, Dev, Test).
 - Meilleure sécurité grâce au cloisonnement.
 - Gestion fine des coûts par compte.
 - Application de politiques globales cohérentes.
 - Simplification de l'audit et de la conformité.
-

5.11 Budgets et alertes multi-comptes

AWS Budgets peut être configuré au niveau de l'organisation pour :

- Suivre les dépenses par compte ou OU
- Déclencher des alertes par email ou SNS

 [AWS Budgets](#)

5.12 Bonnes pratiques recommandées par AWS

- Créer un **compte de management** dédié, jamais utilisé pour déployer des ressources.
- Utiliser **des OU logiques** (par environnement ou par métier).
- Appliquer les SCP **par OU** plutôt que par compte individuel.
- Restreindre les régions et services non utilisés pour limiter les risques.
- Intégrer AWS Organizations avec **IAM Identity Center** pour la gestion des accès à grande échelle.

 [AWS Multi-Account Strategy](#)

5.13 Points de vigilance

- Les SCP n'annulent pas les politiques IAM, elles les **limitent**.
- L'ordre d'évaluation est : SCP → IAM → Permissions effectives.
- Toujours tester les SCP dans un **compte non critique** avant de les déployer globalement.
- Ne pas donner trop de privilèges au compte Management.

⚠️ **Le Management Account ne doit jamais héberger de workloads applicatifs.** Ce compte dispose de droits sur tous les comptes membres via les SCP. Une compromission du Management Account compromet l'ensemble de l'organisation. Restreignez l'accès au strict minimum, activez MFA, et surveillez-le via CloudTrail au niveau organisationnel. ⚠️

6. Traçabilité et surveillance avec CloudTrail

6.1 Pourquoi surveiller les activités dans AWS ?

En environnement cloud, les utilisateurs peuvent créer, modifier ou supprimer des ressources **en quelques secondes**, sans passer par un processus de validation centralisé comme dans un datacenter traditionnel. Cette vitesse est un atout pour l'agilité, mais elle transforme aussi n'importe quelle erreur de configuration ou action malveillante en un risque qui se matérialise presque instantanément — d'où la nécessité de tout tracer.

L'enjeu de **sécurité** est le plus évident : sur un compte AWS, il faut pouvoir répondre à tout moment à la question *qui a fait quoi, quand et depuis où*, faute de quoi une compromission de compte peut passer inaperçue pendant des semaines. Vient ensuite l'enjeu de **conformité** : de nombreux référentiels

réglementaires (ISO 27001, RGPD, PCI-DSS) exigent explicitement une traçabilité des accès et des modifications, et l'absence de journal d'audit peut à elle seule faire échouer une certification. L'**audit interne** dépend lui aussi directement de cette traçabilité : en cas d'incident (suppression accidentelle d'une base de données, fuite de données), c'est l'historique des actions qui permet de reconstituer la chronologie et d'identifier l'origine du problème en quelques minutes plutôt qu'en plusieurs jours. Enfin, cette même traçabilité sert à l'**optimisation** de la gouvernance cloud : en observant qui utilise réellement quels services, une équipe peut ajuster ses permissions IAM au principe du moindre privilège plutôt que de les laisser trop larges par précaution.

Pour cela, AWS fournit **CloudTrail**, un service de **traçabilité des actions** dans un compte ou une organisation.

6.2 AWS CloudTrail — Définition

CloudTrail est le service AWS qui enregistre toutes les actions réalisées via la console, les API et la CLI.

Chaque événement contient :

- Date et heure de l'action
- Identité de l'utilisateur (ou rôle IAM)
- Adresse IP d'origine
- Service AWS concerné
- Action exécutée (ex. `ConsoleLogin`, `RunInstances`, `DeleteBucket`)

CloudTrail permet ainsi :

- de **visualiser l'historique des actions** dans la console,
- de **stocker les logs** dans S3,
- et de les **exploiter** dans CloudWatch ou EventBridge pour créer des alertes.

6.3 CloudTrail et AWS Organizations

Lorsque vous utilisez **AWS Organizations**, vous pouvez :

- Activer un **seul trail** au niveau du compte de management
- Appliquer ce trail à **tous les comptes enfants** (OU ou organisation complète)
- Centraliser les logs dans un **seul bucket S3**.

Cela permet de suivre les activités de tous les comptes de votre organisation depuis un **point unique**.

6.4 Exemples d'événements courants enregistrés

Événement CloudTrail	Description	Cas d'usage
ConsoleLogin	Connexion à la console AWS	Détecter les connexions non MFA
RunInstances	Création d'une instance EC2	Suivi de consommation / sécurité
CreateBucket	Création d'un bucket S3	Audit stockage
PutUserPolicy	Modification d'une policy IAM	Traçabilité sécurité IAM
DeleteTrail	Suppression du trail CloudTrail	Détection activité critique

6.5 CloudTrail vs CloudWatch

CloudTrail	CloudWatch
Enregistre les événements historiques	Surveille les métriques et ressources
Trace les actions utilisateurs/API	Crée des alertes sur seuils ou logs
Stocke les logs dans S3	Affiche en temps réel
Focus "qui a fait quoi"	Focus "ce qui se passe"

Les deux sont complémentaires :

- CloudTrail = audit et traçabilité
- CloudWatch = surveillance opérationnelle

6.6 Exploiter les journaux CloudTrail : de la traçabilité à l'analyse

CloudTrail ne se limite pas à enregistrer les actions : il permet aussi de les **analyser**, de les **interroger**, et de **restreindre leur accès** selon les rôles. Pour cela, AWS propose une chaîne d'outils complémentaires, chacun jouant un rôle précis dans le traitement des logs.

Étapes d'intégration : de CloudTrail à Lake Formation

1. Stockage dans S3

Les événements CloudTrail sont automatiquement archivés dans un bucket S3, sous forme de fichiers JSON. Ce stockage est durable, centralisé, et interrogeable.

2. Catalogage avec AWS Glue

Glue agit comme un annuaire technique : il décrit la structure des fichiers (colonnes, types, formats) et les rend accessibles aux outils d'analyse comme Athena. Sans Glue, Athena ne saurait pas comment lire les logs.

3. Analyse avec Amazon Athena

Athena permet d'exécuter des requêtes SQL directement sur les fichiers CloudTrail dans S3. C'est un moteur d'analyse serverless, sans base de données à déployer. On peut par exemple extraire tous les événements `DeleteBucket` du mois ou filtrer par utilisateur IAM.

4. Gouvernance avec AWS Lake Formation

Lake Formation ajoute une couche de sécurité fine : il permet de contrôler **qui peut accéder à quelles données**, jusqu'au niveau colonne. Cela garantit que seuls les rôles autorisés peuvent interroger les logs sensibles.

 [Analyser CloudTrail avec Athena](#)

6.7 AWS Config vs CloudTrail

Si CloudTrail trace les **actions**, AWS Config trace les **états**. Ensemble, ils offrent une vision complète du comportement et de la conformité des ressources AWS.

Fonction	AWS Config	AWS CloudTrail
Ce qui est suivi	État des ressources	Actions API des utilisateurs

Exemple	"Ce bucket est-il toujours privé ?"	"Qui a modifié ce bucket ?"
Type de données	Historique de configuration	Journal d'activité
Objectif	Conformité, détection de dérive	Audit, traçabilité
Intégration	Conformance Packs, remédiation automatique	EventBridge, Athena, SIEM

 [AWS Config Documentation](#)

6.8 Cas d'usage

Un administrateur souhaite identifier tous les appels `PutUserPolicy` effectués par des utilisateurs non autorisés. Il utilise CloudTrail pour collecter les événements, Glue pour cataloguer les logs, Athena pour interroger les données, et Lake Formation pour restreindre l'accès aux résultats.

6.9 Bonnes pratiques CloudTrail

- Activer CloudTrail au niveau de l'organisation.
- Centraliser les logs dans un bucket S3 sécurisé.
- Activer la chiffrement SSE-S3 ou SSE-KMS.
- Mettre en place des alertes EventBridge sur les événements sensibles :
 - Connexion root
 - Connexion sans MFA
 - Suppression de logs
 - Création de ressources critiques
- Définir une politique de rétention et un plan d'audit régulier.

 [AWS CloudTrail Console](#)

 [Documentation CloudTrail](#)

 [ConsoleLogin event](#)

 [CloudTrail + Organizations](#)

⚠ **Ne jamais désactiver CloudTrail en production.** La suppression ou la désactivation d'un trail est elle-même un événement critique enregistré. Configurez des alertes EventBridge sur l'événement `DeleteTrail` et `StopLogging`. En conformité ISO 27001 ou PCI-DSS, les logs CloudTrail doivent être conservés au minimum 1 an et être immuables (activer S3 Object Lock). ⚠

6.10 Comprendre les coûts de CloudTrail et d'AWS Config

La facture dépend des fonctions activées et du volume observé. Il faut donc raisonner sur les **unités de consommation**, puis appliquer les tarifs affichés pour la région et la date de l'estimation.

Service	Principaux facteurs de coût	Point de contrôle
CloudTrail	Types d'événements collectés, volume d'événements, nombre de copies de trails, stockage et analyse des journaux	Sélectionner uniquement les événements nécessaires à l'objectif d'audit ; distinguer événements de gestion et événements de données
AWS Config	Nombre de ressources enregistrées, fréquence des changements, évaluations de règles et éventuels packs de conformité	Limiter l'enregistrement et les règles au périmètre réellement surveillé

Méthode d'estimation : relever les volumes dans le compte, choisir la région dans la page tarifaire officielle, puis calculer séparément la collecte, l'évaluation, l'analyse et le stockage. Une estimation sans région, sans période et sans volume n'est pas exploitable.

 [AWS CloudTrail Pricing](#)  [AWS Config Pricing](#)

7. Points importants et pièges fréquents

Piège courant	Réalité	Solution
"Les SCP donnent des permissions"	Les SCP limitent les permissions, elles ne les	Toujours combiner SCP + politiques IAM.

donnent pas.

"Un Deny peut être contourné par un Allow"	Un Deny explicite est prioritaire. Toujours.	Reconnaître que Deny > Allow dans l'évaluation.
"IAM et Cognito, c'est pareil"	IAM = accès AWS administratif. Cognito = accès application.	Utiliser IAM pour IT, Cognito pour utilisateurs finaux.
"MFA, c'est juste un code SMS"	MFA peut être TOTP, YubiKey, passkey, biométrie.	Proposer plusieurs types selon la sensibilité.
"Pas besoin de fédération si on a IAM"	La fédération centralise la gestion et réduit les comptes statiques.	Préférer la fédération en environnement d'entreprise.
"CloudTrail ralentit AWS"	CloudTrail est activé implicitement et n'impacte pas les perfs.	L'activer sans crainte pour l'audit.
"Un utilisateur sans policy n'a aucun accès"	Correct : le moindre privilège s'applique par défaut.	Toujours attacher une policy minimale.
"On peut récupérer une clé d'accès perdue"	Non. Les clés ne s'affichent qu'à la création.	Conserver les clés en lieu sûr, utiliser AWS Secrets Manager.

8. Choisir la bonne solution d'authentification AWS

AWS propose de nombreux services d'authentification. Voici une carte complète pour savoir lequel choisir.

Trois profils d'authentification distincts se dégagent : les développeurs/services AWS/applications internes s'appuient sur IAM Users, IAM Roles et STS ; les employés de l'entreprise (B2E) passent par IAM Identity Center (SSO), SAML 2.0 ou Directory Service ; les utilisateurs du grand public (B2C) utilisent Cognito User Pools.

Tableau comparatif complet

Solution	Pour qui	Credentials	Durée	MFA	Modèle de facturation
IAM User + Access Key	Cas hérités nécessitant une identité durable	Permanents	Jusqu'à révocation	Oui	IAM n'est pas facturé séparément les services appelés le s
IAM Role + STS	Services AWS, scripts, accès inter-comptes	Temporaires	Configurable dans les limites du rôle	Selon le parcours d'authentification	STS n'est pas facturé séparément les services appelés le s
Cognito User Pool	Utilisateurs d'une application web/mobile	Jeton JWT	Configurable	Oui	Utilisateurs actifs et fonctions choisies
Cognito Identity Pool	Application web/mobile devant obtenir des autorisations AWS temporaires	Identifiants STS	Temporaires	Via le fournisseur d'identité	Dépend des services associés et de leur consommation
IAM Identity Center	Collaborateurs accédant à plusieurs comptes et applications	Session	Configurable	Oui	Vérifier les fonctions et services associés

SAML 2.0	Fédération avec un fournisseur d'identité d'entreprise	Assertion SAML puis session AWS	Configurable	Géré par le fournisseur d'identité	Dépend du fournisseur d'identité et des services associés
Directory Service	Annuaire managé ou connexion à un annuaire existant	Identité d'annuaire	Selon la solution	Selon la solution	Type et taille d'annuaire, contrôleurs région

MAU signifie *Monthly Active User*, ou utilisateur actif mensuel. Cette unité est notamment utilisée pour certaines fonctions de Cognito. Les seuils et tarifs évoluent : l'estimation doit partir du nombre d'utilisateurs actifs, des méthodes d'authentification et des fonctions de sécurité réellement activées.

Arbre de décision

[illegible]

Focus : Cognito User Pool vs Identity Pool

La confusion la plus fréquente en formation :

1 Cognito USER POOL	Cognito IDENTITY POOL
2	
3 "Qui es-tu ?"	"Qu'as-tu le droit de faire dans AWS ?"
4	
5 Gère l'annuaire d'utilisateurs	Échange un token externe contre des
6 (login, mot de passe, attributs,	credentials AWS temporaires (STS)
7 MFA, réinitialisation de mdp)	
8	
9 Émet des JWT :	Accepte en entrée :
10 • Access Token	• Token Cognito User Pool
11 • ID Token	• Token Google / Facebook / Apple
12 • Refresh Token	• Token SAML
13	• Accès anonyme
14	
15 Utilisé par :	Utilisé pour :
16 • Frontend (login page)	• Appeler S3, DynamoDB, etc.
17 • API Gateway (autoriser)	depuis une app mobile
18 • Tout service vérifiant un JWT	• Accès AWS sans backend

Focus : IAM Identity Center vs SAML 2.0

	IAM Identity Center	SAML 2.0 direct
Configuration	Quelques clics dans la console	Configuration manuelle complexe (métadonnées XML)
Multi-comptes	✓ Natif (une entrée = tous les comptes)	✗ Un trust par compte
IdP supportés	Azure AD, Okta, Ping, SCIM	Tout IdP SAML 2.0
Portail web	✓ Inclus (aws.amazon.com/sso)	✗ À construire

Recommandé
pour

Nouvelles organisations AWS

Besoins très spécifiques

 Choisir la bonne solution d'authentification AWS [↗](#)

Ressources

Documentation officielle AWS

- [AWS IAM Documentation](#) [↗](#)
- [IAM Best Practices](#) [↗](#)
- [AWS IAM Identity Center](#) [↗](#)
- [Amazon Cognito Documentation](#) [↗](#)
- [AWS CloudTrail Documentation](#) [↗](#)

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les questions et les propositions restent dans un ordre stable.

| Le quiz interactif est disponible dans la version web du support.

Chapitre 3 — Stockage Amazon S3 et calcul Amazon EC2

Vocabulaire

1 · Choisir un stockage AWS

2 · Stockage objet avec S3

3 · Protéger et optimiser S3

4 · Calcul avec EC2

5 · Instance, AMI, stockage et sécurité

6 · Dimensionnement

7 · Modèles de tarification EC2

8 · Elastic Load Balancing

9 · Auto Scaling

10 · Calcul sans serveur avec Lambda

11 · Architecture d'ensemble

12 · Haute disponibilité

13 · Données sensibles

14 · Points d'attention

Ressources

Quiz du chapitre

Vocabulaire du chapitre

Terme	Définition
Stockage objet	Stockage dans lequel chaque donnée est enregistrée comme un objet identifié par une clé et accompagné de métadonnées.
Bucket S3	Conteneur logique Amazon S3 qui reçoit des objets et porte une partie de leur configuration.
AMI	Amazon Machine Image : modèle utilisé pour lancer une instance EC2 avec un système et une configuration initiale.
Instance EC2	Serveur virtuel fourni par Amazon Elastic Compute Cloud.
EBS	Elastic Block Store : stockage bloc persistant attaché à une instance EC2.
EFS	Elastic File System : système de fichiers réseau managé pouvant être monté par plusieurs clients.
Load balancer	Répartiteur qui distribue les requêtes entre plusieurs cibles disponibles.

Après avoir sécurisé les accès avec IAM au chapitre précédent, les stagiaires disposent des bases nécessaires pour créer et protéger de vraies ressources AWS : ce chapitre aborde les deux briques les plus utilisées du Cloud AWS, le stockage objet S3 et le calcul EC2.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Créer** et configurer un bucket Amazon S3 (chiffrement, versioning, politiques d'accès)
- **Mettre en œuvre** des règles de lifecycle S3 pour optimiser le coût du stockage
- **Manipuler** S3 via la CLI (upload, download, synchronisation, gestion des permissions)
- **Choisir** un type d'instance EC2, une AMI et un mode de stockage adaptés à un besoin donné
- **Configurer** des Security Groups pour contrôler le trafic réseau d'une instance EC2
- **Utiliser** AWS Compute Optimizer pour dimensionner correctement une instance
- **Comparer** les modèles de tarification EC2 (On-Demand, Reserved, Spot, Savings Plans)
- **Lancer et administrer** une instance EC2 via la CLI
- **Déployer** un Elastic Load Balancer pour répartir le trafic entre plusieurs instances
- **Configurer** un groupe Auto Scaling pour adapter dynamiquement la capacité aux besoins
- **Concevoir** une architecture haute disponibilité combinant S3, EC2, ELB et Auto Scaling :::

1. Introduction aux services de stockage AWS

 Vidéo : Introduction to Amazon S3 [↗](#)

1.1 Pourquoi plusieurs services de stockage ?

Le stockage est au cœur de toute infrastructure cloud. AWS propose plusieurs types de stockage, mais **Amazon Simple Storage Service (S3)** est le service le plus emblématique : fiable, scalable et économique.

Créé en 2006, S3 a révolutionné la façon dont les entreprises stockent leurs données en passant d'un modèle de serveur à un modèle **d'espace de stockage à la demande**.

Avant de plonger dans les services techniques, il est essentiel de comprendre **pourquoi AWS propose plusieurs modèles de stockage** et dans quels contextes les utiliser.

Dans une entreprise traditionnelle, le stockage repose sur :

- des **disques durs internes** (pour les postes ou serveurs locaux),
- des **baies NAS/SAN** (pour le stockage partagé),
- parfois des sauvegardes sur bande ou sur site distant.

AWS transpose ces modèles dans le Cloud et les **rend flexibles, évolutifs et disponibles à la demande**.

1.2 Les trois modèles de stockage AWS

Type de stockage	Service AWS	Cas d'usage typique	Analogie utilisateur
Objet	Amazon S3	Sauvegarde, site statique, logs, Data Lake	Dropbox / Google Drive
Bloc	Amazon EBS	Disque de VM, base de données, stockage persistant	Disque dur local
Fichier	Amazon EFS/FSx	Partage réseau, systèmes distribués	NAS / Partage Windows

À retenir : Chaque type de stockage a ses propres performances, coûts et scénarios d'usage. Nous allons détailler S3 et EBS en priorité, car ce sont les services les plus utilisés par les administrateurs AWS en début de carrière.

 [Documentation Amazon S3](#)

2. Amazon S3 : Le stockage objet scalable

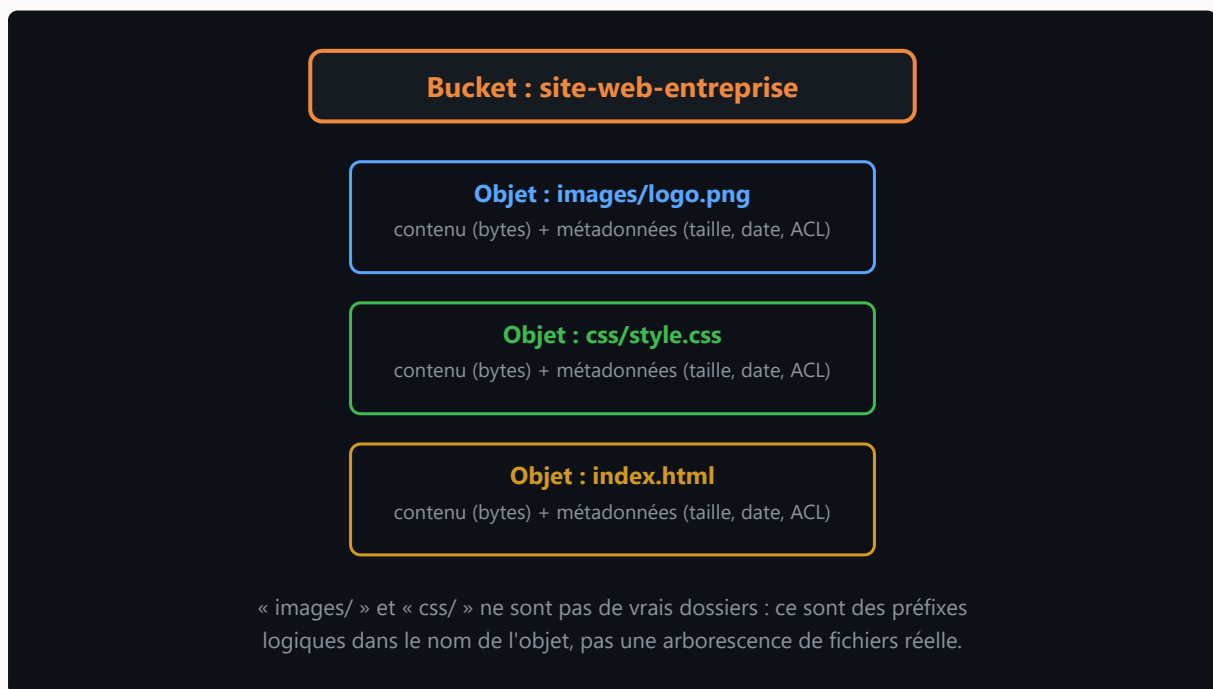
2.1 Qu'est-ce qu'Amazon S3 ?

Amazon S3 (Simple Storage Service) est un service de stockage objet. Il stocke des données sous forme d'objets dans des buckets et fournit différentes classes de stockage. La disponibilité et la durabilité annoncées dépendent de la classe et ne remplacent ni le versioning, ni une stratégie de sauvegarde, ni les contrôles d'accès.

Mais attention : **S3 ne fonctionne pas comme un disque dur classique**. C'est un système de **stockage objet**, ce qui signifie que chaque fichier est stocké avec des informations supplémentaires (appelées **métadonnées**) dans un conteneur appelé **bucket**.

2.2 L'armoire de rangement : analogie avec S3

Imaginez une **armoire de rangement** :



Lecture du schéma. Le bucket constitue l'espace de nommage principal. Chaque objet possède une clé complète, des données et des métadonnées. Les barres obliques visibles dans une clé créent une organisation logique par préfixes, mais pas de véritables répertoires sur un disque.

- L'armoire, c'est le **bucket** : un conteneur dans lequel vous rangez vos fichiers.
- Chaque fichier est un **objet** : il contient le contenu (ex. une image) + des étiquettes (métadonnées) comme son nom, sa date, ses droits d'accès.
- Les "dossiers" que vous voyez dans S3 ne sont pas réels : ce sont juste des **préfixes logiques** dans le nom du fichier (ex. `images/logo.png`).

2.3 Exemple concret

Vous créez un bucket nommé `site-web-entreprise`. Vous y déposez :

- `images/logo.png`
- `css/style.css`
- `js/app.js`

Ces fichiers peuvent ensuite être **consultés via Internet**, sans serveur web, si vous configurez le bucket en mode **site statique**.

 [S3 Static Website Hosting](#)

2.4 Structure interne de S3

Élément	Description
Bucket	Conteneur global (nom unique dans AWS), lié à une région
Objet	Fichier + métadonnées (nom, taille, type, permissions)
Préfixe	Partie du nom qui simule un dossier (ex. <code>images/</code>)
Key	Identifiant unique de l'objet au sein du bucket
Métadonnées	Informations sur l'objet (format, date, ACL, tags)

2.5 Caractéristiques clés de S3

Caractéristique	Description
Durabilité	99,999999999% (11 neuf) grâce à la réplication automatique sur plusieurs zones.
Disponibilité	Haute disponibilité (jusqu'à 99,99% selon la classe).
Évolutivité	Pas de limite pratique en nombre d'objets.
Sécurité	Contrôle fin via IAM, ACL, politiques de bucket et chiffrement.
Coût à l'usage	Payez uniquement pour le stockage et les requêtes.

 [S3 Storage Classes](#)

3. Protéger et optimiser les données S3

Amazon S3 propose plusieurs mécanismes pour **sécuriser vos fichiers, préserver leur historique**, et **réduire les coûts de stockage**. Ces options sont souvent méconnues, mais elles sont essentielles pour bien gérer vos données dans le cloud.

3.1 Chiffrement : protéger les fichiers contre les accès non autorisés

Quand vous stockez un fichier dans S3, vous pouvez demander à AWS de le **chiffrer automatiquement**. Cela signifie que même si quelqu'un accède physiquement au disque, il ne pourra pas lire le contenu sans la clé.

Types de chiffrement et gestion des clés

Acronyme	Signification complète	Description pédagogique
SSE-S3	<i>Server-Side Encryption with Amazon S3-managed keys</i>	Le chiffrement est géré automatiquement par AWS S3 . Vous n'avez rien à configurer.
SSE-KMS	<i>Server-Side Encryption with AWS Key Management Service</i>	Le chiffrement utilise AWS KMS , un service de gestion de clés. Vous définissez et contrôlez les clés.
HTTPS/TLS	<i>HyperText Transfer Protocol Secure / Transport Layer Security</i>	Ce protocole sécurise les échanges entre votre navigateur ou application et AWS.

3.2 Détails des types de chiffrement

SSE (Server-Side Encryption)

Chiffrement effectué **côté serveur**, c'est-à-dire par AWS une fois que les données sont reçues.

SSE-S3 : AWS chiffre les objets S3 avec une clé gérée par le service S3 lui-même.

- **Avantage** : aucune configuration requise.
- **Niveau de sécurité** : standard, suffisant pour de nombreux cas d'usage.

SSE-KMS : AWS chiffre les objets S3 avec une clé gérée par **AWS KMS**, que vous pouvez créer, activer/désactiver, auditer.

- **Avantage** : contrôle granulaire sur les clés.
- **Complexité** : nécessite configuration, permissions IAM, et gestion des quotas KMS.

KMS (Key Management Service)

Service AWS permettant de **créer, stocker et gérer** des clés de chiffrement.

- Utilisé dans **SSE-KMS**, mais aussi pour chiffrer des volumes EBS, des secrets, etc.
- Permet la **rotation automatique**, l'audit via CloudTrail, et l'intégration avec IAM.

HTTPS / TLS

Protocole de **sécurisation des communications réseau**.

- **HTTPS** est HTTP + TLS.
- **TLS (Transport Layer Security)** : protocole de chiffrement qui protège les données en transit.
- Activé **par défaut** dans la console AWS et les SDK/API.

3.3 À retenir sur le chiffrement

- **SSE-S3** : simple, automatique, suffisant pour les données non sensibles.
- **SSE-KMS** : recommandé pour les données sensibles ou les environnements réglementés.
- **HTTPS/TLS** : toujours activé pour sécuriser les échanges réseau.

:::info **SSE-KMS et coûts KMS** — L'utilisation de clés KMS peut générer des appels KMS facturables en plus des opérations S3. Pour un bucket très sollicité, évaluez **S3 Bucket Keys**, qui réduisent le trafic de requêtes de S3 vers KMS. La réduction réelle et les conditions d'éligibilité doivent être vérifiées dans la documentation et la tarification courantes. :::

3.4 Versioning : garder l'historique des fichiers

Le **versioning** permet de conserver toutes les versions d'un fichier, même si vous le modifiez ou le supprimez par erreur.

Exemple :

- Vous téléversez `rapport.pdf`
- Vous le modifiez et téléversez une nouvelle version
- Vous pouvez toujours revenir à la version précédente

C'est utile pour :

- Éviter les pertes accidentelles
- Respecter des exigences réglementaires
- Tracer les modifications

 [S3 Versioning Guide](#)

3.5 Lifecycle Policies : automatiser le nettoyage et l'archivage

Les **politiques de cycle de vie** permettent de définir des règles pour :

- Supprimer automatiquement les fichiers après X jours
- Déplacer les fichiers vers une classe de stockage moins coûteuse
- Archiver dans Glacier pour la conformité

Une règle S3 Lifecycle automatise les transitions



La règle porte sur un préfixe ou des tags et peut distinguer version courante, versions non courantes et marqueurs de suppression.

Lecture du schéma. Une règle sélectionne des objets par préfixe ou par tags, puis applique les actions configurées. Les transitions disponibles, les durées minimales de stockage et les délais de restauration dépendent de la classe choisie. Une expiration est une suppression : elle doit être alignée sur la politique de conservation de l'organisation.

Cela vous aide à **réduire les coûts** sans perdre vos données.

Exemple concret : Un fichier log commence en **Standard** (accès rapide), est déplacé en **Standard-IA** après 30 jours (moins accédé, moins cher), puis archivé en **Glacier** après 90 jours (rarement consulté).

 [S3 Lifecycle Rules](#)

3.6 Classes de stockage : choisir le bon niveau selon l'usage

Amazon S3 propose plusieurs **classes de stockage**, selon la fréquence d'accès et le niveau de disponibilité souhaité.

Classe	Disponibilité	Coût	Temps d'accès	Cas d'usage
Standard	Multi-AZ		Millisecondes	Fichiers actifs, souvent consultés
Standard-IA	Multi-AZ		Millisecondes	Sauvegardes, fichiers rarement lus
One Zone-IA	Mono-AZ		Millisecondes	Données non critiques
Intelligent-Tiering	Automatique		Variable	Accès imprévisible
Glacier / Deep Archive	Multi-AZ	 très bas	Minutes à heures	Archivage long terme, conformité

Moins une donnée est accessible rapidement, moins elle coûte. C'est un **levier puissant pour optimiser votre budget**.

S3 Intelligent-Tiering : Optimisation automatique

S3 Intelligent-Tiering est une classe de stockage qui déplace automatiquement les objets entre quatre niveaux d'accès selon votre modèle de consultation réel.

Niveau	Délai avant bascule	Économie
Frequent Access	0-30 jours	coût = Standard
Infrequent Access	30-90 jours	~40%
Archive Instant	90-180 jours	~70%
Deep Archive	180+ jours	~95%

Avantages :

- Aucune configuration manuelle requise.
- Des frais de surveillance et d'automatisation s'ajoutent pour chaque objet éligible ; leur impact dépend donc fortement du nombre et de la taille des objets.
- Idéal pour les données dont la fréquence d'accès est **imprévisible**.

Cas d'usage :

- Logs d'applications avec accès sporadique.
- Archives de données sans schéma d'accès défini.
- Données de machine learning exploratoires.

Différence avec Lifecycle :

- Lifecycle : vous définissez les règles (ex. "après 90 jours, archiver en Glacier").
- Intelligent-Tiering : AWS observe votre accès réel et adapte automatiquement.

S3 Transfer Acceleration : Optimisation des uploads volumineux

S3 Transfer Acceleration améliore les **vitesse d'upload** vers S3 en utilisant le réseau CloudFront d'AWS.

Fonctionnement :

```

1 Upload standard (lent)
2 Votre ordinateur —————> AWS S3 Région distant
3                               ✗ Slow, high latency
4
5 Avec Transfer Acceleration
6 Votre ordinateur —> CloudFront Edge Location (près de vous)
7
8                               Optimisé (route accélérée AWS)
9                               ▼
10                              AWS S3 Région distant
11                              ✓ Rapide

```

Activation :

```

1 # Activer Transfer Acceleration
2 aws s3api put-bucket-accelerate-configuration \
3     --bucket mon-bucket \
4     --accelerate-configuration Status=Enabled
5
6 # Upload avec accélération

```

```

7  aws s3 cp mon-fichier-gros.zip \
8      s3://mon-bucket/uploads/ \
9      --region eu-west-1

```

:::success **Résultat attendu :**

```

1  # put-bucket-accelerate-configuration : aucun output si succès
2
3  # s3 cp retourne la progression :
4  upload: ./mon-fichier-gros.zip to s3://mon-bucket/uploads/mon-fichier-gros.zip

```

Transfer Acceleration est activé sur le bucket. Les uploads utilisent désormais les Edge Locations CloudFront pour rejoindre le bucket S3, ce qui réduit la latence depuis les clients distants. :::

Coûts :

- Des frais d'accélération s'ajoutent au transfert standard et varient selon le trajet des données.
- Le gain doit être mesuré avec l'outil de comparaison de vitesse AWS avant activation.

S3 comme origine CloudFront — distribuer du contenu statique à grande échelle

Transfer Acceleration optimise l'**upload** vers S3. Le cas d'usage inverse — beaucoup plus fréquent en production — est de distribuer efficacement du contenu **depuis** S3 vers des millions de visiteurs : c'est le rôle de **CloudFront** utilisé comme CDN devant un bucket S3.

```

1  Sans CloudFront (chaque visiteur télécharge depuis S3 directement) :
2  Visiteur Tokyo    —> S3 eu-west-3 (Paris)    ❌ Latence élevée, coût egress
3  Visiteur New York —> S3 eu-west-3 (Paris)    ❌ Latence élevée, coût egress
4  Visiteur Paris    —> S3 eu-west-3 (Paris)    ✅ Rapide (mais N requêtes = N
5
6  Avec CloudFront devant S3 :
7  Visiteur Tokyo    —> Edge Location Tokyo      (cache HIT après 1er accès, < 5
8  Visiteur New York —> Edge Location New York  (cache HIT après 1er accès, < 5
9  Visiteur Paris    —> Edge Location Paris      (cache HIT après 1er accès, <
10
11                      |
12                      ▼ (uniquement au premier accès, cache MISS)
                        S3 eu-west-3 (Paris) — origine unique

```

Pourquoi c'est la bonne pratique, pas juste une option :

- **Coût réduit** : le trafic sortant de S3 vers CloudFront est gratuit (les deux services AWS communiquent via le réseau interne). Seul le trafic CloudFront → visiteur final est facturé, à un

tarif généralement inférieur à l'égress S3 direct.

- **Bucket privé possible** : avec une **Origin Access Control (OAC)**, le bucket S3 n'a besoin d'aucun accès public — seul CloudFront peut le lire. Les visiteurs ne touchent jamais directement S3.
- **Cache réduit la charge S3** : un fichier consulté 100 000 fois par jour ne génère qu'une poignée de requêtes S3 réelles (une par Edge Location, tant que le cache est valide), le reste est servi depuis le cache CloudFront.

Configuration minimale (CLI) :

```
1 # Créer la distribution CloudFront avec S3 comme origine et OAC
2 aws cloudfront create-distribution \
3   --origin-domain-name mon-bucket.s3.eu-west-3.amazonaws.com \
4   --default-root-object index.html
```

:::success **Résultat attendu (extrait) :**

```
1 {
2   "Distribution": {
3     "Id": "E1A2B3C4D5E6F7",
4     "DomainName": "d1111111abcdef8.cloudfront.net",
5     "Status": "InProgress"
6   }
7 }
```

La distribution devient **Deployed** après quelques minutes de propagation sur le réseau mondial CloudFront. Le bucket S3 reste privé — seule cette distribution CloudFront (via son OAC) est autorisée à le lire, configuré automatiquement dans la bucket policy. :::

Amazon CloudFront — Restricting access to S3 [↗](#)

:::warning **Coûts Transfer Acceleration** : cette fonctionnalité ajoute un coût au volume transféré. Ne l'activez qu'après avoir mesuré un gain utile depuis les emplacements réels des clients. Un test de performance et une estimation sur la page tarifaire S3 sont plus fiables qu'un seuil de taille générique. :::

Cas d'usage :

- Uploads de fichiers vidéo ou binaires depuis un client distant.
- Synchronisations multi-sites hautes performances.
- Distributions de fichiers volumineux vers plusieurs régions AWS.

3.7 Stratégies de compartiment (Bucket Policies) : Contrôle d'accès granulaire

Les **bucket policies** sont des documents JSON qui définissent **qui** peut accéder à **quoi** dans un bucket S3.

Structure d'une bucket policy

Une bucket policy est un document JSON attaché directement au bucket (et non à un utilisateur IAM). Elle contrôle qui peut accéder à quoi, y compris des accès publics ou inter-comptes AWS.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "AllowPublicRead",
6        "Effect": "Allow",
7        "Principal": "*",
8        "Action": "s3:GetObject",
9        "Resource": "arn:aws:s3:::mon-bucket/*"
10     },
11     {
12       "Sid": "DenyEncryptedUploads",
13       "Effect": "Deny",
14       "Principal": "*",
15       "Action": "s3:PutObject",
16       "Resource": "arn:aws:s3:::mon-bucket/*",
17       "Condition": {
18         "StringNotEquals": {
19           "s3:x-amz-server-side-encryption": "AES256"
20         }
21       }
22     }
23   ]
24 }
```

Éléments clés :

- **Sid** : identifiant lisible de la règle (ex. "AllowPublicRead")
- **Effect** : Allow ou Deny
- **Principal** : qui a l'accès (* = tout le monde, ou un ARN spécifique)

- **Action** : quelle opération S3 (`s3:GetObject`, `s3:PutObject`, `s3:DeleteObject`, etc.)
- **Resource** : sur quel objet (ARN format)
- **Condition** : contextes additionnels (IP, SSL, chiffrement, etc.)

Cas d'usage 1 : Site web statique public

Pour héberger un site web HTML/CSS sur S3, le bucket doit autoriser la lecture publique. Voici la policy à appliquer — notez `"Principal": "*"` qui signifie “tout le monde sans authentification” :

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "PublicReadGetObject",
6        "Effect": "Allow",
7        "Principal": "*",
8        "Action": "s3:GetObject",
9        "Resource": "arn:aws:s3:::mon-site-web/*"
10     }
11   ]
12 }
```

Effet : Tous les utilisateurs peuvent **lire** les fichiers du bucket (parfait pour un site statique).

⚠️ **Bucket public : risque de fuite de données** — L'utilisation de `"Principal": "*"` rend l'ensemble des objets du bucket accessibles sur Internet **sans authentification**. Ne l'appliquez **jamais** à un bucket contenant des données sensibles (fichiers clients, logs internes, clés, backups). Depuis 2023, AWS bloque par défaut les accès publics sur les nouveaux buckets — cette policy nécessite de désactiver explicitement ce blocage. ⚠️

Cas d'usage 2 : Restreindre à une adresse IP spécifique

Pour un bucket contenant des données sensibles, on limite l'accès au réseau de l'entreprise via la condition `aws:SourceIp`. Tout accès depuis une IP extérieure sera refusé, même avec des credentials IAM valides.

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "RestrictToCompanyIP",
6        "Effect": "Allow",
```



```

7     "Principal": "*",
8     "Action": ["s3:GetObject", "s3:PutObject"],
9     "Resource": "arn:aws:s3:::mon-bucket-prive/*",
10    "Condition": {
11        "IpAddress": {
12            "aws:SourceIp": "203.0.113.0/24"
13        }
14    }
15 }
16 ]
17 }
```

Effet : Seules les IPs du réseau 203.0.113.0/24 peuvent accéder au bucket.

Cas d'usage 3 : Forcer le chiffrement pour tous les uploads

Cette policy est une protection anti-erreur : elle refuse tout upload qui ne spécifie pas le chiffrement côté serveur (SSE-S3). Même si un développeur oublie de configurer le chiffrement dans son code, AWS rejette la requête.

```

1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Sid": "DenyUnencryptedObjectUploads",
6        "Effect": "Deny",
7        "Principal": "*",
8        "Action": "s3:PutObject",
9        "Resource": "arn:aws:s3:::mon-bucket-sensible/*",
10       "Condition": {
11         "StringNotEquals": {
12             "s3:x-amz-server-side-encryption": "AES256"
13         }
14       }
15     }
16   ]
17 }
```

Effet : Toute tentative d'upload sans chiffrement SSE-S3 sera **rejetée**.

Appliquer une bucket policy via CLI

Une fois le fichier JSON rédigé, voici comment l'appliquer, le vérifier, et le supprimer si nécessaire :

```
1  # Créer un fichier policy.json (voir ci-dessus)
2  # Appliquer la policy
3  aws s3api put-bucket-policy \
4      --bucket mon-bucket \
5      --policy file://policy.json
6
7  # Vérifier la policy
8  aws s3api get-bucket-policy --bucket mon-bucket
9
10 # Supprimer la policy
11 aws s3api delete-bucket-policy --bucket mon-bucket
```

:::success **Résultat attendu :**

```
1  # put-bucket-policy : aucun output si succès
2
3  # get-bucket-policy retourne :
4  {
5      "Policy": "{\"Version\":\"2012-10-17\",\"Statement\": [{\"Sid\":\"AllowPubli
6  }
7
8  # delete-bucket-policy : aucun output si succès
```

...

Point important : Les bucket policies s'ajoutent aux **ACL (Access Control Lists)**, il faut les deux pour une sécurité complète.

3.8 Bonnes pratiques S3

- Activez le **versioning** dès que vous stockez des fichiers importants.
- Utilisez **SSE-S3** pour un chiffrement simple et automatique.
- Créez des **règles de cycle de vie** pour archiver ou supprimer les fichiers inutilisés.
- Choisissez la **classe de stockage** adaptée à chaque type de données.
- Appliquez des **bucket policies** pour restreindre l'accès selon le principe du moindre privilège.
- Utilisez **Intelligent-Tiering** si l'accès est imprévisible.
- Activez **Transfer Acceleration** pour les uploads volumineux critiques.

4. Amazon EC2 : La couche de calcul AWS

4.1 Introduction à EC2

 Vidéo : [Lancer sa première machine virtuelle Windows avec EC2](#) 

Après avoir stocké nos données avec Amazon S3, nous allons voir comment **les traiter, les héberger ou les exécuter** grâce à **Amazon Elastic Compute Cloud (EC2)**.

EC2 est l'un des premiers services historiques d'AWS (2006). Il permet de **louer de la puissance de calcul à la demande**, avec une flexibilité inégalée par rapport aux serveurs physiques traditionnels.

 [Documentation officielle Amazon EC2](#) 

Amazon EC2 (Elastic Compute Cloud) est le service AWS qui permet de créer des **machines virtuelles** dans le cloud, appelées **instances EC2**.

4.2 Pourquoi utiliser EC2 ?

EC2 reprend le principe familier d'un serveur physique — un système d'exploitation, du CPU, de la RAM, du stockage, une carte réseau — mais en supprime toutes les contraintes matérielles, ce qui explique son adoption massive comme brique de calcul de base sur AWS.

Le **lancement est rapide** : là où commander, recevoir et configurer un serveur physique prenait des semaines, une instance EC2 est prête à l'emploi en quelques clics ou quelques lignes de CLI, avec un système d'exploitation déjà installé. Le service est aussi **flexible** : vous choisissez la puissance de calcul, le système d'exploitation, le type de stockage attaché et la configuration réseau, et vous pouvez faire évoluer ces choix a posteriori si les besoins changent — un projet peut commencer sur une petite instance et migrer vers une plus puissante sans réinstallation. Le modèle est **économique** parce que la facturation suit la consommation réelle plutôt qu'un investissement matériel figé : vous payez à l'heure ou à la seconde pour ce qui tourne, et vous pouvez arrêter une instance dès qu'elle n'est plus utile pour cesser d'être facturé. Enfin, EC2 est nativement **connecté** au reste de l'écosystème AWS : une instance peut lire et écrire dans un bucket S3, s'authentifier via un rôle IAM sans stocker de clé d'accès, et vivre dans un VPC dont vous contrôlez entièrement le découpage réseau — cette intégration native évite d'avoir à recoller manuellement des briques hétérogènes comme sur une infrastructure on-premise.

4.3 Les composants essentiels d'une instance EC2

Composant	Rôle dans l'architecture EC2
Instance EC2	Machine virtuelle hébergée chez AWS
AMI	Image système (Linux, Windows, etc.) utilisée comme modèle
Type d'instance	Détermine la puissance (CPU, RAM, réseau)
Security Group	Pare-feu virtuel qui contrôle les accès réseau
EBS	Disque dur virtuel attaché à l'instance
EFS	Système de fichiers partagé entre plusieurs instances
Key Pair	Clé SSH utilisée pour se connecter à l'instance en toute sécurité

Lors du lancement d'une instance, vous devez choisir chacun de ces éléments.

5. Choisir le bon type d'instance, AMI, stockage et sécurité

5.1 Types d'instances EC2

AWS propose plusieurs familles d'instances, selon le type de charge à traiter :

Famille	Usage recommandé	Exemple d'application
t / t4g	Usage général, burst occasionnel	Serveur web, environnement de test
m	Charges équilibrées CPU/Mémoire	Application métier, ERP
c	Calcul intensif	Simulation scientifique, traitement d'image
r	Mémoire importante	Base de données en mémoire, Redis
g / p	GPU (accélération graphique/IA)	Intelligence artificielle, machine learning

Dans un environnement de formation, le type d'instance est imposé par le lab. L'éligibilité au Free Tier dépend du plan du compte, de sa date de création et des types actuellement marqués comme éligibles dans la console AWS.

⚠ **Types d'instances coûteux** — Les familles accélérées par GPU et les instances à très grande capacité peuvent avoir un coût horaire élevé. Le type autorisé pendant la formation est celui indiqué dans le lab. En entreprise, le choix doit être vérifié avec AWS Pricing Calculator et les tarifs de la région avant déploiement. ⚠

Explication des suffixes de type :

- **t3** : type t (général), génération 3
- **micro**, **small**, **medium** : taille croissante
- **xlarge** ou **2xlarge** : très puissants, pour les charges importantes
- le **g** que l'on trouve dans **t4g**, **m6g**, **c6g**, etc. signale généralement une instance équipée d'un processeur **AWS Graviton** fondé sur l'architecture ARM. Le rapport performance/prix dépend de la charge. Les binaires et images doivent être compatibles avec l'architecture choisie ; un composant compilé uniquement pour x86 doit être recompilé ou remplacé.

5.2 AMI (Amazon Machine Image)

L'AMI est le **système d'exploitation** de votre machine EC2.

📺 Vidéo : [AMI — Amazon Machine Images](#)

Types d'AMI

- **Public AMI** : proposées par AWS (Linux, Windows, Ubuntu, etc.)
- **Custom AMI** : créées par vous (ex. avec des logiciels préinstallés)
- **Marketplace AMI** : proposées par des éditeurs tiers (ex. WordPress, SAP)

L'AMI détermine ce que contient votre machine au démarrage.

Classification des AMI

Les AMI peuvent être classifiées dans les grandes catégories suivantes :

AMI persistantes (basées sur EBS)

- L'ensemble du filesystem est stocké sur EBS (Elastic Block Store).

- EBS fonctionne de manière similaire à un **NAS (Network Attached Storage)** et permet le partage des données sur le réseau.
- Ces volumes ne sont associés à aucun type de matériel spécifique, ce qui les rend très pratiques pour transférer des données entre zones ou d'une région à une autre.
- Les AMI persistantes sont configurées avec un ou plusieurs volumes EBS.

AMI volatiles (basées sur S3)

- Contrairement aux AMI persistantes, les AMI volatiles stockent leurs données via le service AWS S3 (Simple Storage Service).
- Ces AMI **ne peuvent pas être transférées** depuis une zone de disponibilité ou région vers une autre.

Il est souvent utile en entreprise de créer ses propres AMI afin de pouvoir déployer plus rapidement des instances EC2 correspondant aux besoins spécifiques. Vous pouvez enregistrer le disque contenant cette AMI après lancement de la machine EC2 et après avoir ajouté les spécificités de l'ensemble de vos machines.

5.3 Stockage associé à EC2

EBS — Elastic Block Store

- **Disque attaché** à une instance EC2.
- **Persiste** même si l'instance est arrêtée.
- Permet les **snapshots** (sauvegardes incrémentales).
- Idéal pour le stockage de données d'application.
- Peut être attaché/détaché dynamiquement.

Cas d'usage : serveur web avec base de données, ERP, systèmes de fichiers importants.

EFS — Elastic File System

- **Système de fichiers partagé** entre plusieurs instances.
- Montable sur **plusieurs instances EC2 simultanément**.
- Idéal pour les architectures distribuées.
- Escalabilité automatique sans gestion de capacité.
- Compatible avec NFS (Network File System).

Cas d'usage : cluster d'applications, déploiement multi-serveurs, stockage partagé.

Avantages :

- Haute disponibilité multi-AZ.

- Performance prédictible et constante.
- Paiement à l'usage (pas de provisionnement anticipé).

FSx — Managed File Systems

Amazon FSx propose des systèmes de fichiers managés, avec deux options principales :

FSx for Windows File Server :

- Compatible **Active Directory** et **SMB** (partages Windows).
- Idéal pour les environnements Windows d'entreprise.
- Partages de fichiers compatibles avec les domaines Windows.

SMB (Server Message Block) est le protocole de partage de fichiers natif de Windows — c'est exactement ce que vous utilisez quand vous accédez à un dossier partagé via `\\serveur\dossier` sur un réseau d'entreprise. FSx for Windows reproduit ce protocole nativement dans AWS, ce qui permet à des applications Windows existantes de continuer à fonctionner sans modification.

FSx for Lustre :

- Optimisé pour le **high-performance computing (HPC)** et machine learning.
- Très haute performance pour les grandes quantités de données.

Lustre est un système de fichiers distribué open source conçu à l'origine pour les supercalculateurs : il répartit un même fichier sur plusieurs serveurs de stockage pour permettre à des milliers de machines de le lire et l'écrire simultanément à très haut débit. C'est ce qui en fait le choix de référence pour l'entraînement de modèles de machine learning sur de gros volumes de données ou les simulations scientifiques (météo, génomique, calcul financier).

Comparatif EFS vs FSx :

Critère	EFS	FSx (Windows)	FSx (Lustre)
Protocole	NFS (Linux/Unix)	SMB (Windows)	Lustre (HPC)
OS Support	Linux/Unix	Windows	Linux/HPC
Active Directory	Non	✓ Oui	Non
Performance	Modérée, extensible	Haute	Très haute (HPC)
Coût	Bas à moyen	Moyen-élevé	Élevé

Idéal pour

Linux distribué

Partages Windows
d'entreprise

Calcul
scientifique/IA

Conseil pratique :

- **EFS** : première option pour Linux si pas besoin de domaine.
- **FSx for Windows** : environnement Windows avec Active Directory.
- **FSx for Lustre** : uniquement si performance HPC requise.

 [EBS Documentation](#)  [EFS Documentation](#)  [FSx Documentation](#)

5.4 Sécurité EC2

Security Groups : pare-feu virtuel

Les **Security Groups** sont des pare-feux virtuels qui :

- Autorisent ou bloquent le trafic,
- Sont **stateful** (les réponses sont automatiquement autorisées),
- Peuvent être appliqués à plusieurs instances.

Exemple de règles

Direction	Port	Source/Destination
Inbound	22 (SSH)	192.168.1.100/32
Inbound	80 (HTTP)	0.0.0.0/0
Inbound	443 (HTTPS)	0.0.0.0/0
Outbound	Tout	par défaut

Règle entrante exemple :

- Autoriser le port **22 (SSH)** uniquement depuis une IP précise.
- Autoriser les ports **80 et 443 (HTTP/HTTPS)** depuis n'importe où.

Règle sortante (par défaut) :

- Autoriser tout trafic sortant.

Key Pairs : accès SSH sécurisé

Les **Key Pairs** servent à sécuriser l'accès SSH :

- **Clé publique** enregistrée dans AWS et stockée dans l'instance au démarrage.
- **Clé privée** conservée localement par l'administrateur.

Flux de connexion :

1. Vous lancez une instance EC2 et sélectionnez une Key Pair.
2. AWS injecte la clé publique dans `~/.ssh/authorized_keys` de l'instance.
3. Depuis votre ordinateur, vous utilisez votre clé privée pour vous connecter en SSH.
4. L'authentification par clé est plus sécurisée qu'un mot de passe (impossible à craquer par brute force).

5.5 Options de conformité EC2

Les environnements réglementés (santé, finance, RGPD) ont besoin de **garanties de conformité**. AWS fournit plusieurs mécanismes :

Dedicated Instances

- Instance EC2 qui s'exécute sur **matériel physique dédié**.
- Pas de partage avec d'autres clients AWS.
- Idéal pour les **exigences légales** ou de conformité.
- Coût plus élevé que le partage de matériel.

Dedicated Hosts

- **Serveur physique entier** réservé pour votre compte.
- Contrôle total : vous décidez quelles instances y tournent.
- Utile pour les **licences logicielles** (ex. Windows, SQL Server avec licensing par socket/processeur).
- Exigences réglementaires très strictes.

Instance Store (éphémère)

- Stockage **très rapide** mais **temporaire** sur l'hyperviseur physique.
- **Attention** : données perdues à l'arrêt/redémarrage de l'instance.
- Idéal pour cache, données temporaires, haute performance.
- À éviter pour données persistantes.

⚠️ **Instance Store : perte de données garantie à l'arrêt** — Contrairement à EBS, le stockage instance store **n'est pas persistant**. Toutes les données écrites dessus sont définitivement perdues si l'instance est arrêtée, terminée ou si l'hôte physique tombe en panne. Ne stockez jamais de données de production, de bases de données ou de fichiers importants sur instance store sans sauvegarde préalable vers S3 ou EBS. ⚠️

Encrypted EBS Volumes

- Les volumes EBS peuvent être chiffrés avec **AWS KMS**.
- Le chiffrement est **transparent** pour l'application.
- Utile pour la conformité HIPAA, PCI-DSS, ISO 27001.

Cas d'usage conformité :

- Données médicales → Dedicated Instance + EBS chiffré + Audit CloudTrail.
- Données financières → Dedicated Host + KMS + VPC isolé.
- Données RGPD → Région EU + Versioning S3 + Chiffrement.

6. AWS Compute Optimizer — Dimensionnement optimal

6.1 Qu'est-ce que AWS Compute Optimizer ?

AWS Compute Optimizer est un service qui **analyse vos patterns d'utilisation** des instances EC2 et recommande des types plus optimisés en coût et performance.

	Instance actuelle	Recommandation
Type	t3.large (trop puissante)	t3.small (plus économique)
CPU utilisation	5%	—
Mémoire	12%	—
Coût mensuel	80 \$	~48 \$ (économie 60%)
Performance	—	identique
Risque	—	très faible (marges CPU)

Compute Optimizer applique cette analyse automatiquement à partir des métriques CloudWatch collectées.

6.2 Fonctionnement

1. **Collecte** : Compute Optimizer récupère les métriques CloudWatch (CPU, mémoire, réseau) sur **14 jours minimum**.
2. **Analyse** : Machine Learning compare votre utilisation réelle avec les capacités des autres types.
3. **Recommandation** : Propose des types économiquement viables.
4. **Confiance** : Indique un score de confiance (low, medium, high).

6.3 Types de recommandations

Recommandation	Bénéfice	Risque	Exemple
Downsizer	Économies importantes	Risque d'augmenter le CPU > 100%	t3.large → t3.small
Upgrade	Meilleure performance	Légère augmentation de coût	m5.large → m5.xlarge
Switch Family	Meilleure performance/\$	Changement d'architecture	t3.large → m6i.large
Aucune recommandation	Instance bien dimensionnée	N/A	✅ Garder tel quel

6.4 Activation et utilisation

Compute Optimizer analyse l'utilisation réelle de vos instances (CPU, mémoire, réseau) sur 14 jours et suggère le type le mieux adapté. La commande suivante affiche ces recommandations sous forme de tableau comparatif.

```
1 # Vérifier les recommandations Compute Optimizer
2 aws compute-optimizer get-ec2-instance-recommendations \
3     --region eu-west-1 \
4     --query 'instanceRecommendations[].{
5         Instance:instanceArn,
```

```

6         Current:currentInstanceType,
7         Recommended:recommendationOptions[0].instanceType,
8         Savings:recommendationOptions[0].savingsOpportunity.estimatedMonthlySav
9         ConfidenceLevel:currentInstanceType
10     }' \
11     --output table

```

:::success **Résultat attendu :**

```

1  -----
2  |                                     GetEc2InstanceRecommendations
3  +-----+-----+-----+-----+-----+
4  |          Instance                  | Current  | Recommended| Savings  | Confi
5  +-----+-----+-----+-----+-----+
6  |  arn:aws:ec2:eu-west-1:123:instance | t3.large | t3.small   | 47.82    | t3.1
7  |  arn:aws:ec2:eu-west-1:123:instance | m5.xlarge| m5.large   | 62.40    | m5.x
8  +-----+-----+-----+-----+-----+

```

Si aucune recommandation n'apparaît, Compute Optimizer manque encore de données (il lui faut au minimum 30h d'activité sur les instances). :::

6.5 Cas d'usage

- **Optimisation de coûts** : identifier toutes les instances surdimensionnées.
- **Gouvernance cloud** : politiques de rightsizing automatisées.
- **Migration** : recommandations pour basculer vers une architecture nouvelle.
- **Audit FinOps** : justification des dépenses EC2.

Avantage clé : Compute Optimizer s'appuie sur **12-14 jours de données réelles**, pas sur des hypothèses théoriques.

7. Options de tarification AWS EC2

AWS propose plusieurs modèles de tarification pour s'adapter aux besoins techniques et budgétaires des entreprises. Le choix dépend du niveau de prévisibilité des workloads, du budget disponible, et de la tolérance aux interruptions.

7.1 On-Demand (À la demande)

- **Paiement à l'heure** ou à la seconde pour la capacité de calcul utilisée, sans engagement à long terme.
- **Idéal pour** les charges de travail à court terme, les tests et le développement.
- **Pas de paiement anticipé** ni d'engagement minimum.
- **Prix plus élevé** que les autres options mais offre une flexibilité maximale.
- **Recommandé pour** les applications ne pouvant pas être interrompues et ayant des charges de travail imprévisibles.

Exemple : Vous avez un pic de trafic imprévu. Vous lancez des instances On-Demand pour répondre à la demande, puis les arrêtez après le pic.

7.2 Savings Plans

- **Engagement de consommation horaire** sur une période de 1 ou 3 ans.
- **Réduction variable** par rapport au tarif à la demande selon le plan, la durée et le mode de paiement.
- **Deux types principaux :**
 - **Compute Savings Plans :** Flexibilité maximale couvrant EC2, Fargate et Lambda, avec support multi-familles d'instances, tailles et régions.
 - **EC2 Instance Savings Plans :** Réductions plus importantes mais limité à une famille d'instances dans une région spécifique.
- **Options de paiement flexibles** impactant le taux de réduction :
 - **No Upfront :** Aucun paiement initial.
 - **Partial Upfront :** Paiement partiel initial.
 - **Full Upfront :** Paiement total initial offrant les meilleures réductions.

7.3 Instances Spot (À prix réduit)

- **Utilisation de la capacité EC2 inutilisée** d'AWS.
- **Remise variable** par rapport au prix à la demande, en échange d'un risque d'interruption.
- **Les instances peuvent être interrompues** avec un préavis de 2 minutes si AWS a besoin de la capacité.
- **Idéal pour :**
 - Les charges de travail tolérantes aux interruptions.
 - Le calcul haute performance (HPC).
 - Les jobs batch (traitement par lots).
 - Les workloads flexibles en termes de début et de fin.

Best practices pour la résilience :

- Utiliser les **groupes d'auto-scaling** pour gérer les interruptions automatiquement.
- Implémenter via **EC2 Spot Fleet** ou **EC2 Spot Instances Requests**.
- Concevoir l'application pour tolérer les interruptions.

⚠ **Instances Spot : interruption en 2 minutes** — AWS peut récupérer vos instances Spot avec seulement **2 minutes de préavis** lorsque la capacité est nécessaire. Ne jamais utiliser des instances Spot pour des workloads critiques sans tolérance aux interruptions (bases de données de production, serveurs web sans état de session externalisé). Toujours prévoir un mécanisme de sauvegarde ou de checkpoint des données en cours de traitement. ⚠

7.4 Reserved Instances (RI)

- **Engagement** sur une instance spécifique pour **1 ou 3 ans**.
- **Remise variable** par rapport au prix à la demande, contre un engagement de durée et de configuration.
- **Différences principales** avec les Savings Plans :
 - Les RI sont liées à une instance spécifique (type, taille, région, zone).
 - Les Savings Plans sont basés sur un engagement de consommation en dollars (plus flexibles).
 - Les RI peuvent être vendues sur le **AWS RI Marketplace**, pas les Savings Plans.

7.5 Comparatif synthétique

Critère	On-Demand	Reserved	Spot	Savings Plans
Engagement	Aucun	1 ou 3 ans	Aucun	1 ou 3 ans
Réduction potentielle	Référence	Variable selon l'engagement	Variable selon la capacité disponible	Variable selon l'engagement
Flexibilité	✓ ✓ ✓	✗	✓ ✓	✓ ✓
Risque d'interruption	✗	✗	✓ ✓ ✓	✗
Idéal pour	Dev/Test	Prod stable	Batch/CI	Prod optimisée

Décision : il n'existe pas de modèle universellement meilleur. La charge stable favorise un engagement ; la charge interruptible favorise Spot ; l'incertitude favorise le paiement à la demande. La décision doit s'appuyer sur les métriques réelles.

7.6 Cas métier : Choisir la meilleure option tarifaire

Cas 1 : Site e-commerce avec trafic prévisible

- **Charge** : trafic stable, pics prévisibles en fin d'année.
- **Infrastructure** : 10 instances t3.large en continu, +20 during soldes.
- **Recommandation** : **Savings Plans (Compute)** pour les 10 instances permanentes + **Spot** pour les 20 supplémentaires pendant les soldes.
- **Validation économique** : comparer dans AWS Pricing Calculator le socle engagé, la capacité à la demande et le renfort Spot avec les tarifs du jour.

Cas 2 : Environnement de développement/test

- **Charge** : variable, utilisation heures de travail uniquement.
- **Infrastructure** : 2-4 instances selon le sprint en cours.
- **Recommandation** : **On-Demand** uniquement (pas d'engagement, flexibilité totale).
- **Économie** : aucune, mais coûts minimaux et liberté maximale.

Cas 3 : Job batch nocturne de traitement

- **Charge** : lance chaque nuit des instances pour 4h, puis arrêt.
- **Infrastructure** : 50 instances c5.2xlarge pour le parallélisme.
- **Recommandation** : **Spot instances** avec **Spot Fleet** (demande auto-scaling de remplacement).
- **Économie attendue** : à estimer avec le tarif Spot observé, le tarif à la demande de référence et le coût des interruptions. Le pourcentage varie selon la famille, la région et la capacité disponible.

```
1  # Configuration Spot Fleet pour job batch
2  aws ec2 request-spot-fleet \
3      --spot-fleet-request-config '{
4          "IamFleetRole": "arn:aws:iam::xxxxx:role/fleet",
5          "SpotPrice": "0.10",
6          "TargetCapacity": 50,
7          "LaunchSpecifications": [{
8              "ImageId": "ami-xxxxx",
9              "InstanceType": "c5.2xlarge",
10             "KeyName": "ma-cle"
11         }]
```

12 }

:::success **Résultat attendu :**

```
1 {
2   "SpotFleetRequestId": "sfr-0a1b2c3d4e5f6789a",
3   "SpotFleetRequestState": "submitted"
4 }
```

:::

Cas 4 : Application critiques 24/7 avec charge non prévisible

- **Charge** : pas de pattern clair, augmentations soudaines.
- **Infrastructure** : 4-30 instances selon la demande.
- **Recommandation** : **Savings Plans (mélange)** pour la charge de base + **On-Demand** pour les pics.
- **Avantage** : si charge explose au-delà des prévisions, On-Demand absorbe sans coupure.

7.7 Outil : AWS Pricing Calculator

```
1 URL : https://calculator.aws/
2 1. Sélectionner la région
3 2. Ajouter EC2 : type, nombre, durée
4 3. Sélectionner option (On-Demand, Reserved, Spot)
5 4. Voir l'estimation mensuelle/annuelle
6 5. Exporter en PDF pour justifier budgets
```

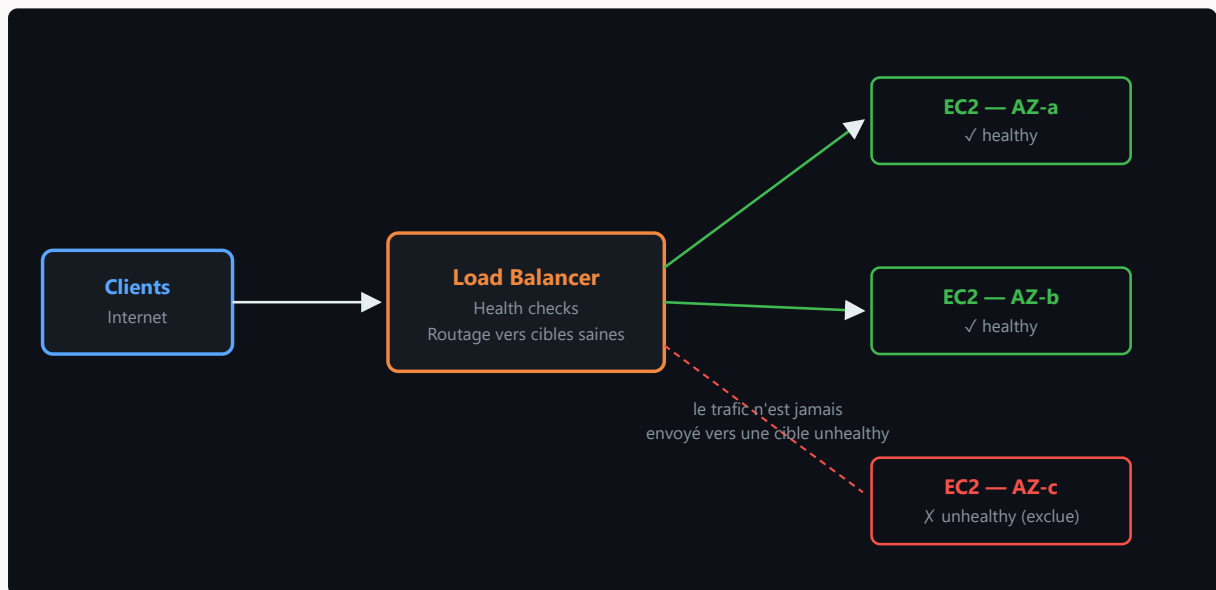
 [EC2 Pricing](#)

8. Elastic Load Balancing (ELB) — Répartition du trafic

8.1 Pourquoi un Load Balancer ?

Un **Load Balancer** agit comme un répartiteur de trafic. Il reçoit les requêtes des clients et les distribue vers les instances EC2 disponibles, selon des règles de routage et de santé (**health checks**).

Architecture simple



Lecture du schéma. Le répartiteur reçoit le trafic client et l'envoie uniquement aux cibles déclarées saines par les contrôles d'état. Les instances sont réparties sur plusieurs zones de disponibilité afin qu'une défaillance de zone n'interrompe pas nécessairement le service.

8.2 Types de Load Balancer AWS

Type	Cas d'usage typique	Protocole	Niveau OSI
ALB (Application Load Balancer)	Applications web, microservices	HTTP/HTTPS	Couche 7 (Application)
NLB (Network Load Balancer)	Faible latence, TCP	TCP/UDP	Couche 4 (Transport)
GLB (Gateway Load Balancer)	Appliances réseau (firewall, inspection)	IP	Couche 3 (Réseau)

ALB (Application Load Balancer) : conçu pour les applications web. Il fonctionne au niveau **HTTP/HTTPS** (couche 7 du modèle OSI) et permet un **routage avancé** (par URL, en-tête, hostname, etc.).

NLB (Network Load Balancer) : adapté aux applications nécessitant une **faible latence**. Il fonctionne au niveau **TCP** (couche 4), idéal pour les bases de données ou les services temps réel.

GLB (Gateway Load Balancer) : utilisé pour intégrer des **appliances réseau** comme des pare-feu ou des outils d'inspection. Il fonctionne au niveau **IP**.

8.3 Fonctionnement du Load Balancer

- Le Load Balancer **vérifie l'état** des instances via des **health checks** (tests de disponibilité).
- Il **répartit les requêtes** vers les instances **saines** uniquement.
- Il **s'adapte automatiquement** à l'ajout ou la suppression d'instances via **Auto Scaling**.

Health Check - Exemple :

```
1 Chaque 30 secondes :  
2 1. LB envoie requête GET http://instance:80/health  
3 2. Instance répond HTTP 200 OK  
4 3. Instance est marquée "saine"  
5  
6 Si pas de réponse ou erreur 5xx :  
7 1. Instance marquée "défaillante"  
8 2. Pas plus de trafic envoyé vers elle
```

 [Elastic Load Balancing Documentation](#)

9. Auto Scaling — Adaptation dynamique des ressources

9.1 Qu'est-ce qu'Auto Scaling ?

Un **Auto Scaling Group (ASG)** est un groupe d'instances EC2 géré automatiquement. Il peut être associé à un Load Balancer pour garantir que :

- Les nouvelles instances sont automatiquement **enregistrées** auprès du Load Balancer.
- Les instances défaillantes sont **retirées** du pool.
- Le trafic est toujours dirigé vers les **ressources disponibles**.

9.2 Politiques de scaling — Fondamentaux

Les politiques définissent **quand et comment** ajouter ou retirer des instances.

Scale-out (Agrandissement)

- 1 Charge CPU dépasse 70% pendant 5 min
- 2 ▼
- 3 Ajouter 2 instances supplémentaires
- 4 ▼
- 5 Attendre que les instances démarrent
- 6 ▼
- 7 Health check OK : instances intégrées au LB

Scale-in (Réduction)

- 1 Charge CPU chute à 30% pendant 10 min
- 2 ▼
- 3 Retirer 1 instance
- 4 ▼
- 5 Attendre que les requêtes actuelles finissent
- 6 ▼
- 7 Fermer l'instance, libérer les ressources

9.3 Configuration d'Auto Scaling

Un ASG typique comporte :

Min Size : 2 instances · Max Size : 10 instances · Desired Capacity : 4 instances Launch Template : my-ami-config · Load Balancer : my-alb

Scaling Policies : Target CPU 70% · Scale out +2 instances/5 min · Scale in -1 instance/10 min

Paramètres clés :

- **Min Size** : minimum d'instances (au moins 2 pour la haute disponibilité)
- **Max Size** : limite supérieure pour éviter les coûts explosifs
- **Desired Capacity** : nombre d'instances cible en ce moment
- **Launch Template** : modèle (AMI, type, security group, etc.) pour les nouvelles instances

9.4 Métriques CloudWatch et politiques de scaling avancées

Auto Scaling peut se baser sur **plusieurs métriques CloudWatch**, pas seulement CPU.

Métriques disponibles

Métrique	Source	Cas d'usage typique
CPU Utilization	CloudWatch	Charge générale, serverless
NetworkIn / NetworkOut	CloudWatch	Applications réseau intensives
ALB Target Count	CloudWatch + ELB	Nombre de requêtes traitées
Request Count Per Target	CloudWatch + ELB	Répartition de charge par instance
Target Response Time	CloudWatch + ELB	Dégradation de performance
Memory Utilization	CloudWatch Agent	Applications mémoire-intensives
Queue Depth (SQS)	SQS	Traitement asynchrone

Exemple de politique de scaling multi-métriques

On crée ici deux politiques indépendantes sur le même ASG : l'une réagit au CPU, l'autre au débit réseau. AWS les évalue en parallèle et déclenche le scaling dès que l'une d'elles est satisfaite.

```
1  # Politique 1 : Scale out si CPU > 75% pendant 2 minutes
2  aws autoscaling put-scaling-policy \
3      --auto-scaling-group-name mon-asg \
4      --policy-name cpu-scale-out \
5      --policy-type TargetTrackingScaling \
6      --target-tracking-configuration '{
7          "TargetValue": 75.0,
8          "PredefinedMetricSpecification": {
9              "PredefinedMetricType": "ASGAverageCPUUtilization"
10         },
11         "ScaleOutCooldown": 120,
12         "ScaleInCooldown": 300
13     }'
14
15 # Politique 2 : Scale out si Network > 1 Gbps
16 aws autoscaling put-scaling-policy \
17     --auto-scaling-group-name mon-asg \
18     --policy-name network-scale-out \
19     --policy-type TargetTrackingScaling \
20     --target-tracking-configuration '{
21         "TargetValue": 70.0,
```

```

22         "CustomizedMetricSpecification": {
23             "MetricName": "NetworkOut",
24             "Namespace": "AWS/EC2",
25             "Statistic": "Average"
26         }
27     }'

```

:::success **Résultat attendu :**

```

1  # put-scaling-policy (cpu-scale-out) retourne :
2  {
3      "PolicyARN": "arn:aws:autoscaling:eu-west-1:123456789012:scalingPolicy:a1b2
4      "Alarms": [
5          {
6              "AlarmName": "TargetTracking-mon-asg-AlarmHigh-cpu-scale-out",
7              "AlarmARN": "arn:aws:cloudwatch:eu-west-1:123456789012:alarm:Target
8          }
9      ]
10 }
11
12 # put-scaling-policy (network-scale-out) retourne de même avec un ARN différent

```

...

Cooldown Periods (délais entre actions)

- **ScaleOutCooldown** (120-300s) : attend avant la prochaine augmentation.
 - Évite les oscillations rapides (scaling de “ping-pong”).
 - Laisse le temps aux instances de démarrer.
- **ScaleInCooldown** (300-900s) : plus long que scale-out.
 - Garantit l'équilibre avant réduction.
 - Préserve la performance en cas de pics rapides.

Exemple réaliste :

```

1  T=0s      CPU = 80% → Déclenche scale-out (+2 instances)
2  T=120s    Instances démarrent (cool-down scale-out)
3  T=180s    CPU = 60% → Pourrait déclencher scale-in MAIS...
4  T=240s    Attendre le cooldown scale-in
5  T=540s    CPU toujours < 30% → Scale-in (-1 instance)

```

Conseil : Définir des cooldowns asymétriques (court pour scale-out, long pour scale-in) pour favoriser la disponibilité.

 [Auto Scaling EC2](#)

9.5 Avantages combinés Load Balancer + Auto Scaling

- **Résilience** : les instances défaillantes sont automatiquement **remplacées**.
- **Scalabilité** : le nombre d'instances s'adapte à la **charge** en temps réel.
- **Performance** : le trafic est réparti de manière **optimale** entre les ressources disponibles.
- **Économie** : vous payez uniquement pour les ressources utilisées.
- **Sécurité** : le Load Balancer peut gérer les **certificats SSL/TLS** pour sécuriser les communications.

10. AWS Lambda — Le calcul sans serveur

10.1 Pourquoi Lambda, quand on a déjà EC2 et Auto Scaling ?

Vous venez de voir comment EC2 et Auto Scaling permettent d'adapter dynamiquement une flotte de serveurs à la charge. Mais même avec Auto Scaling, une instance EC2 minimale **tourne en permanence** — vous la payez même quand elle ne traite aucune requête.

AWS Lambda pousse le modèle serverless plus loin : au lieu de faire tourner un serveur en continu, vous déployez une **fonction** — un bloc de code — qu'AWS exécute uniquement quand un événement le déclenche (requête HTTP, fichier déposé sur S3, message dans une file, tâche planifiée...). Entre deux exécutions, **aucune ressource ne tourne, donc rien n'est facturé**.

	EC2 (même avec Auto Scaling)	Lambda
Ce que vous gérez	OS, runtime, mises à jour, capacité	Uniquement votre code
Facturation	À l'heure/seconde tant que l'instance tourne	À l'exécution (durée × mémoire allouée)
Charge nulle	Coût minimal non nul (au moins 1 instance)	0 \$ — aucune exécution, aucun coût

Démarrage	Minutes (boot instance) ou secondes (déjà démarrée)	Millisecondes à quelques secondes (cold start)
Durée d'exécution max	Illimitée	15 minutes par exécution

10.2 Fonctionnement d'une fonction Lambda

Une fonction Lambda est un paquet de code (Python, Node.js, Java, Go, etc.) associé à une configuration : mémoire allouée (128 Mo à 10 Go), timeout maximal, et un ou plusieurs **triggers** — les événements qui la déclenchent.



Le CPU alloué est proportionnel à la mémoire configurée — une fonction à 1 769 Mo de RAM obtient l'équivalent d'un vCPU complet. AWS gère entièrement l'infrastructure sous-jacente : vous ne choisissez ni AMI, ni type d'instance, ni Security Group pour la fonction elle-même.

10.3 Créer et invoquer une fonction Lambda en CLI

:::info Une activité pratique permet d'approfondir la création et l'invocation de fonctions Lambda. :::

Cette séquence crée une fonction Lambda Python minimale, l'invoque manuellement, puis vérifie les logs d'exécution dans CloudWatch.

```

1 # 1. Écrire le code de la fonction (fichier local)
2 cat > lambda_function.py << 'EOF'
3 def lambda_handler(event, context):
4     nom = event.get('nom', 'monde')
5     return {
6         'statusCode': 200,
7         'body': f'Bonjour, {nom} ! Fonction exécutée avec succès.'
8     }
9 EOF

```

```
10
11 # 2. Empaqueter le code en ZIP (format attendu par Lambda)
12 zip function.zip lambda_function.py
13
14 # 3. Créer le rôle IAM que la fonction va assumer (permissions d'exécution)
15 aws iam create-role \
16     --role-name formation-lambda-role \
17     --assume-role-policy-document '{
18     "Version": "2012-10-17",
19     "Statement": [{
20         "Effect": "Allow",
21         "Principal": {"Service": "lambda.amazonaws.com"},
22         "Action": "sts:AssumeRole"
23     }]
24     }'
25
26 # 4. Attacher la policy minimale pour écrire les logs CloudWatch
27 aws iam attach-role-policy \
28     --role-name formation-lambda-role \
29     --policy-arn arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole
30
31 # 5. Créer la fonction Lambda
32 aws lambda create-function \
33     --function-name formation-bonjour \
34     --runtime python3.12 \
35     --role arn:aws:iam::123456789012:role/formation-lambda-role \
36     --handler lambda_function.lambda_handler \
37     --zip-file fileb://function.zip \
38     --memory-size 128 \
39     --timeout 10
40
41 # 6. Invoquer la fonction avec un événement de test
42 aws lambda invoke \
43     --function-name formation-bonjour \
44     --payload '{"nom": "Formation AWS"}' \
45     --cli-binary-format raw-in-base64-out \
46     reponse.json
47
48 cat reponse.json
```


:::success **Résultat attendu :**

```
1 {
2   "StatusCode": 200,
3   "ExecutedVersion": "$LATEST"
4 }
```

Contenu de `reponse.json` :

```
1 { "statusCode": 200, "body": "Bonjour, Formation AWS ! Fonction exécutée avec su
```

La fonction s’est exécutée en quelques centaines de millisecondes. Aucune instance EC2 n’a été provisionnée — Lambda a alloué l’environnement d’exécution le temps de traiter cette seule invocation, puis l’a libéré. :::

```
1 # 7. Consulter les logs d'exécution (CloudWatch Logs, créés automatiquement)
2 aws logs tail /aws/lambda/formation-bonjour --follow
```

:::info **Le rôle IAM est la seule “sécurité réseau” de Lambda par défaut.** Contrairement à EC2, une fonction Lambda n’a pas de Security Group tant qu’elle n’est pas explicitement rattachée à un VPC (`--vpc-config`). Une Lambda simple qui n’a besoin que d’appeler d’autres services AWS (S3, DynamoDB) via leurs API n’a généralement pas besoin d’être dans un VPC — le rôle IAM suffit à contrôler ce qu’elle a le droit de faire. :::

10.4 Comment estimer le coût de Lambda ?

Lambda facture principalement le **nombre de requêtes** et la **durée d’exécution pondérée par la mémoire allouée**. D’autres postes peuvent s’ajouter : concurrence provisionnée, stockage éphémère supplémentaire, journaux CloudWatch, transfert réseau et services déclencheurs.

```
1 consommation de calcul = nombre d'exécutions × durée moyenne × mémoire allouée
2 coût total = requêtes + calcul + options Lambda + services associés
```

Cette formule explique le modèle sans figer un tarif. Pour comparer Lambda à EC2, utilisez le même trafic et la même exigence de disponibilité : Lambda convient souvent aux charges intermittentes, tandis qu’une capacité durable correctement dimensionnée peut être plus économique pour une charge continue. Le résultat doit être confirmé dans AWS Pricing Calculator.

 [AWS Lambda Pricing](#)  [AWS Lambda Documentation](#) 

11. Architecture complète : Illustration e-commerce

Scénario réaliste — montée en charge pendant les soldes, puis retour à la normale :

11.1 Avant les soldes (charge normale)

Les clients normaux passent par l'Application Load Balancer, qui répartit le trafic sur trois instances EC2 (EC2-1, EC2-2, EC2-3). L'Auto Scaling Group est configuré avec Min=2, Max=10, Desired=3.

11.2 Pendant les soldes (pic de trafic)

Le trafic est multiplié par 5, le CPU moyen passe à 85% — cela déclenche le scale-out : 4 instances supplémentaires sont ajoutées derrière l'Application Load Balancer, portant le total à 7 instances actives (Desired Capacity = 7).

11.3 Après les soldes (retour à la normale)

- 1 Trafic revient à la normale
- 2
- 3 CPU chute à 40%
- 4
- 5 Déclenchement du scale-in
- 6
- 7 Retirer 4 instances
- 8
- 9 Desired Capacity = 3
- 10 Coûts réduits

12. Bonnes pratiques — Architecture hautement disponible

12.1 Architecture résiliente S3

Multi-région :

- S3 est déjà multi-AZ au sein d'une région.
- Pour une résilience maximale, activer la **réplication cross-région** (CRR).
- Utile pour respecter des exigences de conformité (RGPD, etc.).

Versioning + Lifecycle :

- Toujours activer le versioning sur les buckets de production.
- Combiner avec des règles de cycle de vie pour éviter les coûts exponentiels.
- Exemple : garder 30 versions actives, archiver le reste en Glacier.

Monitoring et alertes :

- CloudWatch pour surveiller les métriques S3.
- CloudTrail pour auditer les accès et modifications.
- S3 Access Analyzer pour vérifier les politiques d'accès.

12.2 Architecture résiliente EC2

Load Balancer + Auto Scaling minimum :

- Toujours au minimum 2 instances (haute disponibilité).
- Répartir sur **plusieurs zones de disponibilité (AZ)**.
- Configurer les health checks correctement.

Sécurité en couches :

- Security Groups : bloquer les ports inutiles.
- IAM Roles : donner uniquement les permissions nécessaires.
- Subnets privés pour les instances sans accès Internet.

Sauvegardes :

- Snapshots EBS réguliers (quotidiens ou hebdomadaires).
- Externaliser les sauvegardes sur S3.
- Tester la restauration régulièrement !

12.3 Optimisation des coûts

S3 Coûts :

- Utiliser **Intelligent-Tiering** si l'accès est imprévisible.

- Activer les **lifecycle policies** agressives pour archiver.
- Monitorer la bande passante (les téléchargements hors AWS coûtent cher).

⚠ **Attention aux postes de coût S3** — Le stockage n'est qu'une partie de la facture. Les requêtes, les transitions de classe, la récupération d'archives, la surveillance des objets et le transfert sortant peuvent devenir dominants. Pour un lac de données composé de nombreux petits objets, comptez les opérations autant que les gigaoctets. Utilisez **Cost Explorer** et les rapports de coûts pour identifier les postes réels. ⚠

EC2 Coûts :

- Étudier les **Savings Plans** pour les charges stables, après analyse de l'utilisation réelle.
- **Spot** pour les job batch ou CI/CD tolérants aux interruptions.
- AWS Compute Optimizer pour dimensionner correctement.
- Éteindre les ressources de dev/test en fin de journée.

Estimateur AWS :

- Utiliser le **Pricing Calculator** pour estimer les coûts futurs.
- Vérifier les coûts inattendus via la **Cost Explorer**.

12.4 Performance et scalabilité

S3 Performance :

- Utiliser des **préfixes intelligents** pour éviter les goulots d'étranglement (ex. `2024/03/24/log-xxxxx`).
- Activer **S3 Transfer Acceleration** pour les uploads volumineux (MultiPart Upload).
- CloudFront comme CDN pour la distribution worldwide.

EC2 Performance :

- **Monitoring continu** : CPU, mémoire, réseau, disque.
- **Auto Scaling** sur **multiple métriques** : CPU, mémoire, débit réseau.
- Usar **Read Replicas** pour les bases de données.
- **Connexion pooling** pour les applications critiques.

13. Conformité et sécurité pour les données sensibles

Les environnements soumis à des réglementations (RGPD, HIPAA, PCI-DSS) nécessitent des garanties strictes.

13.1 Frameworks de conformité AWS

Framework	Objectif	Services AWS applicables
RGPD	Protection des données personnelles UE	Encryption, Data Residency, CloudTrail
HIPAA	Confidentialité des données santé	Dedicated Instance, Encrypted EBS, Audit logs
PCI-DSS	Sécurité des données cartes bancaires	VPC isolé, Encryption, Firewall
ISO 27001	Gestion de la sécurité informatique	IAM, KMS, CloudTrail, Monitoring

13.2 Bonnes pratiques de conformité pour S3

-  Chiffrement : SSE-KMS (clés maîtrisées)
-  Versioning : actif (trace des modifications)
-  Bucket Policy : restreint à IP/domaine
-  Logging : S3 Access Logs dans un bucket séparé
-  CloudTrail : audit API dans le compte AWS
-  MFA Delete : protection contre la suppression
-  Block Public : tous les accès publics bloqués
-  Lifecycle : archivage des données obsolètes
-  Réplication CRR : backup multi-région

13.3 Bonnes pratiques de conformité pour EC2

-  Dedicated Instance : pas de partage d'hôte physique
-  EBS chiffré : SSE-KMS pour tous les volumes
-  Security Group : minimaliste (moindre privilège)
-  IAM Role : permissions spécifiques au rôle
-  CloudWatch Agent : logs applicatifs
-  VPC privé : pas d'accès Internet direct

-  Snapshots EBS : conservés X années
-  Patch Management : système à jour
-  Monitoring : alertes sur anomalies

13.4 Exemple : Architecture RGPD multi-région

- 1 Région EU (Ireland)
- 2 - VPC Privé
- 3 - Subnets privés (applications)
- 4 - Subnet public (NAT Gateway seulement)
- 5 - Security Group très restrictif
- 6 - S3 Bucket
- 7 - Versioning activé
- 8 - SSE-KMS (clé EU managée)
- 9 - Bucket Policy : IP/IAM restrictifs
- 10 - CloudTrail logging
- 11 - EC2 Instances
- 12 - Dedicated Instance
- 13 - EBS chiffré KMS
- 14 - Snapshots quotidiens → S3
- 15 - CloudWatch + CloudTrail
- 16
- 17 Région EU (Frankfurt) – Backup
- 18 - S3 Réplication CRR du bucket EU-Ireland
- 19 - Préservé 7 ans (conformité)

13.5 Audit et certification

AWS Artifact : plateforme d’AWS pour les certifications de conformité. Les rapports téléchargés (SOC 2, ISO 27001) servent à prouver à vos clients ou auditeurs qu’AWS respecte les normes de sécurité.

- 1 # Dans la console AWS → Security, Identity & Compliance → Artifact
- 2 # Télécharger :
- 3 # - AWS Compliance Summary
- 4 # - SOC 2 Type II reports
- 5 # - ISO 27001 certificates
- 6 # Utile pour les audits externes

CloudTrail pour l'audit — ces commandes permettent de lister les trails actifs et de rechercher les actions effectuées sur une ressource précise (ici, un bucket S3) :

```
1 aws cloudtrail describe-trails --region eu-west-1
2 aws cloudtrail list-events \
3     --region eu-west-1 \
4     --max-results 50 \
5     --lookup-attributes AttributeKey=ResourceName,AttributeValue=mon-bucket
```

:::success **Résultat attendu :**

```
1  # describe-trails retourne :
2  {
3      "trailList": [
4          {
5              "Name": "management-events-trail",
6              "S3BucketName": "my-cloudtrail-logs-bucket",
7              "IncludeGlobalServiceEvents": true,
8              "IsMultiRegionTrail": true,
9              "HomeRegion": "eu-west-1",
10             "TrailARN": "arn:aws:cloudtrail:eu-west-1:123456789012:trail/manage
11             "LogFileValidationEnabled": true
12         }
13     ]
14 }
15
16 # list-events retourne des événements du type :
17 {
18     "Events": [
19         {
20             "EventId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
21             "EventName": "PutObject",
22             "ReadOnly": "false",
23             "EventTime": "2024-05-17T14:23:05+00:00",
24             "Username": "stagiaire-demo",
25             "Resources": [
26                 {
27                     "ResourceType": "AWS::S3::Object",
28                     "ResourceName": "mon-bucket/documents/rapport.pdf"
29                 }
30             ]
31         }
32     ]
33 }
```

```

30         ]
31     }
32 ]
33 }
```

...

14. Points importants et pièges fréquents

Piège	Réalité	Conséquence
S3 a une structure de dossiers	Non ! C'est du stockage objet, les "dossiers" sont juste des préfixes dans les noms	Impossible de renommer les dossiers, penser en clés, pas en hiérarchies
Versioning S3 ne prend pas de place supplémentaire	Faux ! Chaque version est stockée complètement	Les coûts explosent vite si vous versionnez des fichiers volumineux
Les instances EC2 garderont leurs données après arrêt	Seulement si vous utilisez EBS persistant	Les données en instance store (stockage éphémère) sont perdues à l'arrêt
On-Demand est toujours la meilleure option tarifaire	Non : sa flexibilité se paie, mais elle évite un engagement inadapté	Comparer paiement à la demande, engagements et Spot avec la charge réelle
Auto Scaling remplace les instances défaillantes instantanément	Non, il faut le temps de démarrage (2-5 min)	Configurer les health checks correctement et accepter un délai
Toute IP EC2 est durable	Non, les IPs publiques changent à l'arrêt/redémarrage	Utiliser Elastic IP pour les IPs stables ou les DNS

Un Security Group “ouvert”
(0.0.0.0/0) sur tous les ports
est OK si la machine n’a rien
à cacher

Non ! C’est une faille
de sécurité

Les scanners de ports
peuvent découvrir la
machine, minimiser
l’exposition

EBS et S3 sont
interchangeables

Non ! EBS est un
disque (bloc), S3 est du
stockage objet

Choisir le bon service
selon le cas d’usage

15. Ressources

Documentation officielle AWS

- [AWS Compute Optimizer](#) 
- [CloudTrail Documentation](#) 
- [AWS Compliance](#) 
- [Artifact Console](#) 
- [S3 Intelligent-Tiering](#) 
- [S3 Transfer Acceleration](#) 
- [Bucket Policies Guide](#) 
- [AWS Lambda Documentation](#) 

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les questions et les propositions restent dans un ordre stable.

Le quiz interactif est disponible dans la version web du support.

Chapitre 4 — Amazon VPC et bases de données AWS

Vocabulaire

1 · Bases de données managées

2 · Réseau privé avec VPC

3 · DNS avec Route 53

4 · Mise en cache avec ElastiCache

5 · Points d'attention

Ressources

Quiz du chapitre

Vocabulaire du chapitre

Terme	Définition

Base relationnelle	Base organisée en tables liées et interrogée généralement avec SQL.
NoSQL	Famille de modèles non relationnels, par exemple clé-valeur ou document, conçus pour des accès spécifiques et une distribution horizontale.
RDS	Relational Database Service : service AWS managé pour plusieurs moteurs de bases relationnelles.
Aurora	Moteur relationnel managé par AWS, compatible avec MySQL ou PostgreSQL selon l'édition choisie.
DynamoDB	Base NoSQL clé-valeur et document entièrement managée par AWS.
VPC	Virtual Private Cloud : réseau virtuel logiquement isolé dans AWS.
CIDR	Notation qui décrit un bloc d'adresses IP au moyen d'une adresse réseau et d'une longueur de préfixe.
Subnet	Sous-réseau d'un VPC, limité à une seule zone de disponibilité.
Table de routage	Ensemble de règles qui détermine la prochaine destination d'un paquet réseau.

:::info Les instances EC2 et les buckets S3 déployés au chapitre précédent doivent maintenant s'intégrer dans un réseau maîtrisé et s'appuyer sur des bases de données managées : ce chapitre couvre les deux piliers d'une architecture AWS mature, le réseau (VPC) et la donnée persistante (RDS, Aurora, DynamoDB).

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Expliquer** l'intérêt des bases de données managées face à une base auto-administrée
- **Déployer** une base Amazon RDS en haute disponibilité (Multi-AZ) et en sécuriser l'accès
- **Différencier** Amazon Aurora d'une base RDS classique en termes de performance et de résilience
- **Utiliser** Amazon DynamoDB pour un cas d'usage NoSQL à forte scalabilité
- **Planifier** une migration de base de données avec AWS DMS
- **Concevoir** une VPC avec subnets publics et privés, table de routage et passerelle Internet/NAT
- **Sécuriser** le trafic réseau avec des Security Groups et des Network ACLs
- **Interconnecter** plusieurs VPC avec le VPC Peering et AWS Transit Gateway

- **Utiliser** un VPC Endpoint pour accéder à un service AWS sans transiter par Internet
- **Configurer** une zone DNS et des enregistrements avec Amazon Route 53, dont des politiques de routage avancées
- **Mettre en place** un cluster ElastiCache (Redis) pour accélérer l'accès aux données fréquemment lues :::

1. Bases de données dans AWS — Du service géré à la scalabilité

1.1 La révolution des bases managées

 Vidéo : [AWS Database Services Overview](#) 

Quand on parle de **bases de données dans le Cloud**, la question centrale n'est plus « Où vais-je installer un serveur ? » mais plutôt « Quel type de données vais-je stocker, et quel accès dois-je offrir ? ».

Dans un environnement traditionnel (**on-premise**), administrer une base de données impliquait :

- **Installation manuelle** du moteur (MySQL, PostgreSQL, Oracle...)
- **Configuration** des paramètres (mémoire, cache, compression...)
- **Sauvegardes régulières** et tests de restauration
- **Maintenance** des patchs de sécurité
- **Réplication** pour la haute disponibilité
- **Surveillance** 24h/24 (CPU, mémoire, disque, connexions)
- **Escalade** : augmenter la taille du disque, du CPU, de la mémoire
- **Gestion des droits** d'accès pour chaque application

Autant de tâches qui détournent les équipes de leur **valeur métier réelle** : créer des applications performantes et sécurisées.

Avec **Amazon RDS** et les services managés AWS, ce paradigme s'inverse : **AWS administre l'infrastructure, vous administrez vos données.**

L'abstraction du matériel

Imaginez une **maison en location** vs. une **maison en propriété** :

- **En propriété** (on-prem) : vous gérez tout — le toit, la plomberie, l'électricité, les réparations. Si la toiture fuit, c'est à vos frais et vos équipes doivent intervenir.

- **En location** (RDS) : le propriétaire assure la structure, le toit, les murs. Vous ne vous occupez que du décor intérieur (vos données).

Avec RDS :

- Vous définissez simplement : quel moteur ? (MySQL, PostgreSQL, Oracle, SQL Server, Aurora) — quelle taille ? (t3.small, m5.xlarge...) — combien de stockage ? (100 Go, 5 To...)
- AWS crée l'instance, configure le stockage EBS, met en place le monitoring, gère les snapshots, applique les patches.
- Vous accédez à votre base via un endpoint standard (`mondb.xxxxx.eu-west-1.rds.amazonaws.com:3306`).

1.2 Amazon RDS — Bases relationnelles managées

Amazon RDS (Relational Database Service) est le service managé AWS pour les **bases de données structurées** (SQL). Il supporte plusieurs moteurs :

Moteur	Compatibilité	Cas d'usage
MySQL	Open source, très répandu	Applications web classiques
PostgreSQL	Open source, très avancé	Applications critiques, PostGIS, données complexes
MariaDB	Fork MySQL, meilleure performance	Alternative MySQL
Oracle	Propriétaire, très coûteux en on-prem	Migrations legacy, applications critiques
SQL Server	Windows, intégration Active Directory	Environnements Microsoft
Amazon Aurora	Natif AWS, ultra-performant	Haute disponibilité, haute scalabilité

Caractéristiques clés

Fonctionnalité	Description
Haute disponibilité	Multi-AZ : réplication synchrone sur une autre zone de disponibilité avec basculement automatique
Sauvegardes automatisées	Sauvegardes quotidiennes, conservées jusqu'à 35 jours, restauration à un instant T
Sécurité intégrée	Chiffrement au repos (AWS KMS) et en transit (SSL/TLS), isolation réseau (VPC, Security Groups), audit (CloudTrail)
Scalabilité verticale	Augmenter CPU/RAM sans interruption (dans certains cas)
Read Replicas	Jusqu'à 5 réplicas de lecture asynchrones pour répartir les lectures
Maintenance automatisée	Patches et mises à jour sans intervention manuelle

Types de stockage EBS pour RDS

RDS s'appuie sur **Amazon EBS** pour son stockage sous-jacent. Vous choisissez le type de disque selon vos besoins :

Type	Performance	Coût	Cas d'usage
General Purpose (gp3)	3 000 à 16 000 IOPS	Modéré	Production standards, dev/test
Provisioned IOPS (io1/io2)	Jusqu'à 64 000 IOPS	Élevé	Bases critiques à fort volume transactionnel
Magnetic (standard)	100-200 IOPS	Bas	Archivage, données froides (déprécié)

IOPS (Input/Output Operations Per Second) = nombre d'opérations de lecture/écriture par seconde. Un IOPS élevé garantit des temps de réponse courts pour les transactions.

Exemple concret — Plateforme de vidéo à la demande (VOD)

Supposons une plateforme qui stocke **les métadonnées** de ses contenus : titres, descriptions, dates de diffusion, catégories, droits d'accès. Les **requêtes sont complexes** (jointures sur plusieurs tables), les données sont structurées, et la **cohérence** est critique.

Avec on-prem (avant cloud) :

- 1 - Installation MySQL manuelle (4h)
- 2 - Scripts de sauvegarde réseau + test restauration (8h)
- 3 - Monitoring 24h/24 avec alertes (contrat SLA)
- 4 - Augmentation disque lors des pics → risque de downtime
- 5 - Réplication vers DataCenter secondaire (investissement)
- 6 → Coût total 5 ans : ~100 000 €, équipe IT 2 personnes

Avec RDS AWS :

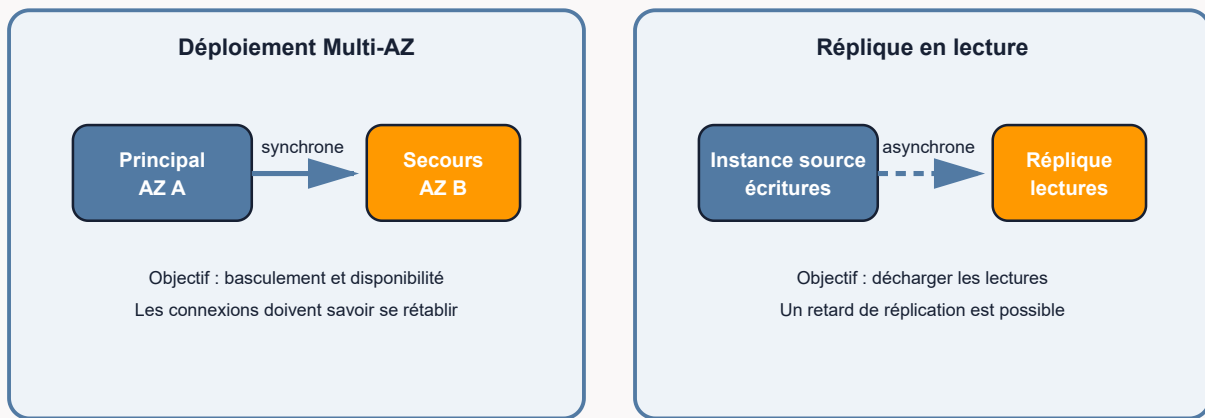
- 1 - Déploiement en 3 minutes via console AWS
- 2 - Option Multi-AZ pour automatiser le basculement, avec une interruption à tolérer
- 3 - Snapshots automatiques (jusqu'à 35 jours)
- 4 - Augmentation CPU/stockage sans interruption
- 5 - Read Replicas pour diffuser les lectures (reports analytiques)
- 6 → Coût annuel : ~1 500 \$, gestion minimal (0,1 FTE)

1.3 Haute disponibilité et résilience dans RDS

Multi-AZ (Availability Zones) — Résilience automatique

Dans un déploiement RDS Multi-AZ avec une instance de secours, les modifications du principal sont répliquées de manière synchrone vers une autre zone de disponibilité. AWS peut basculer vers cette instance lors de certains incidents ou opérations de maintenance. Les autres variantes Multi-AZ peuvent utiliser plusieurs instances lisibles : il faut vérifier le comportement du moteur et du type de déploiement choisis.

Disponibilité et montée en charge ne répondent pas au même besoin



Lecture du schéma. À gauche, la réplication synchrone et le basculement visent la disponibilité. À droite, la réplication asynchrone permet d'envoyer des lectures vers un autre endpoint, avec un retard possible. Une réplique en lecture ne remplace donc pas automatiquement un déploiement Multi-AZ.

Bénéfice : la réplication synchrone réduit le risque de perte de données et le basculement évite une reconstruction manuelle complète. **À savoir :** le nom de l'endpoint reste stable, mais les connexions en cours sont interrompues. L'application doit savoir se reconnecter et tolérer le délai de basculement.

⚠ Coût Multi-AZ RDS — Attention au budget

Une configuration Multi-AZ ajoute des ressources et augmente donc la facture. Selon le moteur et le type de déploiement, l'architecture et la facturation diffèrent : instance de secours non lisible ou cluster comportant plusieurs instances. Ne déduisez pas le prix avec un coefficient générique.

Décision : activez Multi-AZ lorsque l'objectif de disponibilité le justifie, y compris hors production si l'environnement doit tester les basculements. Comparez les options dans la console ou dans AWS Pricing Calculator. ⚠

Read Replicas — Répartition de la charge de lecture

Un **Read Replica** est une **copie asynchrone** de votre base, destinée à répartir les **lectures** (SELECT) sans surcharger la Primary.

Cas d'usage : Reporting, Analytics, Exports.

Une instance source peut répliquer ses modifications vers une ou plusieurs répliques en lecture, selon les capacités et quotas du moteur. L'application utilise l'endpoint propre à chaque réplique pour les requêtes de lecture. Le retard de réplication doit être surveillé : une lecture immédiate après une écriture peut ne pas encore voir la nouvelle valeur.

Différence clé Read Replica vs Multi-AZ :

- **Multi-AZ** : mécanisme de disponibilité et de basculement ; les possibilités de lecture dépendent du type de déploiement.
- **Read Replica** : réplication asynchrone, endpoint séparé et retard variable ; la promotion éventuelle doit être intégrée au plan de reprise.

Commande AWS CLI :

```
1  # Créer un Read Replica pour reportings
2  aws rds create-db-instance-read-replica \
3      --db-instance-identifiant formation-db-analytics \
4      --source-db-instance-identifiant formation-db \
5      --db-instance-class db.t3.small \
6      --availability-zone eu-west-1b
7
8  # Créer un Replica inter-région (DR)
9  aws rds create-db-instance-read-replica \
10     --db-instance-identifiant formation-db-dr \
11     --source-db-instance-identifiant formation-db \
12     --source-region eu-west-1 \
13     --region us-east-1 \
14     --db-instance-class db.t3.small
```

:::success **Résultat attendu :**

```
1  {
2      "DBInstance": {
3          "DBInstanceIdentifier": "formation-db-analytics",
4          "DBInstanceClass": "db.t3.small",
5          "Engine": "mysql",
6          "DBInstanceStatus": "creating",
7          "ReadReplicaSourceDBInstanceIdentifier": "formation-db",
8          "AvailabilityZone": "eu-west-1b",
9          "MultiAZ": false,
10         "StorageType": "gp3",
11         "Endpoint": {
12             "Address": "formation-db-analytics.abc123.eu-west-1.rds.amazonaws.c
13             "Port": 3306
14         }
15     }
16 }
```

1.4 Sécurité et supervision RDS

Chiffrement

Type	Description
Au repos	Les données sur disque EBS sont chiffrées via AWS KMS
En transit	TLS à configurer et, selon le moteur, à imposer aux connexions clientes

⚠ **Important** : on ne transforme pas directement une instance RDS non chiffrée en instance chiffrée par un simple interrupteur. Le parcours habituel consiste à créer une copie chiffrée d'un snapshot puis à restaurer une nouvelle instance, avec une stratégie de bascule adaptée.

Supervision et alertes

Outil	Rôle
CloudWatch	Métriques de base (CPU, RAM, connexions, IOPS)
Performance Insights	Requêtes coûteuses, sessions actives
CloudTrail	Audit : qui a modifié la configuration
Enhanced Monitoring	Metrics granulaires de l'OS

1.5 Amazon Aurora — Performance et résilience supérieures

Amazon Aurora est le **moteur de base de données propriétaire AWS**, conçu d'emblée pour le Cloud. Compatible avec **MySQL** et **PostgreSQL** mais offre des performances bien supérieures.

Architecture distribuée

Aurora découple le **calcul** (instances) du **stockage** (volume distribué). Les écritures sont répliquées **4 fois** sur 3 AZ.

Performances

Métrique	vs MySQL	vs PostgreSQL
Throughput max	5×	3×
Basculement automatique	30 sec	30 sec
Réplicas de lecture	15 (vs 5)	15 (vs 5)

Comparaison détaillée : RDS MySQL/PostgreSQL vs Aurora

Aspect	RDS MySQL/PostgreSQL	Aurora
Architecture	Stockage EBS couplé à l'instance	Calcul découplé du stockage distribué
Réplication	Synchrone (Multi-AZ) / Asynchrone (Replicas)	4 copies sur 3 AZ (natif)
Failover automatique	30 sec si Multi-AZ activé	30 sec (inclus, pas surcoût)
Réplicas de lecture	5 max	15 max
Coût Multi-AZ	+50 % sur l'instance	0 € (inclus)
Scaling en écriture	Impossible (Master unique)	Impossible (Master unique)
Scaling en lecture	Avec Replicas (asynchrones)	Avec Replicas (synchrone, <1ms latence)
Serverless	Non (ne s'adapte pas)	Oui (Aurora Serverless v2)
Performance requêtes complexes	Bonne	Excellente (optimisation native AWS)
Coût stockage	Paiement par Go alloué	Paiement à l'utilisation (auto-scaling)
Certification AWS	Sujet SAA-C03	Sujet SAA-C03 (fortement recommandé)

SAA-C03 est le code de l'examen AWS Certified Solutions Architect – Associate, la certification détaillée au Chapitre 1 (section 1.3) et reprise au Chapitre 5 (section 9) — le comparatif RDS/Aurora ci-dessus fait partie des sujets fréquemment évalués dans cet examen.

Mode Serverless et Provisioned

Aurora offre **deux modèles de déploiement** :

Modèle	Principe	Avantages	Cas d'usage
Provisioned (classique)	Vous choisissez la taille (ex. <code>db.r6g.xlarge</code>)	Coût fixe, performance prévisible	Application de reporting critique, charge stable (12h/jour)
Serverless V2 (moderne)	Scaling auto de 0.5 à 1000+ ACU (Aurora Compute Units)	Paiement à l'utilisation, scaling en $\approx$ 5 sec	API imprévisible, pics aléatoires, environnements de développement

Créer un cluster Aurora en CLI

:::info Une activité pratique permet d'approfondir le déploiement d'un cluster Aurora. :::

Cette séquence crée un cluster Aurora MySQL complet avec une instance writer, une instance reader, du chiffrement activé et une fenêtre de sauvegarde automatique. Aurora est un cluster — pas une instance unique — ce qui explique les deux commandes distinctes (cluster + instance).

```
1 # 1. Créer un Aurora MySQL Cluster (2 instances : writer + reader)
2 aws rds create-db-cluster \
3   --db-cluster-identifier formation-aurora-cluster \
4   --engine aurora-mysql \
5   --engine-version 8.0.mysql_aurora.3.02.0 \
6   --master-username admin \
7   --master-user-password '<MOT_DE_PASSE_FOURNI_HORS_DU_FICHER>' \
8   --database-name appdb \
9   --vpc-security-group-ids sg-12345678 \
10  --db-subnet-group-name my-db-subnet-group \
11  --storage-encrypted \
12  --backup-retention-period 7 \
13  --preferred-backup-window "03:00-04:00" \
```

```
14 --preferred-maintenance-window "mon:04:00-mon:05:00" \  
15 --tag-specifications 'ResourceType=cluster,Tags=[{Key=Name,Value=FormationAur  
16  
17 # 2. Créer les instances Aurora (Writer)  
18 aws rds create-db-instance \  
19 --db-instance-identifiant formation-aurora-writer \  
20 --db-instance-class db.r6g.large \  
21 --engine aurora-mysql \  
22 --db-cluster-identifiant formation-aurora-cluster \  
23 --publicly-accessible false  
24  
25 # 3. Créer instance Reader (lecture seulement)  
26 aws rds create-db-instance \  
27 --db-instance-identifiant formation-aurora-reader \  
28 --db-instance-class db.r6g.large \  
29 --engine aurora-mysql \  
30 --db-cluster-identifiant formation-aurora-cluster \  
31 --promotion-tier 2 \  
32 --publicly-accessible false  
33  
34 # 4. Créer Aurora Serverless V2 (auto-scaling)  
35 aws rds create-db-cluster \  
36 --db-cluster-identifiant formation-aurora-serverless \  
37 --engine aurora-postgresql \  
38 --engine-version 14.6 \  
39 --engine-mode provisioned \  
40 --serverlessv2-scaling-configuration 'MinCapacity=0.5,MaxCapacity=16' \  
41 --master-username admin \  
42 --master-user-password '<MOT_DE_PASSE_FOURNI_HORS_DU_FICHER>' \  
43 --database-name appdb \  
44 --vpc-security-group-ids sg-12345678 \  
45 --db-subnet-group-name my-db-subnet-group  
46  
47 # 5. Attendre disponibilité  
48 aws rds wait db-cluster-available \  
49 --db-cluster-identifiant formation-aurora-cluster  
50  
51 # 6. Récupérer les endpoints  
52 aws rds describe-db-clusters \  
53 --db-cluster-identifiant formation-aurora-cluster \  

```

```

54 --query 'DBClusters[0].[DBClusterEndpoint,ReaderEndpoint]'
55 # Résultat :
56 # Writer : formation-aurora-cluster.xxxx.eu-west-1.rds.amazonaws.com (écritures
57 # Reader : formation-aurora-cluster-ro.xxxx.eu-west-1.rds.amazonaws.com (lectur
58
59 # 7. Modifier le scaling Serverless (augmenter capacité max)
60 aws rds modify-db-cluster \
61 --db-cluster-identifiant formation-aurora-serverless \
62 --serverlessv2-scaling-configuration 'MinCapacity=1,MaxCapacity=32' \
63 --apply-immediately
64
65 # 8. Ajouter un Read Replica dans une autre région
66 aws rds create-db-cluster \
67 --db-cluster-identifiant formation-aurora-dr \
68 --engine aurora-mysql \
69 --source-region eu-west-1 \
70 --region us-east-1 \
71 --replication-source-identifiant arn:aws:rds:eu-west-1:123456789012:cluster:fo

```

:::success **Résultat attendu :**

```

1  {
2    "DBCluster": {
3      "DBClusterIdentifier": "formation-aurora-cluster",
4      "Status": "creating",
5      "Engine": "aurora-mysql",
6      "EngineVersion": "8.0.mysql_aurora.3.02.0",
7      "DBClusterEndpoint": "formation-aurora-cluster.cluster-abc123.eu-west-1
8      "ReaderEndpoint": "formation-aurora-cluster.cluster-ro-abc123.eu-west-1
9      "MultiAZ": true,
10     "StorageEncrypted": true,
11     "BackupRetentionPeriod": 7
12   }
13 }

```

...

Résultat attendu : `create-db-cluster` retourne le JSON du cluster (état `creating`). `rds wait db-cluster-available` bloque jusqu'à ce que le cluster soit prêt (peut prendre 5-10 min). `describe-db-clusters` affiche les endpoints writer et reader — deux URLs distinctes.

À retenir : Aurora et les moteurs RDS classiques répondent à des contraintes différentes. Le choix dépend du moteur compatible, du profil d'entrées-sorties, de la disponibilité, de la capacité, des fonctions attendues et du coût total.

Comparer le coût de RDS et d'Aurora

Poste à mesurer	RDS	Aurora
Calcul	Classe et nombre d'instances, durée de fonctionnement	Instances provisionnées ou capacité Aurora Serverless v2
Stockage	Volume provisionné, type de stockage et performances configurées	Volume consommé et configuration choisie
Haute disponibilité	Déploiement mono-AZ ou Multi-AZ	Stockage distribué ; nombre d'instances à définir pour la disponibilité du calcul
Entrées-sorties	Dépend du type de stockage et de sa configuration	Dépend de l'édition de tarification Aurora choisie
Sauvegarde et transfert	Rétention, snapshots, copies et transferts	Rétention, snapshots, copies et transferts

Méthode : mesurer la charge, construire les deux variantes dans AWS Pricing Calculator, puis tester les performances et le basculement. La mention « serverless » ne signifie ni arrêt total automatique dans toutes les configurations, ni coût nul au repos.

 [AWS RDS Pricing](#)  [AWS Aurora Pricing](#)

1.6 Amazon DynamoDB — NoSQL à ultra-haute scalabilité

DynamoDB est une **base NoSQL managée** de type clé-valeur et document. Les performances dépendent notamment de la conception des clés, de la taille des items, du mode de capacité et de la distribution de la charge. AWS la conçoit pour fournir une latence de l'ordre de la milliseconde à grande échelle, sous réserve d'un modèle de données adapté.

Structure d'une table DynamoDB

- 1 Chaque attribut peut être :
- 2 - Chaîne (S)
- 3 - Nombre (N)
- 4 - Binaire (B)
- 5 - Ensemble (SS, NS, BS)
- 6 - Map (document imbriqué)
- 7 - Liste
- 8
- 9 Aucune contrainte de schéma : vous pouvez ajouter des attributs par ligne.

Modèles de tarification

Modèle	Débit garanti	Facturation	Cas d'usage
Provisionné	À définir (RCU/WCU)	Fixe + dépassement	Charge prédictible
À la demande	Capacité ajustée par le service, soumise aux quotas et aux partitions	Selon les requêtes traitées	Charge variable

- **RCU** (Read Capacity Unit) : 1 RCU = une lecture fortement cohérente d'un item ≤ 4 Ko
- **WCU** (Write Capacity Unit) : 1 WCU = une écriture d'un item ≤ 1 Ko

Méthode d'estimation : comptez les lectures et écritures, leur taille, le niveau de cohérence, le stockage, les sauvegardes et les index secondaires. Le mode à la demande suit l'activité ; le mode provisionné réserve une capacité et peut utiliser l'auto-scaling. Le meilleur choix dépend du profil réel et doit être recalculé avec le tarif régional courant.

 [Amazon DynamoDB Pricing](#)

DynamoDB vs RDS

Aspect	RDS (SQL)	DynamoDB (NoSQL)
Schéma	Structuré, relationnel	Flexible, semi-structuré
Requêtes	Complexes (jointures)	Simple (accès par clé)

Scalabilité	Verticale surtout	Horizontale, ultra-massive
Latence	ms-s	ms

⚠️ Hot Partitions DynamoDB — Erreur de conception fréquente

DynamoDB distribue vos données sur des partitions selon la **Partition Key**. Si vous choisissez une clé avec peu de valeurs distinctes (ex. : `status = "active"/"inactive"`, ou une date comme `2026-05-17`), toutes les requêtes frappent la **même partition** → throttling, latence explosive.

Symptômes : erreurs `ProvisionedThroughputExceededException`, latences P99 > 500ms.

Solutions :

- Choisir une Partition Key à **haute cardinalité** (UUID utilisateur, ID produit unique)
- Ajouter un **suffixe aléatoire** (write sharding) : `user_id#1`, `user_id#2`
- Utiliser un **Global Secondary Index** avec une clé mieux distribuée
- Passer en mode **On-Demand** pour absorber les pics sans throttling ⚡

1.7 Migration de bases de données avec AWS DMS

AWS DMS (Database Migration Service) est un service managé de déplacement et de réplication de données entre des moteurs pris en charge. Il peut effectuer une copie initiale, répliquer ensuite les changements ou combiner les deux. DMS réduit la durée d'indisponibilité potentielle, mais ne garantit pas une migration sans interruption : la bascule applicative, la validation et le retour arrière restent à concevoir.

Architecture de migration DMS

Une migration s'appuie sur un endpoint source, un endpoint cible et une configuration de réplication ou une instance de réplication selon le mode DMS choisi. La connectivité réseau, les autorisations, les types de données compatibles et la capacité doivent être validés avant le transfert.

Processus de migration par étapes

Phase 1 — Full Load (copie complète) : DMS copie les tables sélectionnées. Les objets de schéma, index, contraintes, procédures et fonctions ne sont pas tous recréés de la même manière ; une migration hétérogène peut nécessiter AWS Schema Conversion Tool ou une conversion manuelle.

Phase 2 — CDC (Change Data Capture) : pour les moteurs compatibles, DMS lit les journaux de transactions de la source et applique les changements à la cible. La latence varie avec la charge, le

réseau, la capacité de réplication et la cible ; elle doit être surveillée.

Phase 3 — Cutover (basculement applicatif) :

1. Arrêter ou geler les écritures selon la stratégie retenue.
2. Vérifier que le retard de réplication est compatible avec le RPO.
3. Rediriger les connexions vers la cible.
4. Vérifier les logs applicatifs.
5. Garder un plan de rollback armé.

Types de migrations DMS

Type	Exemple	Point d'attention
Homogène	MySQL → RDS for MySQL	Versions, paramètres, extensions et temps de bascule
Hétérogène	Oracle → Aurora PostgreSQL	Conversion du schéma, du code SQL et des types de données
Schéma complexe	Moteur propriétaire → PostgreSQL	Compatibilité fonctionnelle, tests et réécriture éventuelle

Créer une tâche DMS en CLI

:::info Une activité pratique permet d'approfondir AWS Database Migration Service. :::

DMS fonctionne en 3 objets : un **endpoint source** (base existante), un **endpoint cible** (base AWS), et une **instance de réplication** (le moteur qui exécute la migration). On les crée dans cet ordre, puis on démarre la tâche.

```
1  # =====
2  # 1. Créer les Endpoints source et cible
3  # =====
4
5  # Endpoint SOURCE (Base existante on-prem/RDS)
6  aws dms create-endpoint \
7    --endpoint-identifiant oracle-source \
8    --endpoint-type source \
9    --engine-name oracle \
10   --server-name oracle.company.local \
```

```
11  --port 1521 \  
12  --database-name PRODDb \  
13  --username migration_user \  
14  --password '<SECRET_SOURCE_NON_VERSIONNE>' \  
15  --ssl-mode require \  
16  --extra-connection-attributes 'trustServerCertificate=false'  
17  
18  # Endpoint TARGET (Aurora PostgreSQL)  
19  aws dms create-endpoint \  
20  --endpoint-identifier aurora-target \  
21  --endpoint-type target \  
22  --engine-name aurora-postgresql \  
23  --server-name formation-aurora.xxxxx.eu-west-1.rds.amazonaws.com \  
24  --port 5432 \  
25  --database-name appdb \  
26  --username migration_user \  
27  --password '<SECRET_CIBLE_NON_VERSIONNE>' \  
28  --ssl-mode require  
29  
30  # =====  
31  # 2. Créer une instance DMS (compute qui exécute la migration)  
32  # =====  
33  
34  aws dms create-replication-instance \  
35  --replication-instance-identifier formation-dms-instance \  
36  --replication-instance-class dms.c6i.xlarge \  
37  --allocated-storage 200 \  
38  --vpc-security-group-ids sg-12345678 \  
39  --replication-subnet-group-identifier my-dms-subnet-group \  
40  --multi-az \  
41  --engine-version 3.4.7 \  
42  --publicly-accessible false \  
43  --tag-specifications 'ResourceType=rep,Tags=[{Key=Name,Value=FormationDMS}]'  
44  
45  # Attendre disponibilité (5-10 min)  
46  aws dms wait replication-instance-available \  
47  --filters 'Name=replication-instance-id,Values=formation-dms-instance'  
48  
49  # =====  
50  # 3. Valider connexions (test avant migration)
```

```
51  # =====
52
53  # Tester accès source
54  aws dms test-connection \
55      --replication-instance-arn arn:aws:dms:eu-west-1:123456789012:rep:formation-d
56      --endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:oracle-source
57
58  # Tester accès cible
59  aws dms test-connection \
60      --replication-instance-arn arn:aws:dms:eu-west-1:123456789012:rep:formation-d
61      --endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:aurora-target
62
63  # =====
64  # 4. Créer tâche DMS (FULL LOAD + CDC)
65  # =====
66
67  aws dms create-replication-task \
68      --replication-task-identifiant oracle-to-aurora-migration \
69      --source-endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:oracle-source \
70      --target-endpoint-arn arn:aws:dms:eu-west-1:123456789012:ep:aurora-target \
71      --replication-instance-arn arn:aws:dms:eu-west-1:123456789012:rep:formation-d
72      --migration-type cdc \
73      --table-mappings '{
74          "rules": [
75              {
76                  "rule-type": "selection",
77                  "rule-name": "include-all-tables",
78                  "object-locator": {
79                      "schema-name": "%",
80                      "table-name": "%"
81                  },
82                  "rule-action": "include"
83              }
84          ]
85      }' \
86      --replication-task-settings '{
87          "TargetMetadata": {
88              "TargetSchema": "",
89              "SupportsCascadeDelete": true,
90              "FullLobMode": false,
```

```

91     "LobChunkSize": 64,
92     "LobMaxSize": 32
93 },
94 "FullLoadSettings": {
95     "TargetSchema": "",
96     "CreatePkAfterFullLoad": false,
97     "StopTaskCachedChangesNotApplied": false,
98     "StopTaskCachedChangesApplied": false,
99     "MaxFullLoadSubTasks": 8,
100    "TransactionConsistencyTimeout": 600,
101    "CommitRate": 50000
102 },
103 "Logging": {
104     "EnableLogging": true,
105     "LogComponents": [
106         {
107             "Id": "SOURCE_UNLOAD",
108             "Severity": "LOGGER_SEVERITY_DEFAULT"
109         }
110     ]
111 },
112 "ChangeProcessingDdlHandlingPolicy": {
113     "HandleSourceTableDropped": true,
114     "HandleSourceTableTruncated": true,
115     "HandleSourceTableAltered": true
116 }
117 }' \
118 --tags Key=Project,Value=Migration Key=Type,Value=Oracle2Aurora
119
120 # =====
121 # 5. Attendre tâche et monitorer statut
122 # =====
123
124 # Status : creating → modifying → ready → running → stopped
125 aws dms describe-replication-tasks \
126     --filters 'Name=replication-task-arn,Values=arn:aws:dms:eu-west-1:12345678901

```

:::success **Résultat attendu :**

```
1 {
```

```

2     "ReplicationTasks": [{
3         "ReplicationTaskIdentifier": "oracle-to-aurora-migration",
4         "Status": "running",
5         "MigrationType": "cdc",
6         "ReplicationTaskStats": {
7             "FullLoadProgressPercent": 100,
8             "ElapsedTimeMillis": 18000000,
9             "TablesLoaded": 50,
10            "TablesQueued": 0,
11            "TablesErrored": 0,
12            "TablesLoading": 0
13        }
14    }]
15 }

```

...

```

1  # Voir les tables migrées (status, nb rows)
2  aws dms describe-table-statistics \
3      --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-auro
4      --query 'TableStatistics[*].[SchemaName,TableName,FullLoadRows,FullLoadErrorR

```

...success **Résultat attendu :**

```

1  [
2      ["PRODDb", "CONTENT_META", 2500000, 0, 1250, 340, 12],
3      ["PRODDb", "RIGHTS_TABLE", 180000, 0, 45, 8, 1],
4      ["PRODDb", "CALENDAR_SLOTS", 95000, 0, 320, 120, 5],
5      ["PRODDb", "USERS", 50000, 0, 88, 15, 0]
6  ]

```

...

```

1  # =====
2  # 6. Commandes gestion tâche
3  # =====
4
5  # Arrêter tâche (sans supprimer)
6  aws dms stop-replication-task \
7      --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-auro

```

```

8
9  # Relancer tâche
10 aws dms start-replication-task \
11     --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-auro
12     --start-replication-task-type resume-processing
13
14 # Redémarrer complet (FULL LOAD + CDC)
15 aws dms start-replication-task \
16     --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-auro
17     --start-replication-task-type cdc
18
19 # Supprimer tâche
20 aws dms delete-replication-task \
21     --replication-task-arn arn:aws:dms:eu-west-1:123456789012:task:oracle-to-auro

```

Pièges et bonnes pratiques DMS

Piège	Solution
Oublier full load avant CDC	Toujours : <code>migration-type cdc</code> (inclut full load)
Schema incomplet après migration	Procs stockées, triggers = manuels. DMS copie juste data
CDC lag important	Vérifier capacité DMS instance, réduire <code>MaxFullLoadSubTasks</code>
Incompatibilités types données	Utiliser Schema Conversion Tool (SCT) avant DMS pour préparer
Oublier transaction logs source	Source doit activer binary logs (MySQL) / redo logs (Oracle)
Cutover sans validation données	Test requêtes applicatives sur cible avant redirection

2. Amazon VPC — Concevoir un réseau privé sécurisé

2.1 Du réseau on-prem au réseau virtuel

Définition — Qu'est-ce qu'une VPC ?

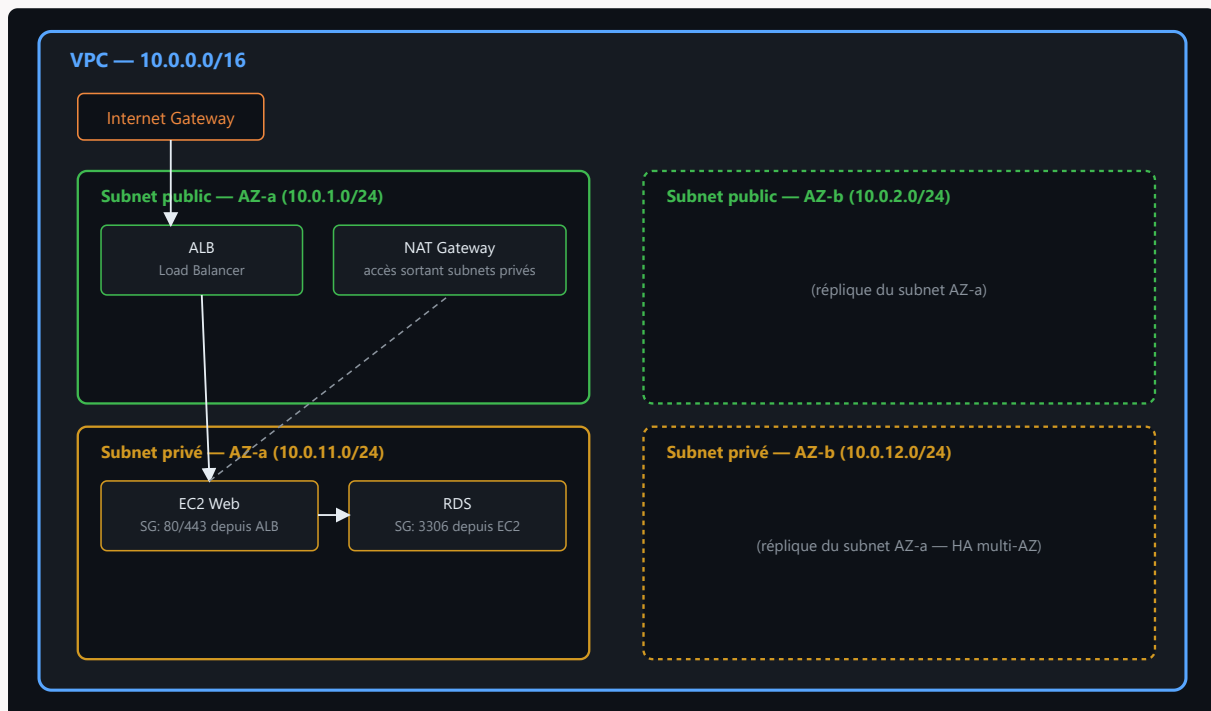
Amazon VPC (Virtual Private Cloud) est un **réseau privé virtuel isolé** que vous créez dans AWS. C'est votre **datacenter logique**, entièrement contrôlé, où vous déployez vos instances EC2, vos bases RDS, vos Load Balancers, etc.

Une VPC vous offre :

- **Isolation logique** : vos ressources ne sont pas visibles aux autres comptes AWS.
- **Contrôle total** : vous définissez les adresses IP, les routes, les pare-feux.
- **Flexibilité** : ajouter ou retirer des subnets, des passerelles à volonté.

2.2 Composants clés d'une VPC

Schéma architectural complet d'une VPC sécurisée



Lecture du schéma. Le VPC est découpé en sous-réseaux publics et privés répartis sur plusieurs zones de disponibilité. Les composants exposés reçoivent le trafic entrant ; les bases restent dans les sous-

réseaux privés. Les routes et les groupes de sécurité contrôlent des aspects différents : chemin réseau pour les premières, autorisation des flux pour les seconds.

Flux de trafic :

1. Internet → ALB (IGW ouvre l'accès)
2. ALB → EC2 Web (Security Group + règles subnet)
3. EC2 → RDS (Security Group DB ouvre port 3306)
4. EC2 → Internet (via NAT Gateway, pour updates)

1. CIDR Block (Classless Inter-Domain Routing)

Une VPC commence par une **plage d'adresses IP privées**. Par exemple, `10.0.0.0/16` signifie :

- 1 `10.0.0.0/16` = 65 536 adresses disponibles (10.0.0.0 à 10.0.255.255)
- 2
- 3 Adresses réservées AWS :
- 4 `10.0.0.0` = Adresse réseau (réservée)
- 5 `10.0.0.1` = Gateway AWS (réservée)
- 6 `10.0.0.2` = DNS AWS (réservé)
- 7 `10.0.0.3` = Réserve pour l'avenir
- 8 `10.0.0.4-...` = EC2, RDS, ALB, etc.

Plages privées standards (RFC 1918) :

- `10.0.0.0/8` (la plus courante, 16 M d'adresses)
- `172.16.0.0/12` (1 M d'adresses)
- `192.168.0.0/16` (65 536 adresses)

2. Subnets (Sous-réseaux)

Un subnet est une **subdivision logique** d'une VPC, liée à une **zone de disponibilité (AZ)** spécifique.

Subnet public : sa table de routage contient une route vers une Internet Gateway. Une ressource n'est toutefois joignable depuis Internet que si elle possède aussi une adresse publique et si ses contrôles de sécurité autorisent le trafic. **Subnet privé** : sa table de routage ne contient pas de route directe vers une Internet Gateway. Une route vers une NAT Gateway est une option pour les connexions sortantes IPv4, pas une propriété obligatoire du subnet privé.

3. Internet Gateway (IGW)

L'Internet Gateway est la **passerelle de sortie vers Internet**.

Une Internet Gateway n'est pas facturée à l'heure. Les adresses publiques et les transferts de données associés aux ressources restent des postes de coût distincts.

4. NAT Gateway

Permet aux instances **privées** d'accéder à Internet **de manière sécurisée** sans être exposées.

Une NAT Gateway cumule une facturation horaire et une facturation au volume traité. Le transfert de données et l'adresse IPv4 publique peuvent également intervenir. Le tarif varie avec la région.

Coût des adresses IPv4 publiques — Point important depuis 2024

Depuis le **1er février 2024**, AWS facture **toutes les adresses IPv4 publiques**, y compris celles attachées à une instance EC2 en cours d'exécution.

Les adresses IPv4 publiques utilisées dans AWS sont facturées. Le prix exact et les éventuelles exceptions se vérifient sur la page officielle de tarification VPC.

Conséquence d'architecture : inventorier les adresses avec Public IP Insights, retirer celles qui ne sont pas nécessaires et privilégier les accès privés, les points de terminaison VPC, Systems Manager ou IPv6 lorsque le besoin le permet. Réduire les IPv4 ne doit pas conduire à centraliser tous les flux sur une ressource unique non résiliente.

 [AWS — Annonce facturation IPv4 \(2023\)](#) 

AWS = plateforme 100 % API — À quoi servent vraiment les Elastic IPs ?

Vous n'avez jamais besoin d'une IP publique pour piloter AWS. Créer une instance EC2, configurer un VPC, déployer une Lambda — tout cela se fait via l'API AWS, que vous passiez par la console web, la CLI ou un SDK.

```
1  Vous (navigateur / terminal / code)
2
3      HTTPS → api.aws.amazon.com
4      (pas besoin d'IP publique sur vos ressources)
5
6      ▼
7      AWS Control Plane (infrastructure interne AWS)
8
9      ▼
10     Votre ressource (EC2, RDS, Lambda...)
11     dans votre VPC privé
```

Le **Control Plane** est entièrement géré par AWS et accessible depuis Internet sur ses propres IPs. Vos ressources peuvent très bien vivre dans un subnet privé sans aucune IP publique.

Une instance EC2 sans IP publique reste pleinement fonctionnelle dans le VPC. Elle peut joindre certains services AWS au moyen de points de terminaison VPC compatibles, sous réserve des routes, du DNS, des politiques d'endpoint et des groupes de sécurité requis.

Les cas d'usage légitimes d'une Elastic IP :

Cas d'usage	Pourquoi une EIP est nécessaire
Whitelist IP chez un partenaire	Le firewall du client autorise uniquement votre IP fixe. Si l'instance redémarre avec une nouvelle IP, la connexion est bloquée.
Adresse source fixe attendue par un tiers	Un partenaire peut filtrer les connexions sortantes sur une adresse connue ; l'architecture doit alors fournir cette adresse de manière résiliente.
NAT Gateway	Obligatoire : le NAT Gateway a toujours besoin d'une EIP pour sortir sur Internet au nom des instances privées.
Équipement ou service exigeant une adresse fixe	Certains protocoles ou systèmes hérités ne savent pas utiliser un nom DNS comme point de terminaison.

Les cas où une EIP n'est PAS la bonne réponse :

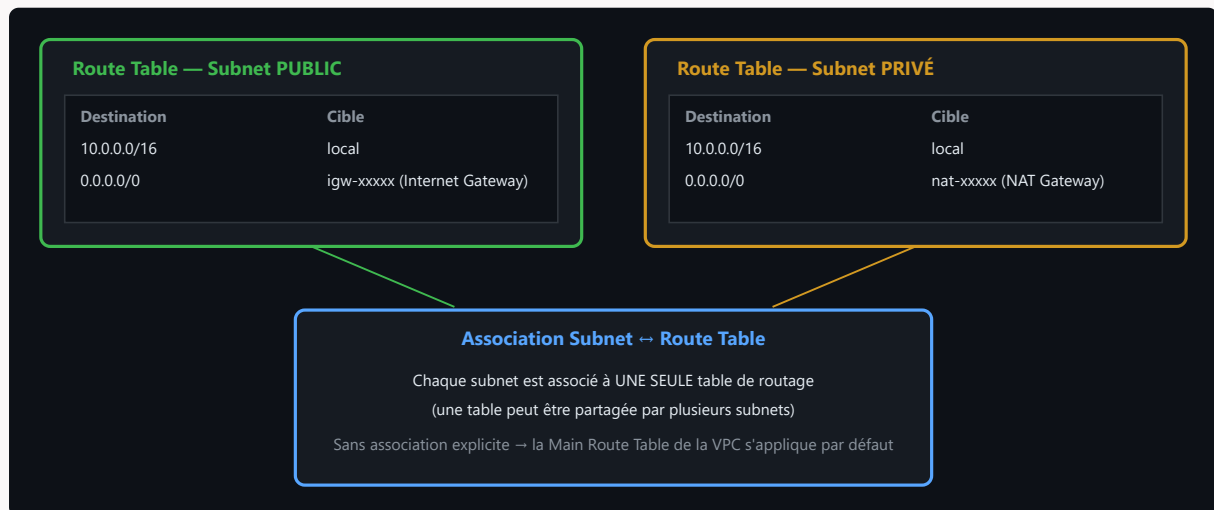
Mauvais réflexe	Meilleure alternative
"Je veux publier plusieurs serveurs web"	→ Load Balancer et nom DNS, avec des cibles privées
"Je veux une URL stable pour mon API"	→ Route 53 + nom de domaine (DNS, pas IP)
"Je veux accéder à mon instance en SSH"	→ Session Manager (SSM) : SSH sans IP publique, sans port 22 ouvert
"Je veux que mes Lambda puissent appeler une API externe"	→ NAT Gateway (une seule EIP pour tout le subnet)

⚠ **Le réflexe à éviter** : assigner une Elastic IP à chaque instance « au cas où ». Chaque exposition doit correspondre à un flux documenté. Pour un service web réparti, l'Application Load Balancer fournit un nom DNS ; il ne reçoit pas directement une Elastic IP. Pour l'administration, Session Manager peut éviter une adresse publique et l'ouverture du port SSH.

👤 [AWS Systems Manager Session Manager](#) 📄 👤 [Elastic IP Addresses — Documentation AWS](#) 📄

5. Route Tables (Tables de routage)

Chaque subnet est associé à une **table de routage** qui définit comment le trafic circule.



Lecture du schéma. Le sous-réseau public possède une route vers l'Internet Gateway. Le sous-réseau privé n'en possède pas ; lorsqu'une sortie Internet est nécessaire, sa route pointe vers une NAT Gateway située dans un sous-réseau public. Le trafic retour suit l'état de la traduction NAT.

2.3 Sécurité réseau — Security Groups et Network ACLs

La sécurité dans une VPC repose sur **deux couches** complémentaires.

Security Groups — Pare-feu au niveau instance

Un **Security Group** est un ensemble de **règles de filtrage** appliquées à une ou plusieurs instances EC2.

Caractéristiques :

- **Stateful** : si vous autorisez les demandes entrantes, les réponses sortantes sont automatiquement autorisées.
- Changements appliqués **immédiatement**.
- Peut être modifié sur une instance en cours d'exécution.

📖 [Groupes de sécurité : pourquoi faire ? Comment ?](#) 📄

Network ACLs — Pare-feu au niveau subnet

Un **Network ACL** est un ensemble de règles appliquées à un **subnet entier**.

Caractéristiques :

- **Stateless** : vous devez définir EXPLICITEMENT les règles entrantes ET sortantes.
- Numérotées (ordre d'évaluation).
- Application par subnet.

Différences clés

Aspect	Security Group	Network ACL
Portée	Instance	Subnet
État	Stateful	Stateless
Défaut	Tout refusé sauf règles	Tout refusé sauf règles

Bonne pratique : utilisez les **Security Groups** pour les **règles fines** (par instance), et les **ACLs** pour les **règles larges** (par subnet).

Comment tout s'imbrique — architecture 3-tiers dans une VPC

Ces briques (subnets, Security Groups, NAT Gateway) ne prennent tout leur sens qu'assemblées avec les ressources RDS/EC2 vues plus haut dans ce chapitre. Voici l'architecture la plus enseignée en SAA-C03 : un serveur web accessible depuis Internet, une base de données qui ne l'est jamais.

```
1  VPC 10.0.0.0/16
2  |
3  |— Subnet PUBLIC (10.0.1.0/24, AZ 1a)
4  |   |— Route Table → 0.0.0.0/0 via Internet Gateway
5  |   |— Application Load Balancer (ALB)
6  |   |   Security Group ALB : inbound 443 depuis 0.0.0.0/0
7  |   |— NAT Gateway (sort vers Internet pour le subnet privé)
8  |
9  |— Subnet PRIVÉ – App (10.0.10.0/24, AZ 1a)
10 |   |— Route Table → 0.0.0.0/0 via NAT Gateway (pas d'IGW direct)
11 |   |— Instance EC2 (serveur web)
12 |       Security Group EC2 : inbound 80 UNIQUEMENT depuis Security Group ALB
13 |
```

```
14 └─ Subnet PRIVÉ – Data (10.0.20.0/24, AZ 1a)
15   └─ Route Table → pas de route vers Internet (isolé)
16   └─ Instance RDS (déployée en section 1 de ce chapitre)
17       Security Group RDS : inbound 3306 UNIQUEMENT depuis Security Group EC
```

Ce que cette architecture garantit :

- Seul l'ALB est exposé sur Internet (subnet public, port 443 ouvert à tous).
- L'EC2 n'accepte du trafic que depuis l'ALB — jamais directement depuis Internet, même si son IP était devinée.
- Le RDS n'accepte du trafic que depuis l'EC2 — la base de données n'a **aucune route vers Internet**, donc même une erreur de configuration Security Group ne peut pas l'exposer.
- Chaque Security Group référence un *autre Security Group* comme source (pas une plage d'IP) — c'est la bonne pratique : si l'EC2 change d'adresse IP (redémarrage, remplacement), la règle reste valide.

2.4 VPC Peering — Connecter plusieurs VPC

VPC Peering établit une **connexion réseau privée** entre deux VPC, permettant aux instances de communiquer comme si elles étaient dans le même réseau.

Caractéristiques

Aspect	Détail
Non-transitif	A ↔ B fonctionne. Mais A ↔ C ne passera pas par B : il faut une peering A-C explicite.
Coût	La connexion de peering n'est pas facturée à l'heure ; le transfert de données applicable reste facturable
Inter-comptes	Deux VPC dans deux comptes AWS différents peuvent être peered
Inter-régions	Deux VPC dans deux régions différentes peuvent être peered

⚠ VPC Peering est non-transitif — Piège architectural classique

Si vous avez 3 VPCs : **Prod** ↔ **Shared** et **Dev** ↔ **Shared**, cela ne signifie PAS que Prod peut parler à Dev via Shared. Le trafic ne transite JAMAIS par un VPC intermédiaire.

Pour interconnecter N VPCs avec transitivity, utilisez **AWS Transit Gateway** (hub centralisé). Avec 4 VPCs, VPC Peering crée 6 connexions à gérer — avec 10 VPCs, c'est 45 connexions. Transit Gateway réduit cela à 1 attachement par VPC. :::

 [Documentation VPC Peering](#)

2.5 Modèle Multi-VPC / Multi-Comptes et AWS Transit Gateway

Limitation du VPC Peering

Quand vos infrastructures deviennent **complexes** (10+ VPCs, plusieurs comptes AWS), le peering classique crée un problème :

```
1  AVEC PEERING CLASSIQUE (non-transitif) :
2  Pour N VPCs :  $N \times (N-1) / 2$  peerings = EXPLOSION combinatoire
3
4  Exemple 4 VPCs :
5  VPC-Prod ↔ VPC-Dev      (1)
6  VPC-Prod ↔ VPC-Staging  (2)
7  VPC-Prod ↔ VPC-Shared   (3)
8  VPC-Dev  ↔ VPC-Staging   (4)
9  VPC-Dev  ↔ VPC-Shared    (5)
10 VPC-Staging ↔ VPC-Shared (6)
11
12      ↓
13      6 peerings manuels ! 🤖
14
15 Problèmes :
16 - Non-transitif : Prod ↔ Dev ↔ Shared = Prod ne voit pas Shared
17 - Gestion complexe : chaque nouveau VPC = 3 peerings à créer
18 - Pas de contrôle centralisé : règles partagées impossibles
```

AWS Transit Gateway — La solution

AWS Transit Gateway est un **hub réseau centralisé** qui connecte **toutes vos VPCs, comptes AWS et réseaux on-prem** via une seule interface.

En architecture *hub-and-spoke*, l’AWS Transit Gateway devient un hub de routage auquel les VPC et certaines connexions réseau s’attachent. Cette centralisation simplifie les relations nombreuses, mais reste soumise aux quotas, aux routes, aux autorisations et au coût du service.

Attachements possibles :

-  VPC (plusieurs)
-  Comptes AWS (via RAM — Resource Access Manager)
-  On-premise (via VPN ou Direct Connect)
-  Transit Gateway externe (inter-régions)

Architecture complète : Multi-Comptes avec Transit Gateway

L’organisation AWS regroupe plusieurs comptes, tous rattachés au même Transit Gateway (`tgw-xxx` , possédé par le Compte Prod) :

Compte	Ressource	Rattachement
PROD (123456789012)	VPC-Prod (<code>10.0.0.0/16</code>)	Attachement TGW
DEV (210987654321)	VPC-Dev (<code>10.1.0.0/16</code>)	Attachement TGW
SHARED (shared-000)	VPC-Shared (<code>10.3.0.0/16</code>)	Attachement TGW
ON-PREMISE	Network (<code>192.168.0.0/16</code>)	Attachement VPN/Direct Connect
MGMT	CloudTrail logs (audit)	—

Le Transit Gateway maintient des route tables distinctes (Production, Dev, Shared) pour contrôler quel trafic peut transiter entre quels VPC.

Avantages Transit Gateway

Aspect	VPC Peering	Transit Gateway
Connexions N VPCs	$N(N-1)/2$ peerings 🧑‍🤝‍🧑	1 attachement par VPC 
Transitif	Non ($A \leftrightarrow B, B \leftrightarrow C \neq A \leftrightarrow C$)	Oui (contrôlé par routing)
Multi-comptes	Oui, avec acceptation de la connexion	Oui, notamment via Resource Access Manager

On-premise	Non	Oui (VPN + Direct Connect)
Policies centralisées	Impossible	Oui (Network Policy)
Coût	Transfert de données applicable	Attachements et traitement de données, selon la région

Configuration AWS CLI — Transit Gateway, principe

Le Transit Gateway est créé une seule fois, puis chaque VPC s’y attache via une **attachment**. La table de routage du TGW détermine quels VPCs peuvent se parler.

```

1  # Créer le Transit Gateway
2  aws ec2 create-transit-gateway \
3      --description "Formation Transit Gateway Hub" \
4      --tag-specifications 'ResourceType=transit-gateway,Tags=[{Key=Name,Value=Form
5  # Résultat : TransitGatewayId = tgw-0123456789abcdef
6
7  # Attacher un VPC au Transit Gateway
8  aws ec2 create-transit-gateway-vpc-attachment \
9      --transit-gateway-id tgw-0123456789abcdef \
10     --vpc-id vpc-prod-12345 \
11     --subnet-ids subnet-prod-1a subnet-prod-1b

```

:::success **Résultat attendu :**

```

1  {"TransitGatewayVpcAttachment": {"State": "pending", "TransitGatewayId": "tgw-0

```

...

:::info **Pour aller plus loin — hors périmètre de cette formation** : partager un Transit Gateway entre plusieurs comptes AWS (via Resource Access Manager) et l’étendre à un réseau on-premise (VPN Site-to-Site) relèvent du niveau Advanced Networking Specialty. Le principe reste le même — un attachement par ressource, une table de routage centralisée — mais la mise en œuvre cross-account est un sujet à part entière. :::

Pièges Transit Gateway

Piège

Solution

TGW par défaut permet tout	Créer des route tables TGW restrictives par environnement
Chaque attachement et volume traité contribue au coût	Compter les attachements, les heures et le trafic dans AWS Pricing Calculator
Association subnet obligatoire	Au moins 1 subnet par AZ pour la résilience
CIDR overlap interdit	VPCs partagés doivent avoir CIDRs différents

2.6 VPC Endpoints — Accès privé aux services AWS

Définition

Un **VPC Endpoint** est une **passerelle privée** qui permet à vos ressources d'accéder à des **services AWS** sans passer par Internet.

Deux types

Type	Services	Fonctionnement	Modèle de coût
Gateway Endpoint	S3, DynamoDB	Cible ajoutée aux tables de routage sélectionnées	Pas de facturation horaire propre à l'endpoint
Interface Endpoint	Services compatibles avec AWS PrivateLink	Interfaces réseau privées dans les subnets choisis	Heures d'endpoint et volume traité, selon la région

 [Documentation VPC Endpoints](#)

2.7 ENI (Elastic Network Interfaces) — Les cartes réseau d’AWS

Qu’est-ce qu’une ENI ?

Une **ENI** est une **carte réseau virtuelle** attachée à une instance EC2. Elle gère vos **adresses IP**, vos **adresses MAC**, vos **Security Groups** et vos **routes réseau**.

Dans une infrastructure **on-prem**, vous aviez des **cartes réseau physiques** (NIC) dans vos serveurs. Sur AWS, c'est exactement la même chose, mais **virtuelle et reconfigurable**.

Anatomie d'une ENI

```
1 Instance EC2 (t3.medium)
2
3 - Primary ENI (eth0) [obligatoire]
4
5   - Primary IP privée : 10.0.1.42 (CIDR subnet)
6
7   - Secondary IP privées : 10.0.1.43, 10.0.1.44 (optionnel)
8
9   - Elastic IP publique : 203.0.113.12 (optionnel)
10
11   - MAC Address : 02:c1:1f:a0:2b:4d (auto-générée)
12
13   - Security Group : sg-12345678
14
15   - Source/Dest Check : ☒ activée (drop trafic non-destiné)
```

 [Comment conserver son IP sur AWS ?](#)

Cas d'usage : Multiple ENIs sur une même instance

Certains scénarios nécessitent **plusieurs ENIs** sur une instance :

```
1 SCÉNARIO : Serveur pare-feu / VPN / Load Balancer
2
3 Instance (m5.xlarge) : 4 ENIs possibles
4
5 - eth0 (Primary) : connectée à VPC public → Internet
6   - 10.0.1.10
7
8 - eth1 (Secondary) : connectée à VPC privée → Apps internes
9   - 10.1.1.10
10
11 - eth2 (Secondary) : connectée à VPC client (peering) → Client network
```

```
12     - 10.2.1.10
13
14 - eth3 (Secondary) : Management/monitoring
15     - 10.0.2.10 (subnet privé)
16
17 Résultat : machine routeur/pare-feu multi-réseaux ! 🔥
```

Configuration ENI via AWS CLI

On crée ici une ENI indépendante puis on l’attache à une instance existante. Cela permet d’ajouter une interface réseau secondaire sans recréer l’instance — utile pour un serveur bastion ou un routeur multi-réseaux.

```
1  # =====
2  # 1. Créer une ENI seule (détachée)
3  # =====
4
5  aws ec2 create-network-interface \
6      --subnet-id subnet-12345678 \
7      --description "Secondary management interface" \
8      --private-ip-addresses PrivateIpAddress=10.0.2.100,Primary=true \
9      --groups sg-mgmt-12345678 \
10     --tag-specifications 'ResourceType=network-interface,Tags=[{Key=Name,Value=EN
11
12  # Résultat : NetworkInterfaceId = eni-0a1b2c3d4e5f6g7h8
13
14  # =====
15  # 2. Attacher une ENI existante à une instance
16  # =====
17
18  aws ec2 attach-network-interface \
19      --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
20      --instance-id i-0123456789abcdef0 \
21      --device-index 1 # eth1 (0=primary/eth0, 1=eth1, etc.)
22
23  # =====
24  # 3. Allouer une Elastic IP et l'attacher à une ENI
25  # =====
26
27  # Allouer Elastic IP
```

```

28 aws ec2 allocate-address \
29     --domain vpc \
30     --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
31     --private-ip-address 10.0.2.100
32
33 # Résultat : PublicIp = 203.0.113.50
34
35 # =====
36 # 4. Ajouter une IP privée secondaire à une ENI
37 # =====
38
39 aws ec2 assign-private-ip-addresses \
40     --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
41     --private-ip-addresses 10.0.2.101 10.0.2.102
42
43 # =====
44 # 5. Changer de Security Group sur une ENI
45 # =====
46
47 aws ec2 modify-network-interface-attribute \
48     --network-interface-id eni-0a1b2c3d4e5f6g7h8 \
49     --groups sg-new-12345678 sg-management-67890
50
51 # =====
52 # 6. Détacher une ENI (reste dans la VPC, peut être réattachée)
53 # =====
54
55 aws ec2 detach-network-interface \
56     --attachment-id eni-attach-12345678
57
58 # =====
59 # 7. Voir toutes les ENI d'une instance
60 # =====
61
62 aws ec2 describe-instances \
63     --instance-ids i-0123456789abcdef0 \
64     --query 'Reservations[0].Instances[0].NetworkInterfaces[*].[NetworkInterfaceId,PrivateIpAddress,PublicIpAddress]'
65
66 # Résultat (exemple) :
67 # [

```

```

68 # ["eni-primary", 0, "10.0.1.42", "203.0.113.50"],      ← eth0 + Elastic IP
69 # ["eni-secondary", 1, "10.0.2.100", null]              ← eth1 (privée)
70 # ]

```

...success **Résultat attendu :**

```

1 [
2     ["eni-0a1b2c3d4e5f00001", 0, "10.0.1.42", "203.0.113.50"],
3     ["eni-0a1b2c3d4e5f00002", 1, "10.0.2.100", null]
4 ]

```

...

Cas d'usage réels : ENI multiples

Scénario	Interfaces	Bénéfice
Routeur/Pare-feu	3-4 ENIs	Connexion à plusieurs VPCs/subnets sans NAT
Haute disponibilité	Primary ENI + failover	IP privée = même, instance change (failover transparent)
Serveur DNS interne	Primary + management	Trafic DNS sur une interface, logs/monitoring sur autre
Load Balancer maison	Multiple NICs	Distribution load par interface réseau
Serveur VPN/bastion	2+ interfaces	Accès de plusieurs subnets via une machine unique

Piège : Source/Destination Check

Par défaut, AWS bloque tout paquet dont l'IP source ou destination ne correspond pas à l'interface. C'est intentionnel pour la sécurité, mais cela empêche un routeur ou un pare-feu de forwarder le trafic. Il faut donc **désactiver cette vérification** sur les instances qui jouent un rôle de routage.

```

1 # ⚠ Par défaut, une ENI REFUSE le trafic non-destiné à elle-même.
2 # Exemple : routeur firewall doit FORWARDER le trafic.
3
4 # Désactiver Source/Destination check

```

```
5 aws ec2 modify-network-interface-attribute \  
6   --network-interface-id eni-12345678 \  
7   --no-source-dest-check  
8  
9 # Résultat : routeur peut maintenant forwarder (forwarding activ )
10  
11 # R activer (s curit )
12 aws ec2 modify-network-interface-attribute \  
13   --network-interface-id eni-12345678 \  
14   --source-dest-check
```

:::success **R sultat attendu :**

```
1 # modify-network-interface-attribute : aucun output si succ s
2  
3 # V rification : aws ec2 describe-network-interface-attribute --network-interfa
4 {
5   "NetworkInterfaceId": "eni-12345678",
6   "SourceDestCheck": {
7     "Value": false
8   }
9 }
```

La v rification Source/Destination est d sactiv e (`Value: false`). L'interface peut d sormais forwarder du trafic dont elle n'est pas la destination finale — comportement requis pour une instance jouant le r le de routeur ou de pare-feu. :::

3. Amazon Route 53 — DNS Intelligent

3.1 Fondamentaux du DNS

Qu'est-ce que le DNS ?

Le **DNS (Domain Name System)** est un syst me **mondial d centralis ** qui traduit des **noms lisibles** (`www.example.com`) en **adresses IP** (`93.184.216.34`).

3.2 Amazon Route 53 — Service DNS managé

Définition

Amazon Route 53 est le **service DNS managé** d’AWS. Il permet de **résoudre les noms de domaine**, de **diriger le trafic intelligemment**, et d’assurer la **haute disponibilité**.

Le nom « Route 53 » vient du **port 53**, utilisé par le protocole DNS.

Capacités

Route 53 combine plusieurs fonctionnalités :

Fonctionnalité	Description
Registrar	Acheter et gérer des domaines (.com, .fr, .io, etc.)
Résolution DNS	Traduire noms → IP
Health Checks	Vérifier si une ressource est disponible
Routage intelligent	Diriger le trafic selon latence, géolocalisation, poids, failover
Alias Records	Lier un domaine à une ressource AWS (ALB, CloudFront, S3)

Types d’enregistrements DNS courants

Type	Exemple	Rôle
A	example.com → 93.184.216.34	IPv4
AAAA	example.com → 2606:2800:...	IPv6
CNAME	www.example.com → example.com	Alias
MX	example.com → mail.example.com	Serveur mail
TXT	example.com → v=spf1...	Enregistrement texte
NS	example.com → ns1.route53...	Serveurs DNS

3.3 Politiques de routage — Diriger le trafic intelligemment

Route 53 n'est pas un simple DNS classique. C'est un **routeur de trafic applicatif**.

Politique Simple

Cas de base : un domaine pointe vers **une seule adresse IP**.

Politique Pondérée

Distribuer le trafic en pourcentage entre plusieurs ressources.

Cas d'usage : déploiement progressif, A/B testing, migration progressive.

Politique Latence

Router les utilisateurs vers la ressource **la plus rapide** (latence réseau minimale).

Cas d'usage : applications globales, réduction latence.

Politique Failover (Basculement)

En cas de panne détectée, router vers une ressource de secours.

Cas d'usage : haute disponibilité, reprise après sinistre.

3.4 Health Checks et Monitoring

Route 53 peut évaluer des contrôles d'intégrité et utiliser leur état dans certaines politiques de routage. Le basculement n'existe que si les enregistrements, les contrôles et la stratégie de routage ont été configurés pour cela.

Types de Health Checks

Type	Description	Fréquence
HTTP/HTTPS	Effectue une requête GET, attend 2xx/3xx	Toutes les 30s
TCP	Établit une connexion TCP	Toutes les 10s
Calculated	Combine plusieurs health checks	Toutes les 30s

4. Amazon ElastiCache — Mise en cache distribuée

4.1 Pourquoi une couche cache ?

Imaginez une **base de données RDS** qui reçoit **1 000 requêtes par seconde** pour lire les **mêmes 10 utilisateurs**. Chaque requête demande 5–10 ms à la base. Résultat : **goulot d'étranglement**, latence élevée, coût RDS énorme.

Solution : placez un **cache rapide** devant la base. Les 1 000 requêtes frappent le cache (**< 1 ms**) au lieu de la base.

Amazon ElastiCache est le service managé AWS pour placer un **cache distribuée** haute performance devant vos applications.

4.2 Deux moteurs : Redis vs Memcached

Redis (Remote Dictionary Server)

- 1 Cas d'usage : Sessions utilisateur, panier e-commerce, rankings, pubsub
- 2 Structure : Chaînes, listes, ensembles, hashes, streams, géo-spatial
- 3 Persistance : RDB + AOF (journalisation)
- 4 Clustering : Oui, avec failover automatique
- 5 Transactions : MULTI/EXEC
- 6 TTL (durée de vie clé) : Oui

Redis conserve d'abord les données en mémoire. Dans ElastiCache, la résilience repose notamment sur les nœuds de réplication, Multi-AZ et les sauvegardes selon la configuration choisie. Un cache ne doit pas devenir l'unique copie d'une donnée métier durable : l'application doit pouvoir le reconstruire depuis le système de référence.

Analogie : un **dictionnaire magique ultra-rapide** qui se souvient des modifications.

Memcached

- 1 Cas d'usage : Cache objet simple (résultats DB, pages HTML)
- 2 Structure : Chaînes et blobs uniquement
- 3 Persistance : Non (tout volatil)
- 4 Clustering : Oui, mais pas de failover
- 5 Transactions : Non
- 6 TTL : Oui

Analogie : un **panier à oublier** ultra-simple — parfait pour des données éphémères.

4.3 Cas d'usage typiques

Cas d'usage	Moteur	Raison
Session utilisateur	Redis	Besoin de persistance, expiration TTL
Panier e-commerce	Redis	Structures complexes (hash), transactions
Leaderboard	Redis	Opérations set triées (ZSET)
Cache HTML statique	Memcached	Simple, volatil, très rapide
Résultats requête DB	Redis	Contrôle TTL par clé, publish/subscribe
Real-time counters	Redis	Opérations atomiques (INCR)

4.4 Architecture ElastiCache

Cluster Mode Disabled (simple, old-school)

L'application (EC2, Lambda) envoie ses requêtes `GET cache_key` vers le nœud ElastiCache Redis primary (`eu-west-1a`), qui réplique de façon asynchrone vers un nœud Replica standby en lecture seule (`eu-west-1b`).

Limitation : une seule shard, donc un seul nœud — le CPU de ce nœud unique plafonne la capacité totale.

Cluster Mode Enabled (production, sharding)

L'application (EC2, Lambda) hache chaque clé pour la router vers l'une des trois shards : Shard 1 (clés 1-3), Shard 2 (clés 4-6) ou Shard 3 (clés 7-10). Chaque shard a son propre primary node, répliqué vers un Replica correspondant (eu-west-1b).

Bénéfice : parallélisation et scalabilité linéaire — ajouter des shards augmente la capacité totale, contrairement au mode Cluster Disabled.

4.5 Commandes Redis essentielles

```
1  # Installation (macOS via Homebrew)
2  brew install redis
3
4  # Lancer serveur Redis local (développement)
5  redis-server
6
7  # Client Redis (dans un autre terminal)
8  redis-cli
9
10 # ——— CHAÎNES (Strings) ———
11 SET nom "Alice"                # Stocker
12 GET nom                        # Récupérer → "Alice"
13 APPEND nom " Dupont"          # Ajouter → "Alice Dupont"
14 STRLEN nom                     # Longueur → 12
15 INCR compteur                 # Incrémenter (atomique)
16 DECR compteur                 # Décrémenter
17
18 # ——— LISTES (Lists) ———
19 RPUSH queue "tache1"          # Ajouter à droite
20 RPUSH queue "tache2" "tache3" # Multiple
21 LPOP queue                     # Retirer de gauche
22 LLEN queue                     # Longueur
23 LRANGE queue 0 -1             # Tout afficher
24
25 # ——— HASHES (Objets) ———
26 HSET user:100 nom "Alice"      # Stocker champ
27 HSET user:100 email "alice@ex.com" age 28
28 HGET user:100 nom              # Récupérer → "Alice"
29 HGETALL user:100               # Tous les champs
```

```

30 HDEL user:100 age                # Supprimer champ
31
32 # ——— ENSEMBLES TRIÉS (Sorted Sets, ZSET) ———
33 ZADD leaderboard 100 "Alice"      # Score 100 → Alice
34 ZADD leaderboard 150 "Bob" 200 "Charlie"
35 ZRANGE leaderboard 0 -1          # Ordre croissant
36 ZREVRANGE leaderboard 0 -1       # Ordre décroissant (top)
37 ZRANK leaderboard "Alice"        # Position → 0 (première)
38
39 # ——— EXPIRATION ———
40 SET session:user123 "data"
41 EXPIRE session:user123 3600      # Expirer dans 1 heure
42 TTL session:user123              # Temps restant → 3599
43
44 # ——— TRANSACTIONS ———
45 MULTI
46 SET clé1 "valeur1"
47 SET clé2 "valeur2"
48 EXEC                             # Atomique : tout ou rien
49
50 # ——— PUBLISH/SUBSCRIBE ———
51 SUBSCRIBE channel:notifications # S'abonner
52 PUBLISH channel:notifications "Hello" # Diffuser

```

4.6 Créer un cluster Redis en CLI

:::info Une activité pratique permet d’approfondir le déploiement d’un cache Redis. :::

On crée ici le type de cluster le plus simple : un nœud Redis unique, sans réplication. En production, on ajouterait un groupe de réplication (`create-replication-group`) avec un nœud primaire et des replicas, mais ce modèle suffit pour comprendre les concepts.

```

1 # 1. Créer un cluster Redis (simple, mode Cluster Mode Disabled)
2 aws elasticache create-cache-cluster \
3   --cache-cluster-id formation-redis-simple \
4   --cache-node-type cache.t3.micro \
5   --engine redis \
6   --engine-version 7.0 \

```

```

7  --num-cache-nodes 1 \
8  --vpc-security-group-ids sg-12345678 \
9  --cache-subnet-group-name my-subnet-group \
10 --tags Key=Name,Value=FormationRedis Key=Env,Value=Dev
11
12 # 2. Attendre que le cluster soit disponible
13 aws elasticache wait cache-cluster-available \
14   --cache-cluster-id formation-redis-simple
15
16 # 3. Récupérer l'endpoint (adresse:port)
17 aws elasticache describe-cache-clusters \
18   --cache-cluster-id formation-redis-simple \
19   --query 'CacheClusters[0].CacheNodes[0].Endpoint'
20 # Résultat exemple : formation-redis-simple.abc123.ng.0001.euw1.cache.amazonaws

```

:::success **Résultat attendu :**

```

1  {
2    "CacheClusters": [{
3      "CacheClusterId": "formation-redis-simple",
4      "CacheClusterStatus": "available",
5      "Engine": "redis",
6      "EngineVersion": "7.0.7",
7      "CacheNodeType": "cache.t3.micro",
8      "CacheNodes": [{
9        "CacheNodeId": "0001",
10       "CacheNodeStatus": "available",
11       "Endpoint": {
12         "Address": "formation-redis-simple.abc123.ng.0001.euw1.cache.am
13         "Port": 6379
14       }
15     }]
16   }]
17 }

```

...

```

1 # 4. Créer un Replication Group (Multi-AZ avec failover auto)
2 aws elasticache create-replication-group \
3   --replication-group-id formation-redis-ha \

```

```
4  --replication-group-description "Redis avec failover" \  
5  --engine redis \  
6  --engine-version 7.0 \  
7  --cache-node-type cache.t3.micro \  
8  --num-cache-clusters 2 \  
9  --automatic-failover-enabled \  
10 --multi-az-enabled \  
11 --vpc-security-group-ids sg-12345678 \  
12 --cache-subnet-group-name my-subnet-group  
13  
14 # 5. Décrire le replication group  
15 aws elasticache describe-replication-groups \  
16   --replication-group-id formation-redis-ha  
17  
18 # 6. Créer un cluster Memcached (simple)  
19 aws elasticache create-cache-cluster \  
20   --cache-cluster-id formation-memcached \  
21   --cache-node-type cache.t3.micro \  
22   --engine memcached \  
23   --engine-version 1.6.17 \  
24   --num-cache-nodes 3 \  
25   --vpc-security-group-ids sg-12345678 \  
26   --cache-subnet-group-name my-subnet-group  
27  
28 # 7. Supprimer un cluster (attention : perte de données)  
29 aws elasticache delete-cache-cluster \  
30   --cache-cluster-id formation-redis-simple
```

4.7 Bonne pratique : Cache-Aside Pattern

Le pattern le plus courant pour intégrer un cache :

```
1  Requête application :  
2  
3  1. Cache.GET(clé) ?  
4    - Si HIT → retourner valeur (< 1 ms)   
5    - Si MISS → aller à 2  
6
```

```
7 2. Requête base de données
8   RDS.SELECT(clé) → résultat
9
10 3. Stocker en cache
11   Cache.SET(clé, résultat, TTL=3600) # Expire en 1 heure
12
13 4. Retourner résultat application
14
15 Avantage : logique simple, contrôle du cache
16 Risque : cache stale (données anciennes) pendant TTL
```

4.8 Comment estimer le coût d'ElastiCache ?

Selon le mode choisi, ElastiCache facture notamment la capacité des nœuds ou des unités de traitement serverless, le stockage de données et de sauvegardes, ainsi que certains transferts. La disponibilité et le partitionnement multiplient les composants à prendre en compte.

Ce qui fait varier la facture :

- **Nombre de nœuds** : un cluster Redis en haute disponibilité (primary + replica) double le coût du nœud seul — exactement comme Multi-AZ sur RDS.
- **Cluster Mode Enabled** (sharding) : chaque shard supplémentaire est un nœud facturé en plus — utile pour la scalabilité, mais le coût grimpe linéairement avec le nombre de shards.
- **Transfert de données** : vérifier les flux entre zones, régions et services dans la page tarifaire courante.

Repère utile : ElastiCache n'est rentable que si le cache réduit suffisamment la charge sur RDS pour permettre une instance RDS plus petite, ou évite d'ajouter des Read Replicas RDS payants. Sur une charge de lecture très répétitive (mêmes clés interrogées des milliers de fois), le calcul est presque toujours favorable ; sur des requêtes peu répétées, le cache n'apporte rien et n'est qu'un coût supplémentaire.

 [Amazon ElastiCache Pricing](#)

5. Points importants et pièges fréquents

5.1 Pièges RDS et Bases de données

Piège	Réalité	Conséquence	Solution
RDS n'est pas auto-scalable en stockage	Faut augmenter manuellement (RDS) ou activer auto-scaling (Aurora)	Saturation disque → downtime	Vérifier "Storage autoscaling" dans RDS config
Multi-AZ RDS ≠ haute disponibilité lue	Multi-AZ = résilience (failover), pas scalabilité lecture	Bottleneck en lecture malgré Multi-AZ	Ajouter Read Replicas (asynchrones)
Chiffrement RDS doit être activé à la création	On ne peut pas l'activer après coup	Recréer l'instance = downtime	Checker "Encrypt at rest" lors création
Read Replica ≠ Multi-AZ	Replica = asynchrone, pour lectures. Multi-AZ = synchrone, failover	Confondre les deux gâche design	Multi-AZ pour haute dispo, Replicas pour scalabilité lecture
Snapshot RDS = backup manuel	Snapshots manuels ne s'auto-suppriment pas	Surcoûts stockage	Supprimer manuellement ou appliquer cycle vie

5.2 Pièges VPC et Réseau

Piège	Réalité	Conséquence	Solution
VPC Peering n'est pas transitif	$A \leftrightarrow B$ et $B \leftrightarrow C$ ne signifie pas $A \leftrightarrow C$	Communication A-C bloquée	Créer peering A-C explicitement ou utiliser Transit Gateway
Security Group ≠ Network ACL	SG = stateful (instance), NACL = stateless (subnet)	Oublier une règle sortante NACL bloque tout	Vérifier ACL entrée ET sortie

Internet Gateway et NAT Gateway ont le même modèle de coût	Une NAT Gateway facture sa durée et le volume traité ; les autres coûts réseau restent à compter	Sous-estimation du budget réseau	Utiliser des endpoints compatibles et mesurer les flux avant de dimensionner la sortie Internet
Security Group par défaut refuse tout	Sauf trafic sortant (autorisé par défaut)	Instances isolées jusqu'à ouverture ingress	Ajouter règles ingress explicites
Security Group : ALLOW vs DENY	SG = whitelist (ALLOW seulement), pas de DENY	Penser NACL pour bloquer IPs spécifiques	NACL pour explicite DENY
Associer route table incorrect	Associer table RT publique à subnet privé = accès internet non sécurisé	Instances "privées" exposées à Internet	Vérifier subnet <→ route table
ENI : Source/Dest Check activé par défaut	ENI refuse le trafic non-destiné à elle (sécurité)	Routeur/pare-feu ne peut pas forwarder	Désactiver source-dest-check pour routeurs
Attach ENI = Device index critique	Device index 0 = primary (obligatoire), 1+ = secondary	Erreur lors attach bloque l'instance	Vérifier device index disponible avant attach
Transit Gateway n'est pas gratuit	Les attachements et les données traitées contribuent au coût	Une topologie centralisée peut devenir coûteuse	Estimer le nombre d'attachements et les flux avant de choisir la topologie
Transit Gateway routing par défaut = tous allowed	TGW fait transiter tous les paquets par défaut	Communication imprévue entre VPCs	Restreindre via Route Tables TGW explicites

VPC CIDR overlap interdit dans Transit Gateway	Tous les VPCs attachés doivent avoir CIDR différents	Adresses en collision = paquets perdus	Planifier CIDR par VPC avant TGW
--	--	--	----------------------------------

5.3 Pièges Route 53 et DNS

Piège	Réalité	Conséquence	Solution
DNS Route 53 a un TTL	Les réponses sont cachées pendant le TTL	Changement DNS peut prendre 24h	Baisser TTL avant changement (300s)
Health Check ≠ Failover automatique	Health check détecte panne, failover redirection	Juste détecter ne suffit pas	Configurer failover + health check
Alias Records ≠ CNAME	Alias = pointeur AWS (gratuit, flexible), CNAME = alias DNS classique	Confondre risque problèmes CNAME root	Toujours Alias pour AWS resources (ALB, CloudFront)

⚠️ **TTL Route 53 trop court = coût de requêtes élevé**

Un TTL très court peut augmenter le nombre de résolutions DNS et donc le coût des requêtes. Le trafic HTTP n'est toutefois pas égal au nombre de résolutions : les résolveurs et les clients mettent les réponses en cache. Mesurez les requêtes DNS réelles au lieu de les déduire directement des requêtes applicatives.

Règle :

- TTL **300s** (5 min) pour la plupart des enregistrements stables
- TTL **60s** maximum pendant une migration ou un failover planifié
- Remonter le TTL à **300-3600s** après stabilisation ⚠️

5.4 Pièges DynamoDB

Piège	Réalité	Conséquence	Solution
DynamoDB provisionned vs on-demand	Mode provisionné = moins cher si prévisible	Charge imprévisible = throttling ou surcoûts	Choisir on-demand si variable, provisionné si stable
Partition key $\neq$ Sort key	Partition = hash (required), Sort = range (optional)	Query sans sort key = full table scan	Bien concevoir partition + sort key
DynamoDB TTL n'est pas immédiat	TTL supprime dans 24-48h après expiration	Données restent visibles brièvement	Ne pas compter sur TTL pour sécurité
Global Secondary Index (GSI) coûte	GSI = throughput supplémentaire à provisionner	Surcoûts si GSI mal utilisés	Bien planifier projections, ne créer que GSI utiles

5.5 Pièges ElastiCache

Piège	Réalité	Conséquence	Solution
Redis vs Memcached	Redis = persistance, structures complexes. Memcached = volatil, simple	Choisir Memcached pour données durables = perte	Redis pour sessions, Memcached pour cache éphémère
Cache-Aside pattern = possibilité cache stale	TTL peut garder données anciennes 1h	Utilisateurs voient données obsolètes	Réduire TTL ou implémenter invalidation manuelle
ElastiCache dans VPC $\neq$ accessible	Besoin Security Group + route table	EC2 ne peut pas accéder cache	Vérifier SG ElastiCache permet

depuis EC2 autre
subnet

EC2, même VPC

Cluster mode
disabled : une
seule shard

Pas de sharding = un
seul nœud max CPU

Bottleneck CPU
même avec plusieurs
replicas

Cluster mode
enabled pour
scalabilité

5.6 Pièges Architecture Générale

Piège	Réalité	Conséquence	Solution
Aurora Serverless = scaling pas instantané	Scaling automatique peut durer 30-60s	Latence pics pendant scaling	Pas idéal real-time, mieux RDS provisionné
VPC Endpoint S3 évite la NAT	Mais doit configurer policies explicites	S3 accès reste privé mais règles complexes	Créer endpoint + bucket policy restrictive
Tous les VPC Endpoints ont le même modèle de coût	Gateway endpoints et interface endpoints sont différents	Une multiplication d'interfaces peut augmenter la facture	Utiliser un gateway endpoint pour S3/DynamoDB et estimer les interfaces PrivateLink nécessaires

Ressources

Documentation officielle AWS

- [Amazon RDS Documentation](#)
- [Amazon DynamoDB Documentation](#)

- [Amazon VPC Documentation](#)
- [Amazon Route 53 Documentation](#)
- [Amazon ElastiCache Documentation](#)
- [AWS Transit Gateway](#)

Quiz interactif du chapitre

Choisissez une réponse : la correction expliquée apparaît immédiatement. Les questions et les propositions restent dans un ordre stable.

Le quiz interactif est disponible dans la version web du support.

Chapitre 5 — Automatisation, supervision et reprise d'activité

Vocabulaire

1 • RTO, RPO et sauvegardes

2 • Principes d'automatisation

3 • Infrastructure as Code

4 • Automatisation opérationnelle

5 • Déploiement avec Elastic Beanstalk

6 • Supervision avec CloudWatch

7 • Évaluation Well-Architected

8 • Files et événements

9 • Repères de certification

Ressources

Quiz du chapitre

Vocabulaire du chapitre

Terme	Définition
RPO	Recovery Point Objective : quantité maximale de données que l'organisation accepte de perdre, exprimée comme un point de reprise dans le temps.
RTO	Recovery Time Objective : délai maximal visé pour rétablir un service après un incident.
Infrastructure as Code	Description versionnée d'une infrastructure dans des fichiers interprétés par un outil de déploiement.

Template CloudFormation	Document JSON ou YAML qui décrit les ressources et leurs propriétés.
Stack CloudFormation	Ensemble de ressources créé et géré comme une unité à partir d'un template.
Drift	Écart entre la configuration déclarée dans le template et l'état réel des ressources.
Métrique	Série de valeurs numériques horodatées utilisée pour observer un système.
Alarme CloudWatch	Règle qui surveille une métrique et change d'état lorsqu'un seuil ou une condition est atteint.

:::info Le réseau et les bases de données mis en place au chapitre précédent constituent une architecture fonctionnelle mais encore déployée manuellement ; ce dernier chapitre referme la formation en automatisant ce déploiement et en donnant les clés pour évaluer et faire évoluer une architecture AWS dans la durée.

Objectifs du chapitre

À l'issue de ce chapitre, les stagiaires seront capables de :

- **Définir** les notions de RTO/RPO et mettre en œuvre une stratégie de sauvegarde avec AWS Backup et les snapshots EBS/RDS
- **Expliquer** les limites du déploiement manuel et l'intérêt de l'Infrastructure as Code
- **Écrire** un template CloudFormation en YAML et le déployer via la CLI
- **Utiliser** AWS Systems Manager pour administrer des instances à distance sans SSH
- **Déployer** une application avec Elastic Beanstalk et la comparer à une architecture serverless (Lambda)
- **Créer** des alarmes et des tableaux de bord CloudWatch pour superviser une infrastructure
- **Appliquer** les six piliers du AWS Well-Architected Framework à une étude de cas concrète
- **Utiliser** AWS Compute Optimizer pour ajuster le dimensionnement des ressources
- **Découpler** des composants applicatifs avec Amazon SQS et Amazon SNS
- **Concevoir** une architecture microservices sans serveur avec API Gateway et Step Functions, et justifier les choix de découplage
- **Situer** les certifications AWS (Cloud Practitioner à Solutions Architect Professional) et les domaines couverts par la SAA-C03 :::

1. RTO/RPO et Récupération de Sauvegarde

1.1 Définitions essentielles

Avant d'automatiser une infrastructure, il faut comprendre deux concepts critiques pour la **continuité de service** :

RTO (Recovery Time Objective) — Temps d'Indisponibilité Acceptable

RTO = combien de temps maximum l'application peut-elle rester indisponible avant que l'impact métier devienne intolérable ?

1 Exemples concrets :

2

3 Service	RTO typical	Raison
4 -----	-----	-----
5 Site e-commerce (Amazon)	5 minutes	Chaque minute sans vente = perte
6 Application interne (RH)	8 heures	Métier critique mais moins urgent
7 Service vidéo (Netflix)	30 minutes	Perte d'utilisateurs, mais pas urgent
8 Système bancaire	15 minutes	Réglementation stricte
9 API partenaire (non-vital)	4 heures	Impact mineur sur le business

RPO (Recovery Point Objective) — Quantité de Données Perdable

RPO = combien de données suis-je prêt à perdre en cas de sinistre ?

1 Exemples concrets :

2

3 Application	RPO	Raison
4 -----	-----	-----
5 E-commerce actif	5 minutes	Transactions en temps réel = critique
6 Logs d'application	1 jour	Données historiques, non urgentes
7 Données de client (CRM)	1 heure	Important pour relancer les clients
8 Backups archivés	30 jours	Archive long terme, peu critique

Relation RTO ↔ RPO

```
1 Scénario : Serveur RDS tombe en panne à 10:00
2
3 Stratégie 1 (RTO court, RPO court)
4   - Sauvegarde automatique toutes les 10 minutes
5   - Multi-AZ activé (failover < 2 minutes)
6   - RTO = 2 minutes, RPO = 10 minutes
7   - Coût : ★★★★★ (cher)
8
9 Stratégie 2 (RTO moyen, RPO moyen)
10  - Sauvegarde quotidienne (minuit)
11  - Backup lisible rapidement (1 heure pour restaurer)
12  - RTO = 1 heure, RPO = 24 heures
13  - Coût : ★★ (raisonnable)
14
15 Stratégie 3 (RTO long, RPO long)
16  - Sauvegarde hebdomadaire
17  - Pas de failover automatique
18  - RTO = 8 heures, RPO = 7 jours
19  - Coût : ★ (très bon marché)
```

1.2 Stratégies AWS pour Atteindre RTO/RPO

Technologie	RTO	RPO	Coût	Cas d'usage
Multi-AZ	< 2 min	≈ 0 min	Moyen	Haute dispo critique
Snapshots EBS	15-30 min	1 jour	Faible	Backup régulier
AWS Backup	1-4 heures	1-24 heures	Faible-Moyen	Backup centralisé
Read Replicas (RDS)	5-10 min	≈ 0 min	Moyen	Failover rapide BD
AWS Glacier	1-12 heures	Sans limite	Très faible	Archive long terme
Lambda + S3	10-60 min	1 heure	Très faible	Backup custom

1.3 AWS Backup — Service Centralisé de Sauvegarde

AWS Backup est un **service managé** pour centraliser et automatiser les sauvegardes de ressources AWS.

Ressources sauvegardables par AWS Backup

- 1 Compute & Storage:
- 2 ✓ Amazon EC2 instances
- 3 ✓ Amazon EBS volumes
- 4 ✓ Amazon EFS (Elastic File System)
- 5
- 6 Database:
- 7 ✓ Amazon RDS databases (MySQL, PostgreSQL, Oracle, SQL Server)
- 8 ✓ Amazon DynamoDB tables
- 9 ✓ Amazon Aurora databases
- 10
- 11 Backup Store:
- 12 ✓ AWS Storage Gateway
- 13 ✓ VMware vSphere

Déployer AWS Backup via CLI

On crée d'abord un **Backup Vault** (le coffre où seront stockées les sauvegardes), puis un **Backup Plan** (la planification) avec ses règles de rétention, et enfin une **Backup Selection** (les ressources à sauvegarder). Ces trois objets forment un système complet.

```
1  # 1. Créer un Backup Vault (conteneur pour les sauvegardes)
2  aws backup create-backup-vault \
3      --backup-vault-name mon-vault-production \
4      --region eu-west-3
5
6  # Output : BackupVaultArn
7
8  # 2. Créer un Backup Plan (planification automatique)
9  # Sauvegarder toutes les instances EC2 chaque jour à minuit
10 cat > backup-plan.json << 'EOF'
11 {
12     "BackupPlanName": "SauvegardeQuotidienne",
13     "Rules": [
14         {
15             "RuleName": "Sauvegarde Quotidienne",
16             "TargetBackupVaultName": "mon-vault-production",
```

```

17     "ScheduleExpression": "cron(0 0 ? * * *)",
18     "StartWindowMinutes": 60,
19     "CompletionWindowMinutes": 120,
20     "Lifecycle": {
21         "DeleteAfterDays": 30,
22         "MoveToColdStorageAfterDays": 7
23     }
24 }
25 ]
26 }
27 EOF
28
29 # 3. Créer le plan
30 aws backup create-backup-plan \
31     --backup-plan file://backup-plan.json \
32     --region eu-west-3

```

:::success **Résultat attendu :**

```

1  {
2      "BackupPlanId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
3      "BackupPlanArn": "arn:aws:backup:eu-west-3:123456789012:backup-plan:a1b2c3d
4      "CreationDate": "2026-03-24T10:15:00.000Z",
5      "VersionId": "NDEzMWVlNzgtMTk2My00NjMxLWJlOTYt"
6  }

```

...

```

1  # 4. Assigner des ressources au plan
2  aws backup create-backup-selection \
3      --backup-plan-id mon-plan-123 \
4      --backup-selection file://selection.json \
5      --region eu-west-3
6
7  # 5. Vérifier les backups créés
8  aws backup list-recovery-points-by-resource \
9      --resource-arn "arn:aws:ec2:eu-west-3:123456789:instance/i-12345678" \
10     --region eu-west-3
11
12 # 6. Restaurer une instance EC2 à partir d'un backup

```

```
13 aws backup start-restore-job \  
14 --recovery-point-arn "arn:aws:backup:eu-west-3:123456789:recovery-point:..."\  
15 --iam-role-arn "arn:aws:iam::123456789:role/AWSBackupDefaultRole" \  
16 --metadata key1=value1,key2=value2 \  
17 --region eu-west-3
```

:::success **Résultat attendu :**

```
1  # list-recovery-points-by-resource :  
2  {  
3      "RecoveryPoints": [  
4          {  
5              "RecoveryPointArn": "arn:aws:backup:eu-west-3:123456789012:recovery  
6              "CreationDate": "2026-03-24T00:05:12.000Z",  
7              "Status": "COMPLETED",  
8              "BackupSizeInBytes": 8589934592,  
9              "BackupVaultName": "mon-vault-production"  
10         }  
11     ]  
12 }  
13  
14 # start-restore-job :  
15 {  
16     "RestoreJobId": "restore-job-0abc123def456"  
17 }
```

...

:::warning **Coûts AWS Backup** : la facture dépend du volume protégé, du type de stockage, des restaurations, des copies interrégions ou intercomptes et de la durée de rétention. Relevez ces paramètres dans le plan de sauvegarde, puis appliquez les tarifs officiels de la région. Vérifiez régulièrement la consommation dans **Cost Explorer** et supprimez les rétentions sans justification métier. :::

Bonnes pratiques AWS Backup

- 1 ✓ Planifier les sauvegardes en dehors des heures de pic
- 2 ✓ Utiliser des Backup Vaults séparés pour Prod/Staging/Dev
- 3 ✓ Tester régulièrement les restaurations (RTO réel)
- 4 ✓ Définir une rétention appropriée (30j prod, 7j dev)
- 5 ✓ Combiner avec CloudWatch Events pour alertes

1.4 Snapshots EBS et RDS — Sauvegardes Point-in-Time

Snapshot EBS = photo point-in-time d'un volume EC2

Un snapshot EBS capture l'état d'un disque à un instant donné. Il est stocké dans S3 (géré par AWS) et peut servir à restaurer un volume ou à déplacer une instance vers une autre région.

```
1  # Créer un snapshot EBS manuel
2  aws ec2 create-snapshot \
3      --volume-id vol-12345678 \
4      --description "Sauvegarde avant migration" \
5      --region eu-west-3
6
7  # Lister les snapshots
8  aws ec2 describe-snapshots \
9      --owner-ids self \
10     --region eu-west-3
11
12 # Créer un volume à partir du snapshot
13 aws ec2 create-volume \
14     --snapshot-id snap-12345678 \
15     --availability-zone eu-west-3a \
16     --region eu-west-3
```

:::success **Résultat attendu :**

```
1  # create-snapshot :
2  {
3      "SnapshotId": "snap-0abc123def456789a",
4      "VolumeId": "vol-12345678",
5      "State": "pending",
6      "StartTime": "2026-03-24T09:00:00.000Z",
```

```

7     "Progress": "0%",
8     "Description": "Sauvegarde avant migration"
9 }
10
11 # describe-snapshots (après quelques minutes) :
12 {
13     "Snapshots": [
14         {
15             "SnapshotId": "snap-0abc123def456789a",
16             "State": "completed",
17             "Progress": "100%",
18             "VolumeSize": 20
19         }
20     ]
21 }

```

...

Snapshot RDS = sauvegarde complète de base de données

```

1  # Créer un snapshot RDS manuel
2  aws rds create-db-snapshot \
3      --db-instance-identifiant production-db \
4      --db-snapshot-identifiant prod-db-backup-2026-03-24 \
5      --region eu-west-3
6
7  # Lister les snapshots
8  aws rds describe-db-snapshots \
9      --region eu-west-3
10
11 # Restaurer une BD à partir d'un snapshot
12 aws rds restore-db-instance-from-db-snapshot \
13     --db-instance-identifiant production-db-restored \
14     --db-snapshot-identifiant prod-db-backup-2026-03-24 \
15     --region eu-west-3

```

:::success **Résultat attendu :**

```

1  # create-db-snapshot :
2  {
3      "DBSnapshot": {

```

```

4      "DBSnapshotIdentifier": "prod-db-backup-2026-03-24",
5      "DBInstanceIdentifier": "production-db",
6      "Status": "creating",
7      "Engine": "mysql",
8      "AllocatedStorage": 100,
9      "SnapshotCreateTime": "2026-03-24T09:30:00.000Z"
10    }
11  }
12
13  # describe-db-snapshots (après quelques minutes) :
14  {
15    "DBSnapshots": [
16      {
17        "DBSnapshotIdentifier": "prod-db-backup-2026-03-24",
18        "Status": "available",
19        "PercentProgress": 100
20      }
21    ]
22  }

```

...

Résultat attendu : `create-db-snapshot` retourne un JSON avec le `DBSnapshotIdentifier` et le statut `creating`. Quelques minutes plus tard, `describe-db-snapshots` affiche le statut `available`. La restauration (`restore-db-instance-from-db-snapshot`) crée une **nouvelle instance** RDS — pas un remplacement de l'existante.

2. Pourquoi automatiser dans le Cloud ?

2.1 Le déploiement manuel : une source d'erreurs

Dans un environnement traditionnel, déployer une infrastructure peut prendre **des jours, voire des semaines**. Les équipes IT doivent :

- Commander du matériel physique
- Installer les systèmes d'exploitation
- Configurer le réseau et les accès

- Mettre à jour les pare-feu et les politiques de sécurité
- Documenter tous les changements (quand c'est fait...)

Le problème : chaque déploiement manuel génère des **incohérences**. Deux administrateurs ne font jamais exactement la même chose. Certains oublis passent inaperçus :

```
1  Infrastructure créée manuellement :
2      Admin Alice crée un VPC le 10 janvier
3      → Configure 2 subnets, 1 IGW, 1 NAT Gateway
4      → Oublie de documenter les CIDR utilisés
5      → Quitte l'entreprise 6 mois après
6
7      Admin Bob doit déployer l'infrastructure
8      → Cherche la documentation (introuvable)
9      → Recrée un nouveau VPC similaire MAIS différent
10     → Incompatibilité lors du peering : PERTE DE TEMPS
11
12  Résultat : deux "mêmes" infrastructures qui ne sont PAS identiques
```

2.2 L'automatisation dans le Cloud : standardisation et rapidité

Dans le cloud AWS, l'**automatisation** permet de :

- **Standardiser** les environnements → Prod et Dev sont identiques
- **Réduire** les délais de déploiement → Quelques minutes au lieu de semaines
- **Limiter** les erreurs humaines → Le code est testé avant déploiement
- **Faciliter** la montée en charge → Spawner 100 instances avec un clic

```
1  Infrastructure automatisée avec CloudFormation :
2      Jour 1 : Écrire un template YAML (1-2 heures)
3      Jour 2 : Déployer en Prod exactement identique au Dev (2 minutes)
4      Jour 3 : Déployer en Test (2 minutes)
5      Jour 4 : Déployer pour un client différent (2 minutes)
6
7      Avantage : zéro divergence entre les environnements
8                  zéro oubli de configuration
9                  traçabilité complète (versionning Git du template)
```

2.3 Les outils AWS pour l'automatisation

Outil	Fonction	Cas d'usage
CloudFormation	Déploiement d'infrastructures as code (IaC)	Créer VPC, EC2, RDS, S3 en une seule opération
Quick Starts	Templates CloudFormation préconfigurés par AWS	Déployer rapidement une architecture éprouvée (VPC + ELB + RDS)
Systems Manager	Gestion centralisée et automatisation opérationnelle	Exécuter des scripts, patcher les serveurs, inventorier les ressources
Elastic Beanstalk	Déploiement PaaS simplifié d'applications web	Déployer une app Node.js sans gérer l'infrastructure
CLI / SDK	Automatisation par scripts et développement	Orchestrer plusieurs services AWS via Python/Bash

Référence : [AWS Systems Manager Automation](#) Référence : [AWS CloudFormation Documentation](#)

2.4 AWS Quick Starts — Templates Éprouvés

AWS Quick Starts sont des **templates CloudFormation préconfigurés et validés par AWS** pour déployer des architectures complètes en quelques clics.

Qu'est-ce qu'un Quick Start ?

```
1 Quick Start = CloudFormation template complet + documentation + bonnes pratique
2
3 Exemplar :
4 - Déployer une architecture WordPress hautement disponible
5   (VPC + ALB + Auto Scaling + RDS Multi-AZ + CloudFront)
6 - Sans avoir à écrire 500 lignes YAML
7 - Basé sur des best practices AWS
8 - Testé et validé en production
```

Quick Starts courants (exemples)

Quick Start	Qu'il déploie	Temps
WordPress on AWS	VPC, ALB, Auto Scaling, RDS, CloudFront	10-15 min
Kubernetes on AWS	EKS cluster complet avec worker nodes	20-30 min
SQL Server on EC2	Instance EC2 + RDS SQL Server + backup	15 min
Jenkins on AWS	Jenkins Master + Agent instances + monitoring	10 min
Hadoop on AWS	Cluster EMR complet multi-node	20 min

EKS (Elastic Kubernetes Service, déjà présenté au Chapitre 1 section 5) est le service AWS de Kubernetes managé. **Hadoop** est un framework open source historique de traitement de données massives (Big Data) : il répartit le calcul et le stockage sur un grand nombre de machines pour traiter des volumes trop importants pour un seul serveur. **EMR** (Elastic MapReduce) est le service managé AWS qui déploie et exploite des clusters Hadoop (ainsi que des outils compatibles comme Spark) sans que vous ayez à installer et administrer vous-même les serveurs du cluster.

Accéder aux Quick Starts

Les Quick Starts sont accessibles depuis la console CloudFormation ou directement via CLI. Voici comment les trouver et les utiliser :

```
1 # 1. Via la console AWS → CloudFormation → Quick Starts
2 #   https://aws.amazon.com/quickstarts/
3
4 # 2. Chercher un Quick Start par domaine :
5 #   - VPC / Networking
6 #   - Databases & Analytics
```

```
7  # - Business Applications
8  # - DevOps Tools
9
10 # 3. Cliquer sur "Launch" → remplir les paramètres → Create Stack
```

Avantages des Quick Starts

- 1 ✓ Gain de temps : architecture complète en 15 min au lieu de 3-4 heures
- 2 ✓ Bonnes pratiques : design conforme au Well-Architected Framework
- 3 ✓ Haute disponibilité : Multi-AZ, load balancing, failover automatique
- 4 ✓ Maintenance : AWS met à jour les templates régulièrement
- 5 ✓ Support : documentation et troubleshooting fournis
- 6 ✓ Flexibilité : vous pouvez modifier les templates après déploiement

Exemple : Déployer WordPress via Quick Start

Plutôt que de créer manuellement VPC, ALB, RDS et Auto Scaling, on lance une seule commande CloudFormation qui déploie toute l'architecture WordPress en ~10 minutes :

```
1  # 2. Chercher "WordPress"
2  # 3. Cliquer sur "Launch Quick Start"
3  # 4. Remplir les paramètres (taille VPC, taille RDS, DNS, etc.)
4  # 5. Vérifier les options VPC et subnet
5  # 6. Cliquer "Create Stack"
6  # 7. Attendre ~10 minutes
7  # 8. CloudFormation affiche l'URL du site WordPress
8
9  # Vous pouvez aussi utiliser AWS CLI :
10 aws cloudformation create-stack \
11     --stack-name wordpress-production \
12     --template-url "https://s3.amazonaws.com/quickstart-reference/wordpress/latest" \
13     --parameters \
14         ParameterKey=KeyName,ParameterValue=ma-clé-ssh \
15         ParameterKey=InstanceType,ParameterValue=t3.small \
16         ParameterKey=DBInstanceClass,ParameterValue=db.t3.micro \
17     --region eu-west-3
```

:::success **Résultat attendu :**

```
1  {
```

```
2     "StackId": "arn:aws:cloudformation:eu-west-3:123456789012:stack/wordpress-p
3 }
```

La stack `wordpress-production` est en cours de création. Suivez la progression dans la console CloudFormation → Events, ou via `aws cloudformation describe-stack-events --stack-name wordpress-production`. Après ~10 minutes, le statut passe à `CREATE_COMPLETE` et l'URL WordPress est disponible dans les outputs de la stack. ...

3. AWS CloudFormation — Infrastructure as Code

3.1 Qu'est-ce que CloudFormation ?

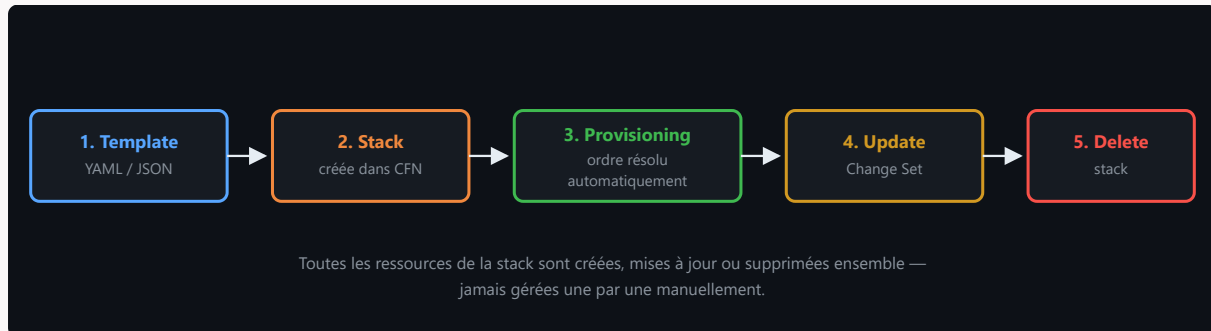
AWS CloudFormation est un service qui permet de **modéliser et déployer des ressources AWS** sous forme de code.

Plutôt que de créer manuellement une VPC, des sous-réseaux, des groupes de sécurité ou des instances EC2 dans la console, on les **décrit dans un template JSON ou YAML** et CloudFormation s'occupe du reste.

```
1 Analogie : CloudFormation est comme une RECETTE DE CUISINE pour construire une
2
3 Recette classique :
4   Ingrédients : 1 VPC, 2 subnets, 1 IGW, 3 EC2
5   Étapes :
6     1. Créer la VPC avec CIDR 10.0.0.0/16
7     2. Créer subnet public 10.0.1.0/24
8     3. Créer subnet privé 10.0.2.0/24
9     4. Ajouter une IGW et l'attacher à la VPC
10    5. Créer 3 instances EC2 dans le subnet public
11    6. Configurer les groupes de sécurité
12
13 Avantage : la recette peut être réutilisée 100 fois identiquement
14            et versionnée dans Git
```

3.2 Fonctionnement simplifié de CloudFormation

1. **Rédiger un template** (JSON/YAML) décrivant les ressources à créer
2. **Créer une stack** dans CloudFormation (via console ou CLI)
3. **AWS provisionne** toutes les ressources **dans le bon ordre** (résout les dépendances automatiquement)
4. **Mettre à jour** la stack pour faire évoluer l'infrastructure (ajout, suppression, modification de ressources)
5. **Supprimer** la stack si plus besoin (CloudFormation supprime toutes les ressources associées)



Lecture du schéma. Le template décrit l'état attendu. CloudFormation analyse les dépendances, appelle les API AWS et regroupe les ressources obtenues dans une stack. Une mise à jour compare la nouvelle déclaration à la stack existante avant d'appliquer les changements nécessaires.

3.3 Exemple simple de template CloudFormation (YAML)

Voici un template minimaliste créant une VPC, un subnet, et une instance EC2 :

```
1  # Version du format CloudFormation (toujours 2010-09-09)
2  AWSTemplateFormatVersion: '2010-09-09'
3
4  # Description brève du template
5  Description: |
6    Infrastructure AWS simple pour débutants
7    Crée une VPC, un subnet public, une IGW et une instance EC2
8
9  # Paramètres (permet de rendre le template réutilisable)
10 # Ici, on paramètre le type d'instance EC2 pour pouvoir changer facilement
11 Parameters:
12   InstanceType:
13     Type: String
```

```
14     Default: t2.micro
15     Description: Type d'instance EC2 (t2.micro, t2.small, etc.)
16     AllowedValues:
17         - t2.micro
18         - t2.small
19         - t3.micro
20
21     # Les ressources à créer
22     Resources:
23         # Ressource 1 : Créer une VPC
24         # Chaque ressource a un identifiant logique (MonVPC) et un type AWS
25         MonVPC:
26             # Type de ressource AWS
27             Type: AWS::EC2::VPC
28             # Propriétés spécifiques
29             Properties:
30                 # CIDR block : plage d'adresses IP pour cette VPC
31                 CidrBlock: 10.0.0.0/16
32                 # Activer le hostname DNS
33                 EnableDnsHostnames: true
34                 # Tags pour identifier facilement
35                 Tags:
36                     - Key: Name
37                       Value: MonVPC-Formation
38
39         # Ressource 2 : Créer un subnet public dans la VPC
40         # !Ref est une fonction intrinsèque qui référence une autre ressource
41         MonSubnetPublic:
42             Type: AWS::EC2::Subnet
43             Properties:
44                 # !Ref MonVPC = ID logique de la VPC créée au-dessus
45                 VpcId: !Ref MonVPC
46                 # Plage d'adresses pour ce subnet (doit être dans le CIDR de la VPC)
47                 CidrBlock: 10.0.1.0/24
48                 # Zone de disponibilité (peut varier, AWS en assigne une par défaut)
49                 AvailabilityZone: eu-west-1a
50                 # Assigner automatiquement une IP publique aux instances
51                 MapPublicIpOnLaunch: true
52                 Tags:
53                     - Key: Name
```

```
54         Value: MonSubnet-Public
55
56     # Ressource 3 : Créer une Internet Gateway (permet l'accès à Internet)
57     MonIGW:
58         Type: AWS::EC2::InternetGateway
59         Properties:
60             Tags:
61                 - Key: Name
62                   Value: MonIGW
63
64     # Ressource 4 : Attacher l'IGW à la VPC
65     # CloudFormation comprend automatiquement que cette ressource dépend de MonVP
66     AttachIGW:
67         Type: AWS::EC2::VPCGatewayAttachment
68         Properties:
69             # Référence à la VPC créée
70             VpcId: !Ref MonVPC
71             # Référence à l'IGW créé
72             InternetGatewayId: !Ref MonIGW
73
74     # Ressource 5 : Groupe de sécurité (pare-feu simplifié)
75     MonSecurityGroup:
76         Type: AWS::EC2::SecurityGroup
77         Properties:
78             # Description obligatoire
79             GroupDescription: Autorise SSH et HTTP
80             # Associer au VPC
81             VpcId: !Ref MonVPC
82             # Règles de trafic entrant
83             SecurityGroupIngress:
84                 # Permettre SSH depuis n'importe quelle IP (⚠ À ÉVITER EN PROD)
85                 - IpProtocol: tcp
86                   FromPort: 22
87                   ToPort: 22
88                   CidrIp: 0.0.0.0/0
89                   Description: SSH access
90             # Permettre HTTP
91             - IpProtocol: tcp
92               FromPort: 80
93               ToPort: 80
```



```
94         CidrIp: 0.0.0.0/0
95         Description: HTTP access
96     Tags:
97         - Key: Name
98           Value: MonSG-Formation
99
100     # Ressource 6 : Instance EC2
101     MonInstance:
102         Type: AWS::EC2::Instance
103         Properties:
104             # AMI ID (Ubuntu 24.04 LTS en eu-west-1)
105             ImageId: ami-0d71ea30463e0ff8d
106             # Type d'instance (utilise le paramètre défini au-dessus)
107             InstanceType: !Ref InstanceType
108             # Placer dans le subnet créé
109             SubnetId: !Ref MonSubnetPublic
110             # Associer le security group
111             SecurityGroupIds:
112                 - !Ref MonSecurityGroup
113             # Script à exécuter au démarrage (user data)
114             UserData:
115                 Fn::Base64: |
116                     #!/bin/bash
117                     # Mises à jour système
118                     apt-get update
119                     apt-get install -y nginx curl
120                     # Démarrer Nginx
121                     systemctl start nginx
122                     systemctl enable nginx
123                     # Créer une page de test
124                     echo "<h1>Bonjour du serveur créé par CloudFormation</h1>" > /var/www
125             Tags:
126                 - Key: Name
127                   Value: MonServeur-Formation
128
129     # Outputs : affiche les résultats après déploiement
130     # Utile pour récupérer les IP, URLs, etc.
131     Outputs:
132         # Affiche l'ID de la VPC créée
133         VPCId:
```

```

134     Description: ID de la VPC
135     # !Ref récupère l'ID physique de la ressource
136     Value: !Ref MonVPC
137     # Export permet à d'autres stacks de référencer cette valeur
138     Export:
139         Name: MonVPC-Id
140
141     # Affiche l'IP publique de l'instance
142     PublicIP:
143         Description: Adresse IP publique de l'instance EC2
144         # !GetAtt récupère un attribut spécifique d'une ressource
145         Value: !GetAtt MonInstance.PublicIp
146         Export:
147             Name: MonServeur-PublicIP
148
149     # Affiche l'URL HTTP pour accéder au serveur
150     WebServerURL:
151         Description: URL pour accéder au serveur web
152         # !Sub remplace les variables ${...} par leurs valeurs
153         Value: !Sub 'http://${MonInstance.PublicIp}'

```

3.3 (suite) — Exemple Production : VPC + Serveur Web Apache

Voici un template **production-ready** déployant une **VPC complète avec serveur web Apache** et un Security Group. C'est le template utilisé dans les ateliers Dawan.

```

1  AWSTemplateFormatVersion: '2010-09-09'
2  Description: |
3      Déploiement d'une VPC + serveur web Apache via CloudFormation
4      - VPC avec CIDR 10.0.0.0/16
5      - 1 subnet public (10.0.1.0/24)
6      - Internet Gateway + route vers l'extérieur
7      - Security Group (SSH + HTTP)
8      - Instance EC2 t3.micro avec Apache2 automatiquement installé
9      - Output : URL publique du serveur web
10
11     # Paramètres pour rendre le template réutilisable
12     Parameters:
13         KeyPairName:

```

```
14     Type: AWS::EC2::KeyPair::KeyName
15     Description: Nom de la paire de clés EC2 existante (pour SSH)
16
17     InstanceType:
18         Type: String
19         Default: t3.micro
20         AllowedValues:
21             - t3.micro
22             - t3.small
23             - t2.micro
24         Description: Type d'instance EC2
25
26     # Les ressources à créer
27     Resources:
28         # VPC
29         FormationVPC:
30             Type: AWS::EC2::VPC
31             Properties:
32                 CidrBlock: 10.0.0.0/16
33                 EnableDnsSupport: true
34                 EnableDnsHostnames: true
35                 Tags:
36                     - Key: Name
37                       Value: Formation-VPC-WebLab
38
39         # Subnet public
40         PublicSubnet:
41             Type: AWS::EC2::Subnet
42             Properties:
43                 VpcId: !Ref FormationVPC
44                 CidrBlock: 10.0.1.0/24
45                 AvailabilityZone: eu-west-3a
46                 MapPublicIpOnLaunch: true
47                 Tags:
48                     - Key: Name
49                       Value: Formation-Subnet-Public
50
51         # Internet Gateway
52         InternetGateway:
53             Type: AWS::EC2::InternetGateway
```

```
54     Properties:
55     Tags:
56         - Key: Name
57           Value: Formation-IGW
58
59     # Attacher IGW à la VPC
60     AttachGateway:
61         Type: AWS::EC2::VPCGatewayAttachment
62         Properties:
63             VpcId: !Ref FormationVPC
64             InternetGatewayId: !Ref InternetGateway
65
66     # Table de routage publique
67     PublicRouteTable:
68         Type: AWS::EC2::RouteTable
69         Properties:
70             VpcId: !Ref FormationVPC
71             Tags:
72                 - Key: Name
73                   Value: Formation-PublicRouteTable
74
75     # Route vers l'IGW (tout ce qui va en dehors du VPC)
76     PublicRoute:
77         Type: AWS::EC2::Route
78         DependsOn: AttachGateway
79         Properties:
80             RouteTableId: !Ref PublicRouteTable
81             DestinationCidrBlock: 0.0.0.0/0
82             GatewayId: !Ref InternetGateway
83
84     # Associer la route table au subnet
85     SubnetRouteTableAssociation:
86         Type: AWS::EC2::SubnetRouteTableAssociation
87         Properties:
88             SubnetId: !Ref PublicSubnet
89             RouteTableId: !Ref PublicRouteTable
90
91     # Security Group (pare-feu)
92     WebServerSecurityGroup:
93         Type: AWS::EC2::SecurityGroup
```

```
94     Properties:
95         GroupDescription: Permettre HTTP et SSH
96         VpcId: !Ref FormationVPC
97         SecurityGroupIngress:
98             # SSH (port 22) - accès depuis n'importe où (⚠ en production : limiter
99             - IpProtocol: tcp
100             FromPort: 22
101             ToPort: 22
102             CidrIp: 0.0.0.0/0
103             Description: SSH access
104             # HTTP (port 80) - serveur web public
105             - IpProtocol: tcp
106             FromPort: 80
107             ToPort: 80
108             CidrIp: 0.0.0.0/0
109             Description: HTTP web server
110         Tags:
111             - Key: Name
112             Value: Formation-WebServerSG
113
114     # Instance EC2
115     WebServerInstance:
116         Type: AWS::EC2::Instance
117         Properties:
118             ImageId: ami-04a92520784b94538 # Amazon Linux 2023 (eu-west-3)
119             InstanceType: !Ref InstanceType
120             KeyName: !Ref KeyPairName
121             SubnetId: !Ref PublicSubnet
122             SecurityGroupIds:
123                 - !Ref WebServerSecurityGroup
124             # Script pour configurer Apache2
125             UserData:
126                 Fn::Base64: !Sub |
127                 #!/bin/bash
128                 # Mise à jour du système
129                 yum update -y
130
131                 # Installer Apache HTTP Server
132                 yum install -y httpd
133
```

```
134     # Activer et démarrer Apache
135     systemctl enable httpd
136     systemctl start httpd
137
138     # Créer une page HTML de test
139     cat > /var/www/html/index.html << 'ENDHTML'
140     <!DOCTYPE html>
141     <html>
142     <head>
143         <title>CloudFormation - Formation AWS</title>
144         <style>
145             body { font-family: Arial, sans-serif; margin: 40px; }
146             h1 { color: #FF9900; }
147         </style>
148     </head>
149     <body>
150         <h1>Bienvenue sur votre serveur web CloudFormation!</h1>
151         <p>Cette instance EC2 a été déployée automatiquement via CloudFor
152         <p><strong>Informations serveur :</strong></p>
153         <ul>
154             <li>Instance ID : ${AWS::StackId}</li>
155             <li>Région : ${AWS::Region}</li>
156             <li>Instance Type : ${InstanceType}</li>
157         </ul>
158     </body>
159     </html>
160     ENDHTML
161
162     Tags:
163     - Key: Name
164       Value: Formation-WebServer
165
166 # Outputs - affiche les résultats utiles après déploiement
167 Outputs:
168   WebServerURL:
169     Description: URL publique du serveur web
170     Value: !Sub 'http://${WebServerInstance.PublicDnsName}'
171     Export:
172       Name: !Sub '${AWS::StackName}-WebServerURL'
173
```

```

174 WebServerPublicIP:
175     Description: Adresse IP publique de l'instance
176     Value: !Sub '${WebServerInstance.PublicIp}'
177
178 SSHCommand:
179     Description: Commande SSH pour se connecter au serveur
180     Value: !Sub 'ssh -i /chemin/vers/cle.pem ec2-user@${WebServerInstance.Publi
181
182 SecurityGroupId:
183     Description: ID du Security Group
184     Value: !Ref WebServerSecurityGroup

```

Déployer ce template

:::info Une activité pratique permet d’approfondir le déploiement de ce template CloudFormation. :::

```

1  # 1. Créer la stack depuis le fichier YAML local
2  aws cloudformation create-stack \
3      --stack-name formation-webserver-lab \
4      --template-body file://vpc-webserver.yaml \
5      --parameters \
6          ParameterKey=KeyPairName,ParameterValue=ma-clé-ssh \
7          ParameterKey=InstanceType,ParameterValue=t3.micro \
8      --region eu-west-3
9
10 # Attendre que la stack soit créée (statut CREATE_COMPLETE)
11 aws cloudformation wait stack-create-complete \
12     --stack-name formation-webserver-lab \
13     --region eu-west-3
14
15 # 2. Récupérer l'URL publique du serveur
16 aws cloudformation describe-stacks \
17     --stack-name formation-webserver-lab \
18     --query 'Stacks[0].Outputs[?OutputKey=='WebServerURL'].OutputValue' \
19     --output text \
20     --region eu-west-3
21
22 # Output : http://ec2-12-34-56-78.eu-west-3.compute.amazonaws.com
23 # → Ouvrir cette URL dans un navigateur pour voir le serveur Apache
24

```

```

25 # 3. Supprimer toute l'infrastructure quand vous avez terminé
26 aws cloudformation delete-stack \
27     --stack-name formation-webserver-lab \
28     --region eu-west-3
29
30 # Attendre la suppression complète
31 aws cloudformation wait stack-delete-complete \
32     --stack-name formation-webserver-lab \
33     --region eu-west-3

```

:::success **Résultat attendu :**

```

1 # create-stack :
2 {
3     "StackId": "arn:aws:cloudformation:eu-west-3:123456789012:stack/formation-w
4 }
5
6 # (après wait stack-create-complete – ~3 à 5 minutes)
7 # describe-stacks → WebServerURL :
8 http://ec2-15-236-78-42.eu-west-3.compute.amazonaws.com
9
10 # Le navigateur affiche la page HTML avec "Bienvenue sur votre serveur web Clou

```

:::

:::warning **IAM requis pour CloudFormation :** Pour déployer ce template, l'utilisateur (ou le rôle IAM) doit avoir les permissions de créer des ressources EC2, VPC, Security Groups. En entreprise, créer un rôle IAM dédié `CloudFormationDeployRole` avec les permissions nécessaires, plutôt que d'utiliser un compte admin. :::

:::danger **delete-stack supprime toutes les ressources :** La commande `delete-stack` détruit la VPC, le subnet, l'IGW, le Security Group ET l'instance EC2 de manière irréversible. Toutes les données stockées sur l'instance EBS seront perdues. Toujours vérifier `--stack-name` avant d'exécuter. :::

Points clés de ce template production

Élément	Explication	Bonne pratique
VPC CIDR 10.0.0.0/16	Classe privée standard pour les VPC	Utiliser RFC 1918 (10.x, 172.16.x, 192.168.x)

Subnet 10.0.1.0/24	Plage de 251 adresses disponibles	Laisser de l'espace pour futurs subnets
IGW + Route 0.0.0.0/0	Rend le subnet public et accessible d'Internet	Nécessaire pour un serveur web public
Security Group restrictif	HTTP (80) + SSH (22) explicites	En prod : limiter SSH à des IPs spécifiques
UserData script	Configure Apache automatiquement au lancement	Évite la configuration manuelle post-lancement
Outputs	Affiche l'URL et l'IP après déploiement	Indispensable pour que l'utilisateur sache comment accéder
Tags	Identification et traçabilité	Permet le suivi des ressources pour la facturation

3.4 Déployer le template avec AWS CLI

Une fois le template rédigé, on peut le déployer depuis la ligne de commande :

:::info **Valider le template avant déploiement** : Avant de créer une stack, il est conseillé de valider la syntaxe YAML avec `aws cloudformation validate-template --template-body file://mon-fichier.yaml`. Cette commande vérifie la syntaxe mais pas la validité des valeurs (AMI ID, type d'instance, etc.). :::

```

1  # 1. Créer la stack (remplacer mon-fichier.yaml par le chemin réel)
2  aws cloudformation create-stack \
3      --stack-name ma-premiere-stack \
4      --template-body file://mon-fichier.yaml \
5      --parameters ParameterKey=InstanceType,ParameterValue=t2.micro \
6      --region eu-west-1
7
8  # Résultat attendu : StackId
9  # Output : arn:aws:cloudformation:eu-west-1:123456789:stack/ma-premiere-stack/g
10
11 # 2. Suivre la progression du déploiement
12 aws cloudformation describe-stack-events \

```

```

13  --stack-name ma-premiere-stack \
14  --region eu-west-1
15
16  # 3. Une fois CREATE_COMPLETE, afficher les outputs
17  aws cloudformation describe-stacks \
18  --stack-name ma-premiere-stack \
19  --query 'Stacks[0].Outputs' \
20  --region eu-west-1
21
22  # 4. Pour mettre à jour la stack (ex. changer le type d'instance)
23  aws cloudformation update-stack \
24  --stack-name ma-premiere-stack \
25  --template-body file://mon-fichier.yaml \
26  --parameters ParameterKey=InstanceType,ParameterValue=t2.small \
27  --region eu-west-1
28
29  # 5. Supprimer la stack (attention : cela supprime TOUTES les ressources)
30  aws cloudformation delete-stack \
31  --stack-name ma-premiere-stack \
32  --region eu-west-1

```

:::success **Résultat attendu :**

```

1  # create-stack :
2  {
3      "StackId": "arn:aws:cloudformation:eu-west-1:123456789012:stack/ma-premiere
4  }
5
6  # describe-stack-events (extrait) :
7  {
8      "StackEvents": [
9          {
10             "StackId": "arn:aws:cloudformation:eu-west-1:...",
11             "EventId": "...",
12             "ResourceStatus": "CREATE_COMPLETE",
13             "ResourceType": "AWS::EC2::Instance",
14             "LogicalResourceId": "MonInstance",
15             "Timestamp": "2026-03-24T10:32:15.000Z"
16         },
17         {

```

```

18         "ResourceStatus": "CREATE_COMPLETE",
19         "ResourceType": "AWS::CloudFormation::Stack",
20         "LogicalResourceId": "ma-premiere-stack"
21     }
22 ]
23 }
24
25 # describe-stacks (Outputs) :
26 [
27     {
28         "OutputKey": "PublicIP",
29         "OutputValue": "54.12.34.56",
30         "Description": "Adresse IP publique de l'instance EC2"
31     },
32     {
33         "OutputKey": "WebServerURL",
34         "OutputValue": "http://54.12.34.56",
35         "Description": "URL pour accéder au serveur web"
36     }
37 ]

```

...

...**danger Attention — delete-stack supprime TOUTES les ressources !** La commande `aws cloudformation delete-stack` détruit définitivement toutes les ressources créées par la stack (instances EC2, VPC, RDS, S3...). Il n'y a pas de corbeille. Assurez-vous d'avoir des sauvegardes et de cibler la bonne stack avant d'exécuter cette commande. ...

3.5 Avantages de CloudFormation

Avantage	Bénéfice pédagogique
Automation complète	Créer/détruire une infrastructure complexe en quelques minutes
Gestion des dépendances	CloudFormation sait que l'IGW doit être créée AVANT d'être attachée à la VPC
Versionning	Stocker les templates dans Git, tracer tous les changements

Reproductibilité	Déployer exactement la même infrastructure en 10 environnements différents
Rollback	Si une erreur survient, CloudFormation annule les changements automatiquement
Coût	CloudFormation est gratuit (on paie seulement les ressources créées)

⚠ **CloudFormation Drift — Divergence de configuration** : Si vous modifiez manuellement des ressources gérées par CloudFormation (via la console ou la CLI), elles entrent en état de **drift** — elles ne correspondent plus au template. CloudFormation ne détecte pas ces écarts automatiquement. Utilisez `aws cloudformation detect-stack-drift --stack-name <nom>` pour identifier les ressources divergentes. Règle d'or : **ne jamais modifier manuellement une ressource gérée par CloudFormation**. ⚠

Référence : [CloudFormation Template Reference](#)

4. AWS Systems Manager — Automatisation opérationnelle

4.1 Qu'est-ce que Systems Manager ?

AWS Systems Manager (SSM) est un service d'**administration centralisée** et d'**automatisation opérationnelle** pour les ressources AWS et hybrides.

Il remplace le service obsolète **OpsWorks** et fournit des outils modernes pour :

- Exécuter des **scripts à distance** sur des instances EC2 (**Run Command**)
- **Patcher** automatiquement les systèmes (**Patch Manager**)
- Stocker des **configurations centralisées** (**Parameter Store**)
- Automatiser des **tâches complexes** (**Automation Documents**)
- Collectionner un **inventaire** de toutes les ressources (**Inventory**)

- 1 Analogie : Systems Manager est comme un TABLEAU DE CONTRÔLE À DISTANCE
- 2
- 3 Avant (sans SSM) :

```
4 Admin doit se connecter à chaque serveur manuellement :
5     ssh ubuntu@10.0.1.100
6     ssh ubuntu@10.0.1.101
7     ssh ubuntu@10.0.1.102
8     ... exécuter la même commande 100 fois
9
10 Avec Systems Manager Run Command :
11 Admin exécute UNE SEULE commande :
12     aws ssm send-command --document-name "AWS-RunShellScript" \
13                             --parameters commands=["apt-get update"]
14 → Appliquée automatiquement à 100 instances simultaneously
```

4.2 Fonctionnalités clés de Systems Manager

Run Command — Exécuter des scripts à distance

Run Command permet d'exécuter des scripts sans accès SSH direct.

Prérequis :

- Instance EC2 doit avoir le rôle IAM `AmazonSSMManagedInstanceCore`
- L'agent SSM est pré-installé sur les AMI récentes

```
1 # Exemple : Installer Apache HTTP Server sur 5 instances
2 aws ssm send-command \
3     --instance-ids i-12345 i-67890 i-abcde i-fghij i-klmno \
4     --document-name "AWS-RunShellScript" \
5     --parameters 'commands=[
6         "apt-get update",
7         "apt-get install -y apache2",
8         "systemctl start apache2",
9         "systemctl enable apache2"
10    ]'
11
12 # Résultat : les 5 instances exécutent les commandes EN PARALLÈLE
13 # Voir les résultats dans CloudWatch Logs ou via CLI :
14 aws ssm get-command-invocation \
15     --command-id <command-id> \
16     --instance-id i-12345
```

...success **Résultat attendu :**

```
1  # send-command :
2  {
3      "Command": {
4          "CommandId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
5          "DocumentName": "AWS-RunShellScript",
6          "Status": "Pending",
7          "TargetCount": 5,
8          "CompletedCount": 0
9      }
10 }
11
12 # get-command-invocation (après exécution) :
13 {
14     "CommandId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
15     "InstanceId": "i-12345",
16     "Status": "Success",
17     "StatusDetails": "Success",
18     "StandardOutputContent": "Reading package lists...\nBuilding dependency tre
19     "StandardErrorContent": ""
20 }
```

...

Résultat attendu : `send-command` retourne un `CommandId`. `get-command-invocation` affiche le `Status` (`InProgress` → `Success`) et les `StandardOutputContent` avec la sortie de chaque commande exécutée sur l'instance.

Parameter Store — Stocker des configurations centralisées

Parameter Store permet de stocker des variables (secrets, chemins d'accès, configurations) de manière centralisée. L'avantage clé : vos applications lisent leurs secrets via l'API SSM, sans jamais avoir de valeur en clair dans le code ou un fichier `.env`.

```
1  # Stocker un secret (ex. mot de passe database)
2  aws ssm put-parameter \
3      --name /prod/database/password \
4      --value '<VALEUR_SECRETE_NON_VERSIONNEE>' \
5      --type "SecureString" \
```

```

6   --description "Mot de passe RDS pour production"
7
8   # Récupérer la valeur dans une application
9   aws ssm get-parameter \
10    --name /prod/database/password \
11    --with-decryption
12
13  # Stocker une configuration simple
14  aws ssm put-parameter \
15    --name /app/api-url \
16    --value "https://api.example.com" \
17    --type "String"

```

:::success **Résultat attendu :**

```

1  # put-parameter :
2  {
3      "Version": 1,
4      "Tier": "Standard"
5  }
6
7  # get-parameter (avec --with-decryption) :
8  {
9      "Parameter": {
10         "Name": "/prod/database/password",
11         "Type": "SecureString",
12         "Value": "<valeur déchiffrée masquée dans le support>",
13         "Version": 1,
14         "LastModifiedDate": "2026-03-24T10:00:00.000Z",
15         "ARN": "arn:aws:ssm:eu-west-3:123456789012:parameter/prod/database/pass
16         "DataType": "text"
17     }
18 }

```

...

Résultat attendu : `put-parameter` retourne un numéro de version (`"Version": 1`). `get-parameter` retourne le JSON avec `"Value"` déchiffré (grâce à `--with-decryption`). Sans ce flag, `SecureString` serait masqué.

Session Manager — Accès shell sécurisé sans SSH

Session Manager permet de se connecter à une instance EC2 **sans port SSH ouvert**, via la console AWS ou CLI. C'est la méthode recommandée pour accéder aux instances en production — zéro clé SSH à gérer, audit complet automatique.

```
1 # Démarrer une session interactive sur une instance
2 aws ssm start-session \
3     --target i-12345
4
5 # AWS configure le tunnel securely et vous connecte au shell
```

:::success **Résultat attendu :**

```
1 Starting session with SessionId: stagiaire-demo-0abc123def456789
2 sh-4.2$
```

Un shell bash s'ouvre directement sur l'instance sans passer par SSH. Toutes les commandes saisies sont journalisées dans CloudTrail. Si la commande échoue avec `TargetNotConnected`, vérifiez que l'agent SSM est actif (`systemctl status amazon-ssm-agent`) et que le rôle IAM `AmazonSSMManagedInstanceCore` est attaché à l'instance. :::

Note : Un shell s'ouvre sur l'instance (`sh-4.2$`). CloudTrail enregistre les appels d'API liés au démarrage et à l'arrêt de la session, mais pas automatiquement chaque commande saisie dans le shell. Pour conserver les données de session, configurez explicitement la journalisation Session Manager vers CloudWatch Logs ou S3. Si la commande échoue avec `TargetNotConnected`, vérifiez l'état de l'agent SSM, la connectivité vers les endpoints SSM et le rôle IAM de l'instance.

Avantages :

- Pas besoin d'ouvrir le port 22 → Sécurité renforcée
- Audit complet des sessions dans CloudTrail
- Gestion centralisée des accès via IAM

Patch Manager — Appliquer les mises à jour automatiquement

Patch Manager scanne les instances et applique les patches de sécurité/OS. On définit d'abord une **Patch Baseline** (quels patches approuver et dans quel délai), puis on associe cette baseline aux instances.

```
1 # Scanner les instances pour les mises à jour manquantes
```



```

2  aws ssm describe-instance-patches \
3      --instance-id i-12345
4
5  # Créer un plan de patch automatique
6  aws ssm create-patch-baseline \
7      --name "MonLieuxPatchLineMonthly" \
8      --operating-system "UBUNTU" \
9      --approval-rules 'PatchRules=[{PatchFilterGroup={PatchFilters=[{Key=CLASSIFIC

```

:::success **Résultat attendu :**

```

1  # describe-instance-patches :
2  {
3      "Patches": [
4          {
5              "Title": "linux-aws-headers-5.15.0-1056",
6              "KBId": "USN-6819-1",
7              "Classification": "SECURITY",
8              "Severity": "Important",
9              "State": "Missing",
10             "InstalledTime": null
11         },
12         {
13             "Title": "libssl3",
14             "Classification": "SECURITY",
15             "Severity": "Critical",
16             "State": "Missing"
17         }
18     ]
19 }
20
21 # create-patch-baseline :
22 {
23     "BaselineId": "pb-0abc123def456789a",
24     "Name": "MonLieuxPatchLineMonthly",
25     "OperatingSystem": "UBUNTU",
26     "CreatedDate": "2026-03-24T10:00:00.000Z"
27 }

```

...

Résultat attendu : `describe-instance-patches` liste les patchs manquants avec leur sévérité (`Critical`, `Important`...). `create-patch-baseline` retourne un `BaselineId` (ex. `pb-0abc123`). Cette baseline s'applique ensuite via une **Maintenance Window** planifiée.

Référence : [AWS Systems Manager Documentation](#)

4.3 AWS OpsWorks — Gestion de configuration avec Chef et Puppet (Contexte Historique)

⚠ Important : AWS OpsWorks en fin de vie

AWS OpsWorks était un service de **gestion de configuration** basé sur les outils open source **Chef** et **Puppet**. Depuis 2023, AWS recommande **fortement** de migrer vers **AWS Systems Manager** pour les nouvelles infrastructures.

Statut actuel :

- ❌ OpsWorks for Chef Automate : fin de vie et désactivé depuis le 5 mai 2024
- ❌ OpsWorks for Puppet Enterprise : fin de vie et désactivé depuis le 5 mai 2024
- ❌ OpsWorks Stacks : fin de vie et désactivé depuis le 26 mai 2024

Qu'était AWS OpsWorks ?

OpsWorks permettait de **déployer et configurer des applications** sur des instances EC2 en utilisant des **scripts de configuration déclaratifs** :



Lecture du schéma. CloudFormation provisionne l'infrastructure déclarée. Systems Manager agit ensuite sur l'exploitation des nœuds et de leurs configurations. OpsWorks correspond à une génération antérieure de services fondés sur Chef ou Puppet ; il est présenté pour reconnaître les architectures historiques, pas comme choix par défaut pour un nouveau projet.

Migration depuis OpsWorks vers Systems Manager

Si vous héritez d'une infrastructure avec OpsWorks :

```
1 1. Exporter les recettes Chef / configurations Puppet
2 → Convertir en shell scripts / PowerShell pour Systems Manager
3
4 2. Utiliser Systems Manager Run Command pour exécuter les scripts
5 aws ssm send-command \
6   --instance-ids i-12345678 \
7   --document-name "AWS-RunShellScript" \
8   --parameters 'commands=["yum install httpd -y","systemctl start httpd"]'
9
10 3. Utiliser Patch Manager pour appliquer les correctifs
11 aws ssm create-patch-baseline \
12   --name "MonLignePatchMigree" \
13   --operating-system "UBUNTU" \
14   --approval-rules ...
15
16 4. Archiver les ressources OpsWorks
17 Archiver les informations nécessaires avant de retirer les dépendances OpsWorks
```

success **Résultat attendu :**

```
1 send-command (migration depuis OpsWorks) :
2 {
3   "Command": {
4     "CommandId": "c3d4e5f6-a7b8-9012-cdef-a12345678901",
5     "DocumentName": "AWS-RunShellScript",
6     "Status": "Pending",
7     "TargetCount": 1
8   }
9 }
```

...

Ressources de migration

- 🔗 [AWS OpsWorks → Systems Manager Migration Guide](https://docs.aws.amazon.com)
- 🔗 [Pourquoi OpsWorks est obsolète](https://aws.amazon.com/fr/blogs/france/migr)

Conclusion pour les stagiaires : Vous ne créez JAMAIS un nouvel OpsWorks stack. Si vous le rencontrez en production, c'est un signal pour moderniser vers Systems Manager.

5. AWS Elastic Beanstalk — Déploiement simplifié d'applications

5.1 Qu'est-ce que Elastic Beanstalk ?

AWS Elastic Beanstalk est une plateforme PaaS managée qui permet de **déployer des applications web** sans gérer l'infrastructure sous-jacente.

Contrairement à CloudFormation où vous décrivez **chaque ressource manuellement**, Beanstalk **abstrait** la complexité : vous uploadez simplement votre code, et Beanstalk s'occupe de :

- Créer/gérer les instances EC2
- Configurer l'Auto Scaling
- Mettre en place le Load Balancer
- Activer le monitoring CloudWatch
- Gérer les mises à jour de l'OS et du runtime

```
1 Analogie : Elastic Beanstalk vs CloudFormation
2
3 CloudFormation = "Je veux décrire exactement mon infrastructure"
4   - Créer VPC, subnets, security groups, EC2, ALB, RDS, etc.
5   - Contrôle total mais responsabilité complète
6
7 Elastic Beanstalk = "Je veux juste déployer mon app, pas m'embêter avec l'infra"
8   - Upload le code (Node.js, Python, Java, .NET)
9   - Beanstalk crée l'infrastructure automatiquement
10  - Mise en échelle automatique en cas de charge
11  - Moins de contrôle mais moins de friction
```

5.2 Runtimes et plateformes supportées

Elastic Beanstalk supporte plusieurs langages et frameworks :

Langage	Framework	Exemple
---------	-----------	---------

Node.js	Express, Fastify	Application web Node.js classique
Python	Flask, Django	Application Flask avec routes
Java	Spring Boot	Application Spring Boot JAR
.NET	ASP.NET Core	Application web .NET
PHP	Laravel, Symfony	Brochure web PHP
Go	Gin, Echo	Microservice Go
Docker	N'importe quel container	Flexibilité maximale

Référence : [Elastic Beanstalk Platforms](#)

5.3 Déployer une app Node.js avec Elastic Beanstalk (CLI)

Elastic Beanstalk gère toute l'infrastructure à votre place : vous fournissez juste le code. La commande `eb create` provisionne automatiquement une instance EC2, un load balancer, un Auto Scaling Group et CloudWatch. Vous n'avez qu'à pousser votre code.

```
1 # 1. Créer une application Node.js simple (app.js)
2 cat > app.js << 'EOF'
3 const express = require('express');
4 const app = express();
5
6 app.get('/', (req, res) => {
7   res.send('Bonjour depuis Elastic Beanstalk !');
8 });
9
10 const PORT = process.env.PORT || 3000;
11 app.listen(PORT, () => {
12   console.log(`Server running on port ${PORT}`);
13 });
14 EOF
15
16 # 2. Créer un package.json
17 cat > package.json << 'EOF'
18 {
```

```
19     "name": "monappbeanstalk",
20     "version": "1.0.0",
21     "description": "Petite app Express sur Beanstalk",
22     "main": "app.js",
23     "scripts": {
24         "start": "node app.js"
25     },
26     "dependencies": {
27         "express": "^4.18.2"
28     }
29 }
30 EOF
31
32 # 3. Initialiser un environnement Elastic Beanstalk
33 eb init -p node.js-18 monappbeanstalk --region eu-west-1
34
35 # 4. Créer et déployer l'environnement
36 # AWS crée automatiquement :
37 #   - Une application Beanstalk
38 #   - Un environnement (Dev, Staging, Prod, etc.)
39 #   - 1+ instances EC2 (t2.micro par défaut)
40 #   - Un Load Balancer
41 #   - Auto Scaling Group
42 #   - Monitoring CloudWatch
43 eb create monappbeanstalk-env --instance-type t2.micro --envvars NODE_ENV=produ
44
45 # 5. Vérifier l'état du déploiement
46 eb status
47
48 # 6. Voir l'URL publique de l'application
49 eb open
50
51 # 7. Voir les logs en temps réel
52 eb logs -f
53
54 # 8. Augmenter le nombre minimum d'instances
55 eb scale 3
56
57 # 9. Deployer une nouvelle version du code
58 # (après modifi app.js)
```

```
59 eb deploy
60
61 # 10. Supprimer l'application et l'environnement
62 eb terminate monappbeanstalk-env
```

:::success **Résultat attendu :**

```
1 # eb create (extrait de la progression) :
2 Creating application version archive "app-v1".
3 Uploading monappbeanstalk/app-v1.zip to S3. This may take a while.
4 Upload Complete.
5 Environment details for: monappbeanstalk-env
6   Application name: monappbeanstalk
7   Region: eu-west-1
8   Deployed Version: app-v1
9   Environment ID: e-abc123defg
10  Platform: arn:aws:elasticbeanstalk:eu-west-1::platform/Node.js 18 running on
11  Tier: WebServer-Standard-1.0
12  CNAME: monappbeanstalk-env.eu-west-1.elasticbeanstalk.com
13  Updated: 2026-03-24 10:45:00.000000+00:00
14  Status: Launching
15  Health: Grey
16  ...
17 INFO: Successfully launched environment: monappbeanstalk-env
18
19 # eb status :
20 Environment details for: monappbeanstalk-env
21   Status: Ready
22   Health: Green
23   CNAME: monappbeanstalk-env.eu-west-1.elasticbeanstalk.com
24
25 # L'application répond "Bonjour depuis Elastic Beanstalk !" à http://monappbean
```

...

:::warning **Cold start Elastic Beanstalk :** Le premier déploiement (`eb create`) prend généralement 5 à 10 minutes car AWS provisionne l'infrastructure complète (EC2, ELB, Auto Scaling Group). Les déploiements suivants (`eb deploy`) sont plus rapides (1 à 3 minutes). Si `eb status` reste sur `Launching` trop longtemps, consultez les logs avec `eb logs`. :::

Résultat attendu : `eb create` affiche la progression en temps réel (création VPC, EC2, Load Balancer...) et se termine avec l'URL publique de l'application. `eb open` ouvre cette URL dans votre navigateur — vous devez voir “Bonjour depuis Elastic Beanstalk !”.

5.4 Avantages et limitations

Avantage	Limitation
Déploiement simple du code	Contrôle limité sur l'infrastructure
Scalabilité automatique	Moins flexible que CloudFormation
Monitoring intégré	Pas adapté aux architectures très complexes
Mises à jour OS automatiques	Coûts potentiellement plus élevés

Référence : [Elastic Beanstalk Documentation](#) 

5.5 Elastic Beanstalk vs Lambda — Quand choisir quoi ?

Ces deux services répondent à la même question — “comment déployer du code sans gérer de serveurs” — mais avec des philosophies radicalement différentes.

	Beanstalk	Lambda
Exécution	EC2 toujours allumé, Load Balancer, Auto Scaling Group, CloudWatch	Conteneur éphémère, créé à la demande et détruit après exécution
Paradigme	Lift & Shift	Event-Driven

Tableau de comparaison

Critère	Elastic Beanstalk	Lambda
Paradigme	PaaS — application toujours en cours	FaaS — fonction déclenchée à la demande

Infrastructure	EC2 + ELB + ASG (gérés automatiquement)	Aucune instance visible
Démarrage	Toujours chaud	Cold start possible (ms à quelques s)

⚠ **Lambda Cold Starts** : Lors du premier appel d'une fonction Lambda (ou après une longue période d'inactivité), AWS doit initialiser le conteneur d'exécution — c'est le **cold start**. La latence peut aller de quelques dizaines de ms (Node.js/Python) à plusieurs secondes (Java). Solutions : **Provisioned Concurrency** (maintient N conteneurs chauds en permanence, payant), ou choisir un runtime léger (Node.js/Python) pour les APIs sensibles à la latence. ⚡ | **Durée max d'exécution** | Illimitée | **15 minutes** | | **Mémoire max** | Celle de l'instance (jusqu'à 384 Go) | **10 Go** | | **Stockage local** | EBS persistant | **/tmp : 10 Go seulement** | | **Langages supportés** | Java, Node.js, Python, Ruby, PHP, Go, .NET | Java, Node.js, Python, Ruby, Go, .NET, Rust + custom runtime | | **Modèle de coût** | Ressources sous-jacentes : instances, équilibrage, stockage et transfert | Requêtes, durée d'exécution, mémoire et services associés | | **Scaling** | Auto Scaling Group (minutes) | Instantané, jusqu'à 1 000 exécutions parallèles | | **État** | Stateful possible (session, fichiers) | Stateless obligatoire | | **Réseau** | VPC natif, Security Groups | VPC optionnel | | **Déploiement** | ZIP, WAR, Docker, `eb deploy` | ZIP, container, `aws lambda update-function-code` |

Comparer les modèles de coût

Elastic Beanstalk n'ajoute pas de frais de service propres, mais crée des ressources facturables. L'estimation doit inclure les instances EC2, l'équilibreur de charge, les volumes EBS, les adresses et le transfert de données. Une capacité maintenue en fonctionnement continue d'être facturée même lorsqu'elle reçoit peu de trafic.

Lambda se calcule à partir du nombre de requêtes et des ressources consommées pendant l'exécution. Il faut aussi compter les services périphériques : API Gateway, journaux CloudWatch, stockage, transfert et éventuelle concurrence provisionnée.

La comparaison correcte utilise le **même scénario de charge** : nombre de requêtes, durée moyenne, mémoire, trafic réseau et disponibilité attendue. Les tarifs et offres gratuites évoluent ; saisissez ces valeurs dans [AWS Pricing Calculator](#) au moment de l'étude.

Quand utiliser lequel ?

- 1 CHOISIR BEANSTALK si :
- 2 ☒ Application web traditionnelle (Django, Express, Spring Boot, WordPress)
- 3 ☒ Traitement long (> 15 minutes)
- 4 ☒ Besoin de sessions persistantes côté serveur

- 5 ✓ Migration d'une app existante ("lift & shift")
- 6 ✓ Besoin d'accès à des fichiers locaux entre requêtes
- 7 ✓ Équipe non familière avec l'architecture event-driven
- 8
- 9 CHOISIR LAMBDA si :
- 10 ✓ API REST légère (avec API Gateway)
- 11 ✓ Traitement d'événements (upload S3, message SQS, stream DynamoDB)
- 12 ✓ Tâches planifiées (cron CloudWatch Events)
- 13 ✓ Traitement de fichiers (redimensionnement images, parsing CSV)
- 14 ✓ Webhooks, notifications, automatisations
- 15 ✓ Trafic très variable (pics et creux importants)
- 16 ✓ Budget serré avec faible volumétrie

💡 **En pratique** : beaucoup d'architectures modernes combinent les deux. Beanstalk pour le frontend/API principale, Lambda pour les traitements en arrière-plan (envoi d'emails, génération de rapports, nettoyage de données).

👤 [Documentation AWS Lambda](#)

6. Amazon CloudWatch — Supervision et alarmes

6.1 Qu'est-ce que CloudWatch ?

Amazon CloudWatch est le service de **monitoring centralisé** d'AWS. Il collecte, stocke et affiche des métriques sur :

- **Instances EC2** : CPU, mémoire réseau, I/O disque
- **Bases RDS** : connexions actives, CPU, I/O
- **Load Balancers** : requêtes/seconde, latence
- **Applications custom** : envoi de métriques via API



Lecture du schéma. Les services et applications publient métriques et journaux dans CloudWatch. Les tableaux de bord servent à observer, tandis que les alarmes évaluent des conditions. Une alarme peut déclencher une notification ou une automatisation, mais elle ne corrige pas un incident sans action associée.

6.2 Créer une alarme CloudWatch (CLI)

```
1  # 1. Créer une alarme sur la métrique CPU d'une instance EC2
2  # L'alarme se déclenche si CPU > 80% pendant 2 périodes consécutives (10 min)
3  aws cloudwatch put-metric-alarm \
4      --alarm-name "MonInstance-CPU-Élevé" \
5      --alarm-description "Alerte CPU élevé instance EC2" \
6      --metric-name CPUUtilization \
7      --namespace AWS/EC2 \
8      --statistic Average \
9      --period 300 \
10     --threshold 80 \
11     --comparison-operator GreaterThanThreshold \
12     --evaluation-periods 2 \
13     --dimensions Name=InstanceId,Value=i-12345
14
15 # 2. Ajouter une action SNS (envoyer un email)
16 # D'abord, créer un sujet SNS
17 aws sns create-topic --name MonTopicAlarmes
18 # Output : TopicArn: arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes
19
20 # S'abonner au sujet (recevoir les notifications)
21 aws sns subscribe \
```

```

22  --topic-arn arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes \
23  --protocol email \
24  --notification-endpoint admin@example.com
25
26  # 3. Modifier l'alarme pour envoyer une notification
27  aws cloudwatch put-metric-alarm \
28  --alarm-name "MonInstance-CPU-Élevé" \
29  --alarm-description "Alerte CPU élevé instance EC2" \
30  --metric-name CPUUtilization \
31  --namespace AWS/EC2 \
32  --statistic Average \
33  --period 300 \
34  --threshold 80 \
35  --comparison-operator GreaterThanThreshold \
36  --evaluation-periods 2 \
37  --dimensions Name=InstanceId,Value=i-12345 \
38  --alarm-actions arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes
39
40  # 4. Lister toutes les alarmes
41  aws cloudwatch describe-alarms
42
43  # 5. Supprimer une alarme
44  aws cloudwatch delete-alarms --alarm-names "MonInstance-CPU-Élevé"

```

:::success **Résultat attendu :**

```

1  # sns create-topic :
2  {
3      "TopicArn": "arn:aws:sns:eu-west-1:123456789012:MonTopicAlarmes"
4  }
5
6  # sns subscribe :
7  {
8      "SubscriptionArn": "pending confirmation"
9  }
10 # → Un email est envoyé à admin@example.com avec un lien de confirmation
11
12 # put-metric-alarm : (pas de sortie si succès – code HTTP 200)
13
14 # describe-alarms (extrait) :

```

```

15  {
16      "MetricAlarms": [
17          {
18              "AlarmName": "MonInstance-CPU-Élevé",
19              "AlarmDescription": "Alerte CPU élevé instance EC2",
20              "StateValue": "OK",
21              "MetricName": "CPUUtilization",
22              "Threshold": 80.0,
23              "Period": 300,
24              "EvaluationPeriods": 2
25          }
26      ]
27  }

```

...

6.3 Envoyer des métriques custom depuis une application

Applications peuvent envoyer des métriques CloudWatch pour monitorer des KPI métier :

```

1  # Exemple : envoyer une métrique custom "OrdersPerMinute"
2  aws cloudwatch put-metric-data \
3      --namespace "MonApplication" \
4      --metric-name "OrdersPerMinute" \
5      --value 42 \
6      --unit Count \
7      --timestamp 2025-03-24T14:30:00Z

```

:::success **Résultat attendu :**

```

1  # put-metric-data : pas de sortie si succès (HTTP 200)
2  # La métrique est visible dans CloudWatch Console sous "MonApplication > Orders
3  # après environ 1 minute de délai d'ingestion.

```

...

```

1  # Ou dans un script Python :
2  import boto3
3

```

```

4  cloudwatch = boto3.client('cloudwatch')
5
6  # Envoyer une métrique custom
7  cloudwatch.put_metric_data(
8      Namespace='MonApplication',
9      MetricData=[
10         {
11             'MetricName': 'PedidosProcessados',
12             'Value': 42,
13             'Unit': 'Count',
14             'Timestamp': datetime.utcnow()
15         }
16     ]
17 )

```

6.4 Tableaux de bord CloudWatch

CloudWatch permet de créer des **dashboards** personnalisés affichant plusieurs métriques :

```

1  # Créer un dashboard JSON
2  cat > dashboard.json << 'EOF'
3  {
4      "widgets": [
5          {
6              "type": "metric",
7              "properties": {
8                  "metrics": [
9                      [ "AWS/EC2", "CPUUtilization", { "stat": "Average" } ],
10                     [ ".", "NetworkIn", { "stat": "Sum" } ]
11                 ],
12                 "period": 300,
13                 "stat": "Average",
14                 "region": "eu-west-1",
15                 "title": "Métriques EC2"
16             }
17         }
18     ]
19 }

```

```
20 EOF
21
22 # Créer le dashboard
23 aws cloudwatch put-dashboard \
24     --dashboard-name "MonDashboard" \
25     --dashboard-body file://dashboard.json
```

:::success **Résultat attendu :**

```
1 # put-dashboard :
2 {
3     "DashboardValidationMessages": []
4 }
5 # Le tableau de bord "MonDashboard" est maintenant visible dans la console Clou
6 # Accès : CloudWatch → Dashboards → MonDashboard
```

...

Référence : [CloudWatch Documentation](#) 

6.5 AWS SDK pour Développeurs — Automatisation Programmatique

Jusque-là, nous avons utilisé la **CLI AWS** pour exécuter des commandes manuellement. Mais les **SDK AWS** permettent d'intégrer AWS directement dans du code applicatif (Python, Node.js, Java, Go, etc.).

Qu'est-ce que le SDK AWS ?

SDK = kit de développement fourni par AWS dans plusieurs langages pour interagir avec les services AWS par programmation.

```
1 Analogie : CLI vs SDK
2
3 CLI (AWS CLI)
4   ↓
5 Outil en ligne de commande
6 Exécute des commandes manuellement ou dans des scripts bash
7 Exemple : aws ec2 describe-instances
8
9 SDK (boto3, SDK.js, SDK.java)
```

```
10     ↓
11     Bibliothèque logicielle intégrée dans votre code
12     Votre application Python/Node/Java appelle AWS directement
13     Exemple : ec2_client.describe_instances()
```

SDK AWS Disponibles

Langage	Nom SDK	Cas d'usage
Python	boto3	Data science, Lambda, backend
JavaScript/Node.js	AWS SDK for JavaScript	Applications web, serverless
Java	AWS SDK for Java	Enterprise, Spring Boot
Go	AWS SDK for Go	CLI tools, microservices
C#/.NET	AWS SDK for .NET	Windows, applications d'entreprise
PHP	AWS SDK for PHP**	Applications web, Laravel

Introduction à boto3 (Python)

boto3 est la **SDK AWS officielle pour Python**. Elle est utilisée dans :

- Scripts d'automatisation
- Applications Lambda
- Tâches cron de maintenance
- Outils de gestion d'infrastructure

Installation de boto3

```
1  # Installer boto3
2  pip install boto3
3
4  # Vérifier l'installation
5  python3 -c "import boto3; print(boto3.__version__)"
```

:::success **Résultat attendu :**

```
1  Collecting boto3
2    Downloading boto3-1.34.69-py3-none-any.whl (139 kB)
```



```
3 Successfully installed boto3-1.34.69 botocore-1.34.69 s3transfer-0.10.1
4 1.34.69
```

⋮

Exemple 1 : Lister les instances EC2

```
1 import boto3
2
3 # Créer un client EC2
4 ec2_client = boto3.client('ec2', region_name='eu-west-3')
5
6 # Lister toutes les instances
7 response = ec2_client.describe_instances()
8
9 # Parcourir les instances
10 for reservation in response['Reservations']:
11     for instance in reservation['Instances']:
12         instance_id = instance['InstanceId']
13         instance_type = instance['InstanceType']
14         state = instance['State']['Name']
15
16         # Afficher les informations
17         print(f"Instance : {instance_id}")
18         print(f"  Type : {instance_type}")
19         print(f"  État : {state}")
20         print()
```

Résultat :

```
1 Instance : i-0123456789abcdef0
2   Type : t3.micro
3   État : running
4
5 Instance : i-0987654321abcdef0
6   Type : t3.small
7   État : stopped
```

Exemple 2 : Créer une snapshot EBS

```

1  import boto3
2
3  # Créer un client EC2
4  ec2_client = boto3.client('ec2', region_name='eu-west-3')
5
6  # Créer un snapshot du volume vol-12345678
7  response = ec2_client.create_snapshot(
8      VolumeId='vol-12345678',
9      Description='Sauvegarde avant migration',
10     TagSpecifications=[
11         {
12             'ResourceType': 'snapshot',
13             'Tags': [
14                 {'Key': 'Name', 'Value': 'backup-migration-2026-03-24'},
15                 {'Key': 'Environment', 'Value': 'production'}
16             ]
17         }
18     ]
19 )
20
21 # Afficher l'ID du snapshot créé
22 snapshot_id = response['SnapshotId']
23 progress = response['Progress']
24
25 print(f"Snapshot créé : {snapshot_id}")
26 print(f"Progression : {progress}")

```

Exemple 3 : Arrêter une instance EC2

```

1  import boto3
2
3  # Créer un client EC2
4  ec2_client = boto3.client('ec2', region_name='eu-west-3')
5
6  # Arrêter une instance
7  instance_id = 'i-0123456789abcdef0'
8
9  response = ec2_client.stop_instances(InstanceIds=[instance_id])
10
11 # Vérifier que l'arrêt est en cours

```

```

12 for instance in response['StoppingInstances']:
13     print(f"Instance {instance['InstanceId']} est en cours d'arrêt")
14     print(f"État précédent : {instance['PreviousState']['Name']}")
15     print(f"État courant : {instance['CurrentState']['Name']}")

```

Exemple 4 : Créer une alarme CloudWatch

```

1 import boto3
2
3 # Créer un client CloudWatch
4 cloudwatch_client = boto3.client('cloudwatch', region_name='eu-west-3')
5
6 # Créer une alarme si CPU > 80%
7 cloudwatch_client.put_metric_alarm(
8     AlarmName='CPU-Haute-Production',
9     ComparisonOperator='GreaterThanThreshold',
10    EvaluationPeriods=2,
11    MetricName='CPUUtilization',
12    Namespace='AWS/EC2',
13    Period=300, # 5 minutes
14    Statistic='Average',
15    Threshold=80.0,
16    ActionsEnabled=True,
17    AlarmActions=['arn:aws:sns:eu-west-3:123456789:AlertesProduction'],
18    Dimensions=[
19        {
20            'Name': 'InstanceId',
21            'Value': 'i-0123456789abcdef0'
22        }
23    ]
24 )
25
26 print("Alarme CloudWatch créée avec succès")

```

Bonnes pratiques boto3

- 1 ✓ Utiliser des variables d'environnement ou des profils AWS pour les credential
- 2 ✓ Gérer les erreurs avec `try/except`
- 3 ✓ Utiliser des context managers ou des sessions boto3
- 4 ✓ Documenter chaque appel `API` avec un commentaire

- 5 ✓ Tester en environnement non-production d'abord
- 6 ✓ Utiliser des rôles IAM appropriés (pas de clés d'accès root)

7. AWS Well-Architected Framework — Mise en pratique

Le Chapitre 1 a introduit les six piliers du **AWS Well-Architected Framework** (Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability) avec leurs bonnes pratiques respectives. Maintenant que vous avez manipulé IAM, S3, EC2, Lambda, RDS, VPC et CloudFormation, vous disposez de tous les services nécessaires pour appliquer concrètement ce framework à un cas réel.

:::info **Besoin d'un rappel des 6 piliers ?** Retournez au Chapitre 1, section 7 — définitions, questions clés et bonnes pratiques par pilier y sont détaillées. :::

7.1 Cas d'étude : Application “CloudPizza”

Imaginons une application de commande de pizzas. Appliquons les 6 piliers :

Pilier	Décision	Justification
Operational Excellence	Déployer via CloudFormation + CI/CD	Répétabilité, traçabilité
Security	IAM par service, KMS pour BDD, bucket S3 privé	Moindre privilège, conformité
Reliability	Multi-AZ, ALB, RDS Multi-AZ, snapshots EBS quotidiens	RTO 1h, RPO 1h
Performance	Lambda et DynamoDB peuvent retirer la gestion de serveurs	Mise à l'échelle gérée, dans les quotas et avec un modèle de données adapté
Cost Optimization	Reserved Instances pour serveurs stables, Spot pour batch	Réduire 40% des coûts

7.2 Rappel — AWS Compute Optimizer et le pilier Cost Optimization

Le Chapitre 3 a détaillé le fonctionnement d'**AWS Compute Optimizer** (collecte CloudWatch, analyse ML, recommandations chiffrées) — c'est l'outil concret qui alimente le pilier **Cost Optimization** vu ci-dessus : dans le cas CloudPizza, c'est lui qui permettrait de vérifier a posteriori que les Reserved Instances choisies sont bien dimensionnées à l'usage réel.

:::info **Besoin d'un rappel du fonctionnement de Compute Optimizer ?** Retournez au Chapitre 3, section 7 — commande CLI complète, exemple de sortie JSON et cas concret `t3.large → t3.small` y sont détaillés. :::

8. Services complémentaires — Queues et événements

8.1 Amazon SQS — File d'attente de messages

Amazon SQS (Simple Queue Service) est une **file d'attente de messages** complètement gérée.

Cas d'usage : Découpler des composants d'une application.

- 1 Analogie : SQS est comme une BOÎTE AUX LETTRES
- 2
- 3 Sans SQS (couplage fort) :
- 4 Producteur (app web) → appelle directement Consommateur (worker)
- 5 Si worker est en panne → producteur attend → demande utilisateur bloquée
- 6
- 7 Avec SQS (découplage) :
- 8 Producteur → envoie message dans SQS → retour immédiat
- 9 Consommateur → consomme messages quand il est prêt (même en panne, pas de pe:

Voici la séquence complète : créer la queue, envoyer un message, le lire, puis le supprimer. La suppression explicite est obligatoire — SQS ne supprime pas automatiquement un message après lecture (pour éviter la perte en cas d'échec).

```
1  # Créer une queue SQS
2  aws sqs create-queue --queue-name MonQueue
3
4  # Envoyer un message
5  aws sqs send-message \
6    --queue-url https://sqs.eu-west-1.amazonaws.com/123456789/MonQueue \
7    --message-body "Bonjour depuis CloudFormation"
8
9  # Consommer un message (avec delete)
10 aws sqs receive-message \
11    --queue-url https://sqs.eu-west-1.amazonaws.com/123456789/MonQueue \
12    --max-number-of-messages 1
13
14 # Supprimer le message de la queue
15 aws sqs delete-message \
16    --receipt-handle <receipt-handle>
```

:::success **Résultat attendu :**

```
1  # create-queue :
2  {
3    "QueueUrl": "https://sqs.eu-west-1.amazonaws.com/123456789012/MonQueue"
4  }
5
6  # send-message :
7  {
8    "MD5OfMessageBody": "9c7c6f0f3f748bdfa5a5e6e7c8d9e0f1",
9    "MessageId": "msg-0abc123def456789a"
10 }
11
12 # receive-message :
13 {
14   "Messages": [
15     {
16       "MessageId": "msg-0abc123def456789a",
17       "ReceiptHandle": "AQEB...longstring...",
```

```

18         "MD5OfBody": "9c7c6f0f3f748bdfa5a5e6e7c8d9e0f1",
19         "Body": "Bonjour depuis CloudFormation"
20     }
21 ]
22 }
23
24 # delete-message : pas de sortie si succès (HTTP 200)

```

...

8.2 Amazon SNS — Notifications pubsub

Amazon SNS (Simple Notification Service) est un service de **notifications pub/sub**.

- 1 Différence SQS vs SNS :
- 2 SQS : 1 producteur → 1 queue → 1 consommateur (FIFO ou parallèle)
- 3 SNS : 1 producteur → N abonnés (email, SMS, SQS, Lambda, HTTP)

Voici comment créer un topic SNS, y abonner une adresse email, et publier un message qui sera envoyé à tous les abonnés simultanément :

```

1 # Créer un sujet SNS
2 aws sns create-topic --name MonTopicAlarmes
3
4 # Ajouter un abonnement email
5 aws sns subscribe \
6     --topic-arn arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes \
7     --protocol email \
8     --notification-endpoint admin@example.com
9
10 # Publier un message
11 aws sns publish \
12     --topic-arn arn:aws:sns:eu-west-1:123456789:MonTopicAlarmes \
13     --message "Alerte : CPU élevé détecté !"

```

:::success **Résultat attendu :**

```

1 # create-topic :
2 {

```

```
3     "TopicArn": "arn:aws:sns:eu-west-1:123456789012:MonTopicAlarmes"
4 }
5
6 # subscribe :
7 {
8     "SubscriptionArn": "pending confirmation"
9 }
10 # → Un email est envoyé avec un lien de confirmation
11
12 # publish :
13 {
14     "MessageId": "pub-0abc123def456789a"
15 }
16 # → Tous les abonnés (email, SQS, Lambda) reçoivent le message instantanément
```

...

Référence : [Amazon SQS](#) Référence : [Amazon SNS](#)

8.3 Architectures découplées, microservices et sans serveur

Le schéma suivant rassemble les briques d'une architecture sans serveur orientée événements. L'objectif n'est pas de mémoriser une succession d'icônes : suivez le trajet d'une requête, puis identifiez les points où l'application absorbe un pic, conserve un état ou isole une panne.

API Gateway reçoit les appels synchrones, tandis qu'une file ou un bus d'événements permet de différer certains traitements. Lambda exécute le code sans serveur à administrer, mais le client reste responsable des permissions IAM, des dépendances, de l'idempotence, des erreurs partielles et du cycle de vie des données.

SQS et SNS résolvent le découplage entre deux composants pris isolément. Une architecture applicative complète va plus loin : elle assemble plusieurs services managés pour qu'aucun composant ne dépende directement de la disponibilité d'un autre, et que chaque brique puisse évoluer, tomber en panne ou être remplacée sans effet domino sur le reste du système. C'est le principe des **microservices** : découper une application monolithique en plusieurs services indépendants, chacun responsable d'une capacité métier précise (paiement, catalogue, notifications), communiquant entre eux par API ou par messages plutôt que par appels de fonction directs en mémoire.

Couplage fort vs couplage faible — le test décisif


```

1 Couplage fort (monolithe classique) :
2   Service A appelle directement Service B (HTTP synchrone ou appel de fonction)
3   → Si B est lent ou en panne, A attend, se bloque, ou échoue en cascade
4   → Faire évoluer B (changer sa techno, sa capacité) impose de coordonner A
5
6 Couplage faible (architecture découplée) :
7   Service A publie un événement/message, sans savoir qui le consommera
8   → B (et C, D...) consomment à leur rythme, indépendamment de la disponibilité
9   → B peut tomber, redémarrer, ou être remplacé sans qu'A ne le sache

```

Le test décisif pour repérer un couplage fort évitable : si le service producteur doit attendre une réponse synchrone du consommateur pour continuer son propre travail, alors qu'il n'a besoin d'aucune information en retour, c'est un candidat naturel au découplage via SQS ou SNS.

Amazon API Gateway — le point d'entrée unifié

Dans une architecture microservices, chaque service pourrait exposer sa propre adresse réseau — mais cela oblige les clients (applications mobiles, sites web, partenaires) à connaître et gérer N adresses différentes, et complique la sécurisation (authentification à répliquer partout). **Amazon API Gateway** résout ce problème en offrant un point d'entrée HTTP unique, qui route chaque requête vers le bon service backend (Lambda, conteneur, serveur EC2) selon l'URL et la méthode appelées.

```

1 Client mobile/web
2   |
3   ▼
4   ┌──────────────────┐
5   │ API Gateway      │ ← authentification centralisée, throttling, cache
6   │ /users    → Lambda A
7   │ /orders   → Lambda B
8   │ /catalog  → ECS Fargate
9   └──────────────────┘

```

API Gateway prend en charge, sans code supplémentaire à écrire dans chaque microservice : l'authentification (intégration IAM, Cognito, ou clé API), la limitation de débit (*throttling*) pour protéger les backends d'un pic de trafic, la mise en cache des réponses, et la transformation de requêtes/réponses (mapping de formats).

```

1 # Créer une API REST
2 aws apigateway create-rest-api --name "MonAPI-Catalogue"
3
4 # Créer une ressource sous la racine

```

```

5  aws apigateway create-resource \
6    --rest-api-id <api-id> \
7    --parent-id <root-resource-id> \
8    --path-part "produits"
9
10 # Associer une méthode GET à une fonction Lambda (intégration proxy)
11 aws apigateway put-integration \
12   --rest-api-id <api-id> \
13   --resource-id <resource-id> \
14   --http-method GET \
15   --type AWS_PROXY \
16   --integration-http-method POST \
17   --uri arn:aws:apigateway:eu-west-1:lambda:path/2015-03-31/functions/arn:aws:1

```

:::success **Résultat attendu (create-rest-api) :**

```

1  {
2    "id": "a1b2c3d4e5",
3    "name": "MonAPI-Catalogue",
4    "createdDate": "2026-07-27T10:00:00Z",
5    "apiKeySource": "HEADER",
6    "endpointConfiguration": {
7      "types": ["EDGE"]
8    }
9  }

```

...

AWS Step Functions — orchestrer plusieurs services dans un workflow

Certains traitements ne se résument pas à un simple message transmis d'un service à l'autre : ils enchaînent plusieurs étapes avec de la logique conditionnelle (si le paiement échoue, annuler la réservation), des étapes parallèles (vérifier le stock et calculer les frais de port en même temps), et des reprises sur erreur. **AWS Step Functions** modélise ce type de workflow sous forme de machine à états (*state machine*), où chaque état est typiquement une fonction Lambda, un appel à un autre service AWS, ou une branche de décision.

```

1  {
2    "Comment": "Traitement d'une commande e-commerce",
3    "StartAt": "VerifierStock",
4    "States": {

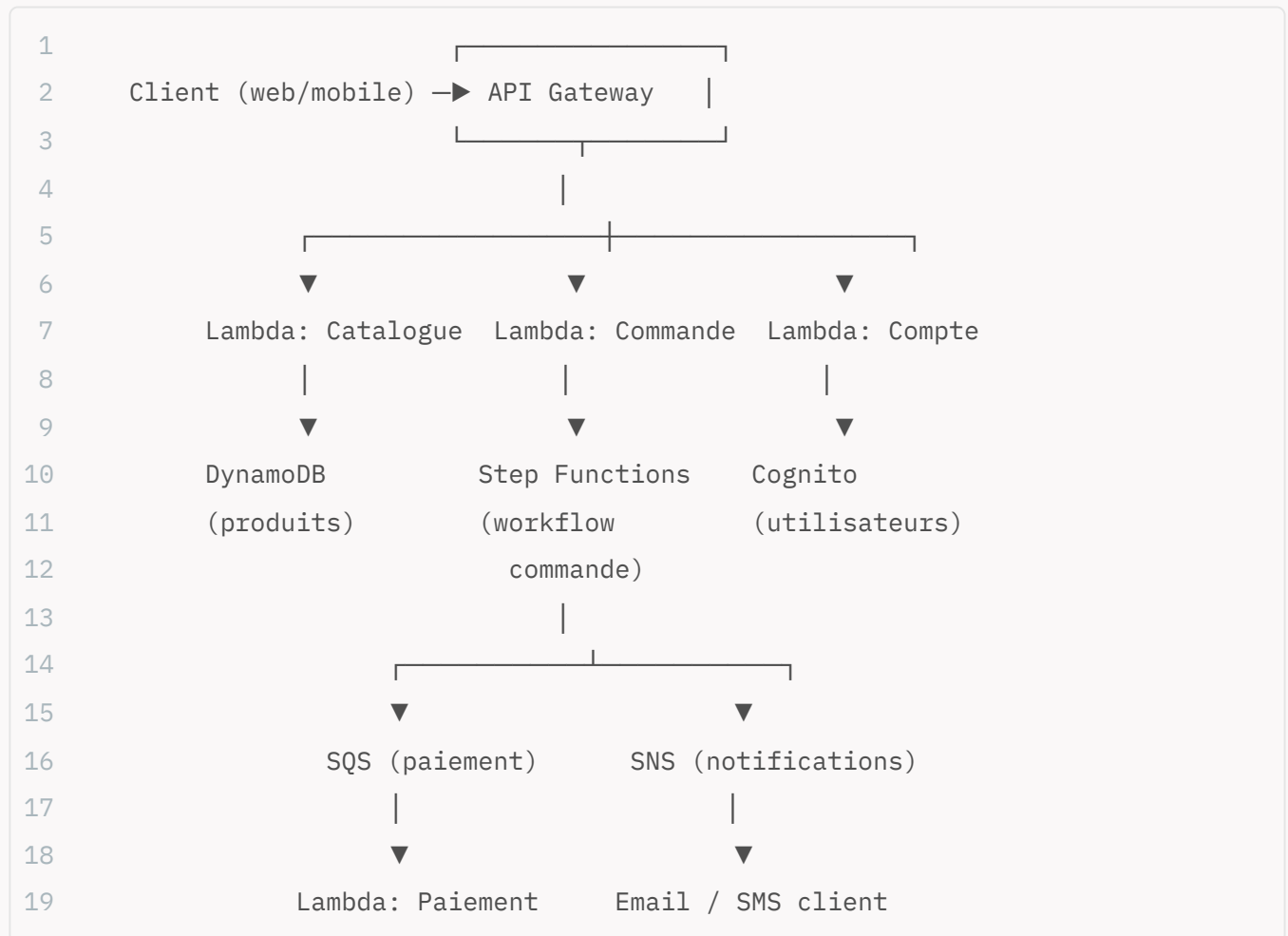
```

```
5     "VerifierStock": {
6         "Type": "Task",
7         "Resource": "arn:aws:lambda:eu-west-1:123456789:function:verifier-stock",
8         "Next": "StockDisponible"
9     },
10    "StockDisponible": {
11        "Type": "Choice",
12        "Choices": [
13            {
14                "Variable": "$.stock_ok",
15                "BooleanEquals": true,
16                "Next": "TraiterPaielement"
17            }
18        ],
19        "Default": "AnnulerCommande"
20    },
21    "TraiterPaielement": {
22        "Type": "Task",
23        "Resource": "arn:aws:lambda:eu-west-1:123456789:function:traiter-paiement",
24        "Catch": [
25            {
26                "ErrorEquals": ["PaielementRefuse"],
27                "Next": "AnnulerCommande"
28            }
29        ],
30        "Next": "ConfirmerCommande"
31    },
32    "ConfirmerCommande": {
33        "Type": "Task",
34        "Resource": "arn:aws:lambda:eu-west-1:123456789:function:confirmer-commande",
35        "End": true
36    },
37    "AnnulerCommande": {
38        "Type": "Task",
39        "Resource": "arn:aws:lambda:eu-west-1:123456789:function:annuler-commande",
40        "End": true
41    }
42 }
43 }
```

L'intérêt par rapport à enchaîner ces appels directement dans le code d'une seule fonction Lambda : chaque état est visible, monitorable et rejouable indépendamment dans la console AWS (Step Functions affiche un diagramme visuel de l'exécution, avec l'état exact atteint en cas d'échec) — un débogage bien plus direct qu'une pile d'appels imbriqués dans les logs CloudWatch d'une fonction monolithique.

Architecture microservices sans serveur complète — exemple

L'illustration ci-dessous assemble les briques vues dans ce chapitre et les précédents en une architecture microservices sans serveur cohérente pour une application de commande en ligne :



Aucun composant de ce schéma ne connaît directement l'adresse réseau d'un autre composant en amont : le client ne connaît que l'URL d'API Gateway, les Lambdas ne connaissent que les ARN des ressources qu'elles invoquent, et la communication asynchrone (SQS/SNS) élimine toute dépendance temporelle stricte entre le traitement de la commande et l'envoi de la notification. C'est cette absence de dépendance directe qui permet à chaque brique d'être mise à l'échelle, remplacée ou de tomber en panne sans effet domino sur le reste de l'architecture — le principe même du découplage appliqué à l'échelle d'un système complet.

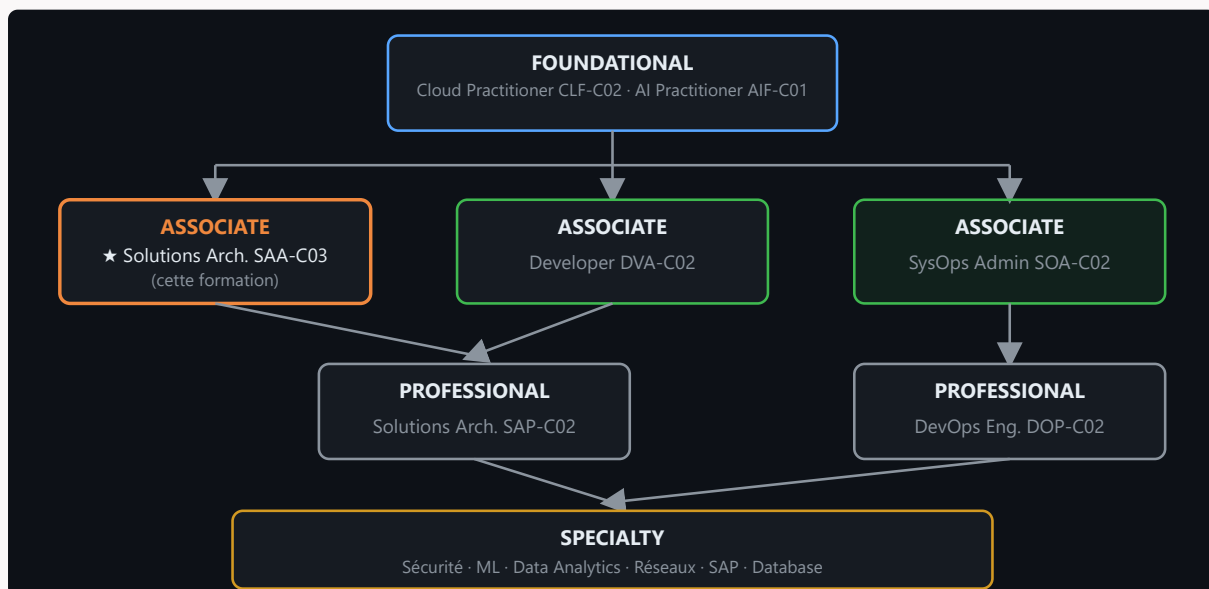
⚠ Piège fréquent : multiplier les microservices sans réel besoin métier augmente la complexité opérationnelle (plus de composants à surveiller, plus de latence réseau entre services, plus de

scénarios d'échec partiel à gérer) sans bénéfice proportionnel. Le découpage en microservices se justifie quand des équipes différentes doivent déployer indépendamment, quand des composants ont des besoins de mise à l'échelle très différents (le service de paiement encaisse un pic le vendredi soir, le catalogue reste stable), ou quand la résilience d'un composant ne doit jamais bloquer les autres. Un monolithe bien structuré reste souvent le bon choix pour une application simple ou une petite équipe. :::

Référence : [Amazon API Gateway](#) Référence : [AWS Step Functions](#)

9. Certifications AWS — Objectif SAA-C03

Le Chapitre 1 a présenté les quatre niveaux de certification AWS (Fundamental, Associate, Professional, Specialty) et pourquoi cette formation cible la **Solutions Architect Associate (SAA-C03)**. Maintenant que vous avez vu l'ensemble des services du programme, voici ce qui compte vraiment pour préparer concrètement cet examen : la pondération réelle des domaines testés.



Lecture du schéma. Les niveaux représentent une progression de profondeur, pas des prérequis obligatoires entre toutes les certifications. Le choix d'un examen dépend du rôle visé et de l'expérience pratique ; les codes et versions d'examen doivent toujours être vérifiés sur le site officiel AWS avant inscription.

:::info **Besoin d'un rappel des 4 niveaux de certification ?** Retournez au Chapitre 1, section 1.3. :::

9.3 Domaines couverts par la SAA-C03

Domaine	Pondération
Design d'architectures résilientes	26 %
Design d'architectures haute performance	24 %
Design d'architectures sécurisées	30 %
Design d'architectures optimisées en coût	20 %

Les quatre piliers de cette formation correspondent exactement à ces quatre domaines.

9.4 Certifications spécialisées

Les certifications **Specialty** valident une expertise approfondie sur un domaine précis. Elles nécessitent généralement une expérience pratique significative.

Specialty	Code	Domaine
Security	SCS-C03	Sécurité des charges de travail et des applications AWS
Advanced Networking	ANS-C01	VPC avancé, Direct Connect, Transit Gateway

Le préfixe du code identifie l'examen et le suffixe **-C0x** sa version. Le catalogue et les codes changent : la liste officielle des guides d'examen reste la seule référence avant une inscription.

Pour aller plus loin : [Parcours de certifications AWS](#) — programme officiel AWS (Solution Architect, Developer, SysOps, DevOps...)

Ressources

Documentation officielle AWS

- [AWS CloudFormation Documentation](#)
- [AWS Systems Manager Documentation](#)

- [AWS Elastic Beanstalk Documentation](#)
- [Amazon CloudWatch Documentation](#)
- [AWS Well-Architected Framework](#)
- [AWS Certification](#)

Quiz interactif du chapitre

Choisissez une ou plusieurs réponses selon la question. La correction expliquée apparaît immédiatement. Les questions et les propositions restent dans un ordre stable.

Le quiz interactif est disponible dans la version web du support.

Vue Graphique

